

Лекции по курсу “Обучение с подкреплением”

Никита Юдин, МФТИ

2026

Contents

1	Лекция 1. Вводная лекция	16
1.1	Команда курса	16
1.2	Правила игры	16
1.3	Обучение с учителем и обучение с подкреплением	17
1.4	Примеры предметных областей	17
1.4.1	Онлайн-реклама	17
1.4.2	Робототехника	17
1.4.3	Видеоигровые среды	18
1.4.4	Другие предметные области	18
1.5	Возникающие проблемы	18
1.6	Пример модели: многорукий бандит	18
1.7	Сравнение типов обучения	19
1.7.1	Обучение с учителем и обучение с подкреплением	19
1.7.2	Обучение без учителя и обучение с подкреплением	19
1.8	Постановка задачи обучения с подкреплением	20
1.9	Формализация	20
1.10	Марковский процесс принятия решений	21
1.11	Метод кросс-энтропии в RL	21
1.11.1	Идея метода кросс-энтропии	21
1.11.2	Схема метода кросс-энтропии	22
1.11.3	Применение метода кросс-энтропии к оптимизации	22
1.12	Метод кросс-энтропии в терминах RL	23
1.12.1	Интерпретация алгоритма	24
1.13	Источники	24
2	Лекция 2. Марковские процессы принятия решений	24
2.1	Марковский процесс принятия решений	24
2.2	MDP. Принятие решений	26
2.3	MDP. Ядро	26

2.4	DMDP. V -функция ценности	27
2.5	DMDP. Q -функция ценности	27
2.6	DMDP. Дисконтированная награда	28
2.7	Ограниченность функций ценности	28
2.8	DMDP. Уравнения Беллмана для фиксированной политики	29
2.9	DMDP. Задача обучения с подкреплением	29
2.10	Уравнение оптимальности Беллмана для V^*	30
2.11	Уравнение оптимальности Беллмана для Q^*	30
2.12	Критерий оптимальности относительно Q -функции	30
2.13	DMDP. Уравнения оптимальности Беллмана	31
2.14	Сдвиг функций ценности и инвариантность оптимальной политики	31
2.14.1	Основное свойство оптимальных V/Q -функций	31
2.14.2	Константный сдвиг функции награды	32
2.14.3	Аддитивное преобразование награды	32
2.15	Предположения для построения алгоритмов	33
2.16	Построение алгоритма обучения политики	33
2.17	Iterative Policy Evaluation	34
2.18	Улучшение политики	35
2.19	Теорема об улучшении политики	35
2.20	Policy Iteration	36
2.21	Value Iteration	37
2.22	Сходимость алгоритма Value Iteration	38
2.23	Value Iteration. Сложность решения	39
2.24	LP-релаксация MDP. AMDP	41
2.25	LP-релаксация MDP. DMDP	43
2.26	Связь решения DMDP с выпуклой оптимизацией	44
2.27	Сглаженный оператор оптимальности Беллмана	45
2.28	DMDP. Пример: задача о разборчивой невесте	45
2.29	Источники	48

3 Лекция 3. Model-free RL, Monte Carlo, Temporal Difference и Q-learning 49

3.1	Что делали ранее	49
3.2	Policy Evaluation, Policy Iteration и Value Iteration	50
3.3	Различные подходы	50
3.3.1	Идея 1: Model-based RL	50
3.3.2	Идея 2: Model-free RL	50
3.4	Model-based RL	50
3.5	Model-free RL	50
3.6	Модификация алгоритмов для model-free RL	51
3.7	Метод Монте-Карло	51

3.7.1	Оценка Q -функции	51
3.7.2	Схема алгоритма Монте-Карло	51
3.7.3	Недостатки алгоритма Монте-Карло	51
3.8	Модификация алгоритма Монте-Карло	52
3.9	Условия Роббинса–Монро	52
3.10	Temporal Difference	53
3.10.1	Идея	53
3.10.2	TD-обновление	53
3.11	Сравнение Monte Carlo, Temporal Difference и Dynamic Programming	54
3.11.1	Temporal Difference	54
3.11.2	Monte Carlo	54
3.11.3	Dynamic Programming	55
3.12	Добавляем улучшение политики	55
3.13	Сходимость Q -обучения	55
3.14	Exploration/Exploitation trade-off	56
3.15	ϵ -жадное исследование среды	56
3.15.1	Идея метода	56
3.15.2	Достоинство	57
3.15.3	Недостаток	57
3.16	Схема алгоритма Q -learning	57
3.17	Буфер в алгоритме Q -обучения	57
4	Лекция 4. Deep Q-learning и улучшения DQN	58
4.1	Что делали ранее	58
4.2	Q -learning	59
4.3	Состояний теперь много	59
4.4	Препроцессинг состояний на практике	60
4.4.1	Action selection frequency	60
4.4.2	Atari-specific preprocessing	60
4.4.3	Standard tricks	60
4.4.4	Reward preprocessing	60
4.5	Проблема большого множества состояний	60
4.6	Возможные реализации DQN	61
4.6.1	Первый вариант	61
4.6.2	Второй вариант	61
4.7	Архитектура DQN	61
4.8	Обучение DQN как задача регрессии	62
4.8.1	MSE loss	62
4.9	Вычисление градиента	62
4.10	Итоговая задача регрессии	63

4.11	Почему обучение сходится к целевому значению	63
4.12	Особенность регрессии в DQN	64
4.13	Схема обучения: один шаг простой итерации	64
4.14	Возможные варианты обновления целевых значений	64
4.14.1	Вариант 1: SGD	64
4.14.2	Вариант 2: обучение до сходимости	64
4.14.3	Вариант 3: что-то между	65
4.15	Декорреляция сэмплов	65
4.16	Target Network	65
4.17	Финальная схема Deep Q-learning	65
4.18	Зачем модифицировать DQN	66
4.19	Природа overestimation	67
4.19.1	Формальное утверждение	67
4.20	Action decoupling	67
4.21	Возможные улучшения: Double и Twin Q-learning	68
4.21.1	Double* Q-learning	68
4.21.2	Double Q-learning	68
4.21.3	Twin Q-learning	68
4.22	Prioritized Experience Replay	68
4.23	Смещение в Prioritized Experience Replay	69
4.24	Importance weights для Prioritized Experience Replay	69
4.25	Техническая деталь: структура SumTree	70
4.26	Noisy Networks	70
4.27	Distributional RL	71
4.28	Dueling DQN	71
4.29	Partially Observable MDP	72
4.30	Multi-step DQN	72
4.31	Передний край науки и индустрии	73
4.31.1	Rainbow DQN, 2017	73
4.31.2	R2D2, 2018	73
4.31.3	Agent57, 2020	74
4.32	Источники	74
5	Лекция 5. Distributional RL, QR-DQN и IQN	74
5.1	Что делали ранее	74
5.2	Deep Q-learning	74
5.3	Проблема vanilla DQN	75
5.4	Double и Twin Q-learning	76
5.4.1	Double* Q-learning	76
5.4.2	Double Q-learning	76

5.4.3	Twin Q-learning	76
5.5	Мотивация Distributional RL	76
5.6	Пример распределения Z^π	77
5.7	Почему иногда важно искать именно распределение	78
5.8	Quantile Regression DQN	78
5.8.1	Идея QR-DQN	78
5.8.2	Аппроксимация распределения дельта-функциями	78
5.9	Distributional Bellman Operator	79
5.9.1	Distributional Bellman Equation	79
5.9.2	Пояснение	79
5.10	Метрика для Z -функций	80
5.11	Замечания про метрики	80
5.12	Wasserstein metric	80
5.12.1	Общий случай	80
5.12.2	Одномерный случай	81
5.13	Свойства Wasserstein metric	81
5.14	Интуиция Wasserstein metric: Earth Mover's Distance	81
5.15	Пример вычисления расстояния Вассерштейна	82
5.16	Distributional Optimality Operator	82
5.16.1	Distributional Optimality Equation	82
5.17	Distributional Optimality Operator может не быть сжатием	83
5.18	Хорошие новости	84
5.19	QR-DQN	84
5.20	QR-DQN: гиперпараметры и предварительные вычисления	85
5.21	QR-DQN: схема алгоритма	85
5.22	Implicit Quantile Networks	86
5.22.1	Идея IQN	86
5.22.2	Сэмплирование через квантильную функцию	86
5.23	IQN: жадная стратегия и функция потерь	87
6	Лекция 6. Exploration, self-supervision, RND и curiosity	88
6.1	Что делали ранее	88
6.2	Жадность — плохое качество	88
6.3	Проблема практического exploration	89
6.4	Среды с разреженной и малоинформативной наградой	89
6.5	Self-supervision	89
6.5.1	Напоминание: MDP	89
6.6	Внешняя и внутренняя награда	90
6.7	Аддитивная мотивация	90
6.8	Раздельное обучение оценочных функций	91

6.9	Почему аналогичное разложение неверно для Q^* и V^*	91
6.10	Почему разложение неверно для Z^π	92
6.11	Разные коэффициенты дисконтирования	92
6.12	Self-supervision на практике	93
6.13	Basic idea: бонус за посещение нового состояния	93
6.13.1	Модификация	93
6.14	Бонусы на основе счётчиков	93
6.15	Пример с хэш-функцией	93
6.16	Недостатки простой идеи	94
6.17	Mario Oracle	94
6.18	Random Network Distillation	94
6.18.1	Идея: детектировать новые состояния	94
6.19	Distillation	95
6.20	Недостатки ошибки регрессии как новизны	95
6.21	Random Network Distillation	95
6.22	Нюансы Random Network Distillation	96
6.23	Curiosity	96
6.23.1	Идея	96
6.24	Curiosity: naive approach	97
6.25	Curiosity: less naive approach	97
6.26	Noisy TV Problem	97
6.27	Требования к эмбедингам	98
6.28	Inverse Dynamics Model	98
6.29	Обучение модели обратной динамики	99
6.30	Почему inverse dynamics помогает	99
6.31	Intrinsic Curiosity Model	100
6.31.1	Идея ICM	100
6.32	ICM как совместное обучение inverse и forward dynamics	100
6.33	Сравнение curiosity-подходов	101
6.34	Наглядный пример	101
6.35	Телевизор и пульт: слабость ICM или нечто большее	101

7 Лекция 7. Policy Gradient 102

7.1	Что делали ранее	102
7.2	Напоминание: V - и Q -функции	103
7.3	Оптимальные функции ценности	103
7.4	On-policy и off-policy	103
7.5	DQN и Policy Gradient	103
7.5.1	DQN	104
7.5.2	Policy Gradient	104

7.6	План вывода Policy Gradient	104
7.7	Первый способ вывода Policy Gradient	105
7.7.1	Постановка задачи	105
7.7.2	Запись через Q -функцию	105
7.8	Трюк производной логарифма	105
7.9	Первое слагаемое	106
7.10	Второе слагаемое	106
7.11	Выражение $\nabla_{\theta} Q$ через $\nabla_{\theta} V$	106
7.12	Рекурсивная формула	107
7.13	Policy Gradient Theorem	107
7.14	Второй способ вывода Policy Gradient	107
7.14.1	Постановка задачи	107
7.14.2	Вычисление градиента	107
7.15	Вероятность траектории	108
7.16	Формула второго способа	108
7.17	Переход к эквивалентности	109
7.18	Проблема наград из прошлого	109
7.19	Теорема о score function	110
7.20	Принцип причинности	110
7.21	Убираем награды из прошлого	111
7.22	Эквивалентная форма через reward-to-go	111
7.23	Теорема об эквивалентности	111
7.24	Физический смысл Policy Gradient	112
7.25	Интерпретация суррогатной функции	113
7.26	Сравнение с supervised learning	113
8	Лекция 8. REINFORCE, baseline, Actor-Critic, GAE и A2C	114
8.1	Оценка Policy Gradient методом Монте-Карло	114
8.1.1	Что будем заменять	114
8.2	REINFORCE	115
8.3	Недостатки REINFORCE	116
8.4	Два типа стохастики	116
8.5	Марковская цепь состояний при фиксированной политике	116
8.6	State visitation frequency	117
8.7	Discounted state visitation distribution	117
8.8	Переписывание ожидания через d^{π}	118
8.9	Примеры использования d^{π}	118
8.10	Уменьшение дисперсии REINFORCE: baseline	118
8.10.1	Мотивация	118
8.10.2	Причина большой дисперсии	119

8.11	Baseline	119
8.12	Оптимальный baseline	120
8.13	Практический baseline и advantage	120
8.14	Схемы Actor-Critic	120
8.15	Actor-Critic и bias-variance trade-off	121
8.15.1	Два крайних варианта	121
8.16	N-шаговая оценка Q -функции	121
8.17	Max trace estimation	122
8.18	Generalized Advantage Estimation	122
8.19	GAE для оборванного rollout	123
8.20	Рекурсивный подсчёт GAE	123
8.21	Обучение критика	124
8.22	Связь таргета критика и advantage	124
8.23	GAE-target для критика	124
8.24	Общий шаг Actor-Critic	125
8.25	Advantage Actor-Critic	125
8.26	Actor-Critic: комментарии	126
8.27	Сходство градиентов актора и критика	127
8.28	Пример с кофеваркой	127
8.29	Пример: A2C и редкий успешный опыт	127
9	Лекция 9. TRPO, trust region и натуральный градиент	128
9.1	Напоминание: Policy Gradient	128
9.2	Можно ли получить более эффективный Policy Gradient?	128
9.3	Соединяя две политики	129
9.4	Relative Performance Identity	129
9.5	Меняем целевую функцию	131
9.6	Суррогатный функционал	131
9.7	Интерпретация суррогатного функционала	132
9.8	Теоретическая граница ошибки	132
9.9	Алгоритм миноризации-максимизации	132
9.10	Практические трудности нижней оценки	133
9.11	Идея TRPO	133
9.12	Разложение Тейлора для модели и условия	134
9.13	Натуральный градиентный подъём	134
9.14	Как обращаться матрицу H	135
9.14.1	Схема	135
9.15	Линейный поиск	136
9.16	Trust Region Policy Optimization	136
9.17	Trust region: пространство параметров или пространство распределений	137

9.18 Проблема большого шага: скатывание с обрыва	137
9.19 Общая схема методов с доверительной областью	138
9.20 Градиентный спуск и натуральный градиентный спуск	138
9.20.1 Обычный градиентный спуск	138
9.20.2 Натуральный градиентный спуск	139
9.21 Информационная матрица Фишера	139
9.22 Направление натурального градиента	139
9.23 Связь натурального градиента и TRPO	140
9.24 Краткая схема TRPO	140
9.25 Итог	141
10 Лекция 10. PPO, clipping, TD(λ), GAE и Retrace	141
10.1 Proximal Policy Optimization: общая идея	141
10.2 PPO: отличие от TRPO в пайплайне	142
10.3 Суррогатный функционал PPO	142
10.4 Клиппирование отношения вероятностей	143
10.5 PPO clipped objective	143
10.6 Интуиция clipped objective	143
10.6.1 Случай положительного advantage	143
10.6.2 Случай отрицательного advantage	144
10.7 Таблица поведения clipped objective	145
10.8 Клиппированная функция потерь критика	145
10.9 Компромисс между смещением и разбросом	146
10.9.1 Монте-Карло оценка	146
10.9.2 N-шаговая оценка	146
10.9.3 Одношаговая оценка	146
10.10 Взгляд назад: идея TD(λ)	147
10.11 Eligibility Traces	147
10.11.1 Онлайн Монте-Карло обновления	147
10.12 TD(1)	148
10.13 TD(0)	149
10.14 TD(λ)	149
10.15 Взгляд вперёд и взгляд назад	150
10.15.1 Взгляд вперёд	150
10.15.2 Взгляд назад	151
10.16 Взгляд вперёд для TD(λ)	151
10.17 Generalized Advantage Estimation	151
10.17.1 Одношаговая невязка	152
10.17.2 Рекурсивная формула	152
10.18 GAE в Advantage Actor-Critic	152

10.19	PPO: особенности реализации	152
10.20	PPO: полный алгоритм	153
10.20.1	Сбор rollout	153
10.20.2	Одношаговые TD-невязки	154
10.20.3	GAE advantage	154
10.20.4	Целевые значения критика	154
10.20.5	Оптимизация по мини-батчам	154
10.21	Источники по PPO и GAE	155
10.22	Value-based методы и Policy Gradient	155
10.23	Off-policy оценка advantage	156
10.24	Коррекция выборкой по значимости	156
10.25	Проблемы importance sampling	157
10.25.1	Затухающий след	157
10.25.2	Взрывающийся след	157
10.26	Общий вид присвоения ценности	158
10.27	Retrace: основная теорема	158
10.28	Retrace: финальный результат	159
10.29	Где используется Retrace	159
10.30	Итог	159
11	Лекция 11. DDPG, TD3 и обучение стохастических политик	160
11.1	Проблема DQN в непрерывных пространствах действий	160
11.2	Архитектура Q-функции для непрерывных действий	160
11.3	Идея: обучить отдельную сеть для аргмаксимума	161
11.4	DDPG как модификация DQN	161
11.5	Таргет в DDPG	162
11.6	Почему это называется Policy Gradient	162
11.7	Deterministic Policy Gradient	162
11.8	Доказательство Deterministic Policy Gradient	163
11.9	Рекурсивное раскрытие второго слагаемого	163
11.10	Суррогатная функция для deterministic policy gradient	164
11.11	Actor-Critic схема для DDPG	164
11.12	Обучение критика в DDPG	165
11.13	Эквивалентность двух выводов DDPG	165
11.14	Exploration в DDPG	165
11.15	Шум Орнштейна–Уленбека	166
11.16	Алгоритм DDPG	166
11.17	Twin Delayed DDPG	168
11.18	Идея двух критиков	168
11.19	Target policy smoothing	168

11.20	Delayed policy updates	168
11.21	Алгоритм TD3	169
11.22	Переход к стохастическим политикам	170
11.23	Репараметризационный трюк	171
11.24	Пример: гауссова политика	171
11.25	Детерминированная политика как частный случай	171
11.26	Policy Gradient с репараметризацией	172
11.27	Суррогат для стохастической политики с репараметризацией	172
11.28	Проблема одномодальной гауссовой политики	172
11.29	Смесь гауссиан	173
11.30	Score-function градиент для стохастического актора	173
11.31	Замена переменной в плотности	174
11.32	Энтропия распределения	174
11.33	Пример: преобразование через $\tanh$	174
11.34	Итог	175
12	Лекция 12. Soft Actor-Critic, Maximum Entropy RL и Soft Q-learning	176
12.1	Soft Actor-Critic: мотивация	176
12.2	Энтропия распределения	176
12.3	Maximum Entropy RL	177
12.4	Эквивалентная форма Maximum Entropy RL	177
12.5	Мягкая награда	178
12.6	Зачем нужен энтропийный бонус	178
12.7	Оптимальная детерминированная стратегия может не существовать	178
12.8	Соглашение про температуру	179
12.9	Мягкие функции ценности	179
12.10	Мягкие уравнения связи	179
12.11	Мягкие уравнения Беллмана	180
12.12	Сжимаемость мягких операторов Беллмана	180
12.13	Сходимость простой итерации	180
12.14	Обучение критика в Maximum Entropy RL	181
12.15	Вспомогательная V -функция	181
12.16	Soft Policy Improvement	182
12.17	Generalized Policy Iteration для Maximum Entropy RL	182
12.18	Связь Soft Policy Improvement с KL-дивергенцией	183
12.19	Вид жадной стратегии	184
12.20	Пример: гауссова политика	184
12.21	Пример: смесь гауссиан	184
12.22	Алгоритм Soft Actor-Critic	185
12.23	Оптимальные мягкие функции ценности	187

12.24	Вид оптимальной стратегии	187
12.25	Оптимальная мягкая V -функция через $\log\text{-sum-exp}$	188
12.26	Мягкое уравнение оптимальности Беллмана	189
12.27	Сжимаемость $\text{soft Bellman optimality operator}$	189
12.28	Критерий оптимальности через мягкую жадность	190
12.29	Soft Q-learning	190
12.30	Связь Soft Q-learning и SAC	191
12.31	$\text{Soft Policy Gradient}$	191
12.32	Итог	192
13	Лекция 13. Imitation Learning, Inverse RL, GAIL и GAIfo	193
13.1	Общая мотивация	193
13.2	Клонирование поведения	193
13.2.1	Мотивация	193
13.2.2	Определение $\text{behavioral cloning}$	194
13.3	Проблема $\text{behavioral cloning}$	194
13.4	DAgger	194
13.5	Пример: $\text{Quadcopter Navigation in the Forest}$	195
13.6	Обратное обучение с подкреплением	195
13.7	Пример IRL в клеточном мире	196
13.7.1	Зелёные клетки	196
13.7.2	Красные клетки	196
13.7.3	Синие клетки	196
13.7.4	Жёлтые клетки	197
13.8	Некорректность задачи IRL	197
13.9	Maximum Entropy предположение в IRL	197
13.10	Связь с $\text{Maximum Entropy RL}$	198
13.11	Ноль в KL -задаче может быть недостижим	199
13.12	$\text{Guided Cost Learning}$	200
13.13	Логарифм правдоподобия	200
13.14	Оптимальная стратегия для текущей награды	201
13.15	Теорема $\text{Guided Cost Learning}$	201
13.16	Интерпретация $\text{Guided Cost Learning}$	202
13.17	Переход к GAIL	202
13.18	Occupancy measure	202
13.19	Ожидание через occupancy measure	203
13.20	Минимаксная форма через occupancy measure	203
13.21	Стратегия через occupancy measure	203
13.22	Энтропийный бонус через occupancy measure	204
13.23	Регуляризация функции награды	204

13.24	Регуляризатор, приводящий к GAIL	204
13.25	Интерпретация GAIL	205
13.26	Оптимизация стратегии в GAIL	206
13.27	GAIL как состязательная схема	206
13.28	Generative Adversarial Imitation from Observation	207
13.29	Пример GAIfo	207
13.30	GAIL без действий	207
13.31	GAIfo: дискриминация пар соседних состояний	208
13.32	Осциллятор measure для пар состояний	208
13.33	Минимаксная задача GAIfo	208
13.34	Сравнение BC, IRL, GAIL и GAIfo	209
13.35	Итог	209

14 Лекция 14. Multi-Armed Bandit, UCB, Thompson Sampling, World Models

	и современные результаты	210
14.1	Multi-Armed Bandit Problem	210
14.1.1	Постановка задачи	210
14.1.2	Обновление стратегии	211
14.2	Regret	211
14.3	Exploration–exploitation dilemma	212
14.4	Простая Monte-Carlo оценка $Q(a)$	212
14.5	Линейный рост regret	212
14.6	ε -жадная стратегия	213
14.7	Алгоритм: наивный бандит	213
14.8	Regret при ε -жадном выборе	214
14.9	Нестационарный бандит	215
14.10	Теорема Лаи–Роббинса	215
14.11	Upper Confidence Bound	216
14.12	Идея confidence interval	216
14.13	Неравенство Хёфдинга	217
14.14	Вывод UCB-бонуса	217
14.15	Практическая форма UCB	218
14.16	Сэмплирование Томпсона	218
14.16.1	Байесовская постановка	218
14.17	Probability matching	219
14.18	Thompson Sampling	219
14.19	Bernoulli-бандиты	220
14.20	Beta prior	220
14.21	Bayesian update для Bernoulli-бандита	220
14.22	Среднее значение выигрыша в Bernoulli-бандите	221

14.23	Планирование	222
14.24	World Models	223
14.25	Общая схема model-based подхода	223
14.26	Модель прямой динамики	224
14.27	Сновидения	224
14.28	Интересные результаты	225
14.29	Optimistic Posterior Sampling for Reinforcement Learning	225
14.30	Regret bound для OPSRL	225
14.31	Теорема: regret bound for OPSRL	226
14.32	Алгоритм OPSRL	226
14.32.1	Optimistic backward induction	227
14.33	Экспериментальные результаты OPSRL	227
14.34	From Dirichlet to Rubin: Optimistic Exploration in RL without Bonuses	228
14.35	Идея BayesUCBVI	228
14.36	Сравнение regret upper bounds	229
14.37	Теорема для BayesUCBVI	229
14.38	Алгоритм BayesUCBVI	229
14.39	Эксперименты BayesUCBVI	230
14.40	Federated UCBVI	230
14.41	Setting: policies and value functions	231
14.42	Q-функция и уравнения Беллмана в Fed-UCBVI	231
14.43	Learning protocol	232
14.44	Federated regret	232
14.45	Communication complexity and cost	233
14.46	Environmental heterogeneity	233
14.46.1	Assumption A-1	233
14.46.2	Assumption A-2	234
14.47	Fed-UCBVI algorithm	234
14.48	Data collection	235
14.49	Synchronization	235
14.49.1	Local Doubling Condition	236
14.49.2	Globally Estimated Doubling Condition	236
14.50	Policy update	236
14.51	Aggregation in Fed-UCBVI	237
14.52	Communication complexity: идея doubling condition	237
14.53	Почему нужен globally estimated doubling	238
14.54	Main results for Fed-UCBVI	238
14.55	Итог	239

15.1	В предыдущих сериях	240
15.2	MCTS: постановка задачи	241
15.3	Наивное построение дерева среды	241
15.4	Дерево MDP	242
15.5	Monte-Carlo Tree Search	242
15.6	MCTS: Selection	242
15.7	TreePolicy как бандитная задача	243
15.8	UCB внутри MCTS	243
15.9	MCTS: Expansion	244
15.10	MCTS: Simulation, или Evaluation	244
15.11	MCTS: Update, или Backpropagation	245
15.12	Применение MCTS: хорошие практики	245
15.13	Итоговый алгоритм MCTS	246
15.13.1	Один шаг MCTS	246
15.14	Планирование для непрерывного управления	248
15.15	Математическая формулировка задачи управления	248
15.16	Прямое дифференцирование	249
15.17	Linear Quadratic Regulator: мотивация	249
15.18	LQR: обозначения	249
15.19	LQR: последний шаг	250
15.20	LQR: ценность последнего состояния	251
15.21	LQR: индуктивный шаг	252
15.22	LQR: обратный проход	253
15.23	LQR: прямой проход	254
15.24	LQR для шумной динамики	255
15.25	Iterative LQR: мотивация	256
15.26	iLQR: локальная аппроксимация	256
15.27	iLQR: обозначения	256
15.28	Перецентрирование в iLQR	257
15.29	Алгоритм iLQR	257
15.30	Сравнение MCTS, LQR и iLQR	258
15.31	Итог	258

1 Лекция 1. Вводная лекция

1.1 Команда курса

Курс посвящён обучению с подкреплением.

Лектор:

- Никита Юдин.

Ассистенты:

- Георгий Акиндинов;
- Дмитрий Былинкин;
- Варвара Руденко;
- Иван Янко.

Telegram-беседа курса:

<https://t.me/+AV4p7VeGGw0N2Uy>

1.2 Правила игры

В курсе предусмотрено не менее пяти лабораторных работ с жёсткими дедлайнами в формате ноутбуков и устный экзамен.

Итоговая оценка по курсу в 10-балльной шкале рассчитывается по формуле:

$$\text{Итоговая оценка} = \lceil 0.3 \cdot \text{Экз} + 0.7 \cdot \text{Лаб} \rceil.$$

Оценке «отлично» соответствует оценка 8 и выше.

Оценке «хорошо» соответствует промежуток:

$$[5, 8).$$

Оценке «удовлетворительно» соответствует промежуток:

$$[3, 5).$$

Помимо баллов необходимо также выполнить следующие условия:

- на «отлично» — оценка за экзамен не меньше 5;
- на «хорошо» — оценка за экзамен не меньше 4;
- на «удовлетворительно» — оценка за экзамен не меньше 3.

1.3 Обучение с учителем и обучение с подкреплением

В задаче обучения с учителем даны объекты и ответы:

$$(x, y).$$

Имеется семейство алгоритмов:

$$a_{\theta}(x) \mapsto y.$$

Также задана функция потерь:

$$L(y, a_{\theta}(x)).$$

Задача обучения с учителем состоит в поиске параметров:

$$\hat{\theta} \in \operatorname{Arg\,min}_{\theta} \{L(y, a_{\theta}(x))\}.$$

В обучении с подкреплением, или reinforcement learning, в чистом виде нет готовых «ответов».

Зато есть:

- среда-симулятор, или environment;
- состояния среды, или states;
- действия, или actions, влияющие на состояния среды;
- награды, или rewards, за совершённые действия.

Основной принцип обучения агента, или стратегии, для оптимального выбора действий — метод проб и ошибок.

1.4 Примеры предметных областей

1.4.1 Онлайн-реклама

Дано:

- сервис видеохостинга;
- метаданные потоковой трансляции: баннеры, видео, клики;
- признаки, применимые для задачи обучения по прецедентам.

Цель — показывать релевантную рекламу на странице пользователю онлайн.

1.4.2 Робототехника

Дано:

- робот;
- неограниченное количество запчастей;
- показания датчиков.

Цель — научить робота ходить.

1.4.3 Видеоигровые среды

Дано:

- дешёвый симулятор;
- неограниченное количество попыток;
- фрейм игры и сопутствующая ему метаянформация.

Цель — научить бота выигрывать.

1.4.4 Другие предметные области

К другим предметным областям относятся:

- диалоговые системы;
- количественные финансы, например оптимизация портфеля;
- глубокое обучение: оптимизация негладких функций, поиск оптимальной архитектуры нейронной сети;
- персонализированная терапия.

1.5 Возникающие проблемы

Один из первых вопросов в обучении с подкреплением: что значит «оптимальное действие»?

Например, в задаче онлайн-рекламы можно оптимизировать разные цели:

- повысить доход с одного пользователя;
- сделать пользователя счастливым, чтобы он пришёл снова.

Ещё одна проблема: следуя исключительно оптимальной в данный момент стратегии, можно никогда не найти стратегию лучше, даже если такая стратегия существует.

Например, невозможно оценить влияние других баннеров, если всё время показывать один и тот же баннер.

1.6 Пример модели: многорукий бандит

Многорукий бандит, или multi-armed bandit, является одной из простейших моделей, близких к обучению с подкреплением.

Примеры применения:

- рекламные баннеры;
- системы рекомендаций;
- разработка лекарств.

Основное действие в многоруком бандите — выбор «рычага» и «нажатие» на него.

Схематично обычный многорукий бандит можно записать как:

$$\text{action} \longrightarrow \text{reward}.$$

В контекстном бандите, или contextual bandit, используется состояние бандита для хранения дополнительной информации, которая влияет на награду за действие. Это приближает взаимодействие с ним к обучению с подкреплением.

Схематично контекстный бандит:

$$\text{state} \longrightarrow \text{action} \longrightarrow \text{reward}.$$

В контекстном бандите агент выбирает действие, например рычаг, получает награду и наблюдает состояние.

В полноценном обучении с подкреплением агент взаимодействует со средой:

$$\text{Agent} \longrightarrow \text{Action} \longrightarrow \text{Environment} \longrightarrow \text{State, Reward} \longrightarrow \text{Agent}.$$

Замечание 1. *Над агентом у нас полный контроль, а среда или бандит для нас является «чёрным ящиком».*

1.7 Сравнение типов обучения

1.7.1 Обучение с учителем и обучение с подкреплением

Обучение с учителем:

- обучаемся аппроксимировать данные ответы;
- требуются корректные ответы;
- настраиваемая модель не влияет на входные данные.

Обучение с подкреплением:

- обучаемся оптимальной стратегии по принципу проб и ошибок;
- требуется отклик на собственные действия агента;
- агент может влиять на входные наблюдения.

1.7.2 Обучение без учителя и обучение с подкреплением

Обучение без учителя:

- учим структуру данных;
- отклик не требуется;
- настраиваемая модель не влияет на входные данные.

Обучение с подкреплением:

- обучаемся оптимальной стратегии по принципу проб и ошибок;
- требуется отклик на собственные действия агента;
- агент может влиять на входные наблюдения.

1.8 Постановка задачи обучения с подкреплением

Введём основные обозначения:

- s_t — состояние среды в момент t ;
- a_t — действие, выбранное в момент t ;
- r_t — награда, полученная в момент t ;
- $\pi(a \mid s, \theta)$ — политика, то есть вероятность совершить действие a из состояния s при параметрах политики θ ;
- $$\tau = \{(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)\}$$

— траектория агента согласно политике;
- $$\gamma \in (0, 1]$$

— дисконтирующий фактор;
- $$p(s' \mid s, a)$$

— вероятность попасть в состояние s' из состояния s с помощью действия a .

Схема взаимодействия агента и среды:

$$\text{Agent} \xrightarrow{a_t} \text{Environment} \xrightarrow{(s_{t+1}, r_t)} \text{Agent}.$$

1.9 Формализация

Вероятность реализации траектории τ при фиксированной политике π :

$$p(\tau \mid \pi) = p(s_0) \prod_{t=0}^T (\pi(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t)).$$

Суммарная награда на момент времени T с учётом дисконтирования на произвольной траектории:

$$R := \sum_{t=0}^T \gamma^t r_t.$$

Общая оптимизационная задача:

$$\mathbb{E}_{p(\tau|\theta)} [R] \rightarrow \max_{\theta}.$$

Здесь θ — параметры политики π .

По умолчанию предполагается:

$$T \rightarrow \infty.$$

1.10 Марковский процесс принятия решений

Марковский процесс принятия решений, или Markov Decision Process, MDP, — это кортеж:

$$(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma),$$

где:

- $\mathcal{A}$ — множество действий;
- $\mathcal{S}$ — множество состояний;
-

$$\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

— функция наград;

-

$$\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$$

— функция вероятности перехода.

Для функции перехода:

$$\mathcal{T}(s, a, s') = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a).$$

То есть вероятность оказаться в следующем состоянии зависит только от предыдущего состояния и действия из него.

1.11 Метод кросс-энтропии в RL

1.11.1 Идея метода кросс-энтропии

Метод кросс-энтропии связан с идеей сэмплированием оценить вероятность редкого события:

$$\mathbb{E}_{p(x)} [f(x) \geq \hat{\gamma}] \approx \frac{1}{M} \sum_{j=1}^M [f(x_j) \geq \hat{\gamma}] \cdot \frac{p(x_j)}{q(x_j \mid \lambda_k)}.$$

Здесь:

- M — количество сэмплов;
-

$$x_j \sim q(x \mid \lambda_k);$$

- λ_k — фиксированный параметр.

Оптимальное распределение имеет вид:

$$q_{\text{opt}}(x) \propto [f(x) \geq \hat{\gamma}] p(x).$$

Это распределение нужно найти.

Замечание 2. Полученная задача есть задача максимума правдоподобия на параметр λ_k .

1.11.2 Схема метода кросс-энтропии

Схема метода кросс-энтропии:

1. Инициализация:

$$\lambda_0, \quad M, \quad \rho \in [0, 1], \quad k := 0.$$

2. Сэмплирование:

$$x_1, \dots, x_M \sim q(x \mid \lambda_k).$$

3. Вычисляем редкое событие:

$$f_j = f(x_j),$$

и сортируем значения по возрастанию:

$$f_{(1)} \leq \dots \leq f_{(M)}.$$

4. Обновляем порог:

$$\hat{\gamma}_k := \min \left(f_{(\max\{\lfloor \rho M \rfloor, 1\})}, \hat{\gamma} \right).$$

5. Обновляем параметр распределения:

$$\lambda_{k+1} := \arg \max_{\lambda} \left(\frac{1}{M} \sum_{j=1}^M [f(x_j) \geq \hat{\gamma}_k] \cdot \frac{p(x_j)}{q(x_j \mid \lambda_k)} \ln q(x_j \mid \lambda) \right).$$

6. Критерий останова:

$$\hat{\gamma}_k = \hat{\gamma}.$$

Если критерий не выполнен, переходим на шаг 2 с

$$k := k + 1.$$

Получение итоговой оценки:

1. Сэмплируем:

$$x_1, \dots, x_M \sim q(x \mid \lambda_k).$$

2. Вычисляем:

$$\frac{1}{M} \sum_{j=1}^M [f(x_j) \geq \hat{\gamma}_k] \cdot \frac{p(x_j)}{q(x_j \mid \lambda_k)}.$$

1.11.3 Применение метода кросс-энтропии к оптимизации

Полученный метод ищет такое распределение $q(x \mid \lambda)$, на котором в среднем $f(x)$ больше некоторого порога чаще всего.

Это является безградиентным методом решения задачи оптимизации:

$$f(x) \rightarrow \max_x.$$

Связь с вероятностью редкого события:

$$f(x) \rightarrow \max_x \iff \mathbb{E}_{p(x)} [f(x) \geq \hat{\gamma}],$$

где

$$\hat{\gamma} \approx \max_x \{f(x)\}, \quad p(x) := q(x \mid \lambda).$$

Оптимизационный метод кросс-энтропии:

1)–3) Как и раньше.

4)

$$\hat{\gamma}_k := f_{(\max\{\lfloor \rho M \rfloor, 1\})}.$$

5)

$$\lambda_{k+1} := \arg \max_{\lambda} \left(\frac{1}{M} \sum_{j=1}^M [f(x_j) \geq \hat{\gamma}_k] \ln q(x_j | \lambda) \right).$$

Динамика кросс-энтропийного метода при оптимизации функции: точки-кандидаты на экстремум генерируются из нормального распределения. Далее распределение постепенно концентрируется около области с лучшими значениями функции.

1.12 Метод кросс-энтропии в терминах RL

Метод кросс-энтропии можно переписать в терминах обучения с подкреплением.

1. Инициализация:

$$\theta_0, \quad M, \quad \rho \in [0, 1], \quad k = 0.$$

2. Сэмплирование траекторий:

$$\tau_1, \dots, \tau_M \sim p(\tau | \theta_k).$$

Получаем M траекторий по политике:

$$\pi(a | s, \theta).$$

3. Вычисляем награды траекторий:

$$R_j = R(\tau_j),$$

и сортируем их по возрастанию:

$$R_{(1)} \leq \dots \leq R_{(M)}.$$

4. Обновляем порог:

$$\hat{\gamma}_k := R_{(\max\{\lfloor \rho M \rfloor, 1\})}.$$

5. Обновляем параметры политики:

$$\theta_{k+1} := \arg \max_{\theta} \left(\frac{1}{M} \sum_{j=1}^M [R(\tau_j) \geq \hat{\gamma}_k] \ln p(\tau_j | \theta) \right).$$

Здесь:

$$\ln p(\tau_j | \theta) = \sum_{i=0}^{|\tau_j|-1} [(s_i, a_i, r_i) \in \tau_j] \ln \pi(a_i | s_i, \theta).$$

6. Делаем:

$$k := k + 1,$$

и переходим на шаг 2, если не выполнен критерий останова.

1.12.1 Интерпретация алгоритма

На каждом шаге алгоритм смотрит на самые успешные, согласно награде, траектории и повышает вероятность совершения действий из этих траекторий.

То есть алгоритм учится совершать действия из лучших траекторий.

Замечание 3. *Алгоритм почти ничего не использует из структуры задачи: как и когда начисляется награда и прочее.*

1.13 Источники

1. S. Ivanov, “Reinforcement learning textbook,” arXiv preprint arXiv:2201.09746, настольная книга по данному курсу, 2022.
2. Z. I. Botev, D. P. Kroese, R. Y. Rubinstein, and P. L’Ecuyer, “The cross-entropy method for optimization,” in *Handbook of Statistics*, vol. 31, pp. 35–59, Elsevier, 2013.

2 Лекция 2. Марковские процессы принятия решений

2.1 Марковский процесс принятия решений

В обучении с подкреплением взаимодействия между агентом и окружающей средой часто описываются марковским процессом принятия решений, или MDP:

$$\text{MDP} = \text{Markov Decision Process}.$$

Различают несколько основных постановок:

- дисконтированный марковский процесс принятия решений:

$$\gamma\text{-Discounted Markov Decision Process, DMDP};$$

- MDP с усреднённым вознаграждением:

$$\text{infinite-horizon Average reward Markov Decision Process, AMDP};$$

- эпизодический марковский процесс принятия решений:

$$H\text{-episodic Markov Decision Process, HMDP};$$

- другие постановки, в том числе частично наблюдаемые.

Марковский процесс принятия решений представляет собой систему, которая со временем

$$t = 0, 1, 2, \dots$$

претерпевает случайные изменения и обозначается кортежем

$$M = (\mathcal{S}, \mathcal{A}, p, r, \gamma),$$

где:

1. $\mathcal{S}$ — пространство состояний,

$$S := |\mathcal{S}|$$

— количество уникальных состояний.

2. $\mathcal{A}$ — пространство действий,

$$A := |\mathcal{A}|$$

— количество уникальных действий.

3.

$$p(s, a; s')$$

— вероятность перехода из состояния $s \in \mathcal{S}$ в момент времени t при действии $a \in \mathcal{A}$ в состояние $s' \in \mathcal{S}$ в момент времени $t + 1$.

При этом

$$\sum_{s' \in \mathcal{S}} p(s, a; s') = 1,$$

и

$$p(s, a; s') \equiv \mathbb{P}(s' \mid s, a).$$

Функция вероятности $p(\cdot)$ вместе с функцией вероятности $\mathbb{P}(a \mid s)$ задают вероятности перехода для марковского ядра, оно же ядро MDP.

4. Функция награды:

$$r_\xi(s, a) : \Omega \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1],$$

причём

$$\mathbb{E}_\xi [r_\xi(s, a)] = r(s, a).$$

В зависимости от постановки задачи функция награды может зависеть от следующего состояния s' :

$$r_\xi(s, a; s') : \Omega \times \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1],$$

$$\mathbb{E}_\xi [r_\xi(s, a; s')] = r(s, a; s').$$

В общем случае предполагается стохастическая природа функции награды в зависимости от случайной величины

$$\xi \in \Omega.$$

При детерминированном вычислении награды относительно фиксированных (s, a) или (s, a, s') обозначение ξ в $r_\xi(\cdot)$ и математическое ожидание по нему опускаются.

Здесь и далее используется предположение об ограниченности награды за каждое действие, поэтому без ограничения общности используются приведённые выше определения функции $r(\cdot)$.

В курсе используются детерминированные относительно своих аргументов награды, если не оговорено иное.

5.

$$\gamma \in (0, 1]$$

— коэффициент дисконтирования для DMDP.

Для AMDP:

$$\gamma = 1.$$

Однако просто положив $\gamma = 1$, из DMDP нельзя получить AMDP: понадобится ещё усреднение суммарной награды агента за взаимодействие с MDP по времени.

Нередко рассматривается более общая форма:

$$M = (\mathcal{S}, \mathcal{A}, p, r, \mu_0, \gamma),$$

где μ_0 — вероятностное распределение начального состояния:

$$s_0 \sim \mu_0.$$

При явном отсутствии μ_0 происходит обуславливание всех вычислений на фиксированное начальное состояние

$$s_0 \in \mathcal{S}.$$

2.2 MDP. Принятие решений

Здесь и далее результаты приводятся для дискретных $\mathcal{S}$ и $\mathcal{A}$ с конечными мощностями. Однако они могут быть обобщены на непрерывный случай заменой суммы по переменной в области её непрерывности на соответствующий интеграл по области.

Стратегией принятия решений, или политикой агента, принимающего решение в MDP, обозначим π .

Политика задаёт вероятностную меру на пространстве действий:

$$\pi(a \mid s) \equiv \mathbb{P}(a \mid s).$$

В общем случае:

$$\hat{a} \sim \pi(\cdot \mid s),$$

или

$$\pi(s) \sim \pi(\cdot \mid s).$$

В случае вырожденного распределения:

$$\hat{a} := \pi(s),$$

где

$$\mathbb{P}(\hat{a} \mid s) = 1.$$

То есть в детерминированном случае политика напрямую сопоставляет состоянию действие:

$$\pi : \mathcal{S} \rightarrow \mathcal{A}.$$

2.3 MDP. Ядро

Введённое распределение позволяет явно записать марковское ядро перехода между состояниями

$$s \mapsto s'.$$

Это ядро также называют ядром MDP или марковским ядром, обусловленным политикой π :

$$P^\pi(s' \mid s) := \sum_{a \in \mathcal{A}} \pi(a \mid s) p(s, a; s').$$

В процессах с конечным количеством состояний марковское ядро можно задать с помощью матрицы:

$$P^\pi = \left(\sum_{a \in \mathcal{A}} \pi(a \mid s) p(s, a; s') \right)_{s \in \mathcal{S}, s' \in \mathcal{S}},$$

где s — строка, а s' — столбец.

В процессе взаимодействия с марковским процессом стратегия π собирает траекторию:

$$\tau_t := (s_0, a_0, r_0, \dots, s_t, a_t, r_t).$$

Правдоподобие траектории длины H выражается следующим образом:

$$\mathbb{P}(\tau_{H-1} \mid \pi) = \mu_0(s_0) \prod_{t=0}^{H-2} (\pi(a_t \mid s_t) p(s_t, a_t; s_{t+1})) \pi(a_{H-1} \mid s_{H-1}).$$

2.4 DMDP. V -функция ценности

Для фиксированной политики π и начального состояния $s_0 = s$ определяется V -функция значений, или функция ценности:

$$V^\pi : \mathcal{S} \rightarrow \mathbb{R}.$$

Она равна дисконтированной сумме будущих вознаграждений:

$$V^\pi(s) := \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a(s_t)) \mid \pi, s_0 = s \right].$$

Здесь:

- s_t — состояние системы в момент времени t ;
- $a(s_t)$ — выбор действия в соответствии с политикой $\pi(\cdot \mid s_t)$.

Это средняя награда по политике π , если агент начинает действовать из состояния s .

Иногда индекс политики опускают:

$$V(s) := V^\pi(s).$$

Замечание 4. В определении V -функции в левой части выражения отсутствует обозначение t в силу однородности MDP.

2.5 DMDP. Q -функция ценности

Схожим образом задаётся Q -функция ценности:

$$Q^\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Она определяется как:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a(s_t)) \mid \pi, s_0 = s, a_0 = a \right].$$

То есть это то же, что V -функция, только теперь из состояния $s_t = s$ обязательно сначала совершается действие $a_t = a$.

Иногда индекс политики опускают:

$$Q(s, a) := Q^\pi(s, a).$$

Замечание 5. В определении V - и Q -функций в левой части выражения отсутствует обозначение t в силу однородности MDP.

2.6 DMDP. Дисконтированная награда

Дисконтированная кумулятивная награда за эпизод длины $H - t$, где $t = 0, \dots, H - 1$, задаётся как:

$$R_t^{H-1} := \sum_{j=t}^{H-1} \gamma^{j-t} r(s_j, a(s_j)).$$

Для бесконечного горизонта:

$$R_t := R_t^\infty := \sum_{j=t}^{\infty} \gamma^{j-t} r(s_j, a(s_j)).$$

При переходе к AMDP, где $\gamma = 1$, наиболее естественным аналогом кумулятивной награды является среднее арифметическое наград по времени:

$$R_t^{H-1} := \frac{1}{H-t} \sum_{j=t}^{H-1} r(s_j, a(s_j)).$$

Для бесконечного горизонта:

$$R_t := R_t^\infty := \lim_{H \rightarrow \infty} \frac{1}{H-t} \sum_{j=t}^{H-1} r(s_j, a(s_j)).$$

2.7 Ограниченность функций ценности

Если награды ограничены:

$$r(s, a) \in [0, 1],$$

то для V^π имеем мажоранту:

$$0 \leq V^\pi(s) \leq \frac{1}{1-\gamma}, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

Q^π -функция обладает той же мажорантой:

$$0 \leq Q^\pi(s, a) \leq \frac{1}{1-\gamma}, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

2.8 DMDP. Уравнения Беллмана для фиксированной политики

Для V^π :

$$\begin{aligned}
 V^\pi(s) &= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a(s_t)) \mid \pi, s_0 = s \right] \\
 &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} p(s, a; s') (r(s, a) + \gamma V^\pi(s')) \\
 &= \mathbb{E}_\pi [Q^\pi(s, a)] \\
 &= \mathbb{E}_\pi [r(s, a)] + \gamma \mathbb{E}_{p, \pi} [V^\pi(s')] \\
 &= \mathbb{E}_\pi [r(s, a)] + \gamma \mathbb{E}_{p, \pi} [Q^\pi(s', a')],
 \end{aligned}$$

где

$$a' \sim \pi(\cdot \mid s').$$

Для Q^π :

$$\begin{aligned}
 Q^\pi(s, a) &= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a(s_t)) \mid \pi, s_0 = s, a_0 = a \right] \\
 &= \sum_{s' \in \mathcal{S}} p(s, a; s') (r(s, a) + \gamma V^\pi(s')) \\
 &= r(s, a) + \gamma \mathbb{E}_p [V^\pi(s')] \\
 &= r(s, a) + \gamma \mathbb{E}_{p, \pi} [Q^\pi(s', a')],
 \end{aligned}$$

где

$$a' \sim \pi(\cdot \mid s').$$

2.9 DMDP. Задача обучения с подкреплением

Цель задачи обучения с подкреплением — поиск политики, позволяющей получить максимальное кумулятивное вознаграждение в долгосрочной перспективе.

В большинстве практических случаев задача RL формулируется как задача оптимизации следующего формата:

$$\pi^* \in \text{Arg max}_{\pi \in \hat{\Pi}} \left\{ \mathbb{E}_{\mathbb{P}(\tau_{H-1} \mid \pi)} [R_0^{H-1}] = \mathbb{E}_{s \sim \mu_0} [V^\pi(s)] \right\}.$$

Множество допустимых политик:

$$\hat{\Pi} := \left\{ \pi \mid \pi(a \mid s) \geq 0, \sum_{\hat{a} \in \mathcal{A}} \pi(\hat{a} \mid s) = 1, a \in \mathcal{A}, s \in \mathcal{S} \right\}.$$

Следующее утверждение задаёт подкласс оптимальных политик, в рамках которого достаточно производить поиск интересующей политики π .

Пусть Π — набор всех нестационарных и рандомизированных политик. Поскольку $V^\pi(s)$ и $Q^\pi(s, a)$ зажаты между 0 и $\frac{1}{1-\gamma}$, существуют конечные значения:

$$V^*(s) := \sup_{\pi \in \Pi} \{V^\pi(s)\},$$

$$Q^*(s, a) := \sup_{\pi \in \Pi} \{Q^\pi(s, a)\}.$$

Существует стационарная детерминированная политика π такая, что

$$\forall s \in \mathcal{S}, a \in \mathcal{A}: \quad V^\pi(s) = V^*(s), \quad Q^\pi(s, a) = Q^*(s, a).$$

Следовательно, такая π является оптимальной политикой.

В данном утверждении можно заменить операцию $\sup$ на $\max$, как минимум в случае наград с достижимыми верхними гранями:

$$V^*(s) = \max_{\pi \in \Pi} \{V^\pi(s)\}.$$

Здесь V^* — оптимальная функция ценности.

Введём обозначение класса всех отображений, описывающих детерминированные политики:

$$\mathcal{A}_{\text{det}} = \{a(\cdot) \mid a : \mathcal{S} \rightarrow \mathcal{A}\}.$$

2.10 Уравнение оптимальности Беллмана для V^*

Если воспользоваться принципом динамического программирования, то удаётся вывести уравнение оптимальности Беллмана для V -функции ценности:

$$\begin{aligned} V^*(s) &= \max_{a(\cdot) \in \mathcal{A}_{\text{det}}} \left\{ \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a(s_t)) \right] \right\} \\ &= \max_{a(\cdot) \in \mathcal{A}_{\text{det}}} \left\{ \mathbb{E} \left[r(s, a(s)) + \gamma \sum_{t=0}^{\infty} \gamma^t r(s_{t+1}, a(s_{t+1})) \right] \right\} \\ &= \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}[V^*(s')]\} \\ &= \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^*(s') \right\}. \end{aligned}$$

2.11 Уравнение оптимальности Беллмана для Q^*

Аналогичные рассуждения можно провести для Q -функции ценности:

$$Q^*(s, a) = \max_{\pi \in \Pi} \{Q^\pi(s, a)\}.$$

Тогда:

$$\begin{aligned} Q^*(s, a) &= \mathbb{E}[r(s, a) + \gamma V^*(s') \mid s_0 = s, a_0 = a] \\ &= r(s, a) + \gamma \mathbb{E} \left[\max_{a' \in \mathcal{A}} \{Q^*(s', a')\} \right] \\ &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{Q^*(s', a')\}. \end{aligned}$$

2.12 Критерий оптимальности относительно Q -функции

Функция Q представляет собой оптимальную функцию ценности Q^* тогда и только тогда, когда она удовлетворяет уравнениям оптимальности Беллмана:

$$\begin{aligned} Q(s, a) &= \mathbb{E} \left[r(s, a(s)) + \gamma \max_{a' \in \mathcal{A}} \{Q(s', a')\} \mid s_0 = s, a_0 = a \right] \\ &= \sum_{s' \in \mathcal{S}} p(s, a; s') \left[r(s, a) + \gamma \max_{a' \in \mathcal{A}} \{Q(s', a')\} \right], \end{aligned}$$

для всех

$$s \in \mathcal{S}, \quad a \in \mathcal{A}.$$

Кроме того, детерминированная политика, определённая как

$$\pi(s) \in \text{Arg max}_{a \in \mathcal{A}} \{Q^*(s, a)\},$$

является оптимальной политикой.

2.13 DMDP. Уравнения оптимальности Беллмана

Для оптимальной политики π^* выполнены следующие соотношения:

1.

$$V^{\pi^*}(s) = \max_{a \in \mathcal{A}} \{Q^{\pi^*}(s, a)\}, \quad \forall s \in \mathcal{S}.$$

2.

$$Q^{\pi^*}(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^{\pi^*}(s'), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

3.

$$V^{\pi^*}(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^{\pi^*}(s') \right\}, \quad \forall s \in \mathcal{S}.$$

4.

$$Q^{\pi^*}(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{Q^{\pi^*}(s', a')\}, \quad s \in \mathcal{S}, a \in \mathcal{A}.$$

Соответствующая этим соотношениям детерминированная политика:

$$\begin{aligned} \pi^*(s) &\in \text{Arg max}_{a \in \mathcal{A}} \{Q^{\pi^*}(s, a)\} \\ &= \text{Arg max}_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^{\pi^*}(s') \right\}. \end{aligned}$$

2.14 Сдвиг функций ценности и инвариантность оптимальной политики

2.14.1 Основное свойство оптимальных V/Q -функций

Если V^* и Q^* — оптимальные функции, то функции

$$V^*(s) := V^*(s) + \alpha,$$

$$Q^*(s, a) := Q^*(s, a) + \alpha,$$

для всех

$$s \in \mathcal{S}, \quad a \in \mathcal{A}, \quad \alpha \in \mathbb{R},$$

также соответствуют оптимальной политике, поскольку:

$$\begin{aligned} \pi^*(s) &\in \text{Arg max}_{a \in \mathcal{A}} \{Q^*(s, a)\} \\ &= \text{Arg max}_{a \in \mathcal{A}} \{Q^*(s, a) + \alpha\} \\ &= \text{Arg max}_{a \in \mathcal{A}} \{Q^*(s, a)\}. \end{aligned}$$

Также:

$$\begin{aligned}\pi^*(s) &= \text{Arg max}_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^*(s') \right\} \\ &= \text{Arg max}_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') (V^*(s') + \alpha) \right\},\end{aligned}$$

где

$$V^*(s') = V^*(s') + \alpha.$$

2.14.2 Константный сдвиг функции награды

Для уравнения Беллмана верна связь с константным сдвигом функции награды:

$$\hat{r}(s, a) := r(s, a) + \alpha(1 - \gamma), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}, \alpha \in \mathbb{R}.$$

Тогда:

$$\begin{aligned}Q^*(s, a) &= Q^*(s, a) + \alpha \\ &= (r(s, a) + \alpha(1 - \gamma)) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{Q^*(s', a') + \alpha\} \\ &= \hat{r}(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{Q^*(s', a')\} \\ &= \hat{r}(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^*(s').\end{aligned}$$

Аналогично для V^* :

$$\begin{aligned}V^*(s) &= \max_{a \in \mathcal{A}} \left\{ (r(s, a) + \alpha(1 - \gamma)) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') (V^*(s') + \alpha) \right\} \\ &= \max_{a \in \mathcal{A}} \left\{ \hat{r}(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^*(s') \right\} \\ &= \max_{a \in \mathcal{A}} \{Q^*(s, a)\}.\end{aligned}$$

2.14.3 Аддитивное преобразование награды

Аддитивное преобразование, не меняющее решение π^* , задаётся как:

$$\hat{r}(s, a) := \beta \left(r(s, a) + f(s) - \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') f(s') \right),$$

где

$$\forall s \in \mathcal{S}, a \in \mathcal{A}, \quad \beta > 0,$$

а

$$f : \mathcal{S} \rightarrow \mathbb{R}$$

— произвольное отображение.

Тогда:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} \{Q^*(s, a)\} = \arg \max_{a \in \mathcal{A}} \{\beta (Q^*(s, a) + f(s))\}.$$

Определим:

$$Q^*(s, a) := \beta (Q^*(s, a) + f(s)).$$

Тогда:

$$\begin{aligned} Q^*(s, a) &= \beta (r(s, a) + f(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{\beta Q^*(s', a')\} \\ &= \hat{r}(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{\beta (Q^*(s', a') + f(s'))\} \\ &= \hat{r}(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') \max_{a' \in \mathcal{A}} \{Q^*(s', a')\}. \end{aligned}$$

Для функции награды, зависящей от $(s, a, s') \in \mathcal{S} \times \mathcal{A} \times \mathcal{S}$, аддитивное преобразование, инвариантное относительно оптимальной политики, выглядит проще:

$$\hat{r}(s, a, s') := \beta (r(s, a, s') + f(s) - \gamma f(s')).$$

Тогда:

$$\begin{aligned} Q^*(s, a) &:= \beta (Q^*(s, a) + f(s)) \\ &= \sum_{s' \in \mathcal{S}} p(s, a; s') \left(\beta (r(s, a, s') + f(s) - \gamma f(s')) + \gamma \max_{a' \in \mathcal{A}} \{\beta (Q^*(s', a') + f(s'))\} \right) \\ &= \sum_{s' \in \mathcal{S}} p(s, a; s') \left(\hat{r}(s, a, s') + \gamma \max_{a' \in \mathcal{A}} \{Q^*(s', a')\} \right). \end{aligned}$$

2.15 Предположения для построения алгоритмов

Будем считать, что о среде известны следующие функции:

•

$$p(s' \mid s, a)$$

— вероятность попасть в состояние s' из состояния s с помощью действия a ;

•

$$r(s, a)$$

— награда за выполнение действия a в состоянии s .

Замечание 6. Эти допущения существенно сужают круг задач, которые можно решать, однако алгоритмы, предложенные в этой лекции, будут оптимальными в данной постановке.

2.16 Построение алгоритма обучения политики

Сам процесс можно разделить на два этапа:

1. оценка качества текущей политики;
2. поиск следующего приближения оптимальной политики.

Дополнительно предположим дискретность и конечность пространств $\mathcal{S}$ и $\mathcal{A}$.
Начнём с оценивания — вычислим $V^\pi(s)$ и $Q^\pi(s, a)$:

$$V^\pi(s) = \underbrace{\mathbb{E}_{\pi(a|s)} [r(s, a)]}_{:=u(s)} + \gamma \underbrace{\mathbb{E}_{\pi(a|s)} \mathbb{E}_{p(s'|s, a)} [V^\pi(s')]}_{:=F(V^\pi(s))}, \quad \forall s \in \mathcal{S}.$$

Представим это выражение в виде матрично-векторных операций.

Здесь:

- V^π — вектор ценностей состояний;
- P — матрица вероятностей;
- u — вектор средних наград за шаг.

Элементы матрицы P :

$$P_{ss'} = \sum_{a \in \mathcal{A}} \pi(a | s) p(s' | s, a) = p(s' | s).$$

Тогда:

$$\begin{aligned} V^\pi(s) &= \sum_{a \in \mathcal{A}} \pi(a | s) r(s, a) + \gamma \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} p(s' | s, a) V^\pi(s') \\ &= u(s) + \gamma \sum_{s' \in \mathcal{S}} V^\pi(s') p(s' | s). \end{aligned}$$

Следовательно:

$$V^\pi = F(V^\pi) = u + \gamma P V^\pi.$$

Полученное отображение F является сжимающим. Для произвольных векторов V и W :

$$\begin{aligned} \|F(V) - F(W)\|_\infty &= \|u + \gamma P V - u - \gamma P W\|_\infty \\ &= \gamma \|P(V - W)\|_\infty \\ &\leq \gamma \|P\|_\infty \|V - W\|_\infty \\ &= \gamma \|V - W\|_\infty. \end{aligned}$$

Здесь:

$$\|P\|_\infty = \max_{x \neq 0} \left\{ \frac{\|Px\|_\infty}{\|x\|_\infty} \right\} = \max_{\|x\|_\infty \leq 1} \max_{s \in \mathcal{S}} \left\{ \sum_{s' \in \mathcal{S}} p(s' | s) x_{s'} \right\} = 1.$$

2.17 Iterative Policy Evaluation

Получаем алгоритм **Iterative Policy Evaluation** для оценки политики π с заданной точностью $\varepsilon > 0$.

1. Инициализировать $V(s)$ для всех $s \in \mathcal{S}$.

2. Повторять в цикле:

(a)

$$\Delta := 0.$$

(b) Для всех $s \in \mathcal{S}$:

$$\begin{aligned}\delta &:= V(s), \\ V(s) &:= \mathbb{E}_{\pi(a|s)} [r(s, a)] + \gamma \mathbb{E}_{\pi(a|s)} \mathbb{E}_{p(s'|s, a)} [V(s')], \\ \Delta &:= \max \{ \Delta, |\delta - V(s)| \}.\end{aligned}$$

(c) Если

$$\Delta \leq \varepsilon,$$

то выход, иначе переход на шаг 2.

2.18 Улучшение политики

Теперь построим процедуру улучшения оценённой политики π .

Для фиксированной политики:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^\pi(s')].$$

Определение 1. Политика $\hat{\pi}$ монотонно лучше политики π , если

$$\hat{\pi} \succeq \pi$$

и

$$V^{\hat{\pi}}(s) \geq V^\pi(s), \quad \forall s \in \mathcal{S}.$$

Изменяя действие для одного состояния $\hat{s} \in \mathcal{S}$, можно произвести улучшение π :

$$\hat{\pi}(a | \hat{s}) = \delta_{\{\arg \max_{\hat{a} \in \mathcal{A}} \{Q^\pi(\hat{s}, \hat{a})\}\}}(a),$$

то есть

$$\hat{\pi}(\hat{s}) := \arg \max_{\hat{a} \in \mathcal{A}} \{Q^\pi(\hat{s}, \hat{a})\}.$$

Для остальных состояний политика не меняется:

$$\hat{\pi}(a | s) = \pi(a | s), \quad \forall s \in \mathcal{S}, s \neq \hat{s}.$$

То есть:

$$Q^\pi(s, \hat{\pi}(s)) \geq V^\pi(s).$$

2.19 Теорема об улучшении политики

Теорема 1 (Теорема об улучшении политики). Пусть π и $\hat{\pi}$ — любая пара детерминированных политик таких, что

$$\forall s \in \mathcal{S} : \quad Q^\pi(s, \hat{\pi}(s)) \geq V^\pi(s). \quad (1)$$

Тогда $\hat{\pi}$ не хуже, чем π , то есть:

$$\forall s \in \mathcal{S} : \quad V^{\hat{\pi}}(s) \geq V^\pi(s). \quad (2)$$

Более того, если в каком-либо состоянии существует строгое неравенство в условии (1), то и (2) должно быть строгим.

Действительно:

$$\begin{aligned}
V^\pi(s) &\leq Q^\pi(s, \hat{\pi}(s)) \\
&= r(s, \hat{\pi}(s)) + \gamma \mathbb{E}_{p(s'|s, a)} [V^\pi(s')] \\
&\leq r(s, \hat{\pi}(s)) + \gamma \mathbb{E}_{p(s'|s, a)} [Q^\pi(s', \hat{\pi}(s'))] \\
&\leq \dots \\
&\leq r(s, \hat{\pi}(s)) + \gamma \mathbb{E}_{p(s'|s, \hat{\pi}(s))} [r(s', \hat{\pi}(s')) + \gamma^2 \dots] \\
&= V^{\hat{\pi}}(s), \quad \forall s \in \mathcal{S}.
\end{aligned}$$

Шаг для обновления всей политики:

$$\pi_{\text{new}}(s) := \arg \max_{a \in \mathcal{A}} \{Q^{\pi_{\text{old}}}(s, a)\}, \quad \forall s \in \mathcal{S}.$$

Если после очередного шага обновления политики получилось, что

$$Q^\pi(s, \hat{\pi}(s)) = V^\pi(s), \quad \forall s \in \mathcal{S},$$

то это означает удовлетворение уравнению Беллмана:

$$\hat{\pi} = \pi,$$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^\pi(s')], \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

2.20 Policy Iteration

Мы получили алгоритм **Policy Iteration**:

1. Инициализируем $V(s)$ и $\pi(a | s)$ для всех $s \in \mathcal{S}$.
2. Оценить $V^\pi(s)$ для текущей π , используя **Iterative Policy Evaluation**.
3. Улучшить политику:

(a)

Flag := True.

(b) Для всех $s \in \mathcal{S}$:

$$a = \pi(s),$$

$$\pi(s) := \arg \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^\pi(s')]\}.$$

Если

$$a \neq \pi(s),$$

то

Flag := False.

4. Если

Flag = True,

то выход, иначе переход на шаг 2.

Описанную процедуру поиска оптимальной политики можно представить как последовательность монотонно улучшающихся политик и функций ценности:

$$\pi_0 \xrightarrow{E} V^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V^{\pi_1} \xrightarrow{I} \dots \xrightarrow{I} \pi^* \xrightarrow{E} V^*.$$

Здесь:

- $\xrightarrow{E}$ обозначает оценку политики;
- $\xrightarrow{I}$ обозначает улучшение политики.

Таким образом получен алгоритм итеративной оптимизации политики.

2.21 Value Iteration

Уравнение Беллмана относительно фиксированной политики по сути решается простым итеративным способом.

Начальное приближение V_0 выбирается произвольно, а каждая последовательная итерация реализуется согласно уравнению Беллмана:

$$V_{t+1}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} p(s, a; s') (r(s, a) + \gamma V_t(s')).$$

Здесь V^π является фиксированной точкой:

$$V_t = V^\pi.$$

Для получения каждого последующего приближения V_{t+1} из V_t при итеративной оценке политики применяется та же операция к каждому состоянию s . Её называют ожидаемым обновлением, а данный алгоритм — итерацией функции ценности.

Для решения уравнения оптимальности Беллмана можно проводить следующую процедуру:

$$V_{t+1}(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V_t(s') \right\}, \quad \forall s \in \mathcal{S}.$$

Политика затем определяется как:

$$\pi_{t+1}(s) \in \text{Arg max}_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V_{t+1}(s') \right\}, \quad \forall s \in \mathcal{S}.$$

Начальное значение V_0 может быть произвольным, а в качестве критерия останова может выступать:

$$\|V_{t+1} - V_t\|_\infty \leq \varepsilon.$$

Алгоритм **Value Iteration** является частным случаем **Policy Iteration**: вместо отдельных шагов оценки политики и улучшения политики делается один объединённый шаг.

Алгоритм:

1. Инициализировать $V(s)$ для всех $s \in \mathcal{S}$.

2. Повторять:

(a)

$$\Delta := 0.$$

(b) Для всех $s \in \mathcal{S}$:

$$\begin{aligned} v &= V(s), \\ V(s) &:= \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V(s')]\}, \\ \Delta &:= \max \{\Delta, |v - V(s)|\}. \end{aligned}$$

(c) Если

$$\Delta \leq \varepsilon,$$

то выход, иначе переход на шаг 2.

Мы по сути схлопнули **Policy Evaluation** и **Policy Improvement**, не вычисляя π явно.

В результате работы алгоритма **Value Iteration** оптимизированная политика вычисляется следующим образом:

$$\pi(s) = \arg \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V(s')]\}.$$

2.22 Сходимость алгоритма Value Iteration

Для оптимальной V -функции выполнено:

$$V^*(s) = V^{\pi^*}(s) = \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^*(s')]\}, \quad \forall s \in \mathcal{S}.$$

Рассмотрим:

$$V^{\pi_{t+1}}(s),$$

где

$$\pi_{t+1}(s) = \arg \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^{\pi_t}(s')]\}.$$

Построим оценку на невязку по V -функции. Пусть

$$V^*(s) \geq V^{\pi_t}(s), \quad t \in \mathbb{Z}_+, \quad s \in \mathcal{S}.$$

Тогда:

$$\begin{aligned} \max_{s \in \mathcal{S}} \{|V^*(s) - V^{\pi_{t+1}}(s)|\} &= \max_{s \in \mathcal{S}} \left\{ \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^*(s')]\} \right. \\ &\quad \left. - \max_{a' \in \mathcal{A}} \{r(s, a') + \gamma \mathbb{E}_{p(s''|s, a')} [V^{\pi_t}(s'')]\} \right\} \\ &\leq \gamma \max_{\substack{s \in \mathcal{S} \\ a \in \mathcal{A}}} \{\mathbb{E}_{p(s'|s, a)} [V^*(s')] - \mathbb{E}_{p(s''|s, a)} [V^{\pi_t}(s'')]\} \\ &= \gamma \max_{\substack{s \in \mathcal{S} \\ a \in \mathcal{A}}} \{\mathbb{E}_{p(s'|s, a)} [V^*(s') - V^{\pi_t}(s')]\} \\ &\leq \gamma \max_{s' \in \mathcal{S}} \{V^*(s') - V^{\pi_t}(s')\} \\ &= \gamma \max_{s' \in \mathcal{S}} \{|V^*(s') - V^{\pi_t}(s')|\}. \end{aligned}$$

Следовательно:

$$\max_{s \in \mathcal{S}} \{|V^{\pi_t}(s) - V^*(s)|\} \leq \gamma^t \max_{s' \in \mathcal{S}} \{|V^{\pi_0}(s') - V^*(s')|\}.$$

В качестве $V^{\pi_0}(s)$ можно взять произвольное начальное приближение, например, тождественную по всем состояниям константу.

Имеем оценку относительно внешних итераций:

$$\max_{s \in \mathcal{S}} \{|V^{\pi_t}(s) - V^*(s)|\} = O(\gamma^t), \quad t \in \mathbb{Z}_+, \gamma \in (0, 1).$$

2.23 Value Iteration. Сложность решения

Введём оператор:

$$T : \mathbb{R}^{\mathcal{S}} \rightarrow \mathbb{R}^{\mathcal{S}},$$

$$T(V)_s := \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V(s') \right\}.$$

Теперь обновление V -функции в **Value Iteration** выражается следующим образом:

$$V_{t+1} := T(V_t),$$

а V -функция кодируется вещественным вектором.

Теорема 2. *Существует DMDP, для которого последовательность оценок V -функции удовлетворяет:*

$$V_0 = 0_S,$$

$$V_{n+1} \in \text{span} \{V_0, \dots, V_n, T(V_0), \dots, T(V_n)\}, \quad n \in \mathbb{Z}_+,$$

со следующим свойством для всех $n = 0, \dots, N-1$:

$$\|V_n - V^*\|_{\infty} \geq \frac{\gamma^n}{1 + \gamma},$$

где

$$V^* = T(V^*).$$

Доказательство. Для произвольного $\gamma \in (0, 1)$ предложим DMDP с N состояниями и одним действием.

Награда для первого состояния:

$$r_1 := 1.$$

Для остальных состояний:

$$r_i := 0, \quad i = 2, \dots, N.$$

Действие из первого состояния оставляет систему в нём же. Действие из $(i+1)$ -го состояния переводит в i -е.

Оптимальное значение V -функции:

$$V^*(i) = \frac{\gamma^{i-1}}{1 - \gamma}.$$

Рассмотрим последовательность векторов $(V_n)_{n \geq 0}$:

$$V_0 = 0_S,$$

$$V_{n+1} \in \text{span} \{V_0, \dots, V_n, T(V_0), \dots, T(V_n)\}, \quad n \geq 0.$$

Докажем через раскрытие рекурсии, что для всех $n \geq 0$ и $i \in \mathcal{S}$:

$$V_n(i) = 0, \quad \text{если } i \geq n + 1.$$

Для $n = 0$ это верно, так как

$$V_0 = 0_S.$$

Предположим, что утверждение верно для

$$V_0, \dots, V_{n-1}.$$

По определению оператора T и в силу того, что

$$r_i = 0, \quad i \geq 2,$$

имеем:

$$T(V_t)_i = 0, \quad \text{если } i \geq t + 2, \quad \forall t \leq n - 1.$$

Следовательно, в силу

$$V_{n+1} \in \text{span} \{V_0, \dots, V_n, T(V_0), \dots, T(V_n)\},$$

замечаем, что

$$V_n(i) = 0, \quad \text{если } i \geq n + 1.$$

Таким образом, рекурсия доказана.

Поскольку $r_1 > 0$ — единственная ненулевая награда, по сути доказано, что для любого метода первого порядка требуется $n - 1$ шагов для распространения награды первого состояния до состояния $1 \leq n \leq N$.

Теперь для $1 \leq n \leq N - 1$:

$$\|V_n - T(V_n)\|_\infty = \|V_n - T(V_n) - (V^* - T(V^*))\|_\infty \quad (3)$$

$$\geq (1 - \gamma) \|V_n - V^*\|_\infty \quad (4)$$

$$= (1 - \gamma) \max_{1 \leq i \leq N} \{|V_n(i) - V^*(i)|\}$$

$$\geq (1 - \gamma) \max_{n+1 \leq i \leq N} \{|V_n(i) - V^*(i)|\}$$

$$\geq (1 - \gamma) \max_{n+1 \leq i \leq N} \{|V^*(i)|\} \quad (5)$$

$$\geq (1 - \gamma) \max_{n+1 \leq i \leq N} \left\{ \frac{\gamma^{i-1}}{1 - \gamma} \right\}$$

$$\geq \gamma^n.$$

Здесь:

- (3) следует из $V^* = T(V^*)$;
- (4) следует из неравенства ниже;

- (5) следует из $V_n(i) = 0$ для $i \geq n + 1$.

Для всех $V, W \in \mathbb{R}^S$:

$$\begin{aligned} (1 - \gamma)\|V - W\|_\infty &\leq \|(I - T)(V) - (I - T)(W)\|_\infty, \\ \|(I - T)(V) - (I - T)(W)\|_\infty &\leq (1 + \gamma)\|V - W\|_\infty. \end{aligned} \quad (6)$$

Следовательно, в силу (6):

$$\begin{aligned} \|V_n - V^*\|_\infty &\geq \frac{1}{1 + \gamma} \cdot \|V_n - T(V_n) - (V^* - T(V^*))\|_\infty \\ &= \frac{1}{1 + \gamma} \cdot \|V_n - T(V_n)\|_\infty \\ &\geq \frac{\gamma^n}{1 + \gamma}. \end{aligned}$$

Утверждение доказано.

Из доказанного утверждения вместе с утверждением о сходимости **Value Iteration** следует оптимальность алгоритма **Value Iteration**:

$$\frac{\gamma^t}{1 - \gamma} = \gamma^t \|V_0 - V^*\|_\infty \geq \|V_t - V^*\|_\infty \geq \frac{\gamma^t}{1 + \gamma}, \quad t \in \mathbb{Z}_+, \gamma \in (0, 1).$$

При

$$V_0 = 0_N$$

получаем:

$$\|V_t - V^*\|_\infty = \Theta(\gamma^t).$$

2.24 LP-релаксация MDP. AMDP

Для AMDP LP-задача вводится следующим образом:

$$V^*(s) = \max_{a(\cdot) \in \mathcal{A}_{\text{det}}} \left\{ \lim_{H \rightarrow \infty} \frac{1}{H} \mathbb{E} \left[\sum_{t=0}^{H-1} r(s_t, a_t(s_t)) \mid s_0 = s \right] \right\}.$$

Здесь H — эпизодическое ограничение, то есть максимальная длина эпизода.

В случае эпизодов конечной длины предел опускается и используется максимальное значение H .

Для политики $\pi(a \mid s)$ можно определить стационарное распределение:

$$\nu^\pi(s') = \sum_{(s,a) \in \mathcal{S} \times \mathcal{A}} p(s, a; s') \pi(a \mid s) \nu^\pi(s), \quad s' \in \mathcal{S}.$$

Ему соответствует вектор вероятностей:

$$\nu^\pi = (\nu^\pi(s))_{s \in \mathcal{S}}.$$

Если MDP равномерно эргодично, то:

$$\begin{aligned} V(\pi) &:= \lim_{H \rightarrow \infty} \frac{1}{H} \mathbb{E} \left[\sum_{t=0}^{H-1} r(s_t, a_t(s_t)) \right] \\ &= \sum_{(s,a) \in \mathcal{S} \times \mathcal{A}} r(s, a) \pi(a \mid s) \nu^\pi(s). \end{aligned}$$

Равномерная эргодичность соответствует:

$$\max_{i=1, \dots, S} \{ \| (P^\pi)^n e_i - \nu^\pi \|_\infty \} \rightarrow 0, \quad n \rightarrow \infty,$$

где

$$e_i^\top = (0, \dots, 0, \underbrace{1}_i, 0, \dots, 0).$$

Вводится распределение действий по состояниям:

$$\mu(s, a) = \nu^\pi(s) \pi(a \mid s).$$

Следовательно, задачу поиска оптимальной политики в AMDP можно переписать как задачу линейного программирования со смыслом оценки ценности политики по распределению μ :

$$\begin{aligned} \max_{\mu \in \Delta_{\mathcal{S} \times \mathcal{A}}} \quad & V(\mu) = \sum_{(s,a) \in \mathcal{S} \times \mathcal{A}} r(s, a) \mu(s, a) = \langle r, \mu \rangle \\ \text{s.t.} \quad & \sum_{b \in \mathcal{A}} \mu(s', b) = \sum_{(s,a) \in \mathcal{S} \times \mathcal{A}} p(s, a; s') \mu(s, a), \quad s' \in \mathcal{S}. \end{aligned}$$

Здесь:

$$\Delta_{\mathcal{S} \times \mathcal{A}} = \left\{ \mu \mid \mu(s, a) \geq 0, \sum_{(s,a) \in \mathcal{S} \times \mathcal{A}} \mu(s, a) = 1 \right\}.$$

Политика восстанавливается по μ :

$$\pi_\mu(a \mid s) = \frac{\mu(s, a)}{\sum_{b \in \mathcal{A}} \mu(s, b)}.$$

Данную LP-задачу можно напрямую переписать в матричной форме:

$$\begin{aligned} \max_{\mu \in \Delta_{\mathcal{S} \times \mathcal{A}}} \quad & \langle r, \mu \rangle, \\ \text{s.t.} \quad & (I_b - P) \mu = 0. \end{aligned}$$

Единичная матрица I_b имеет нестандартный формат: это прямоугольная матрица размера

$$S \times (SA).$$

На каждой строке $s \in \mathcal{S}$ только элементы, соответствующие паре (s, a) , $a \in \mathcal{A}$, равняются единице, остальные элементы данной строки равняются нулю. То есть на каждой строке I_b ровно A единиц.

У матрицы P размера

$$S \times (SA)$$

в каждом столбце $(s, a) \in \mathcal{S} \times \mathcal{A}$ записано распределение:

$$P(\cdot \mid s, a).$$

Для этой LP-задачи напрямую строится двойственная задача, с условием $\mu \geq 0$, которая имеет смысл оценки ценности оптимальной политики через V -функцию:

$$\begin{aligned} \min_{V \in \mathbb{R}, \mathbf{V} \in \mathbb{R}^{|\mathcal{S}|}} \quad & V, \\ \text{s.t.} \quad & r - V \cdot \mathbf{1}_{\mathcal{S}\mathcal{A}} - (I_b - P)^\top \mathbf{V} \leq 0. \end{aligned}$$

Таким образом, имеет место уравнение оптимальности Беллмана со средним вознаграждением:

$$V(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) - V^* + \sum_{s' \in \mathcal{S}} p(s, a; s') V(s') \right\},$$

где

$$V^* = \langle r, \mu^* \rangle.$$

Это уравнение получается из ограничений вида:

$$I_b^\top \mathbf{V} \geq r - V \cdot \mathbf{1}_{\mathcal{S}\mathcal{A}} + P^\top \mathbf{V}.$$

2.25 LP-релаксация MDP. DMDP

Для DMDP LP-задача записывается в следующем виде. Пусть q — распределение начального состояния μ_0 в виде вектора.

Прямая задача:

$$\begin{aligned} \min_{\mathbf{V} \in \mathbb{R}^{|\mathcal{S}|}} \quad & \langle q, \mathbf{V} \rangle, \\ \text{s.t.} \quad & r - (I_b - \gamma P)^\top \mathbf{V} \leq 0. \end{aligned}$$

Ей соответствует двойственная задача:

$$\begin{aligned} \max_{\mu \in \Delta_{\mathcal{S} \times \mathcal{A}}} \quad & \langle r, \mu \rangle, \\ \text{s.t.} \quad & (I_b - \gamma P)\mu = q. \end{aligned}$$

Существует также постановка задачи LP для ограниченного DMDP — **Constrained Markov Decision Process**, или CMDP:

$$\begin{aligned} \max_{\mu \in \Delta_{\mathcal{S} \times \mathcal{A}}} \quad & \langle r, \mu \rangle, \\ \text{s.t.} \quad & (I_b - \gamma P)\mu = q, \quad D\mu \geq c. \end{aligned}$$

По сравнению с предыдущими задачами линейного программирования вводится дополнительно аффинное ограничение вида неравенства:

$$D\mu \geq c.$$

Заметим, что вместо обозначенного ранее распределения

$$\mu(s, a) = \nu^\pi(s) \pi(a \mid s)$$

может быть полезно рассмотреть:

$$\mu(s, a) := \mu^\pi(s, a) = \mathbb{E}_{s_0 \sim \mu_0} \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s, a_t = a \mid s_0) \right].$$

Или даже масштабированную сумму в виде корректно определённой вероятностной меры:

$$\begin{aligned} \mu(s, a) &:= \tilde{\mu}^\pi(s, a) = (1 - \gamma) \mu^\pi(s, a) \\ &= \mathbb{E}_{s_0 \sim \mu_0} \left[(1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s, a_t = a \mid s_0) \right]. \end{aligned}$$

В обоих случаях получается одна и та же политика:

$$\pi(a \mid s) = \frac{\mu^\pi(s, a)}{\sum_{b \in \mathcal{A}} \mu^\pi(s, b)} = \frac{\tilde{\mu}^\pi(s, a)}{\sum_{b \in \mathcal{A}} \tilde{\mu}^\pi(s, b)}.$$

2.26 Связь решения DMDP с выпуклой оптимизацией

Обозначим через

$$v \in \mathbb{R}^S$$

вектор, кодирующий V -функцию ценности.

Тогда решение уравнения

$$v = T(v)$$

соответствует поиску стационарной точки некоторой функции

$$f : \mathbb{R}^S \rightarrow \mathbb{R}$$

со следующим градиентом и субгессианом:

$$\nabla f(v) := v - T(v) = (I - T)(v) =: F(v).$$

$$\nabla^2 f(v) := I - \gamma P^{\pi_v} = \partial F(v) \succeq 0_{S \times S}.$$

Здесь:

$$\begin{aligned} T(v) &= r^{\pi_v} + \gamma P^{\pi_v} v, \\ (r^{\pi_v})_s &:= r(s, a_s^v), \quad s \in \mathcal{S}. \end{aligned}$$

Политика π_v определяется как жадная по текущему v :

$$\pi_v(s) := a_s^v \in \text{Arg max}_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid a, s) V(s') \right\}.$$

Таким образом, можно формально ввести функцию:

$$f(v) := \frac{1}{2} \langle (I - \gamma P^{\pi_v}) v, v \rangle - \langle r^{\pi_v}, v \rangle + \text{const.}$$

Свойства функции $f(\cdot)$: для всех $v, w \in \mathbb{R}^S$ выполнено

$$(1 - \gamma) \|v - w\|_\infty \leq \|\nabla f(v) - \nabla f(w)\|_\infty \leq (1 + \gamma) \|v - w\|_\infty.$$

Также:

$$\frac{1-\gamma}{\sqrt{S}} \|v - w\|_2 \leq \|\nabla f(v) - \nabla f(w)\|_\infty \leq (1+\gamma) \|v - w\|_2.$$

И:

$$\frac{1-\gamma}{\sqrt{S}} \|v - w\|_2 \leq \|\nabla f(v) - \nabla f(w)\|_2 \leq \sqrt{S}(1+\gamma) \|v - w\|_2.$$

С помощью введённых обозначений алгоритм **Policy Iteration** можно переписать как шаг метода Ньютона для решения нелинейного уравнения:

$$F(v) = 0_S, \quad (v, v_0) \in \mathbb{R}^S \times \mathbb{R}^S.$$

Шаг Ньютона:

$$v_{t+1} := v_t - (\partial F(v_t))^{-1} F(v_t), \quad t \in \mathbb{Z}_+.$$

Данная процедура не обладает глобальной сходимостью.

2.27 Сглаженный оператор оптимальности Беллмана

Сглаженный оператор оптимальности Беллмана при $\beta > 0$:

$$T_\beta(v)_s := \frac{1}{\beta} \ln \left(\sum_{a \in \mathcal{A}} \exp \left(\beta \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | a, s) V(s') \right) \right) \right), \quad \forall s \in \mathcal{S}.$$

Если

$$v_\beta^* = T_\beta(v_\beta^*),$$

и

$$v^* = T(v^*),$$

то:

$$\|v_\beta^* - v^*\|_\infty \leq \frac{\gamma \ln(A)}{\beta(1-\gamma)}.$$

Метод Ньютона для решения

$$F_\beta(v) := v - T_\beta(v) = 0_S$$

сходится квадратично для любого начального приближения.

2.28 DMDP. Пример: задача о разборчивой невесте

В некотором царстве, в некотором государстве пришло время принцессе выбирать себе жениха.

В назначенный день явились 1000 царевичей. Более формально, пусть число претендентов достаточно большое, и расстановки претендентов равновероятны.

Их построили в очередь в случайном порядке и стали по одному приглашать к принцессе.

Про любых двух претендентов принцесса, познакомившись с ними, может сказать, какой из них лучше.

Познакомившись с претендентом, принцесса может:

- либо принять предложение, и тогда выбор сделан навсегда;

- либо отвергнуть его, и тогда претендент потерян.

Вопрос: какой стратегии должна придерживаться принцесса, чтобы с наибольшей вероятностью выбрать лучшего?

Оптимальная стратегия невесты:

- пропустить первых $\frac{1}{e}$ претендентов, где

$$e \simeq 2.718;$$

- затем выбрать первого наилучшего.

Среди пропущенных мог быть самый лучший — в таком случае никого лучше невеста уже не встретит.

Такая стратегия позволяет выбрать наилучшего жениха с вероятностью:

$$\frac{1}{e}.$$

Введём управляемую марковскую систему с

$$\gamma = 1.$$

Пространство состояний:

$$\mathcal{S} = \{1, 2, \dots, N, \text{End}\}, \quad N = 1000.$$

Состоянию s соответствует s -й претендент, оказавшийся наилучшим на данный момент.

Возможны два действия:

$$\mathcal{A} = \{\text{не выбрать, выбрать}\}.$$

Обозначим:

$$a_1 = \text{выбрать}, \quad a_2 = \text{не выбрать}.$$

Фиктивное состояние End наступает на следующем шаге:

- после того, как невеста пропустила лучшего жениха;
- или когда невеста сделала свой выбор.

Исходя из этого, можно посчитать функции вознаграждения и вероятности переходов.

Функция награды:

$$r(s = \text{End}, a_1 = \text{выбрать}) = 0,$$

$$r(s, a_2 = \text{не выбрать}) = 0.$$

Если выбираем текущего кандидата:

$$r(s, a_1) = \frac{s}{N}, \quad s = 1, \dots, N.$$

Поскольку:

$$r_{\xi}(s, a_1) = \begin{cases} 1, & \text{с вероятностью } \frac{s}{N}, \\ 0, & \text{с вероятностью } 1 - \frac{s}{N}. \end{cases}$$

Переходные вероятности:

$$p(s, a_1; s') = 0, \quad s' \neq \text{End},$$

$$p(s, a_1; s' = \text{End}) = 1.$$

Для конечного состояния:

$$p(s = \text{End}, a; s' = \text{End}) = 1.$$

Если не выбираем текущего кандидата, вероятность перейти в End:

$$p(s, a_2; s' = \text{End}) = \mathbb{P}(s\text{-й лучший, если известно, что он лучше предыдущих}) = \frac{s}{N}.$$

Для перехода в некоторое состояние s' :

$$\begin{aligned} p(s, a_2; s') &= \mathbb{P}(s'\text{-й} — \text{первый, кто лучше } s\text{-го, если } s\text{-й был лучшим}) \\ &= \frac{\mathbb{P}(s\text{-й лучше предыдущих и } s'\text{-й} — \text{первый, кто лучше } s\text{-го})}{\mathbb{P}(s\text{-й лучше предыдущих})} \\ &= \frac{s}{s'(s' - 1)}. \end{aligned}$$

Поскольку:

$$\mathbb{P}(s\text{-й лучше предыдущих}) = \frac{(s-1)!}{s!} = \frac{1}{s},$$

и

$$\mathbb{P}(s\text{-й лучше предыдущих и } s'\text{-й} — \text{первый, кто лучше } s\text{-го}) = \frac{(s'-2)!}{s'!} = \frac{1}{s'(s' - 1)}.$$

На слайде графы, отвечающие марковской цепи в задаче о разборчивой невесте, соответствуют двум действиям:

- при действии $a = \text{не выбрать}$ возможны переходы вперёд к следующему рекордному кандидату s' с вероятностью

$$p = \frac{s}{s'(s' - 1)},$$

а также переход в End с вероятностью

$$p = \frac{s}{N};$$

- при действии $a = \text{выбрать}$ из любого состояния происходит переход в End с вероятностью 1.

Функция $V^*(s)$ удовлетворяет уравнению Беллмана. В данном случае получившееся выражение называют также уравнением Вальда–Беллмана.

Оптимальная стратегия может быть найдена из условия:

$$a(s) = \arg \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s, a; s') V^*(s') \right\}.$$

Отсюда:

$$V^*(s) = \max \left\{ \frac{s}{N}, \sum_{s'=s+1}^N \frac{s}{s'(s'-1)} V^*(s') \right\}, \quad s = 1, \dots, N-1.$$

Граничное условие:

$$V^*(N) = 1.$$

Если максимум в $V^*(s)$ достигается на первом аргументе, то:

$$a(s) = \text{выбрать}.$$

Если максимум достигается на втором аргументе, то:

$$a(s) = \text{не выбрать}.$$

Оптимальный порог:

$$s^*(N) \simeq \frac{N}{e}.$$

Он задаётся условием:

$$\frac{1}{s^*} + \frac{1}{s^*+1} + \dots + \frac{1}{N-1} \leq 1 \leq \frac{1}{s^*-1} + \frac{1}{s^*} + \dots + \frac{1}{N-1}.$$

Подставим $s^*(N)$ в уравнение на функцию значений:

$$V^* = \frac{s^*(N) - 1}{N} \left(\frac{1}{s^*(N) - 1} + \frac{1}{s^*(N)} + \dots + \frac{1}{N-1} \right) \simeq \frac{1}{e}.$$

Следовательно:

$$V^*(s) = \begin{cases} V^*, & 1 \leq s \leq s^*(N), \\ \frac{s}{N}, & s \geq s^*(N). \end{cases}$$

2.29 Источники

1. S. Ivanov, “Reinforcement learning textbook,” arXiv preprint arXiv:2201.09746, настольная книга по данному курсу, 2022.
2. D. Bertsekas, *Reinforcement learning and optimal control*. Athena Scientific, 2019.
3. V. Goyal and J. Grand-Clement, “A first-order approach to accelerated value iteration,” *Operations Research*, vol. 71, no. 2, pp. 517–535, 2023.
4. M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
5. T. Liu, R. Zhou, D. Kalathil, P. Kumar, and C. Tian, “Policy optimization for constrained mdps with provable fast global convergence,” arXiv preprint arXiv:2111.00552, 2021.
6. J. Grand-Clément, “From convex optimization to mdps: A review of first-order, second-order and quasi-newton methods for mdps,” arXiv preprint arXiv:2104.10677, 2021.

3 Лекция 3. Model-free RL, Monte Carlo, Temporal Difference и Q-learning

3.1 Что делали ранее

В предыдущей лекции были введены понятия Q - и V -функций, с помощью которых можно ввести частичный порядок на политиках.

V -функция — это средняя награда по политике π , если агент начинает действовать в момент времени t из состояния s :

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, \pi \right].$$

Q -функция — это то же, что V -функция, но теперь из состояния $s_t = s$ обязательно сначала совершается действие $a_t = a$:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a, \pi \right].$$

Знать Q^π -функцию очень важно, поскольку мы можем изменить политику так, что новая политика не ухудшится, а в лучшем случае станет лучше:

$$\hat{\pi}(s) = \arg \max_{a \in \mathcal{A}} \{Q^\pi(s, a)\} \implies \hat{\pi} \succeq \pi.$$

Если такое улучшение для всех состояний s политику уже не изменяет, то эта политика оптимальна:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} \{Q^*(s, a)\}.$$

Найти Q - и V -функции помогают рекуррентные формулы на эти функции и на их оптимальные аналоги — уравнения Беллмана:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\pi(a|s)} [Q^\pi(s, a)], \\ Q^\pi(s, a) &= r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^\pi(s')], \\ V^*(s) &= \max_{a \in \mathcal{A}} \{Q^*(s, a)\}, \\ Q^*(s, a) &= r(s, a) + \gamma \mathbb{E}_{p(s'|s, a)} [V^*(s')]. \end{aligned}$$

Если о MDP-задаче известны $p(s' | s, a)$ и $r(s, a)$, а также множества $\mathcal{S}$ и $\mathcal{A}$ конечны, то решение уравнения Беллмана для V -функции сводится к системе линейных алгебраических уравнений:

$$V^\pi = U + \gamma P V^\pi =: F(V^\pi).$$

Это уравнение решается методом простой итерации, так как оператор Беллмана $F(V^\pi)$ является сжимающим с коэффициентом γ .

Такой метод решения называется **Policy Evaluation**.

Если о MDP-задаче известны $p(s' | s, a)$ и $r(s, a)$, а также множества $\mathcal{S}$ и $\mathcal{A}$ конечны, то решение уравнения Беллмана для оптимальной V^* -функции сводится к задаче линейного программирования. Из этого следует существование оптимальной детерминированной политики.

3.2 Policy Evaluation, Policy Iteration и Value Iteration

Policy Evaluation оценивает V -функцию для текущей политики. Алгоритм останавливается, когда V -функция меняется мало.

Policy Iteration оценивает V -функцию для фиксированной политики с помощью **Policy Evaluation**, а затем улучшает политику. Алгоритм останавливается, когда политика перестаёт меняться.

Value Iteration — частный случай **Policy Iteration**: шаг оценки V -функции и изменения политики схлопнуты в один шаг без непосредственного вычисления политики. Получение самой политики является отдельным шагом.

Недостатки. Для применения этих алгоритмов нужно знать о среде слишком много. При этом сама среда должна быть не очень большой.

3.3 Различные подходы

На практике очень часто неизвестно, как устроена среда, то есть неизвестны:

$$p(s' \mid s, a), \quad r(s, a).$$

Поэтому от этого предположения отказываются, но оставляют предположение конечности пространств:

$$|\mathcal{S}| \ll \infty, \quad |\mathcal{A}| \ll \infty.$$

3.3.1 Идея 1: Model-based RL

Можно свести задачу к уже решённой: аппроксимировать модель среды, а затем применять известные методы, например Policy Iteration или Value Iteration.

3.3.2 Идея 2: Model-free RL

Можно сразу учить политику без обучения модели среды.

3.4 Model-based RL

Плюсы. Задача аппроксимации среды может быть сведена к классическому supervised learning: отправляем в среду агента и обучаемся на данных, полученных от него.

Минусы. Переход к supervised learning почти никогда не работает напрямую, поскольку задача получается слишком сложной. Кроме того, нужно подбирать агента, который будет исследовать среду.

3.5 Model-free RL

Плюсы. Учить среду не нужно.

Оказывается, чтобы свести предыдущие алгоритмы к новой задаче, не обязательно учить среду.

Минусы. Нельзя свести задачу к обучению с учителем, поскольку для этого нет готового набора данных с правильными ответами.

3.6 Модификация алгоритмов для model-free RL

В **Policy Iteration** и **Value Iteration** обычно учат V -функцию, поскольку она занимает меньше памяти. Однако ничто не мешает сразу начать учить Q -функцию, а потом менять по ней политику.

Замечание 7. Чтобы выучить Q -функцию в алгоритмах выше, всё равно необходимо знать среду:

$$Q^{\pi_k}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [\mathbb{E}_{a'} [Q^{\pi_k}(s', a')]].$$

Чтобы решить эту проблему, придётся считать Q -функцию методом Монте-Карло по определению.

3.7 Метод Монте-Карло

3.7.1 Оценка Q -функции

Насэмплируем достаточное количество траекторий по политике π и усредним по ним значения Q -функции:

$$Q^{\pi}(s, a) \approx \frac{1}{M} \sum_{i=1}^M R(\tau_i),$$

где τ_i — одна из траекторий по политике π .

Замечание 8. Такой алгоритм будет несмещённой оценкой Q -функции, однако будет иметь очень большую дисперсию. Если и среда, и политика недетерминированы, то оценка есть сумма большого числа случайных величин, и чем длиннее траектория, тем больше дисперсия.

3.7.2 Схема алгоритма Монте-Карло

1. Инициализируем произвольную политику π .
2. В цикле по k :
 - (a) сэмплируем несколько траекторий при фиксированной политике π ;
 - (b) оцениваем $Q^{\pi}(s, a)$, используя метод Монте-Карло;
 - (c) обновляем политику:

$$\pi(s) \leftarrow \arg \max_{a \in \mathcal{A}} \{Q^{\pi}(s, a)\}.$$

3.7.3 Недостатки алгоритма Монте-Карло

- Обновить Q -функцию нельзя, пока эпизод не сыгран до конца. Поэтому метод не подходит для бесконечных игр.
- Не используется MDP-структура, потому что мы отказались от уравнения Беллмана.
- За конечное время точное значение Q -функции теперь не посчитать.

- У алгоритма высокая дисперсия.
- В данном виде алгоритм каждую новую итерацию цикла заново пересчитывает оценки Q -функции, выбрасывая старую оценку.

3.8 Модификация алгоритма Монте-Карло

Рассмотрим онлайн-вычисление среднего:

$$m_k := \frac{1}{k} \sum_{i=1}^k x_i.$$

Тогда:

$$m_k = \frac{k-1}{k} m_{k-1} + \frac{1}{k} x_k.$$

Обозначим:

$$\alpha_k := \frac{1}{k}.$$

Тогда:

$$m_k = (1 - \alpha_k) m_{k-1} + \alpha_k x_k.$$

Это можно интерпретировать как экспоненциальное сглаживание:

$$m_k = \underbrace{(1 - \alpha_k) m_{k-1} + \alpha_k x_k}_{\text{exponential smoothing}}.$$

Также:

$$m_k = m_{k-1} + \alpha_k (x_k - m_{k-1}).$$

Это напоминает стохастический градиентный спуск:

$$m_k = \underbrace{m_{k-1} + \alpha_k (x_k - m_{k-1})}_{\text{approximately stochastic gradient descent}}.$$

Замечание 9. Последнее равенство соответствует оптимизации MSE Loss. В оптимизации шаг обучения часто берут не только равным $\frac{1}{k}$.

3.9 Условия Роббинса–Монро

Теорема 3. Пусть

$$\mathbb{E}[x^2] < +\infty.$$

Если шаг обучения удовлетворяет условиям Роббинса–Монро:

$$\alpha_k \in (0, 1], \quad k \in \mathbb{N},$$

$$\sum_{k=1}^{+\infty} \alpha_k = +\infty,$$

$$\sum_{k=1}^{+\infty} \alpha_k^2 < +\infty,$$

то

$$m_k \xrightarrow[k \rightarrow \infty]{w.p. 1} \mathbb{E}[x].$$

Замечание 10. В применении к RL для сходимости метода Монте-Карло в условии теоремы необходимо добавить условие на политику:

$$\forall s, a : \quad \pi(a \mid s) > 0.$$

Это условие обеспечивает гарантированное посещение агентом всех состояний среды. Подробнее это обсуждается в конце лекции.

3.10 Temporal Difference

3.10.1 Идея

Для фиксированной политики:

$$Q^\pi(s, a) = \mathbb{E}_{s'} [r(s, a) + \gamma \mathbb{E}_{a'} Q^\pi(s', a')].$$

Это можно записать как:

$$Q^\pi(s, a) = \mathbb{E}_{s'} \left[\underbrace{r(s, a) + \gamma \mathbb{E}_{a'} Q^\pi(s', a')}_{f(s', x)} \right].$$

Или, раскрыв ожидание по a' :

$$Q^\pi(s, a) = \mathbb{E}_{s'} \mathbb{E}_{a'} \left[\underbrace{r(s, a) + \gamma Q^\pi(s', a')}_{f(s', a', x)} \right].$$

Теперь, чтобы обновить Q -функцию для пары (s, a) , нужен только один переход:

$$(s, a, r, s', a').$$

Введём целевое значение:

$$y := r + \gamma Q^\pi(s', a').$$

Тогда обновление:

$$Q^\pi(s, a) \leftarrow (1 - \alpha_k) Q^\pi(s, a) + \alpha_k y.$$

3.10.2 TD-обновление

Для перехода

$$(s, a, r, s', a')$$

получаем:

$$Q_{k+1}^\pi(s, a) \leftarrow Q_k^\pi(s, a) + \alpha_k (r + \gamma Q_k^\pi(s', a') - Q_k^\pi(s, a)).$$

Здесь:

$$\underbrace{r + \gamma Q_k^\pi(s', a')}_{\text{Bellman target}}$$

— беллмановская цель, а

$$\underbrace{r + \gamma Q_k^\pi(s', a') - Q_k^\pi(s, a)}_{\text{temporal difference}}$$

— temporal difference.

В этом обновлении:

- s, a фиксированы;

-

$$s' \sim p(s' \mid s, a)$$

— случайная величина, полученная из взаимодействия со средой;

-

$$a' \sim \pi(a' \mid s')$$

— случайная величина из алгоритма.

3.11 Сравнение Monte Carlo, Temporal Difference и Dynamic Programming

Общая форма обновления:

$$Q_{k+1}^{\pi}(s, a) \leftarrow Q_k^{\pi}(s, a) + \alpha_k (y(s, a) - Q_k^{\pi}(s, a)).$$

3.11.1 Temporal Difference

Для Temporal Difference:

$$y(s, a) := r + \gamma Q_k^{\pi}(s', a').$$

Свойства:

- обновление происходит после каждого шага;
- награда распространяется медленно, потому что алгоритм посещает по одной паре состояний за шаг;
- маленькая дисперсия;
- смещённая оценка.

3.11.2 Monte Carlo

Для Monte Carlo:

$$y(s, a) := r + \gamma r' + \gamma^2 r'' + \dots$$

Свойства:

- обновление происходит в конце эпизода;
- награда распространяется быстро, потому что алгоритм посещает несколько состояний в эпизоде разом;
- большая дисперсия;
- несмещённая оценка.

3.11.3 Dynamic Programming

Dynamic Programming использует модель среды и обновляет значения через математическое ожидание по всем возможным следующим состояниям и действиям. По сравнению с Monte Carlo и Temporal Difference, ему требуется знание модели среды.

3.12 Добавляем улучшение политики

Идея такая же, как в **Value Iteration**: будем улучшать оценку Q -функции с помощью TD и тут же улучшать политику.

Начинаем с TD-обновления:

$$Q_{k+1}^\pi(s, a) \leftarrow Q_k^\pi(s, a) + \alpha_k (r + \gamma Q_k^\pi(s', a') - Q_k^\pi(s, a)).$$

Если брать следующее действие по текущей политике:

$$a' = \pi_k(s'),$$

то:

$$Q_{k+1}^\pi(s, a) \leftarrow Q_k^\pi(s, a) + \alpha_k (r + \gamma Q_k^\pi(s', \pi_k(s')) - Q_k^\pi(s, a)).$$

Если политика выбирает жадное действие:

$$\pi_k(s') = \arg \max_{a' \in \mathcal{A}} \{Q_k^\pi(s', a')\},$$

то:

$$\begin{aligned} Q_{k+1}^\pi(s, a) &\leftarrow Q_k^\pi(s, a) + \alpha_k \left(r + \gamma Q_k^\pi \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_k^\pi(s', a')\} \right) - Q_k^\pi(s, a) \right) \\ &= Q_k^\pi(s, a) + \alpha_k \left(r + \gamma \max_{a' \in \mathcal{A}} \{Q_k^\pi(s', a')\} - Q_k^\pi(s, a) \right). \end{aligned}$$

Так получается идея Q -обучения.

3.13 Сходимость Q -обучения

Теорема 4. Пусть в конечном MDP начальная функция

$$Q_0^*(s, a)$$

задана произвольно.

Рассмотрим алгоритм Q -обучения вида:

$$Q_{k+1}^*(s, a) \leftarrow Q_k^*(s, a) + \alpha_k(s, a) \left(r(s, a) + \gamma \max_{a' \in \mathcal{A}} \{Q_k^*(s'_{k,s,a}, a')\} - Q_k^*(s, a) \right),$$

где

$$s'_{k,s,a} \sim p(s' \mid s, a).$$

Тогда, если для всех пар $(s, a) \in \mathcal{S} \times \mathcal{A}$ существует $k \in \mathbb{Z}_+$ такое, что

$$s'_{k,s,a} \sim p(s' \mid s, a),$$

и

$$\alpha_k(s, a) \in (0, 1]$$

удовлетворяет условиям Роббинса–Монро, то

$$Q_k^*(s, a) \xrightarrow[k \rightarrow \infty]{w.p. 1} Q^*(s, a).$$

Замечание 11. Эта теорема точно справедлива, если на каждой итерации обновлять Q -функцию для всех состояний и действий. Но на практике это обычно не выполняется: мы обновляем Q -функцию только в тех состояниях, которые видим.

Однако оказывается, что если каждое состояние можно посетить с положительной вероятностью, то теорема будет работать. Это приводит к проблеме баланса между исследованием и эксплуатацией:

exploration/exploitation trade-off.

3.14 Exploration/Exploitation trade-off

Проблема. Жадная по Q -функции политика с большой вероятностью не посетит все возможные состояния среды.

Вероятность принять жадное действие равна 1, а вероятности остальных действий равны 0. Поэтому сходимость становится весьма условной.

Решение. Если

$$\forall (s, a) \in \mathcal{S} \times \mathcal{A} : \quad \pi(a | s) > 0,$$

то гарантированно рано или поздно агент посетит все состояния.

Этого можно достичь, например, тривиальной равномерной политикой. Тогда теорема о сходимости будет работать.

Итого. Свойство

$$\forall (s, a) \in \mathcal{S} \times \mathcal{A} : \quad \pi(a | s) > 0$$

отвечает за исследование среды.

Жадность политики отвечает за решение задачи.

Нужно найти баланс между этими двумя требованиями.

3.15 ϵ -жадное исследование среды

3.15.1 Идея метода

На каждом шаге агента:

- с вероятностью $1 - \epsilon$ выбирается жадное действие;
- с вероятностью ϵ выбирается совершенно случайное действие.

Для такой политики верно:

$$\forall (s, a) \in \mathcal{S} \times \mathcal{A} : \quad \pi(a | s) > 0.$$

Поэтому будет наблюдаться сходимость.

3.15.2 Достоинство

Метод очень прост в реализации и применяется в RL повсеместно.

3.15.3 Недостаток

Метод часто работает плохо, поскольку идея достаточно наивная.

3.16 Схема алгоритма Q-learning

1. Инициализируем $Q^*(s, a)$ произвольно.

2. Наблюдаем начальное состояние:

$$s_0.$$

3. В цикле по

$$k = 0, 1, 2, \dots$$

выполняем:

(a) Выбираем действие:

$$a_k \sim \varepsilon\text{-greedy}(Q^*(s_k, a)).$$

(b) Наблюдаем:

$$r_k, s_{k+1}.$$

(c) Обновляем Q -функцию. Сначала считаем target:

$$y = r_k + \gamma \max_{a' \in \mathcal{A}} \{Q^*(s_{k+1}, a')\}.$$

Затем обновляем:

$$Q^*(s_k, a_k) \leftarrow (1 - \alpha_k)Q^*(s_k, a_k) + \alpha_k y.$$

Замечание 12. Такой алгоритм гарантированно сходится при выполнении условий посещаемости и условий Роббинса–Монро на шаг обучения.

3.17 Буфер в алгоритме Q-обучения

Q-обучение, в отличие от алгоритма Monte Carlo, является **off-policy** алгоритмом. Это будет подробнее обсуждаться позже.

Off-policy означает, в частности, что алгоритм может обучаться не только на своих действиях в данный момент времени, но и:

- на действиях экспертов;
- на своих старых действиях.

Поэтому удобно добавить в алгоритм буфер. В этот буфер кладутся действия агента и их результаты. Потом во время дальнейшего обучения алгоритм случайно доучивается на сэмплах из буфера.

Идея буфера:

$$(s_k, a_k, r_k, s_{k+1}) \longrightarrow \mathcal{D}.$$

Далее обучение можно проводить по случайным сэмплам:

$$(s, a, r, s') \sim \mathcal{D}.$$

Для каждого такого сэмпла можно делать Q-learning update:

$$Q^*(s, a) \leftarrow (1 - \alpha_k)Q^*(s, a) + \alpha_k \left(r + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a') \right).$$

4 Лекция 4. Deep Q-learning и улучшения DQN

4.1 Что делали ранее

В предыдущей лекции начали работать в условиях отсутствия полной информации о среде и перешли к **model-free** методам.

При этом на среду сохранялись условия конечности:

$$|\mathcal{S}| \ll \infty, \quad |\mathcal{A}| \ll \infty.$$

Было получено, что оценивать Q-функцию можно несколькими способами:

- Monte Carlo;
- Temporal Difference;
- Q-learning.

Для сходимости этих алгоритмов необходимо, чтобы была возможность полного исследования среды:

$$\forall (s, a) \in \mathcal{S} \times \mathcal{A} : \quad \pi(a \mid s) > 0.$$

Решением проблемы исследования может служить ε -greedy алгоритм.

Общая форма обновления Q-функции:

$$Q_{k+1}^\pi(s, a) \leftarrow Q_k^\pi(s, a) + \alpha_k (y(s, a) - Q_k^\pi(s, a)).$$

Для **Temporal Difference**:

$$y(s, a) := r + \gamma Q_k^\pi(s', a').$$

Свойства Temporal Difference:

- обновление происходит после каждого шага;
- награда распространяется медленно, поскольку алгоритм посещает по одной паре состояний;
- маленькая дисперсия;

- смещённая оценка.

Для **Monte Carlo**:

$$y(s, a) := r + \gamma r' + \gamma^2 r'' + \dots$$

Свойства Monte Carlo:

- обновление происходит в конце эпизода;
- награда распространяется быстрее, поскольку алгоритм посещает несколько состояний за один эпизод;
- большая дисперсия;
- несмещённая оценка.

4.2 Q-learning

Идея Q-learning такая же, как в Value Iteration: улучшаем оценку Q-функции с помощью TD и одновременно улучшаем политику.

Начинаем с TD-обновления:

$$Q_{k+1}^\pi(s, a) \leftarrow Q_k^\pi(s, a) + \alpha_k (r + \gamma Q_k^\pi(s', a') - Q_k^\pi(s, a)).$$

Если следующее действие выбирается текущей политикой:

$$a' = \pi_k(s'),$$

то:

$$Q_{k+1}^\pi(s, a) \leftarrow Q_k^\pi(s, a) + \alpha_k (r + \gamma Q_k^\pi(s', \pi_k(s')) - Q_k^\pi(s, a)).$$

Если политика выбирает жадное действие:

$$\pi_k(s') = \arg \max_{a' \in \mathcal{A}} \{Q_k^\pi(s', a')\},$$

то:

$$\begin{aligned} Q_{k+1}^\pi(s, a) &\leftarrow Q_k^\pi(s, a) + \alpha_k \left(r + \gamma Q_k^\pi \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_k^\pi(s', a')\} \right) - Q_k^\pi(s, a) \right) \\ &= Q_k^\pi(s, a) + \alpha_k \left(r + \gamma \max_{a' \in \mathcal{A}} \{Q_k^\pi(s', a')\} - Q_k^\pi(s, a) \right). \end{aligned}$$

4.3 Состояний теперь много

Теперь удалим ограничение на множество состояний среды:

$$|\mathcal{S}| \ll \infty.$$

То есть теперь состояний может быть очень много или они могут быть фактически непрерывными.

Примером таких задач служат игры Atari, которые долгое время использовались как benchmark для RL-алгоритмов.

4.4 Препроцессинг состояний на практике

В задачах Atari и похожих средах обычно используют различные техники препроцессинга.

4.4.1 Action selection frequency

- **Framestack** — несколько последних кадров объединяются в одно состояние.
- **Frameskip** — агент выбирает действие не на каждом кадре, а с некоторым пропуском кадров.
- **MaxAndSkip** — берётся максимум по нескольким последним кадрам, что помогает бороться с мерцанием.
- **Sticky actions** — выбранное действие может с некоторой вероятностью повторяться, что добавляет стохастичность.

4.4.2 Atari-specific preprocessing

- **EpisodicLife** — потеря жизни трактуется как конец эпизода для обучения, хотя сама игра может продолжаться.
- **FireReset** — в играх, где для старта нужно нажать Fire, это действие выполняется при reset.

4.4.3 Standard tricks

- Crop Image — обрезка изображения;
- Rescale:
$$84 \times 84;$$
- Grayscale — перевод изображения в оттенки серого.

4.4.4 Reward preprocessing

Часто награду клипуют:

$$r \in \{-1, 0, 1\}.$$

То есть делают:

$$r \leftarrow \text{clip}(r, -1, 1).$$

4.5 Проблема большого множества состояний

Так как теперь множество состояний может быть очень большим или непрерывным, обычный табличный Q-learning применять нельзя.

Причина: Q-функция больше не может быть описана таблицей:

$$Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Если $\mathcal{S}$ огромно, хранить отдельное значение $Q(s, a)$ для каждой пары (s, a) невозможно.

Решение. Использовать **Deep Q-learning**: искать оптимальную $Q^*(s, a)$ в виде нейронной сети.

$$Q^*(s, a) \approx Q(s, a; \theta),$$

где θ — параметры нейронной сети.

4.6 Возможные реализации DQN

Есть два естественных варианта реализации Q-сети.

4.6.1 Первый вариант

На вход сети подаётся пара (s, a) , а на выходе получается одно число:

$$Q\text{-Net}(s, a) := Q(s, a) \in \mathbb{R}.$$

4.6.2 Второй вариант

На вход сети подаётся только состояние s , а на выходе получается вектор значений для всех действий:

$$Q\text{-Net}(s) := (Q(s, a_1), \dots, Q(s, a_A)) \in \mathbb{R}^A.$$

Что лучше использовать? Если

$$|\mathcal{A}| \ll \infty,$$

то удобнее использовать второй вариант, поскольку так удобнее брать:

$$\arg \max_{a \in \mathcal{A}} Q(s, a).$$

В этом случае один forward pass сети сразу даёт значения всех действий.

4.7 Архитектура DQN

В классическом DQN для Atari на вход подаётся буфер из нескольких кадров. Например, можно использовать:

4-frame buffer

размера

$$[\text{None}, \text{frame}, h, w].$$

Далее применяются свёрточные слои и полносвязные слои, после чего сеть выдаёт Q -значения для действий.

Типичная схема:

$$\text{frames} \longrightarrow \text{conv layers} \longrightarrow \text{dense layers} \longrightarrow Q\text{-values}.$$

Не стоит использовать:

- **max pooling**, поскольку он ведёт к инвариантности сети относительно сдвигов объектов, а в играх положение объекта часто важно;
- **batch norm**, по причинам, близким к проблемам с dropout;
- **dropout**, поскольку train- и test-стадии для алгоритма фактически неотличимы.

4.8 Обучение DQN как задача регрессии

Пусть s, a — входные данные, а

$$f \in \mathbb{R}$$

— искомое значение, которое в идеале хочется брать из уравнения Беллмана:

$$f(s, a) := r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta_k)\} \right],$$

где θ_k — параметры сети на текущем шаге.

Проблема. Как и в табличном Q-learning, математическое ожидание по s' берётся из неизвестного в общем случае распределения:

$$s' \sim p(s' \mid s, a).$$

Нужно обойти эту операцию. Для этого построим оптимизационную задачу, вычислим градиент и поймём, где можно выполнить приближение.

4.8.1 MSE loss

Функция потерь — MSE:

$$\frac{1}{2} (f - y_{\text{pred}})^2.$$

В нашем случае:

$$y_{\text{pred}} = Q^*(s, a; \theta_{k+1}).$$

Итого задача оптимизации:

$$\frac{1}{2} \mathbb{E}_{(s,a,f)} [(f - Q^*(s, a; \theta_{k+1}))^2] \rightarrow \min_{\theta_{k+1}}.$$

Данная задача на самом деле и является Q-обучением.

4.9 Вычисление градиента

Рассмотрим градиент MSE:

$$\nabla_{\theta} \frac{1}{2} (f - Q^*(s, a; \theta))^2.$$

С точностью до направления спуска можно записать:

$$\nabla_{\theta} \frac{1}{2} (f - Q^*(s, a; \theta))^2 = (f - Q^*(s, a; \theta)) \nabla_{\theta} Q^*(s, a; \theta).$$

Подставляя выражение для f , получаем:

$$\begin{aligned} & \left(r + \gamma \mathbb{E}_{s'} \left[\max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta_k)\} \right] - Q^*(s, a; \theta) \right) \nabla_{\theta} Q^*(s, a; \theta) \\ & \approx \left(r + \gamma \max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta_k)\} - Q^*(s, a; \theta) \right) \nabla_{\theta} Q^*(s, a; \theta), \end{aligned}$$

где

$$s' \sim p(s' \mid s, a).$$

То есть математическое ожидание по s' приближается одним сэмплом.

Первая скобка полученного выражения совпадает с TD-ошибкой из алгоритма Q-learning:

$$r + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a'; \theta_k) - Q^*(s, a; \theta).$$

4.10 Итоговая задача регрессии

Если s, a — входные данные, то наблюдаемое целевое значение $y \in \mathbb{R}$ получается как несмещённая оценка f :

$$y(s, a) := r(s, a) + \gamma \max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta_k)\},$$

где

$$s' \sim p(s' \mid s, a).$$

Вывод. С помощью приближения уравнения Беллмана сэмплированием получаем задачу, которая является уже изученным Q-learning, только вместо таблицы приближаем искомое Q^* нейронной сетью.

4.11 Почему обучение сходится к целевому значению

Итого мы решаем задачу с целевыми значениями y , такими что:

$$\mathbb{E}_{s'}[y] = f(s, a).$$

Теорема 5. Пусть $Q_{\theta_{k+1}}(s, a)$ — достаточно ёмкая модель, выборка неограниченно большая, а оптимизатор идеальный.

Тогда решением задачи регрессии выше будет:

$$Q_{\theta_{k+1}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a' \in \mathcal{A}} \{Q_{\theta_k}(s', a')\} \right].$$

Доказательство. Найдём оптимальное значение $Q_{\theta_{k+1}}(s, a)$ для позиции (s, a) , которое минимизирует MSE:

$$\frac{1}{2} \mathbb{E}_{s'} \left[(y(s, a) - Q_{\theta_{k+1}}(s, a))^2 \right].$$

Вспомним, как брали градиент и строили его несмещённую оценку. Приравниваем несмещённую оценку градиента, а значит в среднем и сам градиент, к нулю.

Отсюда:

$$Q_{\theta_{k+1}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a' \in \mathcal{A}} \{Q_{\theta_k}(s', a')\} \right].$$

4.12 Особенность регрессии в DQN

Построенная задача регрессии является нетипичной для классического машинного обучения: целевые значения зависят от аппроксимирующей функции с фиксированными весами:

$$y(s, a) := r(s, a) + \gamma \max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta_k)\}.$$

Важно: градиенты в целевые значения не текут.

То есть при оптимизации по θ значение

$$y(s, a)$$

считается константой.

4.13 Схема обучения: один шаг простой итерации

Один шаг простой итерации выглядит так:

1. Фиксируются веса θ_k , вычисляются целевые значения y_k .
2. Модель $Q^*(s, a; \theta)$ обучается на эти целевые значения.
3. Меняются веса:

$$\theta_k \rightarrow \theta_{k+1},$$

после чего вычисляются новые целевые значения y_{k+1} .

Проблема. Переход со второго пункта на третий обманчиво прост.

Возникают вопросы:

- как долго обучать сеть с фиксированными таргетами;
- как понять, что пора обновить веса сети и пересчитать целевые значения.

4.14 Возможные варианты обновления целевых значений

4.14.1 Вариант 1: SGD

Можно пересчитывать целевые значения после каждого градиентного шага обучения.

Недостаток. Подход нестабилен. Градиентный шаг может немного «поломать» модель, ведь SGD улучшает модель лишь в среднем, но не гарантирует улучшение на каждом конкретном шаге.

4.14.2 Вариант 2: обучение до сходимости

Можно обучать сеть до тех пор, пока веса модели не перестанут значимо меняться.

Недостаток. Ждать сходимости слишком долго.

4.14.3 Вариант 3: что-то между

На практике предлагается выполнять условно 100–1000 итераций градиентного обучения сети, прежде чем менять целевые значения.

4.15 Декорреляция сэмплов

Обучать DQN будем по батчам. Однако есть важный нюанс: при последовательной игре четвёрки

$$(s, a, r, s')$$

часто сильно коррелированы, что ломает обучение сети.

Пути решения.

1. Запуск параллельных агентов.
2. Буфер реплеев.

4.16 Target Network

Предложенный алгоритм фиксации целевых значений и последующего градиентного спуска реализован с помощью **Target Network**.

Target Network генерирует целевые значения, на которых несколько итераций обучается Deep Q-Network. После этого все веса DQN копируются в Target Network, и процесс повторяется.

Веса основной сети:

$$\theta.$$

Веса Target Network часто обозначают:

$$\theta^-.$$

Тогда целевое значение можно записать как:

$$y = r + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a'; \theta^-).$$

Замечание 13. Полученный результат всё так же является *model-free off-policy* алгоритмом, в котором всё так же необходим *exploration*.

4.17 Финальная схема Deep Q-learning

Инициализируем сеть:

$$Q^*(s, a; \theta)$$

произвольно.

Инициализируем Target Network:

$$\theta^- := \theta.$$

Инициализируем буфер:

$$\mathcal{D} = \emptyset.$$

Получаем начальное состояние:

$$s_0.$$

Далее для

$$k = 0, 1, 2, \dots$$

выполняем:

1. Выбрать действие:

$$a_k \sim \varepsilon\text{-greedy}(Q^*(s_k, a; \theta)).$$

2. Пронаблюдать:

$$r_k, \quad s_{k+1}, \quad done_{k+1}.$$

Сохранить переход в буфер:

$$(s_k, a_k, r_k, s_{k+1}, done_{k+1}) \in \mathcal{D}.$$

Здесь:

$$done_{k+1} := \mathbf{1}\{s_{k+1} \text{ терминальное}\}.$$

3. Засэмплировать батч:

$$T = \{(s_{i_j}, a_{i_j}, r_{i_j}, s_{i_j+1}, done_{i_j+1})\}_{j=1}^B$$

из буфера $\mathcal{D}$.

4. Для каждого элемента батча посчитать target:

$$y_j := r_{i_j} + \gamma(1 - done_{i_j+1}) \max_{a' \in \mathcal{A}} \{Q^*(s_{i_j+1}, a'; \theta^-)\}.$$

5. Совершить шаг градиентного спуска:

$$\theta \leftarrow \theta - \frac{\alpha_k}{2B} \sum_{j=1}^B \nabla_{\theta} (Q^*(s_{i_j}, a_{i_j}; \theta) - y_j)^2.$$

Шаг α_k выбирается согласно условиям Роббинса–Монро.

6. Обновить Target Network, если:

$$k \bmod K = 0.$$

Тогда:

$$\theta^- \leftarrow \theta.$$

4.18 Зачем модифицировать DQN

Проблема. Выученная функция склонна к переоцениванию будущей награды, то есть Q -функция может начать неограниченно расти.

Эта проблема называется:

overestimation bias.

Почему это происходит? Причина — оператор максимума в формуле построения целевых значений:

$$y(T) = r + \gamma \max_{a' \in \mathcal{A}} \{Q_{\theta^-}(s', a')\}.$$

4.19 Природа overestimation

При обучении сети ошибка появляется из-за неточности аппроксимации и случайной шумовой ошибки.

Но функции $\max$ это безразлично: она всегда выбирает лучшее значение. Поэтому случайно или аппроксимационно завышенные оценки будут накапливаться.

4.19.1 Формальное утверждение

Пусть s' фиксировано.

Пусть

$$Q^*(s', a')$$

— истинная Q -функция, а

$$Q(s', a') \approx Q^*(s', a')$$

— её приближение.

Пусть:

$$Q(s', a') = Q^*(s', a') + \varepsilon(a'),$$

где

$$\mathbb{P}(\varepsilon(a') > 0) = \mathbb{P}(\varepsilon(a') < 0) = 0.5.$$

Пусть также шум не зависит от выбора a' в вероятностном смысле.

Тогда:

$$\mathbb{P}\left(\max_{a' \in \mathcal{A}} \{Q(s', a')\} > \max_{a' \in \mathcal{A}} \{Q^*(s', a')\}\right) > 0.5.$$

Это означает, что максимум по шумным оценкам систематически склонен выбирать завышенное значение.

4.20 Action decoupling

Идея **action decoupling** — отделить выбор действия от оценки действия.

Обычный максимум можно переписать так:

$$\max_{a' \in \mathcal{A}} \{Q^*(s', a')\} = Q^*\left(s', \arg \max_{a' \in \mathcal{A}} \{Q^*(s', a')\}\right).$$

Здесь:

$$\arg \max_{a' \in \mathcal{A}} \{Q^*(s', a')\}$$

— выбор действия, или action selection.

А внешнее вычисление Q^* для выбранного действия — оценка действия, или action evaluation.

В Double Q-learning эти две операции стараются выполнять с помощью разных сетей или разных наборов параметров.

4.21 Возможные улучшения: Double и Twin Q-learning

4.21.1 Double* Q-learning

Используются две сети:

$$Q_1, \quad Q_2.$$

Целевые значения:

$$\begin{aligned} y_1 &= r + \gamma Q_2 \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_1(s', a')\} \right), \\ y_2 &= r + \gamma Q_1 \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_2(s', a')\} \right). \end{aligned}$$

4.21.2 Double Q-learning

Используются:

$$Q, \quad Q^-,$$

где Q^- — Target Network.

Целевое значение:

$$y = r + \gamma Q^- \left(s', \arg \max_{a' \in \mathcal{A}} \{Q(s', a')\} \right).$$

Здесь основная сеть Q выбирает действие, а Target Network Q^- оценивает выбранное действие.

4.21.3 Twin Q-learning

Используются две сети:

$$Q_1, \quad Q_2.$$

Целевое значение для первой сети:

$$y_1 = r + \gamma \min_{i=1,2} \left\{ Q_i \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_1(s', a')\} \right) \right\}.$$

Целевое значение для второй сети:

$$y_2 = r + \gamma \min_{i=1,2} \left\{ Q_i \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_2(s', a')\} \right) \right\}.$$

Идея Twin Q-learning: брать более осторожную оценку через минимум двух критиков.

4.22 Prioritized Experience Replay

Проблема. В буфере реплеев чаще всего оказываются тривиальные случаи, когда агент сделал шаг и не получил никакой награды.

Это плохо влияет на обучение. С другой стороны, случаи с наградой довольно редки, но именно они на начальных этапах дают агенту подсказки, где искать улучшение.

Решение. Сэмплировать переходы из буфера не равномерно, а приоритезированно.

Для перехода T вводятся веса сэмплирования:

$$P(T) \propto |y(T) - Q^*(s, a; \theta)|^\alpha.$$

Величина

$$|y(T) - Q^*(s, a; \theta)|$$

является модулем TD-ошибки. Чем больше ошибка, тем важнее переход для обучения.

4.23 Смещение в Prioritized Experience Replay

Недостаток. После неконтролируемой замены равномерного сэмплирования на другое может случиться так, что:

$$s' \not\sim p(s' | s, a).$$

Иными словами, приоритезированное сэмплирование приводит к смещению.

Утешение. Этот эффект не так страшен в начале обучения, когда распределение, из которого приходят состояния, всё равно скорее всего не слишком разнообразно.

Однако по ходу обучения этот эффект важно нивелировать. В противном случае процесс обучения может полностью дестабилизироваться или где-нибудь застрять.

4.24 Importance weights для Prioritized Experience Replay

Проблема:

много тривиальных обновлений.

Цель:

распространить подкрепление из будущего в прошлое.

Сэмплирование с приоритетами из буфера:

$$P(T) \propto |y(T) - Q(s, a; \theta)|^\alpha.$$

Но теперь переходы сэмплируются не из исходного равномерного распределения. Поэтому возникает смещение.

Идея коррекции:

$$\mathbb{E}_{T \sim \text{Uniform}} [\text{Loss}(T)] \approx \mathbb{E}_{T \sim P(T)} \left[\left(\frac{1}{P(T)} \right)^{\beta(t)} \text{Loss}(T) \right].$$

Множитель

$$\left(\frac{1}{P(T)} \right)^{\beta(t)}$$

является весом importance sampling.

Для $\beta(t)$ выполняется отжиг:

$$\beta(t) : 0 \rightarrow 1.$$

При

$$\beta(t) = 0$$

оценка смещённая.

При

$$\beta(t) = 1$$

оценка становится несмещённой.

4.25 Техническая деталь: структура SumTree

Для эффективного приоритизированного сэмплирования используется структура данных **SumTree**.

Использование SumTree.

- листья — значения приоритета сэмплирования;
- каждый внутренний узел — сумма двух детей.

Если приоритеты переходов:

$$p_1, p_2, \dots, p_N,$$

то корень дерева хранит сумму:

$$\sum_{i=1}^N p_i.$$

Буферизация и сэмплирование.

1. Сэмплировать из $P(T)$ с помощью равномерного распределения на отрезке:

$$\left[0, \sum_T P(T) \right].$$

2. По выбранному числу пройти по дереву и найти соответствующий лист.
3. Выполнить обновление параметров сети.
4. Пересчитать приоритеты только для текущего батча.
5. Обновить узлы SumTree.

Сложность операций:

$$O(\log N),$$

где N — размер буфера.

4.26 Noisy Networks

Проблема. ε -жадная стратегия наивна:

- появляется неудобный гиперпараметр ε ;
- exploration происходит вне зависимости от состояния.

Решение. Добавить шум на веса сети:

$$w := \mu + \sigma\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

Плюсы.

- нет отдельного гиперпараметра ε ;
- exploration зависит от состояния.

Минусы.

- такая зависимость от состояния плохо интерпретируема;
- вычисления становятся дороже.

4.27 Distributional RL

В стандартном DQN сеть приближает математическое ожидание будущей награды:

$$Q^\pi(s, a) = \mathbb{E}[R_t \mid s_t = s, a_t = a].$$

Идея **Distributional RL**: вместо одного среднего значения приближать распределение будущей случайной награды:

$$Z^\pi(s, a) \sim R_t.$$

Тогда:

$$Q^\pi(s, a) = \mathbb{E}[Z^\pi(s, a)].$$

На слайде перечислены варианты развития этой идеи:

- DQN — предсказывает одно среднее значение;
- C51 — аппроксимирует распределение через категориальное распределение на фиксированной сетке;
- QR-DQN — аппроксимирует квантили распределения;
- IQN — аппроксимирует квантильную функцию.

Подробно Distributional RL разбирается в следующей лекции.

4.28 Dueling DQN

Проблема. Оценка пары состояние–действие требует знания о всех действиях.

Идея Dueling DQN — разложить Q -функцию на ценность состояния и преимущество действия:

$$Q^*(s, a) := V^*(s) + A^*(s, a).$$

Здесь:

- $V^*(s)$ показывает, насколько хорошим является состояние s само по себе;
- $A^*(s, a)$ показывает преимущество действия a в состоянии s .

Внимание. Такое разложение не единственно: добавляется лишняя степень свободы. Например, можно прибавить константу к V и вычесть её из A , не изменив сумму.

Чтобы убрать неоднозначность, используют нормировку:

$$Q_\theta(s, a) := V_\theta(s) + A_\theta(s, a) - \text{mean}_{a \in \mathcal{A}} \{A_\theta(s, a)\}.$$

То есть:

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A_\theta(s, a').$$

На практике сеть разделяется на две головы:

- value-head для $V_\theta(s)$;
- advantage-head для $A_\theta(s, a)$.

Затем эти две оценки объединяются в $Q_\theta(s, a)$.

4.29 Partially Observable MDP

Проблема. Иногда у агента нет доступа к истинному состоянию среды s_t . Вместо этого он получает только наблюдения:

$$o_t \sim p(\cdot \mid s_t).$$

Теоретически такая постановка описывается через **PoMDP**, то есть частично наблюдаемые MDP.

Практика. На практике часто используют рекуррентные сети, например LSTM.

Идея:

$$o_t, o_{t+1}, \dots \longrightarrow \text{LSTM} \longrightarrow h_t.$$

Скрытое состояние LSTM:

$$h_t$$

аккумулирует информацию из истории наблюдений и частично заменяет неизвестное истинное состояние s_t .

В обучении таких моделей используют:

- hidden state burn-in;
- truncated BPTT.

4.30 Multi-step DQN

Проблема. Есть отложенные награды и ошибки вычисления.

Вместо одношагового target:

$$y = r + \gamma \max_{a'} Q(s', a')$$

можно использовать многошаговый target:

$$y := r + \gamma r' + \gamma^2 r'' + \dots + \gamma^N \max_{a^{(N)} \in \mathcal{A}} Q(s^{(N)}, a^{(N)}).$$

Более формально:

$$y := \sum_{n=0}^{N-1} \gamma^n r^{(n)} + \gamma^N \max_{a^{(N)} \in \mathcal{A}} Q(s^{(N)}, a^{(N)}).$$

Внимание.

- Такой подход может значительно помочь.
- В теории он не предназначен для off-policy случая.

Практика. В буфере хранят кортежи вида:

$$\left(s, a, \sum_{n=0}^{N-1} \gamma^n r^{(n)}, s^{(N)}, \bigvee_{n=1}^N done_n \right).$$

Здесь:

$$\bigvee_{n=1}^N done_n$$

означает, что хотя бы на одном из N шагов эпизод завершился.

4.31 Передний край науки и индустрии

4.31.1 Rainbow DQN, 2017

Rainbow DQN объединяет несколько улучшений DQN:

- Double DQN;
- буфер с приоритетами;
- Multi-step DQN;
- Noisy Nets;
- Dueling DQN;
- Distributional RL.

4.31.2 R2D2, 2018

R2D2 включает:

- Double DQN;
- буфер с приоритетами;
- Multi-step DQN;
- LSTM;
- массовую параллелизацию.

4.31.3 Agent57, 2020

Agent57 включает:

- Double DQN;
- буфер с приоритетами;
- Multi-step DQN;
- LSTM;
- массовую параллелизацию;
- Retrace;
- внутреннюю мотивацию;
- управление гиперпараметрами.

4.32 Источники

- *Playing Atari with Deep Reinforcement Learning*, 2013.
- Ссылки на модификации DQN перечислены на предыдущем слайде.

5 Лекция 5. Distributional RL, QR-DQN и IQN

5.1 Что делали ранее

В предыдущей лекции была рассмотрена новая постановка задачи: удалили ограничение на множество состояний среды

$$|\mathcal{S}| \ll \infty.$$

Вместо табличного хранения Q -функции стали использовать нейронную сеть.

Если s, a — входные данные, а $y \in \mathbb{R}$ — наблюдаемые значения, которые получают как несмещённая оценка решения уравнения Беллмана, то решается задача регрессии с целевым значением:

$$y(s, a) := r(s, a) + \gamma \max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta_k)\},$$

где

$$s' \sim p(s' | s, a).$$

5.2 Deep Q-learning

Алгоритм Deep Q-learning:

1. Инициализируем Q -сеть:

$$Q^*(s, a; \theta)$$

произвольно.

2. Инициализируем Target Network:

$$\theta^- := \theta.$$

3. Инициализируем replay buffer:

$$\mathcal{D} = \emptyset.$$

4. Получаем начальное состояние:

$$s_0.$$

5. Для

$$k = 0, 1, 2, \dots$$

повторяем:

(a) Выбрать действие:

$$a_k \sim \varepsilon\text{-greedy}(Q^*(s_k, a; \theta)).$$

(b) Пронаблюдать:

$$r_k, \quad s_{k+1}, \quad done_{k+1}.$$

Сохранить переход:

$$(s_k, a_k, r_k, s_{k+1}, done_{k+1})$$

в буфер $\mathcal{D}$.

(c) Засэмплировать батч переходов:

$$T := (s, a, r, s', done)$$

из $\mathcal{D}$.

(d) Посчитать target:

$$y(T) := r + \gamma(1 - done) \max_{a' \in \mathcal{A}} \{Q^*(s', a'; \theta^-)\}.$$

(e) Совершить шаг градиентного спуска:

$$\theta \leftarrow \theta - \frac{\alpha_k}{B} \sum_T \nabla_{\theta} (Q^*(s, a; \theta) - y(T))^2.$$

(f) Обновить Target Network, если:

$$k \bmod K = 0.$$

Тогда:

$$\theta^- \leftarrow \theta.$$

5.3 Проблема vanilla DQN

У vanilla DQN есть проблема: выученная функция склонна к переоцениванию будущей награды.

То есть Q -функция может начать неограниченно расти. Эта проблема называется:

overestimation bias.

Для борьбы с этим использовались модификации, отделяющие выбор действия от оценки действия.

5.4 Double и Twin Q-learning

5.4.1 Double* Q-learning

Используются две сети:

$$Q_1, \quad Q_2.$$

Целевые значения:

$$\begin{aligned} y_1 &= r + \gamma Q_2 \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_1(s', a')\} \right), \\ y_2 &= r + \gamma Q_1 \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_2(s', a')\} \right). \end{aligned}$$

5.4.2 Double Q-learning

Используются:

$$Q, \quad Q^-,$$

где Q^- — Target Network.

Целевое значение:

$$y = r + \gamma Q^- \left(s', \arg \max_{a' \in \mathcal{A}} \{Q(s', a')\} \right).$$

5.4.3 Twin Q-learning

Используются две сети:

$$Q_1, \quad Q_2.$$

Целевые значения:

$$\begin{aligned} y_1 &= r + \gamma \min_{i=1,2} \left\{ Q_i \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_1(s', a')\} \right) \right\}, \\ y_2 &= r + \gamma \min_{i=1,2} \left\{ Q_i \left(s', \arg \max_{a' \in \mathcal{A}} \{Q_2(s', a')\} \right) \right\}. \end{aligned}$$

5.5 Мотивация Distributional RL

Для фиксированной политики π Q -функция определяется как:

$$Q^\pi(s, a) = \mathbb{E}_\tau \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right].$$

Это можно записать через случайную величину полной дисконтированной награды:

$$Q^\pi(s, a) = \mathbb{E}_\tau [Z^\pi(s, a)],$$

где

$$Z^\pi(s, a) := \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a.$$

Траектория распределена как:

$$\tau \sim p(s_0) \prod_{t=0}^{\infty} (\pi(a_t | s_t) p(s_{t+1} | s_t, a_t)).$$

До этого момента распределение Z^π нас почти не интересовало: мы искали сразу его математическое ожидание.

5.6 Пример распределения Z^π

Рассмотрим пример.

Пусть

$$\gamma = 0.5.$$

Мы находимся в состоянии s_0 и совершаем действие a_0 .

После этого:

- с вероятностью 0.6 попадаем в состояние s_1 и получаем награду 1;
- с вероятностью 0.4 попадаем в состояние s_2 и получаем награду 0.

Далее:

- из состояния s_1 есть два действия:

$$a_{11}, \quad a_{12};$$

действие a_{11} даёт награду 2, а действие a_{12} даёт награду 0; оба действия ведут в терминальное состояние;

- из состояния s_2 есть два действия:

$$a_{21}, \quad a_{22};$$

действие a_{21} даёт награду 1, а действие a_{22} даёт награду 0.

Политика:

$$\begin{aligned} \pi(a_{11} | s_1) &= \pi(a_{12} | s_1) = 0.5, \\ \pi(a_{21} | s_2) &= 0.25, \quad \pi(a_{22} | s_2) = 0.75. \end{aligned}$$

Посчитаем распределение:

$$Z^\pi(s_0, a_0).$$

Случайная величина $Z^\pi(s_0, a_0)$ дискретна. Возможные исходы:

s'	a'	Вероятность	reward-to-go
s_1	a_{11}	$0.6 \cdot 0.5 = 0.3$	$1 + 2\gamma$
s_1	a_{12}	$0.6 \cdot 0.5 = 0.3$	1
s_2	a_{21}	$0.4 \cdot 0.25 = 0.1$	γ
s_2	a_{22}	$0.4 \cdot 0.75 = 0.3$	0

Так как $\gamma = 0.5$, значения reward-to-go равны:

$$1 + 2\gamma = 2, \quad 1, \quad \gamma = 0.5, \quad 0.$$

То есть:

$$Z^\pi(s_0, a_0) = \begin{cases} 2, & \text{с вероятностью } 0.3, \\ 1, & \text{с вероятностью } 0.3, \\ 0.5, & \text{с вероятностью } 0.1, \\ 0, & \text{с вероятностью } 0.3. \end{cases}$$

5.7 Почему иногда важно искать именно распределение

Поиск только математического ожидания Z^π не всегда даёт полную картину.

Например, разные распределения могут иметь одинаковое или близкое математическое ожидание, но радикально отличаться по риску, дисперсии, вероятности больших выигрышей или больших проигрышей.

В частности, если распределение двухмодальное и симметрично относительно нуля, то среднее может быть около нуля, хотя сама случайная величина почти никогда не принимает значения около нуля.

Поэтому полезно искать не только:

$$Q^\pi(s, a) = \mathbb{E}[Z^\pi(s, a)],$$

но и само распределение:

$$Z^\pi(s, a).$$

5.8 Quantile Regression DQN

5.8.1 Идея QR-DQN

Идея: искать распределение Z^π , то есть распределение наград, и таким образом понимать о задаче больше.

Нейронная сеть будет аппроксимировать функцию:

$$Z^\pi(s, a),$$

а оптимальное действие выбирается согласно математическому ожиданию этого распределения:

$$a^* = \arg \max_{a \in \mathcal{A}} \{ \mathbb{E}[Z^\pi(s, a)] \}.$$

Вопрос: каким образом заставить нейросеть возвращать распределение?

5.8.2 Аппроксимация распределения дельта-функциями

В QR-DQN фиксируется число атомов N . Сеть генерирует N дельта-функций.

Их позиции выбирает нейронная сеть, а каждая дельта-функция имеет массу:

$$\frac{1}{N}.$$

То есть распределение аппроксимируется как:

$$Z_\theta(s, a) = \frac{1}{N} \sum_{i=1}^N \delta_{z_i(s, a; \theta)}.$$

Здесь:

$$z_i(s, a; \theta)$$

— позиция i -го атома, предсказываемая сетью.

5.9 Distributional Bellman Operator

5.9.1 Distributional Bellman Equation

Для фиксированной политики π distributional Bellman equation:

$$Z^\pi(s, a) \stackrel{D}{=} r(s, a) + \gamma Z^\pi(S', A'),$$

где

$$S' \sim p(\cdot \mid s, a), \quad A' \sim \pi(\cdot \mid S').$$

Знак

$$\stackrel{D}{=}$$

означает равенство по распределению.

Аналогично тому, как Q -функцию можно интерпретировать как таблицу, $Z^\pi(s, a)$ тоже можно рассматривать как таблицу. В ячейке (s, a) этой таблицы находится случайная величина.

Выражение

$$Z^\pi(S', A')$$

означает, что мы обращаемся к случайной ячейке этой таблицы, причём индексы этой ячейки сами являются случайными величинами:

$$(S', A').$$

Distributional Bellman Operator — это правая часть distributional Bellman equation:

$$\mathcal{T}^\pi Z(s, a) := r(s, a) + \gamma Z(S', A').$$

5.9.2 Пояснение

Слева и справа в distributional Bellman equation записаны два процесса генерации одной и той же случайной величины.

Можно:

- напрямую сэмплировать случайную величину $Z^\pi(s, a)$;
- или сначала сэмплировать s' , потом a' , затем $Z^\pi(s', a')$, и выдать исход:

$$r(s, a) + \gamma Z^\pi(s', a').$$

Эти две процедуры порождения эквивалентны.

Замечание 14. Подобные уравнения называются *recursive distributional equations* и рассматриваются в одном из разделов теории вероятности.

5.10 Метрика для Z -функций

Пусть W — метрика в пространстве вероятностных распределений:

$$\mathcal{P}(\mathbb{R}).$$

Тогда её максимальной формой, или maximal form, называется метрика в пространстве Z -функций:

$$W_{\max}(Z_1, Z_2) := \sup_{s \in \mathcal{S}, a \in \mathcal{A}} \{W(Z_1(s, a), Z_2(s, a))\}.$$

Теорема 6 (без доказательства). *Distributional Bellman Operator является сжимающим оператором в пространстве с метрикой*

$$d(Z_1, Z_2) := \max_{s \in \mathcal{S}, a \in \mathcal{A}} \{W_p(Z_1(s, a), Z_2(s, a))\},$$

где W_p — расстояние Вассерштейна.

5.11 Замечания про метрики

Замечание 15. *Использовать KL -дивергенцию нельзя, потому что она не является метрикой. Кроме того, с KL -дивергенцией *Distributional Bellman Operator* не является сжимающим оператором.*

Замечание 16. *Если в vanilla DQN с replay buffer заменить Q на Z , а $MSELoss$ на*

$$\max_{s \in \mathcal{S}, a \in \mathcal{A}} \{W_p(Z_1(s, a), Z_2(s, a))\},$$

то алгоритм потеряет свойство сходимости.

Причина в том, что в таком случае не будет выполняться условие на сходимость SGD:

$$\mathbb{E}[\nabla \text{loss}] \neq \nabla \mathbb{E}[\text{loss}].$$

5.12 Wasserstein metric

5.12.1 Общий случай

Расстояние Вассерштейна между случайными величинами U и V :

$$W_p(U, V) = \left(\inf_{\mu \in \Gamma(U, V)} \{ \mathbb{E}_{(x, y) \sim \mu} [\|x - y\|_p^p] \} \right)^{1/p}.$$

Здесь

$$\Gamma(U, V)$$

— множество всех совместных распределений, маргиналы которых имеют распределения U и V .

5.12.2 Одномерный случай

Для одномерных случайных величин:

$$W_p(U, V) = \left(\int_0^1 |F_U^{-1}(w) - F_V^{-1}(w)|^p dw \right)^{1/p},$$

где

$$F_U^{-1}, \quad F_V^{-1}$$

— квантильные функции распределений U и V .

5.13 Свойства Wasserstein metric

- Масштабирование:

$$W_p(aU, aV) = |a| W_p(U, V), \quad a \in \mathbb{R}.$$

- Неравенство треугольника:

$$W_p(U, V) \leq W_p(U, X) + W_p(X, V),$$

где X — случайная величина.

- Инвариантность к сдвигу:

$$W_p(x + U, x + V) = W_p(U, V),$$

где x — постоянный вектор.

- Действие линейного оператора:

$$W_p(AU, AV) \leq \|A\|_p W_p(U, V),$$

где A — линейный оператор,

$$\|A\|_p = \sup_{y \neq 0} \left\{ \frac{\|Ay\|_p}{\|y\|_p} \right\}.$$

5.14 Интуиция Wasserstein metric: Earth Mover's Distance

Расстояние Вассерштейна также называется Earth Mover's Distance.

Аналогия следующая. Даны две кучи песка одинакового объёма, но с разными конфигурациями. Они насыпаны по-разному.

Чтобы перенести песчинку массы m на расстояние x , нужно затратить работу:

$$mx.$$

Расстояние Вассерштейна измеряет минимальное количество работы, которое нужно совершить, чтобы перевести конфигурацию первой кучи песка во вторую.

Для дискретных распределений, когда функции распределения и квантильные функции являются ступенчатыми, минимальная работа соответствует площади между функциями распределения.

5.15 Пример вычисления расстояния Вассерштейна

Рассмотрим два распределения.

Первое распределение — честная монетка с исходами 0 и 1:

$$\mathbb{P}(U = 0) = 0.5, \quad \mathbb{P}(U = 1) = 0.5.$$

Вторая случайная величина V принимает значения:

$$0.4 \quad \text{с вероятностью } \theta < 0.5,$$

и

$$0.8 \quad \text{с вероятностью } 1 - \theta.$$

Можно нарисовать функции распределения и посчитать площадь между ними. Также можно рассуждать через перенос песка.

Песок объёма θ в точке 0.4 удобнее перенести в точку 0, так как туда ближе. Работа:

$$0.4\theta.$$

Песок объёма $1 - \theta$ находится в точке 0.8.

В точку 1 нужно перенести только 0.5 песка. Работа для этой части:

$$0.2 \cdot 0.5 = 0.1.$$

Оставшийся объём:

$$1 - \theta - 0.5 = 0.5 - \theta$$

нужно перенести из точки 0.8 в точку 0. Работа:

$$0.8(0.5 - \theta).$$

Итого расстояние Вассерштейна равно:

$$W_1(U, V) = 0.4\theta + 0.8(0.5 - \theta) + 0.1.$$

5.16 Distributional Optimality Operator

5.16.1 Distributional Optimality Equation

Для оптимального случая:

$$Z^*(s, a) \stackrel{D}{=} r(s, a) + \gamma Z^*(S', A'),$$

где

$$S' \sim p(\cdot \mid s, a),$$

а

$$A' \sim \arg \max_{a \in \mathcal{A}} \{\mathbb{E}[Z^*(S', a)]\}.$$

Здесь $\arg \max$ может достигаться в разных точках, поэтому используется случайный выбор A' из множества максимизаторов.

Равенство снова подразумевается по распределению.

Аналогично случаю V/Q -функций, **Distributional Optimality Operator** — это правая часть Distributional Optimality Equation:

$$\mathcal{T}Z(s, a) := r(s, a) + \gamma Z(S', A'),$$

где A' выбирается жадно по математическому ожиданию распределения.

5.17 Distributional Optimality Operator может не быть сжатием

Замечание 17. *Distributional Optimality Operator* может не являться сжимающим оператором в пространстве с метрикой Вассерштейна.

Контрпример. Рассмотрим среду, где под

$$r = \varepsilon \pm 1$$

понимается равновероятная награда:

$$\varepsilon + 1 \quad \text{и} \quad \varepsilon - 1,$$

где

$$\varepsilon > 0, \quad \varepsilon \ll 1.$$

Пусть

$$\gamma = 1.$$

Оптимальная политика состоит в выборе действия a_2 из состояния s_1 , так как математическое ожидание награды максимально.

Рассмотрим таблицу распределений:

	(s_0, a_0)	(s_1, a_1)	(s_1, a_2)
Z^*	$\varepsilon \pm 1$	0	$\varepsilon \pm 1$
Z_0^{our}	$\varepsilon \pm 1$	0	$-\varepsilon \pm 1$

Здесь:

- Z^* — оптимальное распределение;
- Z_0^{our} — начальное приближение, на которое будем действовать оператором $\mathcal{T}$, то есть DOO.

Тогда:

$$d(Z_0^{our}, Z^*) = \max_{(s,a) \in \mathcal{S} \times \mathcal{A}} \{W_1(Z_0^{our}(s, a), Z^*(s, a))\} = 2\varepsilon.$$

Для каждой ячейки:

	(s_0, a_0)	(s_1, a_1)	(s_1, a_2)
$W_1(Z_0^{our}(s, a), Z^*(s, a))$	0	0	2ε

Теперь применим к Z_0^{our} оператор Беллмана:

$$Z_1^{our} := \mathcal{T}(Z_0^{our}).$$

Получаем:

	(s_0, a_0)	(s_1, a_1)	(s_1, a_2)
Z^*	$\varepsilon \pm 1$	0	$\varepsilon \pm 1$
Z_0^{our}	$\varepsilon \pm 1$	0	$-\varepsilon \pm 1$
$\mathcal{T}(Z_0^{our})$	0	0	$\varepsilon \pm 1$

Теперь расстояния до Z^* :

$$\frac{\quad}{W_1(Z^*, \mathcal{T}(Z_0^{our}))} \left| \begin{array}{c|c|c} (s_0, a_0) & (s_1, a_1) & (s_1, a_2) \\ \hline 1 & 0 & 0 \end{array} \right|$$

Следовательно:

$$d(\mathcal{T}(Z_0^{our}), Z^*) = \max(1, 0, 0) = 1.$$

Получили, что за одну итерацию расстояние увеличилось:

$$1 > 2\varepsilon$$

при достаточно малом ε .

Значит сжатия нет.

5.18 Хорошие новости

Несмотря на то, что Distributional Optimality Operator может не быть сжимающим в метрике Вассерштейна, для математических ожиданий сохраняется полезное свойство.

Теорема 7.

$$\|\mathbb{E}[Z_1] - \mathbb{E}[Z_2]\|_\infty \geq \gamma \|\mathbb{E}[\mathcal{T}(Z_1)] - \mathbb{E}[\mathcal{T}(Z_2)]\|_\infty.$$

Использование DOO, хоть и несжимающего, приведёт к оптимальной политике, но гарантированного приближения к настоящему распределению Z не будет.

Найдётся некоторое другое распределение:

$$\hat{Z},$$

такое что:

$$\mathbb{E}[\hat{Z}] = \mathbb{E}[Z].$$

Это происходит из-за того, что Distributional Optimality Equation имеет не единственное решение.

5.19 QR-DQN

Как и в DQN, пусть θ — параметры сети.

Для одного сэмпла:

$$(s, a, r, s')$$

алгоритм QR-DQN делает следующее:

1. Выбрать жадное действие по ожиданию распределения:

$$a^* = \arg \max_{a \in \mathcal{A}} \{\mathbb{E}[Z_\theta(s', a)]\}.$$

2. Посчитать целевое значение:

$$y = r + \gamma Z_\theta(s', a^*).$$

3. Посчитать потери:

$$\text{Loss}(y, Z_\theta(s, a)) \rightarrow \min_\theta,$$

и сделать один шаг стохастической оптимизации, например SGD.

5.20 QR-DQN: гиперпараметры и предварительные вычисления

Гиперпараметры:

- B — размер мини-батчей;
- A — число атомов;
- K — периодичность обновления target-сети;
- $\varepsilon(t)$ — стратегия исследования;
- $z_i(s, a; \theta)$ — нейросеть с параметрами θ ;
- SGD-оптимизатор.

Предварительные вычисления.

Предпочитать середины отрезков квантильной сетки:

$$\tau_i := \frac{\frac{i}{A} + \frac{i+1}{A}}{2}, \quad i = 0, \dots, A-1.$$

Эквивалентно:

$$\tau_i = \frac{2i+1}{2A}.$$

Инициализировать параметры θ произвольно.

Положить:

$$\theta^- := \theta.$$

Пронаблюдать начальное состояние:

$$s_0.$$

5.21 QR-DQN: схема алгоритма

На очередном шаге t :

1. Выбрать действие a_t случайно с вероятностью $\varepsilon(t)$.

Иначе выбрать жадное действие:

$$a_t := \arg \max_{a_t \in \mathcal{A}} \left\{ \sum_{i=0}^{A-1} z_i(s_t, a_t; \theta) \right\}.$$

2. Пронаблюдать:

$$r_t, \quad s_{t+1}, \quad done_{t+1}.$$

3. Добавить пятёрку:

$$(s_t, a_t, r_t, s_{t+1}, done_{t+1})$$

в replay buffer.

4. Засэмплировать мини-батч размера B из буфера.

5. Для каждого перехода:

$$T := (s, a, r, s', done)$$

посчитать target:

$$y(T)_j := r + (1 - done)\gamma z_j \left(s', \arg \max_{a' \in \mathcal{A}} \left(\sum_{i=0}^{A-1} z_i(s', a'; \theta^-) \right), \theta^- \right).$$

6. Посчитать loss:

$$\text{Loss}(\theta) := \frac{1}{BA} \sum_T \sum_{i=0}^{A-1} \sum_{j=0}^{A-1} (\tau_i - \mathbf{1}[z_i(s, a; \theta) < y(T)_j]) (z_i(s, a; \theta) - y(T)_j).$$

7. Сделать шаг градиентного спуска по θ , используя:

$$\nabla_{\theta} \text{Loss}(\theta).$$

8. Если:

$$t \bmod K = 0,$$

то обновить target network:

$$\theta^- \leftarrow \theta.$$

5.22 Implicit Quantile Networks

5.22.1 Идея IQN

Идея: научить нейросеть выдавать произвольные квантили распределения.

Для этого дополнительно подаём на вход:

$$\tau \in (0, 1).$$

Тогда модель:

$$z(s, a, \tau; \theta)$$

неявно, или implicitly, задаёт некоторое распределение на $\mathbb{R}$.

По сути моделируется квантильная функция целиком:

$$F_{Z_{\theta}(s, a)}^{-1}(\tau) := z(s, a, \tau; \theta).$$

5.22.2 Сэмплирование через квантильную функцию

Теорема 8. Пусть F — функция распределения случайной величины X .

Если:

$$\tau \sim U[0, 1],$$

то случайная величина:

$$F^{-1}(\tau)$$

имеет то же распределение, что и X .

Доказательство. Функция распределения равномерной случайной величины при $x \in [0, 1]$ равна:

$$\mathbb{P}(\tau < x) = x.$$

Рассмотрим функцию распределения случайной величины:

$$F^{-1}(\tau).$$

Тогда:

$$\mathbb{P}(F^{-1}(\tau) < x) = \mathbb{P}(\tau < F(x)) = F(x).$$

Следовательно, $F^{-1}(\tau)$ имеет функцию распределения F , то есть то же распределение, что и X .

5.23 IQN: жадная стратегия и функция потерь

Жадную стратегию можно аппроксимировать примерно так:

$$\pi^*(s) := \arg \max_{a \in \mathcal{A}} \left(\sum_{i=0}^N z(s, a, \tau_i; \theta) \right),$$

где

$$\tau_i \sim U[0, 1].$$

То есть ожидание распределения приближается средним по случайно выбранным квантилям.

В качестве функции потерь предлагается использовать тот же квантильный loss, что и в QR-DQN, но в модифицированном виде:

$$\text{Loss}(T, \theta) := \sum_{i=0}^{N'} \frac{1}{N''} \sum_{j=0}^{N''} \text{Loss}_{\tau_i} \left(z(s, a, \tau_i; \theta), r + \gamma z(s', \pi^*(s'), \tau_j; \theta^-) \right),$$

где

$$\tau_i, \tau_j \sim U[0, 1].$$

Квантильный loss:

$$\text{Loss}_{\tau}(c, X) = (\tau - \mathbf{1}[c < X]) (c - X).$$

Здесь:

- $T = (s, a, r, s')$ — transition;
- θ — параметры основной сети;
- θ^- — параметры target network;
- $z(s, a, \tau; \theta)$ — предсказанный τ -квантиль распределения $Z_{\theta}(s, a)$.

6 Лекция 6. Exploration, self-supervision, RND и curiosity

6.1 Что делали ранее

В предыдущей лекции рассматривали distributional RL.

Для фиксированной политики π Q -функция записывается как:

$$Q^\pi(s, a) = \mathbb{E}_\tau \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] = \mathbb{E}_\tau [Z^\pi(s, a)],$$

где траектория распределена как:

$$\tau \sim p(s_0) \prod_{t=0}^{\infty} (\pi(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t)).$$

До этого момента распределение Z^π нас интересовало не сильно: мы искали сразу его математическое ожидание.

Идея distributional RL:

искать не только $\mathbb{E}[Z^\pi(s, a)]$, а всё распределение $Z^\pi(s, a)$.

Оптимальное действие выбирается согласно:

$$a^* = \arg \max_{a \in \mathcal{A}} \{\mathbb{E}[Z^\pi(s, a)]\}.$$

В QR-DQN фиксируется число атомов N , и сеть генерирует N дельта-функций. Их позиции выбирает нейронная сеть, а каждая дельта-функция имеет массу:

$$\frac{1}{N}.$$

То есть:

$$Z_\theta(s, a) = \frac{1}{N} \sum_{i=1}^N \delta_{z_i(s, a; \theta)}.$$

6.2 Жадность — плохое качество

Рассмотрим пример, где агенту нужно двигаться вправо.

Награда задаётся как движение по координате x :

$$r_t = x_{t+1} - x_t.$$

У агента есть, например, два действия:

- упасть;
- сделать шаг вперёд.

Если агент падает, он получает маленькую награду:

$$r_{\text{fall}} = \Delta x.$$

Если агент идёт вперёд, награда пропорциональна движению.

Пусть начальная оценка Q -функции равна:

$$Q(s_0, a_{\text{fall}}) = 0, \quad Q(s_0, a_{\text{step}}) = 0.$$

Если политика жадная и агенту не повезло первым действием выбрать падение, то награда всегда будет Δx , потому что агент не исследует другие варианты.

Вывод. Нужно добавить exploration, аналогично тому, как это делалось в DQN.

6.3 Проблема практического exploration

Несмотря на то что для DQN с ε -жадной стратегией есть теорема о сходимости, на практике сходимость может быть очень плохой.

Причина в том, что в теореме используется бесконечное количество взаимодействий со средой.

В сложных средах, например в трёхмерном Doom, простого случайного исследования может быть катастрофически недостаточно.

6.4 Среды с разреженной и малоинформативной наградой

Задачей поиска будем называть задачу со следующей функцией награды:

$$r(s, a) = \begin{cases} +1, & s \in \mathcal{S}^+, \\ \text{const}, & s \notin \mathcal{S}^+, \end{cases}$$

где:

$$\mathcal{S}^+$$

— множество терминальных целевых состояний, а

$$\text{const} \leq 0$$

— штраф за потерю времени.

В таких задачах агент долго может не получать никакого полезного сигнала. Поэтому стандартный exploration может работать очень плохо.

6.5 Self-supervision

6.5.1 Напоминание: MDP

Марковский процесс принятия решений — это кортеж:

$$(\mathcal{S}, \mathcal{A}, r, T, \gamma),$$

где:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
-

$$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

— функция наград;

-

$$T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$$

— функция вероятности перехода;

-

$$\gamma$$

— коэффициент дисконтирования.

Функция перехода:

$$T(s, a, s') = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a).$$

То есть вероятность оказаться в следующем состоянии зависит только от предыдущего состояния и действия из него.

Для решения проблемы exploration попробуем изменить функцию награды r .

6.6 Внешняя и внутренняя награда

Исходная MDP-задача содержит внешнюю награду:

$$r_{\text{extr}} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Она соответствует исходной задаче.

Введём также вспомогательную задачу, или auxiliary task, с внутренней наградой:

$$r_{\text{intr}} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Эта внутренняя награда нужна для лучшего exploration.

Итоговую награду можно задать аддитивно:

$$r(s, a) := r_{\text{extr}}(s, a) + \alpha r_{\text{intr}}(s, a),$$

где α — масштабирующий гиперпараметр.

Для простоты далее можно считать:

$$\alpha = 1.$$

6.7 Аддитивная мотивация

Если:

$$r(s, a) = r_{\text{extr}}(s, a) + r_{\text{intr}}(s, a),$$

то для фиксированной политики π выполнено:

$$Q^\pi(s, a) = Q_{\text{extr}}^\pi(s, a) + Q_{\text{intr}}^\pi(s, a),$$

и

$$V^\pi(s) = V_{\text{extr}}^\pi(s) + V_{\text{intr}}^\pi(s).$$

Доказательство. По определению:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right].$$

Подставим:

$$r(s_t, a_t) = r_{\text{extr}}(s_t, a_t) + r_{\text{intr}}(s_t, a_t).$$

Тогда:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t (r_{\text{extr}}(s_t, a_t) + r_{\text{intr}}(s_t, a_t)) \right] \\ &= \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{\text{extr}}(s_t, a_t) \right] + \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{\text{intr}}(s_t, a_t) \right] \\ &= Q_{\text{extr}}^\pi(s, a) + Q_{\text{intr}}^\pi(s, a). \end{aligned}$$

Аналогично доказывается разложение для V^π .

6.8 Раздельное обучение оценочных функций

Если в алгоритме учиться оценочная функция V^π или Q^π , то можно учить оценочные функции каждого слагаемого в награде отдельно.

Например, сеть может иметь отдельные головы:

$$V_{\text{extr}}^\pi(s), \quad V_{\text{intr}}^\pi(s),$$

или:

$$Q_{\text{extr}}^\pi(s, a), \quad Q_{\text{intr}}^\pi(s, a).$$

Трюк можно применять даже в value-based методах, где учиться Q^* , поскольку этот процесс можно интерпретировать как оценивание текущей стратегии:

$$\pi(s) = \arg \max_{a \in \mathcal{A}} \{Q^\pi(s, a)\}.$$

6.9 Почему аналогичное разложение неверно для Q^* и V^*

В общем случае аналогичное разложение для Q^* и V^* неверно.

Причина. Максимум суммы может быть меньше суммы максимумов:

$$\max_a \{f(a) + g(a)\} \leq \max_a f(a) + \max_a g(a).$$

И равенство не обязано выполняться.

Пример. Пусть первое действие даёт внешнюю награду +1, а второе действие даёт внутреннюю награду +1. В остальных случаях награды равны нулю.

Тогда:

$$V_{\text{extr}}^*(s) = +1, \quad V_{\text{intr}}^*(s) = +1.$$

Но оптимальная V -функция для суммы наград равна не 2, а 1, потому что нельзя выбрать два действия одновременно.

То есть между мотивациями приходится выбирать.

6.10 Почему разложение неверно для Z^π

В общем случае аналогичное разложение для Z^π неверно.

Причина в том, что внутренняя и внешняя мотивация могут быть скоррелированы, и этой информации при доступе только к отдельным распределениям:

$$Z_{\text{extr}}^\pi, \quad Z_{\text{intr}}^\pi$$

у нас нет.

Пример. Пусть известно, что для некоторых s, a :

$$Z_{\text{extr}}^\pi(s, a) = \begin{cases} +1, & \text{с вероятностью } 0.5, \\ 0, & \text{с вероятностью } 0.5, \end{cases}$$

и то же самое для:

$$Z_{\text{intr}}^\pi(s, a).$$

Если предположить независимость, то распределение суммы:

$$Z_{\text{extr}}^\pi(s, a) + Z_{\text{intr}}^\pi(s, a)$$

имело бы вид:

$$\begin{cases} +2, & \text{с вероятностью } 0.25, \\ +1, & \text{с вероятностью } 0.5, \\ 0, & \text{с вероятностью } 0.25. \end{cases}$$

Однако независимость может быть неверной. Например, внешняя награда $+1$ может получаться тогда и только тогда, когда получается внутренняя награда $+1$.

Тогда распределение суммы будет:

$$\begin{cases} +2, & \text{с вероятностью } 0.5, \\ 0, & \text{с вероятностью } 0.5. \end{cases}$$

Следствие. В общем случае аналогичное разложение для Z^* также неверно.

6.11 Разные коэффициенты дисконтирования

Разложение на внешнюю и внутреннюю мотивацию позволяет использовать разные коэффициенты дисконтирования для разных мотиваций:

$$y_{\text{extr}} := r_{\text{extr}} + \gamma_{\text{extr}} V_{\text{extr}}(s'),$$

$$y_{\text{intr}} := r_{\text{intr}} + \gamma_{\text{intr}} V_{\text{intr}}(s').$$

Также разложение позволяет во внутренней мотивации игнорировать понятие эпизодов, не используя флаги *done* из среды при обучении V_{intr} .

Такой агент учитывает, что если он окажется в терминальном состоянии, произойдёт сброс среды, и он окажется в стартовом состоянии:

$$s_0 \sim p(s_0),$$

для которого значение $V_{\text{intr}}(s)$ может быть высоким.

Это может мотивировать агента прерывать текущий неудачный эпизод ради того, чтобы начать новый, что иногда может быть полезно.

6.12 Self-supervision на практике

В реальных задачах раздвоение MDP означает добавление к основной награде придуманной нами награды за исследование.

Главная сложность — создать хорошую функцию внутренней мотивации агента к исследованиям даже для сред, о которых нам самим известно мало.

6.13 Basic idea: бонус за посещение нового состояния

Самая простая идея: выдавать агенту дополнительную награду за посещение нового состояния среды.

Такие награды могут пересоздаваться перед началом каждого эпизода.

Однако такая функция наград поощряет не умное исследование, а просто исследование всего пространства состояний на каждом эпизоде.

6.13.1 Модификация

Часто на практике делают иначе: не пересоздают награды в посещённых ячейках между эпизодами, а заводят счётчики посещения состояний агентом за всё время обучения.

Когда агент заходит в состояние, его поощряют обратно пропорционально счётчику.

Теоретически такое решение делает MDP-формализм неприменимым, поскольку среда подстраивается под агента. Но на практике такой подход может работать хорошо.

6.14 Бонусы на основе счётчиков

Пусть:

$$h : \mathcal{S} \rightarrow \{0, 1, \dots, N\}$$

— некоторая хэш-функция состояний, называемая oracle.

Пусть:

$$n(i)$$

— счётчик, сколько раз за всё время обучения встретились состояния с хэшем i .

Тогда:

$$r_{\text{intr}}(s, a) := \frac{1}{n(h(s)) + \varepsilon}, \quad \varepsilon > 0,$$

называется нестационарным исследовательским бонусом.

Другой вариант — эпизодичный исследовательский бонус:

$$r_{\text{intr}}(s_t, a_t) := \mathbf{1} [\forall t' < t : h(s_t) \neq h(s_{t'})].$$

То есть агент получает +1, если попал в состояние, хэш которого в данном эпизоде ещё не встречался.

6.15 Пример с хэш-функцией

В табличных MDP, где:

$$|\mathcal{S}| < +\infty,$$

хэш-функция по сути не нужна:

$$h(s) = s.$$

В ряде сред такой oracle можно придумать эвристически.

Например, если у агента есть понятие координат, можно разделить пространство сеткой на условные блоки и награждать агента за посещение большого числа блоков.

6.16 Недостатки простой идеи

Недостатки бонусов за посещение новых состояний:

- Решение невозможно напрямую применить в средах с непрерывным количеством состояний. Среду приходится разбивать на конечное число областей и присваивать награды этим областям, либо иметь дополнительные знания о среде.
- Такой подход награждает все состояния одинаково и не выявляет важные или похожие друг на друга состояния.
- Нужно помнить, в каких состояниях агент уже был.

6.17 Mario Oracle

Пример хорошего reward shaping на конкретной задаче — Mario Oracle.

В такой среде можно поощрять движение агента вправо и наказывать за движение влево.

Например, можно задать внутреннюю награду:

$$r_{\text{intr}}(s_t, a_t) = x_{t+1} - x_t,$$

где x_t — горизонтальная координата агента.

Тогда движение вправо даёт положительную внутреннюю награду, а движение влево — отрицательную.

6.18 Random Network Distillation

6.18.1 Идея: детектировать новые состояния

Рассмотрим задачу регрессии. Обычно ошибка регрессии выше на тех объектах, которые модель ещё не видела или видела редко.

Задача обучения Q -функции тоже является задачей регрессии. Поэтому можно попробовать оценивать новизну состояний через ошибку некоторой регрессионной модели.

Но обучать Q -функцию может быть трудно. Особенно тяжело оценить, насколько хорошо она выучена, то есть нужен корректный target.

Поэтому смотреть на ошибку Q -функции как на новизну — плохая идея.

Другая задача регрессии. Заведём вспомогательную нейронную сеть. Ошибка этой вспомогательной задачи на состояниях и будет использоваться как новизна состояний.

6.19 Distillation

Пусть:

$$\phi : \mathcal{S} \rightarrow \mathbb{R}^d$$

— некоторая функция, строящая эмбединги состояний.

Например, ϕ может быть случайной сетью, или random network: нейросетью со случайно инициализированными весами, которые не обучаются и не изменяются.

Пусть:

$$f : \mathcal{S} \rightarrow \mathbb{R}^d$$

учится предсказывать выход ϕ , используя в качестве обучающей выборки значения ϕ на состояниях, встречавшихся в ходе обучения.

Определение 2. Задача обучения одной нейросети f на входах-выходах другой заданной фиксированной нейросети ϕ :

$$\mathbb{E}_s [\|f(s) - \phi(s)\|_2^2] \rightarrow \min_f,$$

где математическое ожидание $\mathbb{E}_s$ берётся по произвольному буферу, называется дистилляцией, или *distillation*.

Так как ошибка ожидается выше там, где состояния новее, можно использовать её в качестве обучающего сигнала:

$$r_{\text{intr}}(s) := \|f(s) - \phi(s)\|_2^2.$$

Интуиция. Если агент впервые увидел состояние s , модель f ещё не видела, какой эмбединг выдаёт на нём случайная сеть ϕ . Поэтому f , скорее всего, ошибётся.

Такой сигнал мотивирует агента идти в области среды, где обучающаяся сеть плохо предсказывает выход случайно проинициализированной сети.

6.20 Недостатки ошибки регрессии как новизны

У такого метода есть недостатки:

- Большая ошибка может возникать не только на новых состояниях, но и тогда, когда модель f недостаточно гибкая.
- Могут возникнуть большие проблемы, если целевые значения в задаче регрессии являются случайными величинами, то есть target недетерминирован.
- Ещё хуже, если в процессе обучения какие-то целевые значения меняются, то есть target нестационарный.

6.21 Random Network Distillation

Алгоритм **Random Network Distillation**, или RND:

1. Заводим нейросеть, называемую **Target Network**. Инициализируем её веса случайным образом и фиксируем их навсегда.

2. Заводим вторую нейросеть, называемую **Predictor Network**. Её уже обучаем. Она пытается предсказывать выход Target Network на состояниях среды.
3. Невязку между значениями Target Network и Predictor Network на данном состоянии интерпретируем как новизну этого состояния:

$$r_{\text{intr}}(s) := \frac{1}{2} \|\phi_{\text{predictor}}(s) - \phi_{\text{target}}(s)\|_2^2.$$

6.22 Ньюансы Random Network Distillation

У RND есть важные технические нюансы.

- Случайная Target Network должна давать различные ответы на различных состояниях. Для этого нужна аккуратная инициализация.
- Эмбединги состояний желательно нормировать перед подачей в сети.
Для этого перед основным обучением запускают агента в среду, он собирает набор состояний. По этому набору считают среднее и стандартное отклонение, а далее используют их для нормировки при обучении.
После этого статистики лучше не менять, иначе возникнет нестационарность target.
- Первые несколько итераций обучения иногда лучше вообще не использовать внешнюю награду, а обучать агента только на внутренней награде, пока Predictor Network не выйдет на асимптоту обучения.
- Внутреннюю награду можно масштабировать в зависимости от того, какие внутренние награды были на предыдущих шагах. Для этого используют скользящее оценивание статистик по ходу обучения.

6.23 Curiosity

6.23.1 Идея

Вместо того чтобы смотреть на новизну состояния, можно смотреть на то, насколько хорошо агент понимает устройство мира.

Иными словами, можно учить модель среды.

Пусть:

$$f(s, a)$$

— функция, которая по текущим s, a предсказывает следующее состояние s' .

Тогда можно решать задачу:

$$\frac{1}{2} \|f(s, a) - s'\|_2^2 \rightarrow \min_f.$$

Ошибку этой модели будем использовать как внутреннюю награду:

$$r_{\text{intr}}(s, a, s') := \frac{1}{2} \|f(s, a) - s'\|_2^2.$$

То есть агенту не важно, новое это состояние или нет. Если агент может предсказать, что будет дальше, то это состояние ему неинтересно.

Определение 3. Любопытством, или *curiosity*, называется ошибка модели мира агента.

6.24 Curiosity: naïve approach

Наивный подход — учить модель, которая по текущему состоянию s и действию a предсказывает следующее состояние s' прямо в пространстве наблюдений.

Например:

$$s, a \longrightarrow \text{model} \longrightarrow \hat{s}',$$

и оптимизировать:

$$\frac{1}{2} \|\hat{s}' - s'\|_2^2.$$

Если состояния — картинки, это означает генерацию следующей картинки.

Проблемы.

- Генерация картинок — дорогой процесс.
- Модель выучивает большое количество ненужной информации, например как движется вся сцена.
- Модель учится в пространстве пикселей.
- Если среда недетерминирована, предсказание следующего наблюдения может быть принципиально неоднозначным.

6.25 Curiosity: less naïve approach

Более аккуратная идея — предсказывать не следующее состояние в пикселях, а его эмбединг.

Для этого можно взять VAE и обучить его на replay buffer, затем зафиксировать.

На каждом новом состоянии-картинке VAE выдаёт эмбединг состояния:

$$\phi(s).$$

Этот эмбединг отправляется в Forward Dynamics Model.

Forward Dynamics Model по текущему эмбедингу и действию предсказывает эмбединг следующего состояния:

$$f(\phi(s), a) \approx \phi(s').$$

Далее реальное следующее состояние s' также пропускается через VAE, получаем:

$$\phi(s').$$

После этого считаем MSELoss между предсказанным и реальным эмбедингами:

$$\|f(\phi(s), a) - \phi(s')\|_2^2.$$

Вывод. Перевод состояний в эмбединги — полезная идея. Всё, что нужно, — хорошие эмбединги.

6.26 Noisy TV Problem

Определение 4. Шумным телевизором, или *noisy TV*, в среде называются принципиально непредсказуемые из-за стохастичности функции переходов явления.

Проблема шумного телевизора. Часто в средах на фоне происходят случайные бесполезные события, которые никак не помогают игроку достичь цели.

Такие события для RL-агентов вредны, потому что на их восприятие тратится много ресурсов.

Хотелось бы, чтобы такие события отсекались на уровне эмбедингов.

6.27 Требования к эмбедингам

Сформулируем, каким должен быть хороший эмбединг состояния.

Ищем функцию:

$$\phi(s) : \mathcal{S} \rightarrow \mathbb{R}^d,$$

которая переводит состояние в эмбединг и обладает следующими свойствами:

1. не содержит бесполезного шума среды;
2. потенциально может содержать всю информацию, необходимую для оптимального решения задачи;
3. желательно, чтобы латентное пространство эмбедингов было небольшим и в нём можно было считать MSELoss;
4. желательно, чтобы эмбединги не менялись со временем или менялись достаточно медленно.

Последнее требование обычно выполнить не удаётся.

Поэтому уменьшим желания до выполнимых:

1. эмбединг не содержит бесполезного шума среды;
2. потенциально может содержать всю информацию, необходимую для оптимального решения задачи;
3. латентное пространство эмбедингов желательно небольшое, чтобы можно было считать MSELoss.

6.28 Inverse Dynamics Model

Определение 5. Модель, аппроксимирующая распределение:

$$p(a \mid s, s'),$$

называется моделью обратной динамики, или *inverse dynamics model*.

Для модели обратной динамики проблема шумных телевизоров в среде заменяется симметричной проблемой в пространстве действий $\mathcal{A}$.

Из формализма MDP по формуле Байеса следует:

$$p(a \mid s, s') = \frac{p(s' \mid s, a)\pi(a \mid s)}{\int_{\mathcal{A}} p(s' \mid s, a)\pi(a \mid s) da}.$$

6.29 Обучение модели обратной динамики

Рассмотрим модель обратной динамики с сиамской архитектурой.

Пусть:

$$\phi : \mathcal{S} \rightarrow \mathbb{R}^d$$

строит латентное описание состояний.

Функция:

$$g(\phi(s), \phi(s'))$$

пытается восстановить действие a , произошедшее между двумя состояниями, по их латентным описаниям.

Для непрерывного пространства действий задача выглядит так:

$$\mathbb{E}_{s,a,s'} \left[\|g(\phi(s), \phi(s')) - a\|_2^2 \right] \rightarrow \min_{g,\phi}.$$

Для дискретного пространства действий, где задача является классификацией:

$$\mathbb{E}_{s,a,s'} [\ln g(a \mid \phi(s), \phi(s'))] \rightarrow \max_{g,\phi}.$$

Определение 6. Латентные представления состояний, которые учит функция $\phi(s)$ в задаче обучения обратной динамики, называются контролируемым состоянием, или *controllable state*.

Сама функция $\phi(s)$ называется фильтром, или *filter*.

6.30 Почему inverse dynamics помогает

Модель обратной динамики по двум соседним состояниям среды предсказывает действие между ними:

$$(s, s') \longrightarrow a.$$

При этом $\phi(s)$ также обучается.

Чтобы понять, какое действие было совершено, фоновая шумовая информация обычно не нужна. Поэтому она отбрасывается самой моделью.

То есть inverse dynamics model заставляет эмбединг хранить ту часть состояния, которая связана с управляемыми агентом изменениями.

Бонус. На практике к *curiosity* иногда добавляют невязку между предсказанным такой моделью действием и действительно выбранным действием из буфера.

Такие состояния кажутся более интересными, чем те, где предсказания модели и выбор агента совпали.

Замечание к бонусу. Невязка может быть большой не только потому, что состояние интересное.

Например, два различных действия из одного и того же состояния могут приводить в одно и то же состояние.

Пример: упор в стену в игре. Действия «стоять на месте» и «идти в сторону стены» могут приводить к одному и тому же следующему состоянию.

6.31 Intrinsic Curiosity Model

6.31.1 Идея ICM

Модель обратной динамики — не единственный способ достичь желаемого.

Идея Intrinsic Curiosity Model, или ICM:

1. Сначала обучить эмбединги в Inverse Dynamics Model.
2. Затем можно было бы выбросить блок Inverse Dynamics Model, оставив только эмбединги.
3. Вместо выброшенной части вставить модель прямой динамики.
4. Считать MSELoss в латентном пространстве эмбедингов.

Проблема. Не совсем понятно, где и когда учить Inverse Dynamics Model на самом первом шаге.

Решение. Не выбрасывать Inverse Dynamics Model, а носить её с собой и доучивать вместе с моделью прямой динамики.

6.32 ICM как совместное обучение inverse и forward dynamics

Свойство модели обратной динамики наводит на способ защиты от noisy TV.

Берём описание состояний и очищаем их от шумных телевизоров при помощи фильтра:

$$\phi(s).$$

Фильтр переводит состояние в латентное пространство, хранящее только частичную информацию о входе.

Далее модель прямой динамики строится в отфильтрованном латентном представлении:

$$\mathbb{E}_{s,a,s'} \left[\|f(\phi(s), a) - \phi(s')\|_2^2 \right] \rightarrow \min_f.$$

Итоговая модель ICM, например для непрерывных действий:

$$\mathbb{E}_{s,a,s'} \left[\|g(\phi(s), \phi(s')) - a\|_2^2 + \alpha \|f(\phi(s), a) - \phi(s')\|_2^2 \right] \rightarrow \min_{f,g,\phi}.$$

Здесь:

- f — forward dynamics model;
- g — inverse dynamics model;
- ϕ — encoder/filter состояний;
- α — масштабирующий гиперпараметр;
- (s, a, s') — произвольные тройки из любого буфера.

В качестве внутренней мотивации используется только ошибка модели прямой динамики:

$$r_{\text{intr}}(s, a, s') := \|f(\phi(s), a) - \phi(s')\|_2^2.$$

Замечание 18. В ICM градиент затекает во все блоки. Появляется регуляризация модели прямой динамики с помощью модели обратной динамики.

6.33 Сравнение curiosity-подходов

В работе *Large-Scale Study of Curiosity-Driven Learning*, 2018, сравниваются разные варианты curiosity-driven learning.

Сравниваются, в частности:

- предсказание в пикселях;
- VAE features;
- inverse dynamics features;
- random CNN features.

Главная идея таких сравнений: качество внутренней мотивации сильно зависит от того, какие признаки состояния используются для вычисления ошибки предсказания.

6.34 Наглядный пример

Наглядный пример curiosity-driven поведения: агенты могут исследовать среду, не глядя на внешние очки.

Это иллюстрирует идею:

агент может учиться исследовать среду по внутренней мотивации.

6.35 Телевизор и пульт: слабость ICM или нечто большее

Проблема прокрастинации. ICM настроен фильтровать шум, неподконтрольный агенту.

Но если агент может совершать действие и каждый раз получать неожиданный результат, то он может залипнуть на такой части среды.

Например, если у агента есть пульт от телевизора, и переключение канала каждый раз приводит к новому непредсказуемому изображению, агент может бесконечно нажимать кнопки.

Суть проблемы. Эта уязвимость не обязательно привязана конкретно к ICM. Она является более общей концептуальной проблемой.

Может быть, среда устроена так, что нужно долистать до сотого канала и остаться на нём.

Может быть, мы не можем предсказать будущее не потому, что это нерелевантный шум, а потому, что наша предсказательная стратегия ещё недостаточно обучена.

Вывод. Управляемые шумные телевизоры полностью отфильтровать нельзя. Их приходится оставить как содержательную часть проблемы exploration.

7 Лекция 7. Policy Gradient

7.1 Что делали ранее

В предыдущей лекции была поставлена проблема исследования среды: в сложных средах случайное исследование работает плохо, поэтому нужно придумать что-то лучше.

Основная идея:

нужно поощрять умное исследование.

Для этого была введена внутренняя награда. Исходная MDP-задача содержит внешнюю награду:

$$R_{\text{extr}} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R},$$

а вспомогательная задача для exploration содержит внутреннюю награду:

$$R_{\text{intr}} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Итоговая награда может быть записана как:

$$R(s, a) = R_{\text{extr}}(s, a) + \alpha R_{\text{intr}}(s, a),$$

где α — масштабирующий коэффициент.

Были рассмотрены три идеи для улучшения exploration:

1. Наивный бонус за посещение нового состояния.

Агент получает награду за посещение состояния, в котором в данном эпизоде ещё не был.

2. Random Network Distillation.

Новизна состояния оценивается по ошибке на вспомогательной задаче регрессии.

Агенту выгодно идти в состояния, для которых ошибка предсказания выхода фиксированной случайной сети всё ещё велика.

3. Curiosity.

Это шаг в сторону model-based алгоритмов. Агент учит модель мира и пытается предсказать, что будет, если из состояния s совершить действие a .

Если агент хорошо предсказывает следующий переход (s, a, s') , то эта тройка неинтересна. Если предсказание плохое, возникает любопытство.

Также обсуждалась проблема шумных телевизоров.

Неуправляемый шум можно частично отфильтровать, если встроить в агента модель, которая по двум соседним состояниям среды предсказывает действие между ними.

Однако проблема управляемого шума остаётся: агент может залипать на объектах, на которые он может воздействовать, но результаты этих воздействий предсказать не может.

7.2 Напоминание: V - и Q -функции

Определение 7. V -функцией ценности, или *value function*, называется функция:

$$V^\pi(s) = \mathbb{E}_{p(\tau_t|\pi)} [R_t \mid s_t = s].$$

Это средняя награда по политике π , если агент начинает действовать в момент времени t из состояния s .

Определение 8. Q -функцией ценности, или *quality function*, называется функция:

$$Q^\pi(s, a) = \mathbb{E}_{p(\tau_t|\pi)} [R_t \mid s_t = s, a_t = a].$$

Это то же, что и V -функция, но теперь из состояния $s_t = s$ обязательно сначала совершается действие $a_t = a$.

7.3 Оптимальные функции ценности

Обозначим оптимальную политику как:

$$\pi^*.$$

Это политика, на которой максимизируется ожидаемая суммарная награда:

$$\mathbb{E}_{p(\tau|\pi)} [R].$$

Определение 9. Оптимальной V -функцией ценности называется функция:

$$V^*(s) = \max_{\pi} \{ \mathbb{E}_{p(\tau_t|\pi)} [R_t \mid s_t = s] \} = \max_{\pi} \{ V^\pi(s) \} = V^{\pi^*}(s).$$

Определение 10. Оптимальной Q -функцией ценности называется функция:

$$Q^*(s, a) = \max_{\pi} \{ Q^\pi(s, a) \} = Q^{\pi^*}(s, a).$$

7.4 On-policy и off-policy

Определение 11. *On-policy* алгоритмы — это алгоритмы, которые оценивают и улучшают ту же самую политику, которую используют для выбора действий.

То есть:

$$\text{Target Policy} = \text{Behavior Policy}.$$

Определение 12. *Off-policy* алгоритмы — это алгоритмы, которые оценивают и улучшают одну политику, а для выбора действий используют другую политику.

То есть:

$$\text{Target Policy} \neq \text{Behavior Policy}.$$

7.5 DQN и Policy Gradient

Сравним DQN и Policy Gradient.

7.5.1 DQN

DQN обладает следующими свойствами:

- off-policy алгоритм;
- использует одношаговое оценивание политики;
- оценка смещённая;
- использует ϵ -жадную политику;
- учит оптимальную Q -функцию:

$$Q^*.$$

7.5.2 Policy Gradient

Policy Gradient обладает следующими свойствами:

- on-policy алгоритм;
- оценка строится до конца эпизода;
- из-за этого оценка имеет большую дисперсию;
- явно обучается политика как распределение:

$$\pi(a \mid s);$$

- учится функция ценности по текущей политике:

$$V^\pi.$$

7.6 План вывода Policy Gradient

План:

1. Ввести оптимизационную задачу для Policy Gradient.
2. Вывести градиент оптимизируемой функции первым способом.
3. Вывести градиент оптимизируемой функции вторым способом.
4. Доказать эквивалентность двух подходов.
5. Показать физический смысл полученной формулы.

7.7 Первый способ вывода Policy Gradient

7.7.1 Постановка задачи

Рассмотрим θ -параметризованное семейство политик:

$$\pi(a \mid s, \theta),$$

или кратко:

$$\pi_\theta(a \mid s).$$

Будем максимизировать следующий функционал:

$$V^\pi := J(\theta) := \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t \geq 0} \gamma^t r_t \right] \rightarrow \max_{\theta}.$$

7.7.2 Запись через Q -функцию

Для фиксированной политики можно записать:

$$V^\pi := \mathbb{E}_s \mathbb{E}_{a \sim \pi_\theta} [Q^{\pi_\theta}(s, a)].$$

В интегральной форме:

$$V^\pi = \mathbb{E}_s \int_{\mathcal{A}} \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) da.$$

Тогда:

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_s \int_{\mathcal{A}} \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) da \\ &= \mathbb{E}_s \int_{\mathcal{A}} \nabla_\theta [\pi_\theta(a \mid s) Q^{\pi_\theta}(s, a)] da \\ &= \mathbb{E}_s \int_{\mathcal{A}} \nabla_\theta \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) da \\ &\quad + \mathbb{E}_s \int_{\mathcal{A}} \pi_\theta(a \mid s) \nabla_\theta Q^{\pi_\theta}(s, a) da. \end{aligned}$$

Получили два слагаемых.

7.8 Трюк производной логарифма

Используем стандартный трюк:

$$\nabla_\theta p_\theta(x) = p_\theta(x) \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} = p_\theta(x) \nabla_\theta \ln p_\theta(x).$$

Применим его к политике:

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s) \nabla_\theta \ln \pi_\theta(a \mid s).$$

7.9 Первое слагаемое

Рассмотрим первое слагаемое:

$$\mathbb{E}_s \int_{\mathcal{A}} \nabla_{\theta} \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a) da.$$

По трюку производной логарифма:

$$\begin{aligned} & \mathbb{E}_s \int_{\mathcal{A}} \nabla_{\theta} \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a) da \\ &= \mathbb{E}_s \int_{\mathcal{A}} \pi_{\theta}(a \mid s) \nabla_{\theta} \ln \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a) da \\ &= \mathbb{E}_s \mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a)]. \end{aligned}$$

7.10 Второе слагаемое

Второе слагаемое:

$$\mathbb{E}_s \int_{\mathcal{A}} \pi_{\theta}(a \mid s) \nabla_{\theta} Q^{\pi_{\theta}}(s, a) da.$$

Его можно переписать как:

$$\mathbb{E}_s \mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} Q^{\pi_{\theta}}(s, a)].$$

Итого:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_s \mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a) + \nabla_{\theta} Q^{\pi_{\theta}}(s, a)].$$

Замечание 19. Мы смогли выразить градиент V -функции через градиент Q -функции. Теперь попробуем сделать наоборот.

7.11 Выражение $\nabla_{\theta} Q$ через $\nabla_{\theta} V$

Для Q -функции:

$$Q^{\pi_{\theta}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V^{\pi_{\theta}}(s')].$$

Тогда:

$$\nabla_{\theta} Q^{\pi_{\theta}}(s, a) = \nabla_{\theta} r(s, a) + \nabla_{\theta} \gamma \mathbb{E}_{s'} [V^{\pi_{\theta}}(s')].$$

Поскольку награда $r(s, a)$ не зависит от параметров политики θ , имеем:

$$\nabla_{\theta} r(s, a) = 0.$$

Следовательно:

$$\begin{aligned} \nabla_{\theta} Q^{\pi_{\theta}}(s, a) &= \nabla_{\theta} \gamma \int_{\mathcal{S}} V^{\pi_{\theta}}(s') p(s' \mid s, a) ds' \\ &= \gamma \mathbb{E}_{s'} [\nabla_{\theta} V^{\pi_{\theta}}(s')]. \end{aligned}$$

7.12 Рекурсивная формула

Подставляя это в формулу для градиента, получаем:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_s \mathbb{E}_a \mathbb{E}_{s'} [\nabla_{\theta} \ln \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a) + \gamma \nabla_{\theta} V^{\pi_{\theta}}(s')].$$

Это похоже на уравнение Беллмана: в правой части стоит математическое ожидание по действию a , совершаемому из состояния s , и по следующему состоянию s' .

Раскрутим рекурсию до бесконечности.

7.13 Policy Gradient Theorem

В результате получаем:

Теорема 9 (Policy Gradient Theorem).

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) Q^{\pi_{\theta}}(s_t, a_t) \right],$$

где τ — траектория согласно политике π_{θ} .

7.14 Второй способ вывода Policy Gradient

7.14.1 Постановка задачи

Рассмотрим θ -параметризованное семейство политик:

$$\pi(a | s, \theta) = \pi_{\theta}(a | s),$$

которое порождает траектории τ с вероятностями:

$$p_{\theta}(\tau).$$

Будем максимизировать:

$$V^{\pi} := J(\theta) := \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t \geq 0} \gamma^t r_t \right] \rightarrow \max_{\theta}.$$

Обозначим награду на траектории:

$$r(\tau) := \sum_{t \geq 0} \gamma^t r_t.$$

7.14.2 Вычисление градиента

По определению математического ожидания:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)] = \int_{\mathcal{T}} p_{\theta}(\tau) r(\tau) d\tau,$$

где $\mathcal{T}$ — носитель траекторий.

Тогда:

$$\nabla_{\theta} J(\theta) = \int_{\mathcal{T}} \nabla_{\theta} p_{\theta}(\tau) r(\tau) d\tau.$$

Снова используем трюк производной логарифма:

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = p_{\theta}(\tau) \nabla_{\theta} \ln p_{\theta}(\tau).$$

Тогда:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \int_{\mathcal{T}} \nabla_{\theta} p_{\theta}(\tau) r(\tau) d\tau \\ &= \int_{\mathcal{T}} p_{\theta}(\tau) \nabla_{\theta} \ln p_{\theta}(\tau) r(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \ln p_{\theta}(\tau) r(\tau)]. \end{aligned}$$

7.15 Вероятность траектории

Для траектории:

$$\tau = (s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T)$$

вероятность равна:

$$p_{\theta}(\tau) = p_{\theta}(s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T) = p(s_0) \prod_{t=0}^{T-1} (\pi_{\theta}(a_t | s_t) p(s_{t+1} | a_t, s_t)).$$

Логарифм вероятности траектории:

$$\ln p_{\theta}(\tau) = \ln p(s_0) + \sum_{t=0}^{T-1} [\ln \pi_{\theta}(a_t | s_t) + \ln p(s_{t+1} | a_t, s_t)].$$

Градиент логарифма вероятности траектории:

$$\nabla_{\theta} \ln p_{\theta}(\tau) = \nabla_{\theta} \sum_{t=0}^{T-1} \ln \pi_{\theta}(a_t | s_t) + \nabla_{\theta} \sum_{t=0}^{T-1} \ln p(s_{t+1} | a_t, s_t).$$

Второе слагаемое равно нулю, так как динамика среды не зависит от параметров политики θ :

$$\nabla_{\theta} \sum_{t=0}^{T-1} \ln p(s_{t+1} | a_t, s_t) = 0.$$

Поэтому:

$$\nabla_{\theta} \ln p_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t).$$

7.16 Формула второго способа

Подставим это в выражение для градиента:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \ln p_{\theta}(\tau) r(\tau)] \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) \right) r(\tau) \right]. \end{aligned}$$

Так как:

$$r(\tau) = \sum_{\hat{t} \geq 0} \gamma^{\hat{t}} r_{\hat{t}},$$

то можно записать:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{\hat{t} \geq 0} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) r(\tau) \right].$$

Вывод. Более простым способом мы получили похожую формулу, но с полной суммарной наградой за эпизод вместо Q -функции из первого способа.

Математически эти формы эквивалентны как интегралы, но их Монте-Карло оценки могут вести себя по-разному.

7.17 Переход к эквивалентности

Распишем формулу второго способа подробнее:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{\hat{t} \geq 0} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) \right) \left(\sum_{\hat{t} \geq 0} \gamma^{\hat{t}} r_{\hat{t}} \right) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{\hat{t} \geq 0} \sum_{t \geq 0} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) \gamma^{\hat{t}} r_{\hat{t}} \right]. \end{aligned}$$

Выпишем одно из слагаемых этой двойной суммы:

$$j_t := \sum_{\hat{t} \geq 0} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) \gamma^{\hat{t}} r_{\hat{t}}.$$

7.18 Проблема наград из прошлого

Слагаемое:

$$j_t = \sum_{\hat{t} \geq 0} \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) \gamma^{\hat{t}} r_{\hat{t}}$$

отвечает за градиент решения выбрать действие a_t в момент времени t .

Но на него влияют не только награды после принятия этого решения, то есть $\hat{t} \geq t$, но и награды из прошлого:

$$\hat{t} < t.$$

Это плохо, потому что принятое в момент t решение не могло повлиять на награды, полученные раньше.

Пример. До момента времени:

$$t_{\text{near end}} \approx T$$

агент мог выполнять очень хорошие действия, поэтому сумма:

$$\sum_{\hat{t} < t_{\text{near end}}} \gamma^{\hat{t}} r_{\hat{t}}$$

может быть очень большой.

Но решение в момент $t_{\text{near end}}$ может быть ужасным, и после него награда всегда будет минимальной.

Если в градиенте использовать полную награду за эпизод, плохое позднее решение может быть замаскировано большой наградой из прошлого.

7.19 Теорема о score function

Теорема 10. Для произвольного распределения $\pi_\theta(a)$ верно:

$$\mathbb{E}_{a \sim \pi_\theta(a)} [\nabla_\theta \ln \pi_\theta(a)] = 0.$$

Доказательство.

$$\begin{aligned} \mathbb{E}_{a \sim \pi_\theta(a)} [\nabla_\theta \ln \pi_\theta(a)] &= \mathbb{E}_{a \sim \pi_\theta(a)} \left[\frac{\nabla_\theta \pi_\theta(a)}{\pi_\theta(a)} \right] \\ &= \int_{\mathcal{A}} \pi_\theta(a) \frac{\nabla_\theta \pi_\theta(a)}{\pi_\theta(a)} da \\ &= \int_{\mathcal{A}} \nabla_\theta \pi_\theta(a) da \\ &= \nabla_\theta \int_{\mathcal{A}} \pi_\theta(a) da \\ &= \nabla_\theta 1 = 0. \end{aligned}$$

7.20 Принцип причинности

Теорема 11 (Принцип причинности). При $\hat{t} < t$ выполнено:

$$\mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t \mid s_t) \gamma^{\hat{t}} r_{\hat{t}}] = 0.$$

Доказательство. Рассмотрим:

$$\mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t \mid s_t) \gamma^{\hat{t}} r_{\hat{t}}].$$

Так как $\hat{t} < t$, величина:

$$\gamma^{\hat{t}} r_{\hat{t}}$$

определяется прошлым относительно момента t .

Разделим математическое ожидание на прошлое и будущее:

$$\begin{aligned} &\mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t \mid s_t) \gamma^{\hat{t}} r_{\hat{t}}] \\ &= \mathbb{E}_{s_0, a_0, s_1, \dots, s_{\hat{t}}, a_{\hat{t}}} [\gamma^{\hat{t}} r_{\hat{t}} \cdot \mathbb{E}_{s_{\hat{t}+1}, a_{\hat{t}+1}, \dots, s_t, a_t, \dots} [\nabla_\theta \ln \pi_\theta(a_t \mid s_t)]] . \end{aligned}$$

Внутреннее ожидание равно нулю по предыдущей теореме:

$$\mathbb{E} [\nabla_\theta \ln \pi_\theta(a_t \mid s_t)] = 0.$$

Следовательно:

$$\mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t \mid s_t) \gamma^{\hat{t}} r_{\hat{t}}] = 0.$$

7.21 Убираем награды из прошлого

Из формулы, полученной вторым способом, можно убрать все слагаемые с:

$$\hat{t} < t.$$

После взятия математического ожидания они обратятся в ноль.

Кроме того, вычёркивание этих слагаемых уменьшает дисперсию при оценке градиента методом Монте-Карло.

Дисконтирование в среде идёт с самого начала, поэтому:

$$\sum_{\hat{t} \geq t} \gamma^{\hat{t}} r_{\hat{t}} = \gamma^t \sum_{\hat{t} \geq t} \gamma^{\hat{t}-t} r_{\hat{t}}.$$

Обозначим reward-to-go:

$$R_t(\tau) := \sum_{\hat{t} \geq t} \gamma^{\hat{t}-t} r_{\hat{t}}.$$

Тогда:

$$\sum_{\hat{t} \geq t} \gamma^{\hat{t}} r_{\hat{t}} = \gamma^t R_t(\tau).$$

7.22 Эквивалентная форма через reward-to-go

После применения принципа причинности получаем:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) R_t(\tau) \right].$$

Неформально:

$$R_t(\tau)$$

очень похож на Q -функцию, поскольку является её несмещённой Монте-Карло оценкой.

А так как в формуле всё равно берётся математическое ожидание, формулы из первого и второго способа должны совпадать.

7.23 Теорема об эквивалентности

Теорема 12. *Следующие формулы эквивалентны:*

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) Q^{\pi}(s_t, a_t) \right],$$

и

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) R_t(\tau) \right].$$

Доказательство. Начнём со второй формулы:

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t \mid s_t) R_t(\tau) \right] \\
&= \sum_{t \geq 0} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t \mid s_t) R_t(\tau)] \\
&= \sum_{t \geq 0} \mathbb{E}_{a_0, s_1, \dots, s_t, a_t} \mathbb{E}_{s_{t+1}, a_{t+1}, \dots} [\gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t \mid s_t) R_t(\tau)] .
\end{aligned}$$

Вынесем всё, что зависит только от истории до t , из внутреннего ожидания:

$$\nabla_{\theta} J(\theta) = \sum_{t \geq 0} \mathbb{E}_{a_0, s_1, \dots, s_t, a_t} [\gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t \mid s_t) \mathbb{E}_{s_{t+1}, a_{t+1}, \dots} [R_t(\tau)]] .$$

Но:

$$\mathbb{E}_{s_{t+1}, a_{t+1}, \dots} [R_t(\tau)] = Q^{\pi}(s_t, a_t) .$$

Следовательно:

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= \sum_{t \geq 0} \mathbb{E}_{a_0, s_1, \dots, s_t, a_t} [\gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t \mid s_t) Q^{\pi}(s_t, a_t)] \\
&= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t \mid s_t) Q^{\pi}(s_t, a_t) \right] .
\end{aligned}$$

Что и требовалось доказать.

7.24 Физический смысл Policy Gradient

Градиент функционала имеет вид градиента взвешенных логарифмов правдоподобий.

Чтобы это увидеть, рассмотрим суррогатную функцию — другой функционал, который в точке текущих параметров стратегии имеет такой же градиент, как и $J(\theta)$.

Определение 13. *Суррогатная функция:*

$$L_{\tilde{\pi}}(\theta) := \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t \geq 0} \gamma^t \ln \pi_{\theta}(a_t \mid s_t) Q^{\tilde{\pi}}(s_t, a_t) \right] .$$

Здесь есть две политики:

- π_{θ} — политика, которую мы оптимизируем;
- $\tilde{\pi}$ — политика, из которой берутся траектории и относительно которой считается $Q^{\tilde{\pi}}$.

Рассмотрим эту суррогатную функцию в точке θ , где:

$$\pi_{\theta} = \tilde{\pi} .$$

При этом будем менять только θ в π_{θ} , а распределение траекторий и Q -функцию будем считать замороженными.

То есть мы замораживаем:

$$p(\tau \mid \tilde{\pi})$$

и

$$Q^{\tilde{\pi}}(s, a) .$$

Теорема 13.

$$\nabla_{\theta} L_{\tilde{\pi}}(\theta)|_{\tilde{\pi}=\pi_{\theta}} = \nabla_{\theta} J(\theta).$$

Доказательство. Поскольку математическое ожидание по траекториям в суррогатной функции не зависит от θ , градиент можно пронести внутрь:

$$\nabla_{\theta} L_{\tilde{\pi}}(\theta)|_{\tilde{\pi}=\pi_{\theta}} = \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) Q^{\tilde{\pi}}(s_t, a_t) \right] \Big|_{\tilde{\pi}=\pi_{\theta}}.$$

В точке, где:

$$\pi_{\theta} = \tilde{\pi},$$

верно:

$$p(\tau | \tilde{\pi}) \equiv p(\tau | \pi_{\theta}),$$

и

$$Q^{\tilde{\pi}}(s, a) = Q^{\pi}(s, a).$$

Следовательно:

$$\nabla_{\theta} L_{\tilde{\pi}}(\theta)|_{\tilde{\pi}=\pi_{\theta}} = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) Q^{\pi}(s_t, a_t) \right] = \nabla_{\theta} J(\theta).$$

7.25 Интерпретация суррогатной функции

Направление максимизации $J(\theta)$ в текущей точке θ совпадает с направлением максимизации суррогатной функции:

$$L_{\tilde{\pi}}(\theta).$$

То есть в текущей точке можно считать, что мы как будто максимизируем логарифм правдоподобия пар:

$$(s, a),$$

где каждая пара дополнительно имеет вес:

$$Q^{\pi}(s, a).$$

Иными словами, Policy Gradient похож на взвешенное максимальное правдоподобие:

$$\sum_{(s,a)} \ln \pi_{\theta}(a | s) Q^{\pi}(s, a) \rightarrow \max_{\theta}.$$

7.26 Сравнение с supervised learning

В обычных задачах регрессии и классификации в машинном обучении есть выборка:

$$(x, y),$$

и мы максимизируем правдоподобие:

$$\sum_{(x,y)} \ln p(y | x, \theta) \rightarrow \max_{\theta}.$$

В RL готовой выборки нет. Агент сам сэмплирует себе:

$$s$$

как входные данные и:

$$a$$

как примеры выходных данных.

После этого каждой паре:

$$(s, a)$$

выдаётся некоторый кредит доверия, то есть скалярная оценка качества:

$$Q^\pi(s, a).$$

И агент идёт в направлении максимизации:

$$\sum_{(s,a)} \ln p(a \mid s, \theta) Q^\pi(s, a) \rightarrow \max_{\theta}.$$

Физический смысл. Policy Gradient увеличивает вероятность тех действий, которые в данных состояниях получили высокий кредит доверия, и уменьшает вероятность действий с плохой оценкой.

То есть:

$$Q^\pi(s, a) > 0$$

подталкивает политику делать действие a в состоянии s чаще, а маленькое или отрицательное значение $Q^\pi(s, a)$ делает это действие менее предпочтительным.

8 Лекция 8. REINFORCE, baseline, Actor-Critic, GAE и A2C

8.1 Оценка Policy Gradient методом Монте-Карло

Воспользуемся первой формулой для подсчёта Policy Gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) Q^{\pi}(s_t, a_t) \right].$$

В этой формуле заменим всё неизвестное на оценку по Монте-Карло.

8.1.1 Что будем заменять

Во-первых, математическое ожидание:

$$\mathbb{E}_{\tau \sim \pi_{\theta}}$$

заменим на несколько полных игр, сыгранных текущей стратегией π_{θ} .

Поскольку данные собираются текущей стратегией, алгоритм будет **on-policy**.

Во-вторых, истинную Q -функцию:

$$Q^\pi(s_t, a_t)$$

заменяем на reward-to-go:

$$R_t(\tau) = \sum_{\hat{t} \geq t} \gamma^{\hat{t}-t} r_{\hat{t}}.$$

То есть можно сказать, что мы воспользовались второй формой Policy Gradient с Монте-Карло оценкой математического ожидания по траекториям. Это естественно, потому что в предыдущей лекции было показано, что две формы эквивалентны.

8.2 REINFORCE

Алгоритм **REINFORCE** оптимизирует среднюю дисконтированную награду.

Гиперпараметры:

- N — количество игр;
- $\pi(a \mid s, \theta)$ — стратегия с параметрами θ ;
- SGD-оптимизатор.

Инициализация. Произвольно инициализируем параметры:

$$\theta.$$

На очередном шаге k :

1. Играем N игр текущей стратегией:

$$\tau_1, \tau_2, \dots, \tau_N \sim \pi_\theta.$$

2. Для каждого момента времени t в каждой траектории τ_i считаем reward-to-go:

$$R_t(\tau) := \sum_{\hat{t} \geq t} \gamma^{\hat{t}-t} r_{\hat{t}}.$$

3. Считаем оценку градиента:

$$\nabla_\theta J(\pi) := \frac{1}{N} \sum_{\tau} \sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) R_t(\tau).$$

4. Делаем шаг градиентного подъёма по θ , используя оценку:

$$\nabla_\theta J(\pi).$$

8.3 Недостатки REINFORCE

У базового REINFORCE есть два серьёзных недостатка.

1. Для одного шага градиентного подъёма нужно сыграть несколько игр до конца с помощью текущей стратегии.
2. Оценка градиента имеет колоссальную дисперсию.

На практике в сложных задачах дожидаться хороших результатов от такого алгоритма может быть невозможно.

8.4 Два типа стохастики

До этого часто рассматривались функционалы вида:

$$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t \geq 0} \gamma^t f(s_t, a_t) \right],$$

где f — некоторая функция от пар состояние–действие.

В MDP есть два вида стохастики:

1. Внешняя стохастика.

Она связана со случайностью в самой среде и неподконтрольна агенту.

Эта стохастика заложена в функции переходов:

$$p(s' \mid s, a).$$

2. Внутренняя стохастика.

Она связана со случайностью в стратегии самого агента.

Эта стохастика заложена в политике:

$$\pi(a \mid s).$$

В отличие от внешней стохастики, внутренняя стохастика подконтрольна нам при обучении.

Математическое ожидание:

$$\mathbb{E}_{\tau \sim \pi}$$

неудобно тем, что ожидания по внешней и внутренней стохастике чередуются.

При обучении из внешней стохастики мы можем получать только сэмплы. Поэтому полезно переписать функционал так, чтобы он выглядел как ожидание по распределению посещаемых состояний.

8.5 Марковская цепь состояний при фиксированной политике

Теорема 14. *Состояния, которые встречает агент со стратегией π , приходят из некоторой стационарной марковской цепи.*

Доказательство. Выпишем вероятность оказаться на очередном шаге в состоянии s' , если используется стратегия π :

$$p(s' | s) = \int_{\mathcal{A}} \pi(a | s) p(s' | s, a) da.$$

Эта вероятность не зависит от времени и от истории, а зависит только от текущего состояния s .

Следовательно, цепочка состояний образует марковскую цепь.

8.6 State visitation frequency

Пусть начальное состояние s_0 фиксировано.

Обозначим вероятность оказаться в состоянии s в момент времени t при использовании стратегии π как:

$$p(s_t = s | \pi).$$

Определение 14. Для данного MDP и политики π *state visitation frequency* называется величина:

$$\mu^\pi(s) := \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t \geq 0}^T p(s_t = s | \pi).$$

Это частотное распределение состояний, которые посещает агент при политике π .

8.7 Discounted state visitation distribution

Введём дисконтированный счётчик посещения состояний.

Определение 15. Для данного MDP и политики π *discounted state visitation distribution* называется:

$$d^\pi(s) := (1 - \gamma) \sum_{t \geq 0} \gamma^t p(s_t = s | \pi).$$

При дисконтировании отпадают проблемы с нормировкой.

Теорема 15. *State visitation distribution является распределением на множестве состояний:*

$$\int_{\mathcal{S}} d^\pi(s) ds = 1.$$

Доказательство.

$$\begin{aligned} \int_{\mathcal{S}} d^\pi(s) ds &= \int_{\mathcal{S}} (1 - \gamma) \sum_{t \geq 0} \gamma^t p(s_t = s | \pi) ds \\ &= (1 - \gamma) \sum_{t \geq 0} \gamma^t \int_{\mathcal{S}} p(s_t = s | \pi) ds \\ &= (1 - \gamma) \sum_{t \geq 0} \gamma^t \\ &= (1 - \gamma) \frac{1}{1 - \gamma} = 1. \end{aligned}$$

8.8 Переписывание ожидания через d^π

Теорема 16. Для произвольной функции $f(s, a)$ верно:

$$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t \geq 0} \gamma^t f(s_t, a_t) \right] = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [f(s, a)].$$

Доказательство.

$$\begin{aligned} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t \geq 0} \gamma^t f(s_t, a_t) \right] &= \sum_{t \geq 0} \gamma^t \mathbb{E}_{\tau \sim \pi} [f(s_t, a_t)] \\ &= \sum_{t \geq 0} \gamma^t \int_{\mathcal{S}} \int_{\mathcal{A}} p(s_t = s, a_t = a \mid \pi) f(s, a) da ds \\ &= \sum_{t \geq 0} \gamma^t \int_{\mathcal{S}} \int_{\mathcal{A}} p(s_t = s \mid \pi) \pi(a \mid s) f(s, a) da ds \\ &= \sum_{t \geq 0} \gamma^t \int_{\mathcal{S}} p(s_t = s \mid \pi) \mathbb{E}_{a \sim \pi(a|s)} [f(s, a)] ds \\ &= \int_{\mathcal{S}} \sum_{t \geq 0} \gamma^t p(s_t = s \mid \pi) \mathbb{E}_{a \sim \pi(a|s)} [f(s, a)] ds \\ &= \int_{\mathcal{S}} \frac{d^\pi(s)}{1 - \gamma} \mathbb{E}_{a \sim \pi(a|s)} [f(s, a)] ds \\ &= \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [f(s, a)]. \end{aligned}$$

8.9 Примеры использования d^π

Функционал средней дисконтированной награды можно переписать как:

$$J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [r(s, a)].$$

Policy Gradient можно переписать как:

$$\nabla_{\theta} J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [\nabla_{\theta} \log \pi_{\theta}(a \mid s) Q^{\pi}(s, a)].$$

8.10 Уменьшение дисперсии REINFORCE: baseline

8.10.1 Мотивация

При стохастической оптимизации ключевым фактором является дисперсия оценки градиента.

Когда математические ожидания заменяются Монте-Карло оценками, дисперсия увеличивается.

В REINFORCE замена Q -функции, то есть уже проинтегрированных будущих наград, на её Монте-Карло оценку:

$$R_t(\tau)$$

сильно повышает дисперсию.

Однако в текущем виде основной источник дисперсии связан ещё и с тем, что кредит, на который домножается градиент логарифма вероятности, не отцентрирован.

8.10.2 Причина большой дисперсии

Градиент логарифма правдоподобия в среднем равен нулю:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_\theta \log \pi_\theta(a | s)] = 0.$$

Это значит, что если в состоянии s политика задаёт распределение $\pi(a | s)$, то для увеличения вероятностей в одной области пространства действий нужно одни параметры увеличивать, а в другой области уменьшать.

В среднем направление изменения равно нулю.

Но в Монте-Карло оценке мы получаем только одно действие:

$$a \sim \pi(a | s),$$

и направление изменения домножается на кредит:

$$Q^\pi(s, a).$$

Если в одной области $Q^\pi(s, a)$ примерно равно 100, а в другой примерно 1000, то дисперсия величины:

$$\nabla_\theta \log \pi_\theta(a | s) Q^\pi(s, a)$$

может быть очень большой.

Вывод. Кредит нужно центрировать нулём.

8.11 Baseline

Теорема 17. Для произвольной функции:

$$b(s) : \mathcal{S} \rightarrow \mathbb{R},$$

называемой *baseline*, верно:

$$\nabla_\theta J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [\nabla_\theta \log \pi_\theta(a | s) (Q^\pi(s, a) - b(s))].$$

Доказательство. Нужно показать, что добавленное слагаемое равно нулю:

$$\mathbb{E}_{a \sim \pi(a|s)} [\nabla_\theta \log \pi_\theta(a | s) b(s)] = 0.$$

Так как $b(s)$ не зависит от действия a , его можно вынести из ожидания:

$$\begin{aligned} \mathbb{E}_{a \sim \pi(a|s)} [\nabla_\theta \log \pi_\theta(a | s) b(s)] &= b(s) \mathbb{E}_{a \sim \pi(a|s)} [\nabla_\theta \log \pi_\theta(a | s)] \\ &= b(s) \cdot 0 = 0. \end{aligned}$$

Следовательно, *baseline* не меняет математическое ожидание оценки градиента.

Замечание 20. *Baseline может быть произвольной функцией от состояния. Однако утверждение становится неверным, если baseline начинает зависеть от действия a .*

Baseline не меняет среднее значение оценки градиента, но меняет её дисперсию.

8.12 Оптимальный baseline

Теорема 18. *Baseline, максимально снижающий дисперсию Монте-Карло оценок градиента, имеет вид:*

$$b^*(s) := \frac{\mathbb{E}_a [\|\nabla_\theta \log \pi_\theta(a | s)\|_2^2 Q^\pi(s, a)]}{\mathbb{E}_a [\|\nabla_\theta \log \pi_\theta(a | s)\|_2^2]}.$$

Проблема. Практическая ценность результата невысока.

Чтобы использовать такую формулу, нужно знать норму градиента:

$$\|\nabla_\theta \log \pi_\theta(a | s)\|_2$$

для всех действий a .

Даже в дискретных пространствах действий это может быть вычислительно затратно.

8.13 Практический baseline и advantage

На практике часто предполагают слабую вариацию по действию нормы градиента:

$$\|\nabla_\theta \log \pi_\theta(a | s)\|_2^2 \approx \text{const}(a).$$

Тогда:

$$\begin{aligned} b^*(s) &= \frac{\mathbb{E}_a [\|\nabla_\theta \log \pi_\theta(a | s)\|_2^2 Q^\pi(s, a)]}{\mathbb{E}_a [\|\nabla_\theta \log \pi_\theta(a | s)\|_2^2]} \\ &\approx \mathbb{E}_a [Q^\pi(s, a)] = V^\pi(s). \end{aligned}$$

Итого получаем:

$$\nabla_\theta J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [\nabla_\theta \log \pi_\theta(a | s) A^\pi(s, a)].$$

Определение 16. *Для данного MDP advantage-функцией политики π называется:*

$$A^\pi(s, a) := Q^\pi(s, a) - V^\pi(s).$$

Интуитивно $A^\pi(s, a)$ показывает, насколько действие a в состоянии s лучше или хуже среднего действия по политике π .

8.14 Схемы Actor-Critic

Хотим оптимизировать параметры стратегии при помощи формулы градиента, но не хотим каждый раз доигрывать эпизоды до конца.

Для этого вводим вторую нейронную сеть, которая будет оценивать решения агента.

Определение 17. *Нейросеть, моделирующая стратегию:*

$$\pi_\theta(a | s),$$

называется *actor*, или *актор*.

Нейросеть, оценивающая решения актора, называется *critic*, или *критик*.

Алгоритмы, в которых обучается и actor, и critic, называются **Actor-Critic**.

Обычно в качестве критика учат именно V -функцию:

$$V_\phi(s).$$

Возможность не обучать сложную оптимальную Q^* -функцию является одним из преимуществ прямой оптимизации $J(\theta)$.

Из соображений эффективности:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V^\pi(s')] \approx r(s, a) + \gamma V_\phi(s'),$$

где:

$$s' \sim p(s' | s, a).$$

8.15 Actor-Critic и bias-variance trade-off

В формулу градиента будем подставлять некоторую оценку advantage-функции:

$$\Psi(s, a) \approx A^\pi(s, a).$$

Тогда:

$$\nabla_\theta J(\pi) \approx \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi(s)} \mathbb{E}_{a \sim \pi(a|s)} [\nabla_\theta \log \pi_\theta(a | s) \Psi(s, a)].$$

В контексте Policy Gradient речь напрямую идёт о дисперсии и смещении оценок градиента.

8.15.1 Два крайних варианта

$\Psi(s, a)$	Дисперсия	Смещение
$R_t - V_\phi(s)$	высокая	нет
$r(s, a) + \gamma V_\phi(s') - V_\phi(s)$	низкая	большое

Первый вариант использует Монте-Карло оценку до конца эпизода. Он почти не смещён, но имеет высокую дисперсию.

Второй вариант использует одношаговую TD-оценку. Он имеет низкую дисперсию, но может быть смещён из-за ошибки критика.

8.16 N-шаговая оценка Q -функции

Можно построить промежуточный вариант.

N-шаговая оценка Q -функции:

$$Q^\pi(s, a) \approx \sum_{t=0}^{N-1} \gamma^t r^{(t)} + \gamma^N V_\phi(s^{(N)}).$$

Для credit assignment N-шаговая оценка advantage, или N-шаговая временная разность, задаётся как:

$$\Psi^{(N)}(s, a) := \sum_{t=0}^{N-1} \gamma^t r^{(t)} + \gamma^N V_\phi(s^{(N)}) - V_\phi(s).$$

С ростом N дисперсия такой оценки увеличивается, потому что всё больший фрагмент траектории оценивается по Монте-Карло.

Нужно сэмплировать:

$$\begin{aligned} a_{t+1} &\sim \pi(a_{t+1} \mid s_{t+1}), \\ s_{t+2} &\sim p(s_{t+2} \mid s_{t+1}, a_{t+1}), \end{aligned}$$

и так далее до s_{t+N} .

8.17 Max trace estimation

Рассмотрим rollout:

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_N$$

длины N .

Определение 18. Для пары (s_t, a_t) из rollout оценкой максимальной длины, или *max trace estimation*, называется оценка с максимальным заглядыванием в будущее.

Для Q -функции:

$$y^{\text{MaxTrace}}(s_t, a_t) := \sum_{\hat{t}=t}^{N-1} \gamma^{\hat{t}-t} r_{\hat{t}} + \gamma^{N-t} V^{\pi}(s_N).$$

Для *advantage*-функции:

$$\Psi^{\text{MaxTrace}}(s_t, a_t) := y^{\text{MaxTrace}}(s_t, a_t) - V^{\pi}(s_t).$$

8.18 Generalized Advantage Estimation

Решение дилеммы bias-variance trade-off подсказывает теория TD(λ)-оценки.

Идея: заансамблировать N -шаговые оценки разной длины.

Определение 19. GAE-оценкой *advantage*-функции называется ансамбль многошаговых оценок, где оценка длины N берётся с весом λ^{N-1} :

$$\Psi^{\text{GAE}}(s, a) := (1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} \Psi^{(N)}(s, a),$$

где:

$$\lambda \in (0, 1)$$

— гиперпараметр.

При:

$$\lambda = 0$$

GAE соответствует одношаговой оценке.

При:

$$\lambda \rightarrow 1$$

GAE соответствует Монте-Карло оценке Q -функции.

8.19 GAE для оборванного rollout

В формуле выше суммируются все N -шаговые оценки вплоть до конца эпизода.

На практике собранные rollout могут прерваться в середине эпизода.

Пусть для данной пары (s, a) через M шагов rollout обрывается. Если известно состояние:

$$s^{(M)},$$

но эпизод ещё не доигран до конца, используется TD(λ)-форма с доступными слагаемыми:

$$\Psi^{\text{GAE}}(s, a) := \sum_{t=0}^{M-1} \gamma^t \lambda^t \Psi^{(1)}(s^{(t)}, a^{(t)}).$$

Теорема 19. Эта формула эквивалентна следующему ансамблю N -шаговых оценок:

$$\Psi^{\text{GAE}}(s, a) = (1 - \lambda) \sum_{N=1}^{M-1} \lambda^{N-1} \Psi^{(N)}(s, a) + \lambda^{M-1} \Psi^{(M)}(s, a).$$

В такой обрезанной оценке:

$$\lambda = 1$$

соответствует оценке максимальной длины, а:

$$\lambda = 0$$

всё ещё даёт одношаговую оценку.

8.20 Рекурсивный подсчёт GAE

В коде TD(λ)-форма удобна для рекурсивного подсчёта.

Для практического алгоритма нужно также учитывать флаги:

$$done_t.$$

Пусть rollout имеет вид:

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_N.$$

Одношаговая advantage-оценка:

$$\Psi^{(1)}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

Тогда GAE для последнего перехода:

$$\Psi^{\text{GAE}}(s_{N-1}, a_{N-1}) := \Psi^{(1)}(s_{N-1}, a_{N-1}).$$

Для предыдущего:

$$\Psi^{\text{GAE}}(s_{N-2}, a_{N-2}) := \Psi^{(1)}(s_{N-2}, a_{N-2}) + \gamma \lambda (1 - done_{N-2}) \Psi^{\text{GAE}}(s_{N-1}, a_{N-1}).$$

И далее рекурсивно:

$$\Psi^{\text{GAE}}(s_t, a_t) := \Psi^{(1)}(s_t, a_t) + \gamma \lambda (1 - done_t) \Psi^{\text{GAE}}(s_{t+1}, a_{t+1}).$$

В частности:

$$\Psi^{\text{GAE}}(s_0, a_0) := \Psi^{(1)}(s_0, a_0) + \gamma \lambda (1 - done_0) \Psi^{\text{GAE}}(s_1, a_1).$$

Эти формулы похожи на расчёт кумулятивной награды за эпизод, где наградой за шаг выступает:

$$\Psi^{(1)}(s, a).$$

8.21 Обучение критика

Для обучения критика воспользуемся идеей перехода к регрессии, как раньше в DQN.

Нужно методом простой итерации решать уравнение Беллмана:

$$V_{\phi_{k+1}}(s) \leftarrow \mathbb{E}_a [r + \gamma \mathbb{E}_{s'} [V_{\phi_k}(s')]].$$

Поскольку работаем в on-policy режиме, можно поступить так же, как с оценкой Q -функции в формуле градиента: решать многошаговое уравнение Беллмана вместо одношагового.

Например, можно выбрать N -шаговое уравнение и построить целевую переменную:

$$y := r + \gamma r' + \gamma^2 r'' + \dots + \gamma^N V_{\phi_k}(s^{(N)}).$$

8.22 Связь таргета критика и advantage

Если advantage оценивается как:

$$\Psi(s, a) = y - V_{\phi}(s),$$

где y — некоторая оценка Q -функции, то этот же y является target для V -функции.

Используя MSE с таким target, мы учим среднее значение наших оценок Q -функции, то есть baseline.

Если используется GAE-оценка advantage, достаточно убрать baseline:

$$Q^{\pi}(s, a) = A^{\pi}(s, a) + V^{\pi}(s) \approx \Psi^{\text{GAE}}(s, a) + V_{\phi}(s).$$

Значит target для критика можно брать как:

$$y_{\text{critic}}(s, a) = \Psi^{\text{GAE}}(s, a) + V_{\phi}(s).$$

8.23 GAE-target для критика

Теорема 20. *Target:*

$$\Psi^{\text{GAE}}(s, a) + V_{\phi}(s)$$

является несмещённой оценкой правой части ансамбля уравнений Беллмана:

$$V_{\phi}(s) = (1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} [B^N V_{\phi}](s),$$

где B — оператор Беллмана для V -функции.

Доказательство. По определению:

$$\Psi^{(N)}(s, a) + V(s)$$

является несмещённой оценкой правой части N -шагового уравнения Беллмана, то есть несмещённой оценкой:

$$[B^N V^{\pi}](s).$$

Тогда:

$$\begin{aligned} (1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} (\Psi^{(N)}(s, a) + V(s)) &= (1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} \Psi^{(N)}(s, a) + V(s) \\ &= \Psi^{\text{GAE}}(s, a) + V(s). \end{aligned}$$

8.24 Общий шаг Actor-Critic

Делаем несколько шагов взаимодействия со средой, собирая rollout некоторой длины N .

Для каждой пары (s, a) считаем некоторую оценку Q -функции:

$$y(s, a).$$

Например, можно использовать оценку максимальной длины:

$$y^{\text{MaxTrace}}(s_t, a_t) = \sum_{i=t}^{N-1} \gamma^{i-t} r_i + \gamma^{N-t} V_\phi(s_N).$$

Затем оцениваем advantage:

$$\Psi(s, a) := y(s, a) - V_\phi(s).$$

Далее по Монте-Карло оцениваем градиент по параметрам стратегии:

$$\nabla_\theta J(\pi) \approx \frac{1}{N} \sum_{s,a} \nabla_\theta \log \pi_\theta(a | s) \Psi(s, a).$$

Для критика, если критик является V -функцией, используем:

$$\text{Loss}_{\text{critic}}(\phi) = \frac{1}{N} \sum_{s,a} (y(s, a) - V_\phi(s))^2.$$

8.25 Advantage Actor-Critic

Advantage Actor-Critic, или **A2C**, использует actor и critic, а также advantage-оценку для обновления политики.

Гиперпараметры:

- M — количество параллельных сред;
- N — длина rollout;
- $V_\phi(s)$ — нейросеть-критик с параметрами ϕ ;
- $\pi_\theta(a | s)$ — нейросеть-стратегия с параметрами θ ;
- α — коэффициент масштабирования функции потерь критика;
- SGD-оптимизатор.

Инициализация. Инициализируем параметры:

$$(\theta, \phi).$$

На каждом шаге:

1. В каждой из M параллельных сред собрать rollout длины N , используя стратегию π_θ :

$$s_0, a_0, r_0, s_1, \dots, s_N.$$

2. Для каждой пары (s_t, a_t) из каждого rollout посчитать оценку Q -функции максимальной длины, игнорируя зависимость оценки от ϕ :

$$Q(s_t, a_t) := \sum_{\hat{t}=t}^{N-1} \gamma^{\hat{t}-t} r_{\hat{t}} + \gamma^{N-t} V_\phi(s_N).$$

3. Вычислить функцию потерь критика:

$$\text{Loss}_{\text{critic}}(\phi) := \frac{1}{MN} \sum_{s_t, a_t} (Q(s_t, a_t) - V_\phi(s_t))^2.$$

4. Сделать шаг градиентного спуска по ϕ , используя:

$$\nabla_\phi \text{Loss}_{\text{critic}}(\phi).$$

5. Вычислить градиент для актора:

$$\nabla_\theta^{\text{actor}} := \frac{1}{MN} \sum_{s_t, a_t} \nabla_\theta \log \pi_\theta(a_t | s_t) (Q(s_t, a_t) - V_\phi(s_t)).$$

6. Сделать шаг градиентного подъёма по θ , используя:

$$\nabla_\theta^{\text{actor}}.$$

8.26 Actor-Critic: комментарии

Интересно, что идея model-free алгоритмов, которые отказываются обучать модель функции переходов, в итоге всё равно часто приводит к обучению промежуточных оценочных функций.

То есть вместо прямого связывания состояний и действий:

$$s \mapsto a$$

мы учим промежуточную величину:

$$V_\phi(s)$$

или другие оценочные функции.

Интуитивно может казаться, что обучать актора проще, чем оценочную функцию. В реальных задачах человек редко думает в терминах будущей награды.

Однако есть теоретические наблюдения, что эти задачи близки по сложности.

8.27 Сходство градиентов актора и критика

Для актора градиент имеет вид:

$$\nabla_{\theta} \log \pi_{\theta}(a \mid s) \Psi(s, a).$$

Для критика, если продифференцировать MSE:

$$(y(s, a) - V_{\phi}(s))^2,$$

получается градиент вида:

$$\nabla_{\phi} V_{\phi}(s) \Psi(s, a),$$

с точностью до знака и констант.

То есть эти градиенты похожи по смыслу.

Они оба говорят: увеличить соответствующий выход нейронной сети, а кредит предоставляет скаляр:

$$\Psi(s, a),$$

который задаёт масштаб изменения и, главное, знак.

8.28 Пример с кофеваркой

Представим, что человек стоит перед кофеваркой и у него есть два действия:

- получить кофе;
- не получить кофе.

Человек хорошо понимает, какое действие лучше.

Но при этом он не обязан явно оценивать, сколько кофе сможет выпить в будущем при условии, что сейчас выберет первое или второе действие.

То есть человек не обязательно строит явный прогноз значения оценочной функции:

$$Q(s, a).$$

Это иллюстрирует интуицию, что иногда policy learning может казаться более естественным, чем value learning.

8.29 Пример: A2C и редкий успешный опыт

Рассмотрим видеоигру.

В начале обучения агент почти ничего не умеет, действует примерно случайно и всё время падает в первую яму.

Среда всё время выдаёт нулевую награду, и агент продолжает вести себя случайно.

В какой-то момент из-за стохастичности стратегии агент перепрыгивает первую яму и получает монетку:

$$+1.$$

В DQN этот ценный опыт будет сохранён в replay buffer. Постепенно критик сможет выучить, какие действия привели к награде.

В A2C агент делает один небольшой шаг изменения весов моделей и затем выбросит собранные данные, потому что на следующих итерациях он не может переиспользовать их.

Агенту придётся ждать ещё много сессий в игре, пока он снова перепрыгнет яму, чтобы сделать следующий шаг обучения.

Вывод. On-policy actor-critic методы могут быть менее sample-efficient, чем off-policy методы с replay buffer, особенно в задачах с редкими наградами.

9 Лекция 9. TRPO, trust region и натуральный градиент

9.1 Напоминание: Policy Gradient

В предыдущих лекциях рассматривалась оптимизация функционала:

$$J(\pi_\theta) := \mathbb{E}_{T \sim \pi_\theta} \left[\sum_{t \geq 0} \gamma^t r_t \right].$$

Для Policy Gradient была получена оценка:

$$\nabla_\theta J(\pi_\theta) \approx \mathbb{E}_{T \sim \pi_\theta} \left[\sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t | s_t) (Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)) \right].$$

Здесь:

$$Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t) = A^{\pi_\theta}(s_t, a_t),$$

где A^{π_θ} — advantage-функция.

То есть:

$$\nabla_\theta J(\pi_\theta) \approx \mathbb{E}_{T \sim \pi_\theta} \left[\sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t | s_t) A^{\pi_\theta}(s_t, a_t) \right].$$

В этой формуле данные порождены той же политикой, которую мы оптимизируем:

$$T \sim \pi_\theta.$$

Поэтому такая оценка является **on-policy**.

9.2 Можно ли получить более эффективный Policy Gradient?

В форме через discounted state visitation distribution градиент можно записать так:

$$\nabla_\theta J(\pi_\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_\theta}(s)} \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_\theta \log \pi_\theta(a | s) A^{\pi_\theta}(s, a)].$$

Проблема: данные в этой формуле должны быть порождены текущей политикой:

$$\pi_\theta.$$

Предположим теперь, что:

- мы хотим оптимизировать текущую политику π_θ ;
- есть данные, то есть траектории, порождённые старой политикой:

$$\pi^{\text{old}};$$

- значит, мы можем оценить ожидания:

$$\mathbb{E}_{s \sim d^{\pi^{\text{old}}}(s)}, \quad \mathbb{E}_{a \sim \pi^{\text{old}}(a|s)};$$

- можем обучить:

$$V^{\pi^{\text{old}}}(s),$$

и, следовательно, оценить:

$$A^{\pi^{\text{old}}}(s, a).$$

Идея TRPO: использовать эти старые данные и при этом оптимизировать политику эффективнее, чем обычным SGD.

9.3 Соединяя две политики

Попробуем связать функционалы для новой политики π_θ и старой политики π^{old} .

Идея: изменить награду с помощью функции ценности другой политики.

Используем телескопическую сумму.

Для произвольной последовательности a_t с условием:

$$\lim_{t \rightarrow \infty} a_t = 0$$

верно:

$$\sum_{t \geq 0}^{\infty} (a_{t+1} - a_t) = -a_0.$$

Применим это к последовательности:

$$a_t = \gamma^t V^\pi(s_t).$$

Тогда для произвольной траектории:

$$T := s_0, a_0, s_1, a_1, \dots$$

и произвольной политики π получаем:

$$-V^\pi(s_0) = \sum_{t \geq 0} (\gamma^{t+1} V^\pi(s_{t+1}) - \gamma^t V^\pi(s_t)).$$

9.4 Relative Performance Identity

Теорема 21 (Relative Performance Identity). *Для любых двух политик π_θ и π^{old} верно:*

$$V^{\pi_\theta}(s) - V^{\pi^{\text{old}}}(s) = \mathbb{E}_{T \sim \pi_\theta | s_0 = s} \left[\sum_{t \geq 0} \gamma^t A^{\pi^{\text{old}}}(s_t, a_t) \right].$$

Доказательство. Начнём с разности функций ценности:

$$V^{\pi_\theta}(s) - V^{\pi^{\text{old}}}(s) = \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t r_t \right] - V^{\pi^{\text{old}}}(s).$$

Так как $s = s_0$, можно записать:

$$V^{\pi_\theta}(s) - V^{\pi^{\text{old}}}(s) = \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t r_t - V^{\pi^{\text{old}}}(s_0) \right].$$

По телескопической формуле:

$$-V^{\pi^{\text{old}}}(s_0) = \sum_{t \geq 0} \left[\gamma^{t+1} V^{\pi^{\text{old}}}(s_{t+1}) - \gamma^t V^{\pi^{\text{old}}}(s_t) \right].$$

Подставим:

$$\begin{aligned} & V^{\pi_\theta}(s) - V^{\pi^{\text{old}}}(s) \\ &= \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t r_t + \sum_{t \geq 0} \left(\gamma^{t+1} V^{\pi^{\text{old}}}(s_{t+1}) - \gamma^t V^{\pi^{\text{old}}}(s_t) \right) \right] \\ &= \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t \left(r_t + \gamma V^{\pi^{\text{old}}}(s_{t+1}) - V^{\pi^{\text{old}}}(s_t) \right) \right]. \end{aligned}$$

Теперь воспользуемся свойством:

$$\mathbb{E}_x f(x) = \mathbb{E}_x \mathbb{E} f(x),$$

то есть заменим значение $V^{\pi^{\text{old}}}(s_{t+1})$ на ожидание по следующему состоянию:

$$\begin{aligned} & V^{\pi_\theta}(s) - V^{\pi^{\text{old}}}(s) \\ &= \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t \left(r_t + \gamma \mathbb{E}_{s_{t+1}} \left[V^{\pi^{\text{old}}}(s_{t+1}) \right] - V^{\pi^{\text{old}}}(s_t) \right) \right]. \end{aligned}$$

По определению Q -функции старой политики:

$$Q^{\pi^{\text{old}}}(s_t, a_t) = r_t + \gamma \mathbb{E}_{s_{t+1}} \left[V^{\pi^{\text{old}}}(s_{t+1}) \right].$$

Значит:

$$\begin{aligned} V^{\pi_\theta}(s) - V^{\pi^{\text{old}}}(s) &= \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t \left(Q^{\pi^{\text{old}}}(s_t, a_t) - V^{\pi^{\text{old}}}(s_t) \right) \right] \\ &= \mathbb{E}_{T \sim \pi_\theta | s_0=s} \left[\sum_{t \geq 0} \gamma^t A^{\pi^{\text{old}}}(s_t, a_t) \right]. \end{aligned}$$

Что и требовалось доказать.

9.5 Меняем целевую функцию

Из Relative Performance Identity можно получить выражение для изменения функционала:

$$J(\pi_\theta) - J(\pi^{\text{old}}).$$

Можно записать:

$$J(\pi_\theta) - J(\pi^{\text{old}}) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_\theta}(s)} \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[A^{\pi^{\text{old}}}(s, a) \right].$$

Плюс этой формулы:

$$A^{\pi^{\text{old}}}(s, a)$$

можно оценить по старым данным, потому что это advantage старой политики.

Минус: ожидания берутся по новой политике:

$$d^{\pi_\theta}(s), \quad \pi_\theta(a | s).$$

Для ожидания по действиям:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)}$$

можно использовать выборку по значимости.

Но остаётся проблема:

$$\mathbb{E}_{s \sim d^{\pi_\theta}(s)}.$$

Мы не знаем, как сэмплировать состояния из d^{π_θ} , не запуская новую политику.

9.6 Суррогатный функционал

Введём суррогатный функционал, где вместо распределения состояний новой политики используется распределение состояний старой политики:

$$J(\pi_\theta) - J(\pi^{\text{old}}) \approx L_{\pi^{\text{old}}}(\theta).$$

Определим:

$$L_{\pi^{\text{old}}}(\theta) := \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi^{\text{old}}}(s)} \mathbb{E}_{a \sim \pi^{\text{old}}(a|s)} \left[\frac{\pi_\theta(a | s)}{\pi^{\text{old}}(a | s)} A^{\pi^{\text{old}}}(s, a) \right].$$

Здесь:

$$\frac{\pi_\theta(a | s)}{\pi^{\text{old}}(a | s)}$$

— correction ratio, то есть коэффициент выборки по значимости.

С этим функционалом уже можно работать, потому что:

- данные порождены старой политикой π^{old} ;
- не требуется свежий критик;
- advantage старой политики можно оценить на старых данных.

9.7 Интерпретация суррогатного функционала

Суррогатный функционал указывает направление улучшения старой политики.

Если бы мы оптимизировали θ при фиксированной π^{old} , то политика училась бы выбирать действие:

$$\arg \max_{a \in \mathcal{A}} A^{\pi^{\text{old}}}(s, a).$$

Если оптимизировать θ на фиксированных данных, то идеальная политика стала бы:

$$\pi_{\theta}(a \mid s) = 1,$$

если $A^{\pi^{\text{old}}}(s, a)$ максимально, и:

$$\pi_{\theta}(a \mid s) = 0$$

иначе.

То есть суррогат действительно толкает новую политику в сторону действий, которые были лучше среднего по старой политике.

9.8 Теоретическая граница ошибки

Хотим оптимизировать не исходный функционал:

$$J(\pi_{\theta}) - J(\pi^{\text{old}}),$$

а его приближение:

$$L_{\pi^{\text{old}}}(\theta).$$

Возникает вопрос: насколько мы ошибаемся?

Теорема 22. *Верна оценка:*

$$|J(\pi_{\theta}) - J(\pi^{\text{old}}) - L_{\pi^{\text{old}}}(\theta)| \leq C \text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta}),$$

где:

$$\text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta}) := \max_{s \in \mathcal{S}} \text{KL}(\pi^{\text{old}}(\cdot \mid s) \parallel \pi_{\theta}(\cdot \mid s)),$$

а:

$$C := \frac{4\gamma \max_{s \in \mathcal{S}, a \in \mathcal{A}} |A^{\pi^{\text{old}}}(s, a)|}{(1 - \gamma)^2}.$$

Из этой теоремы следует нижняя оценка:

$$J(\pi_{\theta}) - J(\pi^{\text{old}}) \geq L_{\pi^{\text{old}}}(\theta) - C \text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta}).$$

9.9 Алгоритм миноризации-максимизации

Получили вариационную нижнюю оценку для нашего функционала:

$$J(\pi_{\theta}) - J(\pi^{\text{old}}) \geq L_{\pi^{\text{old}}}(\theta) - C \text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta}).$$

Это приводит к схеме **minorization-maximization**.

Миноризация. Построить новую нижнюю оценку для функционала.

В нашем случае это означает обновить старую политику:

$$\pi^{\text{old}} \leftarrow \pi_{\theta}.$$

Максимизация. Оптимизировать нижнюю оценку:

$$L_{\pi^{\text{old}}}(\theta) - C \text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta})$$

произвольным способом.

Смысл. Если точно оптимизировать эту нижнюю оценку, то можно получить гарантию монотонного улучшения.

9.10 Практические трудности нижней оценки

Формально можно поставить задачу:

$$L_{\pi^{\text{old}}}(\theta) - C \text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta}) \rightarrow \max_{\theta}.$$

Но на практике возникают проблемы:

- критик неидеален;
- приходится использовать тот critic, который есть;
- работать с $\text{KL}_{\max}$ неудобно;
- поэтому $\text{KL}_{\max}$ заменяют на средний KL:

$$\mathbb{E}_s \text{KL}(\pi^{\text{old}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s));$$

- константа C неизвестна;
- её можно сделать гиперпараметром;
- но на деле она часто получается очень большой.

Из-за этого прямой штраф:

$$-C \text{KL}_{\max}(\pi^{\text{old}} \parallel \pi_{\theta})$$

может сделать оптимизацию неудобной.

9.11 Идея TRPO

Вместо штрафа за большое изменение политики будем явно ограничивать изменение политики.

Идея:

оптимизировать $L_{\pi^{\text{old}}}(\theta)$ при условии, что π_{θ} не изменится сильно относительно π^{old} .

То есть ставится задача:

$$\begin{cases} L_{\pi^{\text{old}}}(\theta) \rightarrow \max_{\theta}, \\ \mathbb{E}_{s \sim d^{\pi^{\text{old}}}(s)} \text{KL}(\pi^{\text{old}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)) \leq \delta. \end{cases}$$

Здесь:

$$L_{\pi^{\text{old}}}(\theta)$$

— модель функционала, а условие:

$$\mathbb{E}_{s \sim d^{\pi^{\text{old}}}(s)} \text{KL}(\pi^{\text{old}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)) \leq \delta$$

— доверительная область этой модели.

Алгоритм **TRPO**, или **Trust Region Policy Optimization**, приблизительно решает эту задачу.

9.12 Разложение Тейлора для модели и условия

TRPO использует разложение Тейлора для целевой функции и ограничения.

Обозначим:

$$\begin{aligned} d &:= \theta - \theta^{\text{old}}, \\ g &:= \nabla_{\theta} L_{\pi^{\text{old}}}(\theta) \big|_{\theta = \theta^{\text{old}}}. \end{aligned}$$

Линейное приближение суррогатного функционала:

$$L_{\pi^{\text{old}}}(\theta) \approx g^T d.$$

Значит локальная задача максимизации:

$$g^T d \rightarrow \max_d.$$

Для KL-дивергенции первое слагаемое в разложении равно нулю, поэтому используется разложение второго порядка:

$$\mathbb{E}_{s \sim d^{\pi^{\text{old}}}(s)} \text{KL}(\pi^{\text{old}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)) \approx \frac{1}{2} d^T H d \leq \delta.$$

Здесь H — гессиан KL-дивергенции по параметрам политики.

Получаем приближённую задачу:

$$\begin{cases} g^T d \rightarrow \max_d, \\ \frac{1}{2} d^T H d \leq \delta. \end{cases}$$

9.13 Натуральный градиентный подъём

Решение приближённой задачи имеет направление:

$$d \propto H^{-1} g.$$

Тогда обновление параметров имеет вид:

$$\theta_{k+1} = \theta_k + \alpha H^{-1} g.$$

Это называется **натуральный градиентный подъём**.

Он похож на методы второго порядка, потому что в обновлении участвует матрица:

$$H,$$

то есть матрица второго порядка для KL-дивергенции.

9.14 Как обращать матрицу H

Пусть:

$$h$$

— размерность параметров θ .

Тогда:

$$H \in \mathbb{R}^{h \times h}$$

— большая матрица.

Явно обращать её дорого или невозможно.

Вместо обращения матрицы решают систему линейных алгебраических уравнений:

$$Hd = g.$$

Для этого используется метод сопряжённых градиентов.

9.14.1 Схема

1. Вычислить g , как в обычном градиентном спуске.
2. Инициализировать d_0 произвольно.
3. Для:

$$i = 0, 1, 2, \dots, M$$

метод сопряжённых градиентов приближённо вычисляет d_{i+1} с помощью g и произведения:

$$Hd_i.$$

4. С помощью линейного поиска найти шаг:

$$\alpha_k.$$

5. Обновить параметры:

$$\theta_{k+1} = \theta_k + \alpha_k d_M.$$

Важная деталь: для метода сопряжённых градиентов не обязательно явно хранить H ; достаточно уметь вычислять произведение:

$$Hd_i.$$

9.15 Лине́йный поиск

После того как найдено направление:

$$d_k,$$

нужно определить шаг:

$$\alpha_k.$$

Обновление:

$$\theta_{k+1} = \theta_k + \alpha_k d_k.$$

Шаг α_k связан с параметром доверительной области:

$$\delta.$$

На практике используется backtracking line search: уменьшаем α_k , пока не выполняются условия:

$$L_{\pi^{\text{old}}}(\theta_{k+1}) > 0,$$

и:

$$\mathbb{E}_s \text{KL}(\pi^{\text{old}}(\cdot | s) \| \pi_{\theta_{k+1}}(\cdot | s)) \leq \delta.$$

Например, можно уменьшать α_k вдвое, пока условия не станут выполнены.

9.16 Trust Region Policy Optimization

Итак, TRPO решает приближённую задачу:

$$\begin{cases} L_{\pi^{\text{old}}}(\theta) \rightarrow \max_{\theta}, \\ \text{KL}(\pi^{\text{old}} \| \pi_{\theta}) \leq \delta. \end{cases}$$

Плюсы.

- Метод робастен.
- Ограничение на KL предотвращает слишком большие изменения политики.
- Это снижает риск того, что после одного большого шага политика резко испортится.

Минусы.

- Актора и критика не получится просто архитектурно скрестить в одну стандартную схему обучения.
- Метод вычислительно дорогой.
- Имплементация сложная.

9.17 Trust region: пространство параметров или пространство распределений

В обычном градиентном спуске шаг часто ограничивается в пространстве параметров, например через евклидову норму:

$$\|d\|_2 \leq \delta.$$

Но для вероятностных моделей важнее не то, насколько изменились параметры, а то, насколько изменилось распределение.

Рассмотрим две пары гауссиан:

$$\mathcal{N}(0, 0.2), \quad \mathcal{N}(1, 0.2),$$

и:

$$\mathcal{N}(0, 10), \quad \mathcal{N}(1, 10).$$

В обоих случаях евклидово расстояние между параметрами среднего одинаковое:

$$|\mu_1 - \mu_2| = 1.$$

Если учитывать также стандартные отклонения, евклидово расстояние параметров можно записать как:

$$\sqrt{(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2}.$$

Но статистическое отличие распределений в этих двух примерах совсем разное.

Для узких гауссиан изменение среднего на 1 резко меняет распределение.

Для широких гауссиан изменение среднего на 1 меняет распределение гораздо слабее.

Поэтому для trust region в policy optimization разумнее измерять расстояние в пространстве распределений, например через:

$$\text{KL}(\mathcal{N}(\mu_1, \sigma_1) \parallel \mathcal{N}(\mu_2, \sigma_2)),$$

а не только через евклидово расстояние между параметрами.

9.18 Проблема большого шага: скатывание с обрыва

Обычный policy-gradient шаг имеет вид:

$$\theta_{k+1} = \theta_k + \alpha \hat{g}_k.$$

Если шаг слишком велик, возникают проблемы:

- большие шаги могут привести к плохой политике;
- после этого данные начинают собираться уже плохой политикой;
- качество может коллапсировать.

Поэтому нужен метод, который измеряет расстояние не в пространстве параметров, а в пространстве распределений политик.

TRPO делает именно это через ограничение на KL-дивергенцию.

9.19 Общая схема методов с доверительной областью

Рассмотрим задачу оптимизации:

$$G(\theta) := G(q_\theta(x)) \rightarrow \min_{\theta}.$$

Общая схема методов оптимизации с доверительной областью:

1. Начать с произвольного:

$$\theta_0.$$

2. Для:

$$k = 0, 1, 2, \dots$$

выполнять:

(a) Построить модель, то есть локальную аппроксимацию $G(\theta)$ около θ_k :

$$m_k(\theta) \approx G(\theta).$$

(b) Выбрать область доверия, где модель считается достаточно точной:

$$r(\theta, \theta_k) \leq \delta.$$

(c) Найти θ_{k+1} как решение задачи:

$$\begin{cases} m_k(\theta) \rightarrow \min_{\theta}, \\ r(\theta, \theta_k) \leq \delta. \end{cases}$$

9.20 Градиентный спуск и натуральный градиентный спуск

Пусть снова:

$$G(\theta) := G(q_\theta(x)) \rightarrow \min_{\theta}.$$

9.20.1 Обычный градиентный спуск

Локальная линейная модель:

$$G(\theta_k + d) \approx G(\theta_k) + \nabla_{\theta} G(\theta_k)^T d.$$

Обычный градиентный спуск ограничивает шаг в евклидовой норме:

$$\begin{cases} G(\theta_k) + \nabla_{\theta} G(\theta_k)^T d \rightarrow \min_d, \\ \|d\|_2 \leq \delta. \end{cases}$$

Решение имеет направление:

$$d \propto -\nabla_{\theta} G(\theta_k).$$

9.20.2 Натуральный градиентный спуск

В натуральном градиентном спуске ограничение задаётся не в пространстве параметров, а в пространстве распределений:

$$\begin{cases} G(\theta_k) + \nabla_{\theta} G(\theta_k)^T d \rightarrow \min_d, \\ \text{KL}(q_{\theta_k}(x) \| q_{\theta_k+d}(x)) \leq \delta. \end{cases}$$

То есть шаг ограничивается тем, насколько изменилось распределение:

$$q_{\theta}(x).$$

9.21 Информационная матрица Фишера

Используем разложение Тейлора для ограничения:

$$\text{KL}(q_{\theta_k}(x) \| q_{\theta_k+d}(x)) \leq \delta.$$

Первый член в разложении равен нулю:

$$\nabla_d \text{KL}(q_{\theta_k}(x) \| q_{\theta_k+d}(x))|_{d=0} = 0.$$

Определение 20. Пусть $F(\theta_k)$ — информационная матрица Фишера распределения $q_{\theta}(x)$ в точке θ_k :

$$F(\theta_k) := -\mathbb{E}_{x \sim q_{\theta}(x)} [\nabla_{\theta}^2 \log q_{\theta}(x)].$$

Тогда приближение KL-дивергенции второго порядка:

$$\text{KL}(q_{\theta_k}(x) \| q_{\theta_k+d}(x)) \approx \frac{1}{2} d^T F(\theta_k) d.$$

9.22 Направление натурального градиента

После разложения Тейлора получаем задачу:

$$\begin{cases} G(\theta_k) + \nabla_{\theta} G(\theta_k)^T d \rightarrow \min_d, \\ d^T F(\theta_k) d \leq \delta. \end{cases}$$

Решение имеет направление:

$$d \propto -F^{-1}(\theta_k) \nabla_{\theta} G(\theta_k).$$

Поэтому метод натурального градиента обновляет параметры так:

$$\theta_{k+1} = \theta_k - \alpha F^{-1}(\theta_k) \nabla_{\theta} G(\theta_k),$$

где α определяется выбором доверительной области:

$$\delta.$$

9.23 Связь натурального градиента и TRPO

TRPO можно понимать как применение идеи натурального градиента к policy optimization.

В обычной оптимизации можно ограничивать шаг так:

$$\|d\|_2 \leq \delta.$$

Но в policy optimization параметры θ задают распределение действий:

$$\pi_\theta(a \mid s).$$

Поэтому естественнее ограничивать не:

$$\|\theta - \theta^{\text{old}}\|_2,$$

а статистическое расстояние между политиками:

$$\text{KL}(\pi^{\text{old}}(\cdot \mid s) \parallel \pi_\theta(\cdot \mid s)).$$

Именно это и делает TRPO: строит локальную модель улучшения политики и оптимизирует её внутри доверительной области по KL.

9.24 Краткая схема TRPO

Одна итерация TRPO концептуально выглядит так:

1. Собрать данные старой политикой:

$$\pi^{\text{old}}.$$

2. Оценить:

$$A^{\pi^{\text{old}}}(s, a).$$

3. Построить суррогат:

$$L_{\pi^{\text{old}}}(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi^{\text{old}}}} \mathbb{E}_{a \sim \pi^{\text{old}}} \left[\frac{\pi_\theta(a \mid s)}{\pi^{\text{old}}(a \mid s)} A^{\pi^{\text{old}}}(s, a) \right].$$

4. Найти направление натурального градиентного подъёма:

$$d \approx H^{-1}g.$$

5. Подобрать шаг α с помощью backtracking line search так, чтобы:

$$L_{\pi^{\text{old}}}(\theta_{k+1}) > 0,$$

и:

$$\mathbb{E}_s \text{KL}(\pi^{\text{old}}(\cdot \mid s) \parallel \pi_{\theta_{k+1}}(\cdot \mid s)) \leq \delta.$$

6. Обновить параметры:

$$\theta_{k+1} = \theta_k + \alpha d.$$

7. Положить:

$$\pi^{\text{old}} \leftarrow \pi_{\theta_{k+1}},$$

и повторить.

9.25 Итог

Главная идея лекции: обычный Policy Gradient является on-policy и может быть неэффективным, а слишком большие шаги по параметрам политики могут разрушить обучение.

TRPO решает эту проблему через:

- суррогатный функционал, использующий данные старой политики;
- коррекцию выборкой по значимости:

$$\frac{\pi_{\theta}(a | s)}{\pi^{\text{old}}(a | s)};$$

- ограничение на изменение политики через KL-дивергенцию;
- натуральный градиентный шаг в пространстве распределений.

Смысл trust region в RL: не позволять новой политике слишком резко отличаться от старой, потому что иначе агент начнёт собирать данные плохой политикой, и качество обучения может коллапсировать.

10 Лекция 10. PPO, clipping, TD(λ), GAE и Retrace

10.1 Proximal Policy Optimization: общая идея

В предыдущей лекции рассматривался TRPO. Его идея состояла в том, чтобы оптимизировать суррогатный функционал старой политики, но не давать новой политике слишком далеко уйти от старой.

Суррогатный функционал имел вид:

$$L_{\pi^{\text{old}}}(\theta) = \mathbb{E}_{s \sim d^{\pi^{\text{old}}}(s)} \mathbb{E}_{a \sim \pi^{\text{old}}(a|s)} \left[\frac{\pi_{\theta}(a | s)}{\pi^{\text{old}}(a | s)} A^{\pi^{\text{old}}}(s, a) \right].$$

В TRPO использовалось ограничение на изменение политики:

$$\text{KL}(\pi^{\text{old}} \| \pi_{\theta}) \leq \delta.$$

Или, в штрафной форме, можно было рассматривать оптимизацию:

$$L_{\pi^{\text{old}}}(\theta) - C \text{KL}(\pi^{\text{old}} \| \pi_{\theta}) \rightarrow \max_{\theta}.$$

Proximal Policy Optimization, или **PPO**, сохраняет ту же общую идею:

не давать новой политике слишком сильно отличаться от старой.

Но вместо сложного ограничения через trust region и натуральный градиент PPO использует более простой механизм — **клиппирование**.

10.2 PPO: отличие от TRPO в пайплайне

TRPO концептуально делает одно большое constrained optimization обновление на данных, собранных старой политикой:

$$\pi^{\text{old}}.$$

PPO устроен практичнее:

1. Собираем батч данных старой политикой:

$$\pi^{\text{old}}.$$

2. Фиксируем для этого батча:

$$\pi^{\text{old}}(a_t \mid s_t), \quad V^{\text{old}}(s_t).$$

3. Несколько эпох оптимизируем actor и critic на этом же батче с помощью обычного SGD/Adam.
4. При этом ограничиваем изменение политики через клипированный objective.

То есть PPO позволяет переиспользовать собранный батч несколько раз:

several epochs of SGD optimization.

Это делает PPO гораздо проще в реализации, чем TRPO.

10.3 Суррогатный функционал PPO

Введём отношение вероятностей новой и старой политики:

$$\rho(\theta) := \frac{\pi_\theta(a \mid s)}{\pi^{\text{old}}(a \mid s)}.$$

Тогда обычный суррогатный функционал можно записать как:

$$L_{\pi^{\text{old}}}(\theta) := \mathbb{E}_{s,a} \left[\rho(\theta) A^{\pi^{\text{old}}}(s, a) \right],$$

где ожидание берётся по данным, собранным старой политикой:

$$s \sim d^{\pi^{\text{old}}}(s), \quad a \sim \pi^{\text{old}}(a \mid s).$$

Если:

$$\rho(\theta) > 1,$$

то новая политика увеличила вероятность действия a в состоянии s относительно старой политики.

Если:

$$\rho(\theta) < 1,$$

то новая политика уменьшила вероятность этого действия.

10.4 Клиппирование отношения вероятностей

PPO ограничивает отношение вероятностей:

$$\rho(\theta) = \frac{\pi_{\theta}(a \mid s)}{\pi^{\text{old}}(a \mid s)}$$

на интервале:

$$[1 - \varepsilon, 1 + \varepsilon].$$

Определим клиппированное отношение:

$$\rho^{\text{clip}}(\theta) := \text{clip}(\rho(\theta), 1 - \varepsilon, 1 + \varepsilon).$$

То есть:

$$\rho^{\text{clip}}(\theta) = \begin{cases} 1 - \varepsilon, & \rho(\theta) < 1 - \varepsilon, \\ \rho(\theta), & 1 - \varepsilon \leq \rho(\theta) \leq 1 + \varepsilon, \\ 1 + \varepsilon, & \rho(\theta) > 1 + \varepsilon. \end{cases}$$

Клиппированный суррогат можно было бы записать как:

$$L_{\pi^{\text{old}}}^{\text{clip}}(\theta) := \mathbb{E}_{s,a} \left[\rho^{\text{clip}}(\theta) A^{\pi^{\text{old}}}(s, a) \right].$$

Однако в PPO используется не просто клиппированный суррогат, а минимум исходного и клиппированного членов.

10.5 PPO clipped objective

Основной actor-objective в PPO:

$$L^{\text{PPO}}(\theta) := \mathbb{E}_{s,a} \left[\min \left(\rho(\theta) A^{\pi^{\text{old}}}(s, a), \rho^{\text{clip}}(\theta) A^{\pi^{\text{old}}}(s, a) \right) \right].$$

Иногда дополнительно записывают KL-регуляризатор:

$$\mathbb{E}_{s,a} \left[\min \left(\rho(\theta) A^{\pi^{\text{old}}}(s, a), \rho^{\text{clip}}(\theta) A^{\pi^{\text{old}}}(s, a) \right) \right] - C \text{KL}(\pi^{\text{old}} \parallel \pi_{\theta}) \rightarrow \max_{\theta}.$$

Но в стандартном PPO часто KL-регуляризатор не используют явно. Главным ограничителем выступает клиппирование.

10.6 Интуиция clipped objective

Рассмотрим знак advantage.

10.6.1 Случай положительного advantage

Пусть:

$$A^{\pi^{\text{old}}}(s, a) \geq 0.$$

Это означает, что действие a в состоянии s было лучше среднего по старой политике.

Тогда мы хотим увеличить вероятность этого действия:

$$\pi_{\theta}(a \mid s) \uparrow .$$

То есть хотим увеличить:

$$\rho(\theta).$$

Но если:

$$\rho(\theta) > 1 + \varepsilon,$$

то вероятность уже выросла слишком сильно. Дальнейший рост не должен поощряться.

В этом случае:

$$\rho^{\text{clip}}(\theta) = 1 + \varepsilon.$$

Поскольку advantage положительный, минимум выбирает клипированный член:

$$\min(\rho A, \rho^{\text{clip}} A) = (1 + \varepsilon) A.$$

Этот член больше не зависит от θ через ρ , поэтому градиент по actor становится нулевым.

Если же:

$$\rho(\theta) < 1 - \varepsilon,$$

то вероятность хорошего действия стала слишком маленькой. В этом случае градиент остаётся таким же, как у исходного суррогата, и политика будет подталкиваться обратно.

10.6.2 Случай отрицательного advantage

Пусть:

$$A^{\pi^{\text{old}}}(s, a) < 0.$$

Это означает, что действие a хуже среднего по старой политике.

Тогда мы хотим уменьшить вероятность этого действия:

$$\pi_{\theta}(a \mid s) \downarrow .$$

То есть хотим уменьшить:

$$\rho(\theta).$$

Но если:

$$\rho(\theta) < 1 - \varepsilon,$$

то вероятность плохого действия уже уменьшилась слишком сильно. Дальнейшее уменьшение не должно поощряться.

Так как advantage отрицательный, минимум работает наоборот: выбирается тот член, который даёт более консервативное значение.

В результате при слишком сильном уменьшении вероятности плохого действия градиент обнуляется.

10.7 Таблица поведения clipped objective

Для типичного значения:

$$\varepsilon = 0.2$$

интервал клиппирования:

$$[0.8, 1.2].$$

Поведение можно резюмировать так:

Знак advantage	Желаемое направление	Значение $\rho(\theta)$	Градиент
$A^{\pi^{\text{old}}}(s, a) \geq 0$	$\pi_{\theta}(a s) \uparrow$	$\rho(\theta) > 1.2$	0
$A^{\pi^{\text{old}}}(s, a) \geq 0$	$\pi_{\theta}(a s) \uparrow$	$\rho(\theta) < 0.8$	тот же
$A^{\pi^{\text{old}}}(s, a) < 0$	$\pi_{\theta}(a s) \downarrow$	$\rho(\theta) > 1.2$	тот же
$A^{\pi^{\text{old}}}(s, a) < 0$	$\pi_{\theta}(a s) \downarrow$	$\rho(\theta) < 0.8$	0

Смысл. PPO разрешает политике двигаться в полезную сторону, но только пока изменение вероятности действия не стало слишком большим.

Как только политика слишком сильно изменила вероятность конкретного действия, градиент по этому примеру обнуляется.

10.8 Клиппированная функция потерь критика

В PPO часто клиппируют не только actor-objective, но и critic-loss.

Обычная функция потерь критика:

$$\text{Loss}(\phi) := (y(s, a) - V_{\phi}(s))^2.$$

Здесь:

$$y(s, a)$$

— целевое значение для регрессии.

Перепишем ошибку через старое значение критика:

$$\begin{aligned} \text{Loss}(\phi) &= (y(s, a) - V_{\phi}(s))^2 \\ &= (y(s, a) - V^{\text{old}}(s) + V^{\text{old}}(s) - V_{\phi}(s))^2. \end{aligned}$$

Клиппированная версия ограничивает изменение предсказания критика относительно старого значения:

$$\text{Loss}^{\text{clip}}(\phi) := (y(s, a) - V^{\text{old}}(s) + \text{clip}(V^{\text{old}}(s) - V_{\phi}(s), -\hat{\varepsilon}, \hat{\varepsilon}))^2.$$

В реализации часто используют максимум из двух функций потерь:

$$\max(\text{Loss}(\phi), \text{Loss}^{\text{clip}}(\phi)).$$

Интуиция. Клиппирование критика запрещает $V_{\phi}(s)$ слишком резко уходить от старого значения:

$$V^{\text{old}}(s).$$

Это стабилизирует обучение, потому что critic используется для вычисления advantage и targets.

10.9 Компромисс между смещением и разбросом

Пусть дана траектория по политике π :

$$s, r, s', r', s'', r'', \dots, s^{(M)}.$$

Пусть есть приближение:

$$V^\pi(s).$$

Нужно оценить advantage для пары (s, a) , то есть понять:

хорошее ли решение было принято?

Для actor используется градиент вида:

$$\nabla := \rho(\theta) \nabla_\theta \log \pi_\theta(a | s) \Psi(s, a),$$

где:

$$\Psi(s, a)$$

— оценка advantage.

Для critic целевое значение для регрессии:

$$y_Q := \Psi(s, a) + V(s).$$

Разные оценки $\Psi(s, a)$ дают разный компромисс между смещением и разбросом.

10.9.1 Монте-Карло оценка

$$\Psi^{(\infty)}(s, a) := r + \gamma r' + \gamma^2 r'' + \dots - V(s).$$

У неё:

смещение = 0, разброс высокий.

10.9.2 N-шаговая оценка

$$\Psi^{(N)}(s, a) := r + \gamma r' + \dots + \gamma^N V(s^{(N)}) - V(s).$$

У неё:

смещение промежуточное, разброс промежуточный.

10.9.3 Одношаговая оценка

$$\Psi^{(1)}(s, a) := r + \gamma V(s') - V(s).$$

У неё:

смещение высокое, разброс низкий.

Проблема. Нужно выбирать:

$$N.$$

Чем больше N , тем меньше bootstrapping через V , но больше шум Монте-Карло.

Чем меньше N , тем меньше дисперсия, но больше зависимость от ошибки критика.

10.10 Взгляд назад: идея TD(λ)

Рассмотрим N-шаговое обновление:

$$V(s) \leftarrow V(s) + \alpha \Psi^{(N)}(s, a).$$

Одношаговая TD-невязка:

$$\Psi^{(1)}(s, a) = r + \gamma V(s') - V(s).$$

Для следующего перехода:

$$\Psi^{(1)}(s', a') = r' + \gamma V(s'') - V(s').$$

Посмотрим, как из одношаговых невязок получить двухшаговую.

Сумма:

$$\Psi^{(1)}(s, a) + \gamma \Psi^{(1)}(s', a')$$

равна:

$$\begin{aligned} & (r + \gamma V(s') - V(s)) + \gamma (r' + \gamma V(s'') - V(s')) \\ &= r + \gamma r' + \gamma^2 V(s'') - V(s). \end{aligned}$$

Это ровно двухшаговая невязка:

$$\Psi^{(2)}(s, a) = r + \gamma r' + \gamma^2 V(s'') - V(s).$$

В общем случае:

$$\Psi^{(N)}(s, a) = \sum_{t=0}^N \gamma^t \Psi^{(1)}(s^{(t)}, a^{(t)}).$$

Замечание 21. В некоторых соглашениях сумма для N-шаговой оценки пишется до $N-1$. Конкретный индекс зависит от того, считается ли $\Psi^{(1)}$ первым шагом. Важна сама идея: многошаговая невязка раскладывается в дисконтированную сумму одношаговых TD-невязок.

10.11 Eligibility Traces

Идея eligibility traces: использовать текущую одношаговую TD-ошибку для обновления $V(s)$ не только в текущем состоянии, но и во всех состояниях, которые недавно посещались.

Введём eligibility trace:

$$e(s).$$

Это скалярный коэффициент в обновлении:

$$\forall s : \quad V(s) \leftarrow V(s) + \alpha e(s) \Psi^{(1)}.$$

10.11.1 Онлайн Монте-Карло обновления

Для варианта, похожего на онлайн Монте-Карло:

1. В начале каждого эпизода:

$$\forall s : \quad e(s) := 0.$$

2. После посещения состояния s :

$$e(s) \leftarrow e(s) + 1.$$

3. После каждого шага:

$$\forall s : e(s) \leftarrow \gamma e(s).$$

Интуиция. Если состояние посещалось недавно, его след $e(s)$ большой. Значит текущая TD-ошибка сильно обновит оценку этого состояния.

Если состояние посещалось давно, след успел затухнуть, и обновление будет слабым.

10.12 TD(1)

Алгоритм TD(1):

1. Вход: политика π .

2. Инициализировать $V(s)$ произвольно.

3. Инициализировать:

$$e(s) = 0.$$

4. Наблюдать начальное состояние:

$$s_0.$$

5. Для:

$$k = 0, 1, 2, \dots$$

повторять:

(a) Выбрать действие:

$$a_k \sim \pi.$$

(b) Пронаблюдать:

$$r_k, \quad s_{k+1}.$$

(c) Посчитать одношаговую TD-невязку:

$$\Psi^{(1)} := r_k + \gamma V(s_{k+1}) - V(s_k).$$

(d) Увеличить след текущего состояния:

$$e(s_k) \leftarrow e(s_k) + 1.$$

(e) Обновить все состояния:

$$\forall s : V(s) \leftarrow V(s) + \alpha e(s) \Psi^{(1)}.$$

(f) Затухание следов:

$$\forall s : e(s) \leftarrow \gamma e(s).$$

10.13 TD(0)

TD(0) можно записать в том же виде через eligibility traces, но после каждого шага все следы обнуляются:

$$\forall s : e(s) \leftarrow 0 \cdot \gamma e(s).$$

То есть алгоритм TD(0):

1. Вход: политика π .

2. Инициализировать $V(s)$ произвольно.

3. Инициализировать:

$$e(s) = 0.$$

4. Наблюдать:

$$s_0.$$

5. Для:

$$k = 0, 1, 2, \dots$$

повторять:

(a) Выбрать:

$$a_k \sim \pi.$$

(b) Пронаблюдать:

$$r_k, \quad s_{k+1}.$$

(c) Посчитать:

$$\Psi^{(1)} := r_k + \gamma V(s_{k+1}) - V(s_k).$$

(d) Увеличить:

$$e(s_k) \leftarrow e(s_k) + 1.$$

(e) Обновить:

$$\forall s : V(s) \leftarrow V(s) + \alpha e(s) \Psi^{(1)}.$$

(f) Обнулить следы:

$$\forall s : e(s) \leftarrow 0.$$

Смысл. TD(0) обновляет фактически только текущее состояние.

TD(1) распространяет текущую ошибку на всю недавнюю историю.

10.14 TD(λ)

TD(λ) интерполирует между TD(0) и TD(1).

Алгоритм:

1. Вход: политика π .

2. Инициализировать $V(s)$ произвольно.

3. Инициализировать:

$$e(s) = 0.$$

4. Наблюдать:

$$s_0.$$

5. Для:

$$k = 0, 1, 2, \dots$$

повторять:

(a) Выбрать действие:

$$a_k \sim \pi.$$

(b) Пронаблюдать:

$$r_k, \quad s_{k+1}.$$

(c) Посчитать одношаговую TD-невязку:

$$\Psi^{(1)} := r_k + \gamma V(s_{k+1}) - V(s_k).$$

(d) Увеличить след текущего состояния:

$$e(s_k) \leftarrow e(s_k) + 1.$$

(e) Обновить все состояния:

$$\forall s : \quad V(s) \leftarrow V(s) + \alpha e(s) \Psi^{(1)}.$$

(f) Затупить следы:

$$\forall s : \quad e(s) \leftarrow \lambda \gamma e(s).$$

При:

$$\lambda = 0$$

получаем TD(0).

При:

$$\lambda = 1$$

получаем TD(1).

10.15 Взгляд вперёд и взгляд назад

TD(λ) можно понимать двумя эквивалентными способами.

10.15.1 Взгляд вперёд

Взгляд вперёд означает:

дать оценку настоящему по известному будущему.

То есть мы спрашиваем:

хорошо ли решение или плохо на основе результата?

В этом подходе мы смотрим на будущие награды и будущие значения V .

10.15.2 Взгляд назад

Взгляд назад означает:

обновить прошлые оценки с помощью настоящей информации.

То есть мы спрашиваем:

какие решения в прошлом оказали значительное влияние на настоящее?

Eligibility traces — это именно взгляд назад.

10.16 Взгляд вперёд для TD(λ)

Для пары (s, a) можно рассмотреть сумму одношаговых TD-невязок с коэффициентами:

$$(\gamma\lambda)^t.$$

Тогда:

$$\sum_{t=0}^{\infty} (\gamma\lambda)^t \Psi^{(1)}(s^{(t)}, a^{(t)})$$

эквивалентна взвешенному ансамблю N -шаговых оценок:

$$(1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} \Psi^{(N)}(s, a).$$

То есть:

$$\sum_{t=0}^{\infty} (\gamma\lambda)^t \Psi^{(1)}(s^{(t)}, a^{(t)}) = (1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} \Psi^{(N)}(s, a).$$

Смысл. TD(λ) не выбирает одно конкретное N .

Вместо этого он смешивает все N -шаговые оценки:

$$\Psi^{(1)}, \Psi^{(2)}, \Psi^{(3)}, \dots$$

с геометрически убывающими весами:

$$1 - \lambda, \quad (1 - \lambda)\lambda, \quad (1 - \lambda)\lambda^2, \dots$$

10.17 Generalized Advantage Estimation

Пусть для пары (s, a) нам известно будущее только на T шагов вперёд.

Тогда используем обрезанную сумму:

$$\Psi^{\text{GAE}}(s, a) := \sum_{t=0}^T (\gamma\lambda)^t \Psi^{(1)}(s^{(t)}, a^{(t)}).$$

Это и есть практическая форма **Generalized Advantage Estimation**, или **GAE**.

10.17.1 Одношаговая невязка

Для момента времени t :

$$\Psi^{(1)}(s_t, a_t) = r_t + \gamma(1 - done_{t+1})V_\phi(s_{t+1}) - V_\phi(s_t).$$

Флаг $done_{t+1}$ нужен, чтобы обнулить bootstrap через $V_\phi(s_{t+1})$, если эпизод завершился.

10.17.2 Рекурсивная формула

На практике GAE удобно считать назад по rollout:

$$\Psi^{\text{GAE}}(s_t, a_t) = \Psi^{(1)}(s_t, a_t) + \lambda\gamma(1 - done_{t+1})\Psi^{\text{GAE}}(s_{t+1}, a_{t+1}).$$

Для последнего перехода rollout:

$$\Psi^{\text{GAE}}(s_{N-1}, a_{N-1}) := \Psi^{(1)}(s_{N-1}, a_{N-1}).$$

Затем для:

$$t = N - 2, N - 3, \dots, 0$$

считается:

$$\Psi^{\text{GAE}}(s_t, a_t) := \Psi^{(1)}(s_t, a_t) + \lambda\gamma(1 - done_{t+1})\Psi^{\text{GAE}}(s_{t+1}, a_{t+1}).$$

10.18 GAE в Advantage Actor-Critic

Длинные развёртки дают богатые GAE-ансамбли.

Но в A2C rollout обычно короткие.

Поэтому часто выбирают:

$$\lambda = 1.$$

Тогда GAE превращается в оценку максимальной длины внутри rollout, которую иногда называют:

max-trace estimation.

10.19 PPO: особенности реализации

Ключевые элементы PPO:

- клиппинг функции потерь политики;
- клиппинг функции потерь критика;
- GAE.

Другие важные практические приёмы:

- нормализация и клиппинг награды;
- нормализация и клиппинг наблюдений, особенно в задачах непрерывного управления;

- ортогональная инициализация слоёв;
- отжиг параметра клиппинга ϵ ;
- нормализация advantage в мини-батчах;
- энтропийная функция потерь;
- оптимизатор Adam;
- отжиг learning rate;
- функция активации tanh;
- клиппинг градиента.

Нормализация награды. Один из вариантов: делить награды на скользящее стандартное отклонение собранных кумулятивных наград.

Нормализация advantage. В каждом мини-батче:

$$\Psi^{\text{GAE}} \leftarrow \frac{\Psi^{\text{GAE}} - \text{mean}(\Psi^{\text{GAE}})}{\text{std}(\Psi^{\text{GAE}})}.$$

Это стабилизирует масштаб actor-gradient.

10.20 PPO: полный алгоритм

Инициализировать actor:

$$\pi(a \mid s, \theta)$$

и critic:

$$V_{\phi}(s).$$

Для:

$$k = 0, 1, 2, \dots$$

повторять.

10.20.1 Сбор rollout

Собрать rollout длины N по текущей политике:

$$s_0, a_0, r_0, s_1, done_1, a_1, \dots, s_N, done_N.$$

При сборе данных сохранить вероятности действий старой политики:

$$\pi^{\text{old}}(a_t \mid s_t) := \pi(a_t \mid s_t, \theta).$$

Также сохранить старые значения критика:

$$V^{\text{old}}(s_t) := V_{\phi}(s_t).$$

10.20.2 Одношаговые TD-невязки

Вычислить:

$$\Psi^{(1)}(s_t, a_t) := r_t + \gamma(1 - done_{t+1})V_\phi(s_{t+1}) - V_\phi(s_t).$$

10.20.3 GAE advantage

Инициализировать для последнего перехода:

$$\Psi^{\text{GAE}}(s_{N-1}, a_{N-1}) := \Psi^{(1)}(s_{N-1}, a_{N-1}).$$

Для:

$$t = N - 2, N - 3, \dots, 0$$

посчитать:

$$\Psi^{\text{GAE}}(s_t, a_t) := \Psi^{(1)}(s_t, a_t) + \lambda\gamma(1 - done_{t+1})\Psi^{\text{GAE}}(s_{t+1}, a_{t+1}).$$

10.20.4 Целевые значения критика

Целевые значения критика:

$$y(s_t) := \Psi^{\text{GAE}}(s_t, a_t) + V_\phi(s_t).$$

Составить выборку:

$$(s_t, a_t, \Psi^{\text{GAE}}(s_t, a_t), y(s_t), \pi^{\text{old}}(a_t | s_t), V^{\text{old}}(s_t)).$$

10.20.5 Оптимизация по мини-батчам

Пройти по выборке n_{epochs} раз, сэмплируя мини-батчи размера B .

Для каждого мини-батча:

1. Нормализовать advantage в мини-батче:

$$\Psi^{\text{GAE}} \leftarrow \frac{\Psi^{\text{GAE}} - \text{mean}(\Psi^{\text{GAE}})}{\text{std}(\Psi^{\text{GAE}})}.$$

2. Вычислить веса выборки по значимости:

$$\rho(s, a, \theta) := \frac{\pi(a | s, \theta)}{\pi^{\text{old}}(a | s)}.$$

3. Вычислить клипированные веса:

$$\rho^{\text{clip}}(s, a, \theta) := \text{clip}(\rho(s, a, \theta), 1 - \varepsilon, 1 + \varepsilon).$$

4. Определить два actor-члена:

$$L_1(s, a, \theta) := \rho(s, a, \theta)\Psi^{\text{GAE}}(s, a),$$

$$L_2(s, a, \theta) := \rho^{\text{clip}}(s, a, \theta)\Psi^{\text{GAE}}(s, a).$$

5. Обновить actor:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \frac{1}{B} \sum_{s,a} \min(L_1(s, a, \theta), L_2(s, a, \theta)).$$

6. Для critic посчитать:

$$\text{Loss}_1(s, \phi) := (y(s) - V_{\phi}(s))^2.$$

7. Посчитать клипированную critic-loss:

$$\text{Loss}_2(s, \phi) := (y(s) - V^{\text{old}}(s) - \text{clip}(V_{\phi}(s) - V^{\text{old}}(s), -\hat{\epsilon}, \hat{\epsilon}))^2.$$

8. Обновить critic:

$$\phi \leftarrow \phi - \alpha \nabla_{\phi} \frac{1}{B} \sum_s \max(\text{Loss}_1(s, \phi), \text{Loss}_2(s, \phi)).$$

10.21 Источники по PPO и GAE

В лекции упоминались следующие источники:

- *Proximal Policy Optimization Algorithms*;
- *Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO*;
- *High-Dimensional Continuous Control Using Generalized Advantage Estimation*;
- Sutton, Barto — *Reinforcement Learning: An Introduction*, глава 12.

10.22 Value-based методы и Policy Gradient

Сравним value-based методы, такие как DQN, и policy-gradient методы.

Value-based методы	Policy Gradient
Off-policy: можно использовать replay buffer.	On-policy: данные батча часто становятся бесполезны после обновления параметров.
Обучается оптимальная функция: $Q^*(s, a)$. Это сложный промежуточный шаг.	Политика обучается напрямую. Обычно достаточно обучать $V^{\pi}(s)$, что проще.
Есть проблемы с исследованием среды, потому что value iteration работает с детерминированными жадными политиками.	Есть более естественное исследование за счёт сэмплирования из $\pi(a s)$.
Обычно используются одношаговые целевые значения.	Используются многошаговые или бесконечношаговые целевые значения. Можно применять GAE и для актора, и для критика.

10.23 Off-policy оценка advantage

Пусть дана траектория:

$$s_0, r_0, s_1, r_1, s_2, r_2, \dots, s_M$$

по политике поведения:

$$\mu.$$

Пусть требуется оценить advantage для целевой политики:

$$\pi.$$

То есть:

$$\mu \neq \pi.$$

Есть приближение:

$$V^\pi(s),$$

и нужно выполнить credit assignment для пары:

$$(s_0, a_0)$$

в off-policy режиме.

Хотелось бы воспользоваться GAE:

$$\sum_{t \geq 0} (\gamma \lambda)^t \Psi^{(1)}(s_t, a_t).$$

Но проблема в том, что каждое слагаемое:

$$\Psi^{(1)}(s_t, a_t)$$

зависит от случайных величин:

$$a_0, s_0, a_1, s_1, \dots, s_{t+1},$$

которые были порождены не целевой политикой π , а поведением μ .

Замечание 22. Если:

$$\pi(a_0 \mid s_0) = 0,$$

то корректно оценить ценность действия a_0 для политики π по такой траектории не получится: целевая политика никогда бы это действие не выбрала.

10.24 Коррекция выборкой по значимости

Для off-policy оценки можно использовать importance sampling.

Для траектории под μ корректирующий множитель до момента t :

$$\prod_{\hat{t}=1}^t \frac{\pi(a_{\hat{t}} \mid s_{\hat{t}})}{\mu(a_{\hat{t}} \mid s_{\hat{t}})}.$$

Тогда оценка:

$$\Psi = \sum_{t \geq 0} (\gamma \lambda)^t \left(\prod_{\hat{t}=1}^t \frac{\pi(a_{\hat{t}} \mid s_{\hat{t}})}{\mu(a_{\hat{t}} \mid s_{\hat{t}})} \right) \Psi^{(1)}(s_t, a_t).$$

По соглашению:

$$\prod_{\hat{t}=1}^0 \equiv 1.$$

Если записать correction ratio с динамикой среды, то получится:

$$\prod_{\hat{t}=1}^t \frac{\pi(a_{\hat{t}} | s_{\hat{t}}) p(s_{\hat{t}+1} | s_{\hat{t}}, a_{\hat{t}})}{\mu(a_{\hat{t}} | s_{\hat{t}}) p(s_{\hat{t}+1} | s_{\hat{t}}, a_{\hat{t}})}.$$

Функции переходов сокращаются:

$$p(s_{\hat{t}+1} | s_{\hat{t}}, a_{\hat{t}}),$$

и остаётся только отношение политик:

$$\frac{\pi(a_{\hat{t}} | s_{\hat{t}})}{\mu(a_{\hat{t}} | s_{\hat{t}})}.$$

10.25 Проблемы importance sampling

На практике такая оценка непрактична из-за очень высокой дисперсии.

10.25.1 Затухающий след

Если:

$$\mu(a | s) \gg \pi(a | s),$$

то:

$$\frac{\pi(a | s)}{\mu(a | s)}$$

мал.

Такой множитель быстро зануляет вклад будущих слагаемых.

Типичная ситуация: поведенческая политика μ делает примитивные случайные действия, которые целевая политика π почти никогда не совершает.

С этим ничего хорошего сделать нельзя: если целевая политика сама почти не делает такие действия, траектория действительно малоинформативна для оценки π .

10.25.2 Взрывающийся след

Если:

$$\mu(a | s) \ll \pi(a | s),$$

то:

$$\frac{\pi(a | s)}{\mu(a | s)}$$

может быть огромным.

Это происходит, когда μ выбрала действие с малой вероятностью под μ , но это действие вероятно для π .

Именно это является причиной большой дисперсии.

10.26 Общий вид присвоения ценности

Перепишем оценку advantage в общем виде:

$$\Psi = \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t c_i \right) \Psi^{(1)}(s_t, a_t),$$

где:

$$\prod_{i=1}^0 \equiv 1.$$

Коэффициенты:

$$c_i$$

— это коэффициенты отжига следа.

Разные методы соответствуют разным c_i .

Название оценки	Коэффициенты c_i	Проблема
GAE	λ	только on-policy
Одношаговая	0	большое смещение
Выборка по значимости	$\lambda \frac{\pi(a_i s_i)}{\mu(a_i s_i)}$	легко взрывается

10.27 Retrace: основная теорема

Общая форма:

$$\Psi = \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t c_i \right) \Psi^{(1)}(s_t, a_t), \quad \prod_{i=1}^0 \equiv 1.$$

Теорема 23 (Retrace). В on-policy режиме можно выбирать произвольные коэффициенты:

$$c_i \in [0, 1].$$

В off-policy режиме можно выбирать произвольные коэффициенты из интервала:

$$c_i \in \left[0, \frac{\pi(a_i | s_i)}{\mu(a_i | s_i)} \right].$$

Следствия.

- С затухающим следом ничего не подделаешь: если отношение вероятностей маленькое, траектория действительно плохо подходит для целевой политики.
- Взрывающийся след можно контролировать: если вес importance sampling больше 1, его можно клиппировать.

10.28 Retrace: финальный результат

Retrace выбирает:

$$c_i := \lambda \min \left(1, \frac{\pi(a_i | s_i)}{\mu(a_i | s_i)} \right).$$

Тогда:

$$\Psi = \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t c_i \right) \Psi^{(1)}(s_t, a_t), \quad \prod_{i=1}^0 \equiv 1.$$

Смысл. Retrace похож на off-policy версию GAE, но с безопасными коэффициентами следа.

Если отношение:

$$\frac{\pi(a_i | s_i)}{\mu(a_i | s_i)}$$

меньше 1, оно используется как есть.

Если оно больше 1, оно обрезается до 1.

Дополнительно весь след масштабируется параметром:

$$\lambda.$$

10.29 Где используется Retrace

Retrace используется:

- в off-policy RL алгоритмах для теоретически корректных многошаговых целевых значений;
- часто с $\lambda = 1$, потому что след всё равно быстро затухает из-за клипированных importance ratios;
- в распределённых on-policy RL системах, где данные о градиенте от некоторых серверов могут задерживаться на несколько итераций обновления.

10.30 Итог

В этой лекции были рассмотрены три связанные идеи.

РРО. РРО упрощает TRPO. Вместо сложного trust-region constrained optimization используется клипированный суррогат:

$$\mathbb{E} [\min (\rho A, \rho^{\text{clip}} A)].$$

Он запрещает политике слишком сильно менять вероятность действий на одном батче данных.

TD(λ) и GAE. TD(λ) и GAE решают проблему выбора горизонта N при credit assignment.

Вместо выбора одного N они используют смесь многошаговых оценок:

$$(1 - \lambda) \sum_{N=1}^{\infty} \lambda^{N-1} \Psi^{(N)}.$$

На практике GAE считается рекурсивно назад по rollout:

$$\Psi^{\text{GAE}}(s_t, a_t) = \Psi^{(1)}(s_t, a_t) + \lambda \gamma (1 - \text{done}_{t+1}) \Psi^{\text{GAE}}(s_{t+1}, a_{t+1}).$$

Retrace. Retrace переносит идею многошаговых следов в off-policy режим.

Он использует коэффициенты:

$$c_i = \lambda \min \left(1, \frac{\pi(a_i | s_i)}{\mu(a_i | s_i)} \right),$$

что позволяет контролировать взрывающиеся importance weights.

11 Лекция 11. DDPG, TD3 и обучение стохастических политик

11.1 Проблема DQN в непрерывных пространствах действий

В DQN есть принципиальное ограничение: алгоритм плохо переносится на непрерывные пространства действий.

Причина в том, что в DQN постоянно нужно считать операторы максимума и аргмаксимума:

$$\max_a Q_\theta(s, a), \quad \arg \max_a Q_\theta(s, a).$$

Они нужны в двух местах:

- для жадного выбора действия;
- для построения таргета в задаче регрессии.

В дискретном пространстве действий это просто: можно посчитать $Q_\theta(s, a)$ для всех действий и выбрать максимум.

Но если:

$$a \in \mathbb{R}^A,$$

то перебрать все действия невозможно.

11.2 Архитектура Q-функции для непрерывных действий

В непрерывном пространстве действий единственная естественная архитектура критика — подавать действие на вход вместе с состоянием:

$$Q_\theta(s, a).$$

Тогда для фиксированного состояния s можно попытаться найти максимум по действию через градиентный подъём по входу модели.

Инициализируем действие случайно:

$$a_0.$$

Затем делаем итерации:

$$a_{k+1} = a_k + \alpha \nabla_a Q_\theta(s, a)|_{a=a_k}.$$

Проблема. Это дорого и неудобно, потому что для каждого выбора действия и для каждого таргета нужно решать внутреннюю оптимизационную задачу по a .

11.3 Идея: обучить отдельную сеть для аргмаксимума

Пусть есть нейросеть:

$$\pi_\omega(s),$$

которая принимает состояние s и выдаёт действие.

Хотим, чтобы она приближала аргмаксимум текущей аппроксимации Q -функции:

$$\pi_\omega(s) \approx \arg \max_a Q_\theta(s, a).$$

Такую сеть понятно как обучать: нужно максимизировать значение критика на действии, которое выдаёт актор:

$$Q_\theta(s, \pi_\omega(s)) \rightarrow \max_\omega.$$

То есть актор π_ω учится выбирать действия, которые критик Q_θ считает хорошими.

11.4 DDPG как модификация DQN

Теперь можно оставить схему DQN почти неизменной.

Единственная модификация: каждый раз, когда нужно посчитать максимум или аргмаксимум по действию, мы используем сеть:

$$\pi_\omega(s).$$

В DQN для таргета нужно было считать:

$$\max_{a'} Q_{\theta^-}(s', a').$$

В DDPG это заменяется на:

$$Q_{\theta^-}(s', \pi_{\omega^-}(s')).$$

Здесь:

$$\theta^-$$

— параметры target-сети критика, а:

$$\omega^-$$

— параметры target-сети актора.

То есть для target-сети критика тоже нужна старая версия вспомогательной функции:

$$\pi_{\omega^-}(s).$$

11.5 Таргет в DDPG

Для перехода:

$$T := (s, a, r, s')$$

таргет в DQN-подобной записи:

$$y(T) := r + \gamma Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta^-}(s', a') \right).$$

В DDPG аргмаксимум приближается target-актором:

$$\arg \max_{a'} Q_{\theta^-}(s', a') \approx \pi_{\omega^-}(s').$$

Итого:

$$y(T) \approx r + \gamma Q_{\theta^-}(s', \pi_{\omega^-}(s')).$$

С учётом терминальных состояний:

$$y(T) := r + \gamma(1 - done)Q_{\theta^-}(s', \pi_{\omega^-}(s')).$$

11.6 Почему это называется Policy Gradient

Название **Deep Deterministic Policy Gradient** может сбить с толку: пока что алгоритм выглядит как DQN с дополнительной сетью для поиска аргмаксимума.

Однако та же схема получается и из policy gradient подхода, если рассмотреть детерминированные политики.

11.7 Deterministic Policy Gradient

Теорема 24 (Deterministic Policy Gradient). *В непрерывных пространствах действий при дифференцируемости Q -функции по действиям:*

$$\nabla_{\theta} J(\pi_{\theta}) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_{\theta}}(s)} \left[(\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\pi}(s, a)|_{a=\pi_{\theta}(s)} \right].$$

Смысл. Обычный policy gradient для стохастической политики использует:

$$\nabla_{\theta} \log \pi_{\theta}(a | s).$$

А deterministic policy gradient использует chain rule через действие:

$$\theta \longrightarrow \pi_{\theta}(s) \longrightarrow Q^{\pi}(s, \pi_{\theta}(s)).$$

То есть параметры политики меняются так, чтобы действие $\pi_{\theta}(s)$ двигалось в сторону увеличения Q -функции.

11.8 Доказательство Deterministic Policy Gradient

Начнём с:

$$\nabla_{\theta} V^{\pi_{\theta}}(s).$$

По связи V - и Q -функций для детерминированной политики:

$$V^{\pi_{\theta}}(s) = Q^{\pi_{\theta}}(s, \pi_{\theta}(s)).$$

Тогда:

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \nabla_{\theta} Q^{\pi_{\theta}}(s, \pi_{\theta}(s)).$$

При изменении θ меняется не только действие:

$$\pi_{\theta}(s),$$

но и сама функция:

$$Q^{\pi_{\theta}}.$$

По правилу цепочки:

$$\begin{aligned} \nabla_{\theta} Q^{\pi_{\theta}}(s, \pi_{\theta}(s)) &= (\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\pi}(s, a)|_{a=\pi_{\theta}(s)} \\ &\quad + \nabla_{\theta} Q^{\pi_{\theta}}(s, a)|_{a=\pi_{\theta}(s)}. \end{aligned}$$

Здесь второе слагаемое — это градиент $Q^{\pi_{\theta}}$ по параметрам стратегии, которую эта Q -функция оценивает, при фиксированном действии a .

Размерности:

$$\begin{aligned} \nabla_{\theta} Q^{\pi_{\theta}}(s, a) &\in \mathbb{R}^{d \times 1}, \\ \nabla_a Q^{\pi_{\theta}}(s, a) &\in \mathbb{R}^{A \times 1}, \\ \nabla_{\theta} \pi_{\theta}(s) &\in \mathbb{R}^{A \times d}, \end{aligned}$$

где:

$$A$$

— размерность пространства действий, а:

$$d$$

— размерность параметров θ .

11.9 Рекурсивное раскрытие второго слагаемого

Для второго слагаемого используем QV -уравнение:

$$Q^{\pi_{\theta}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V^{\pi_{\theta}}(s')].$$

Так как награда и динамика среды не зависят от параметров политики θ , получаем:

$$\nabla_{\theta} Q^{\pi_{\theta}}(s, a) = \gamma \mathbb{E}_{s'} [\nabla_{\theta} V^{\pi_{\theta}}(s')].$$

Следовательно:

$$\begin{aligned}\nabla_{\theta} V^{\pi_{\theta}}(s) &= (\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\pi}(s, a)|_{a=\pi_{\theta}(s)} \\ &\quad + \gamma \mathbb{E}_{s'} [\nabla_{\theta} V^{\pi_{\theta}}(s')].\end{aligned}$$

Раскрывая эту рекурсию вдоль траектории, получаем:

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \mathbb{E}_{T \sim \pi|s_0=s} \left[\sum_{t \geq 0} \gamma^t (\nabla_{\theta} \pi_{\theta}(s_t))^{\top} \nabla_a Q^{\pi}(s_t, a)|_{a=\pi_{\theta}(s_t)} \right].$$

Теперь применяем эквивалентную форму математического ожидания через discounted state visitation distribution для функции:

$$f(s) = (\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\pi}(s, a)|_{a=\pi_{\theta}(s)}.$$

Тогда:

$$\nabla_{\theta} J(\pi_{\theta}) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_{\theta}}(s)} \left[(\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\pi}(s, a)|_{a=\pi_{\theta}(s)} \right].$$

11.10 Суррогатная функция для deterministic policy gradient

Для deterministic policy gradient можно сразу построить суррогатную функцию:

$$L_{\tilde{\pi}}(\theta) := \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\tilde{\pi}}(s)} [Q^{\tilde{\pi}}(s, \pi_{\theta}(s))].$$

Если взять градиент этой функции по θ , то получится chain rule:

$$\nabla_{\theta} Q^{\tilde{\pi}}(s, \pi_{\theta}(s)) = (\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\tilde{\pi}}(s, a)|_{a=\pi_{\theta}(s)}.$$

Интерпретация. Градиент по параметрам детерминированной стратегии просто проводит policy improvement: сдвигает действия в сторону увеличения Q -функции, используя её градиент по действиям.

11.11 Actor-Critic схема для DDPG

Если строить Actor-Critic схему, то нужно аппроксимировать:

$$Q_{\omega}(s, a) \approx Q^{\pi}(s, a).$$

Дальше мы явно используем градиент критика по действию:

$$\nabla_a Q_{\omega}(s, a) \approx \nabla_a Q^{\pi}(s, a).$$

Для актора делаем шаг градиентного подъёма:

$$\theta \leftarrow \theta + \alpha \mathbb{E}_s \left[(\nabla_{\theta} \pi_{\theta}(s))^{\top} \nabla_a Q^{\pi}(s, a)|_{a=\pi_{\theta}(s)} \right].$$

В практической реализации вместо истинного Q^{π} используется критик:

$$Q_{\omega}.$$

Эквивалентно оптимизируется суррогат:

$$\mathbb{E}_s [Q_{\omega}(s, \pi_{\theta}(s))] \rightarrow \max_{\theta}.$$

Состояния s могут приходить из произвольного распределения, например из replay buffer.

11.12 Обучение критика в DDPG

Критик:

$$Q_{\omega}(s, a) \approx Q^{\pi}(s, a)$$

обучается off-policy по переходам из replay buffer:

$$T := (s, a, r, s').$$

Для обучения используется одношаговый target:

$$y(T) := r + \gamma Q_{\omega^-}(s', \pi_{\theta^-}(s')).$$

С учётом терминальности:

$$y(T) := r + \gamma(1 - \text{done})Q_{\omega^-}(s', \pi_{\theta^-}(s')).$$

Далее минимизируется MSE:

$$(Q_{\omega}(s, a) - y(T))^2 \rightarrow \min_{\omega}.$$

11.13 Эквивалентность двух выводов DDPG

DDPG можно получить двумя путями:

1. как DQN, где аргмаксимум по непрерывному действию аппроксимируется актором;
2. как deterministic policy gradient с критиком $Q_{\omega}(s, a)$.

Эти две схемы полностью эквивалентны.

Идея. Актор в DDPG делает ровно то же, что жадный оператор в DQN:

$$\pi_{\theta}(s) \approx \arg \max_a Q_{\omega}(s, a).$$

Только вместо явного решения задачи максимизации по a мы учим нейросеть, которая сразу выдаёт приближённый максимум.

11.14 Exploration в DDPG

Так как стратегия в DDPG детерминированная:

$$a = \pi_{\theta}(s),$$

возникает та же проблема exploration-exploitation, что и в DQN.

В непрерывных пространствах действий вместо ε -greedy обычно добавляют шум к действию:

$$a_t := \pi(s_t) + \varepsilon_t,$$

где:

$$\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I).$$

Проблема независимого шума. Если один шаг среды — это маленький интервал времени, то независимый шум на каждом шаге может выглядеть как случайное дёрганье управления.

Например, если действие робота — это угол руля, странно исследовать среду, хаотично меняя руль туда-сюда много раз в секунду.

Интуитивно хочется, чтобы исследовательское смещение сохранялось некоторое время.

11.15 Шум Орнштейна–Уленбека

Для более плавного exploration можно использовать коррелированный шум.

Определение 21. *Шум Орнштейна–Уленбека, или Ornstein–Uhlenbeck noise, инициализируется нулём в начале эпизода и задаётся рекурсивно:*

$$\varepsilon_{t+1} := \alpha \varepsilon_t + \hat{\varepsilon}_t, \quad \hat{\varepsilon}_t \sim \mathcal{N}(0, \sigma^2 I),$$

где:

$$\alpha \leq 1$$

и:

$$\sigma$$

— гиперпараметры.

Если:

$$\alpha$$

близко к 1, шум меняется плавно и сохраняет направление смещения.

Это подходит для задач управления, где действие физически не должно резко прыгать на каждом шаге.

11.16 Алгоритм DDPG

Гиперпараметры:

- B — размер мини-батча;
- β — коэффициент экспоненциального сглаживания для target-сетей;
- α, σ — параметры шума;
- $Q_\theta(s, a)$ — critic с параметрами θ ;
- $\pi_\omega(s)$ — детерминированная стратегия с параметрами ω ;
- SGD-оптимизаторы;
- K — периодичность обновления target-сетей.

Инициализация. Инициализировать:

$$\theta, \omega$$

произвольно.

Положить:

$$\theta^- := \theta, \quad \omega^- := \omega.$$

Инициализировать шум:

$$\varepsilon := 0.$$

Пронаблюдать начальное состояние:

$$s_0.$$

На очередном шаге t :

1. Обновить шум:

$$\varepsilon \leftarrow \alpha \varepsilon + \hat{\varepsilon}, \quad \hat{\varepsilon} \sim \mathcal{N}(0, \sigma^2 I).$$

2. Выбрать действие:

$$a_t := \pi_\omega(s_t) + \varepsilon.$$

3. Пронаблюдать:

$$r_t, \quad s_{t+1}, \quad done_{t+1}.$$

4. Добавить переход в replay buffer:

$$(s_t, a_t, r_t, s_{t+1}, done_{t+1}).$$

5. Засэмплировать мини-батч размера B из буфера.

6. Сделать шаг градиентного подъёма по ω :

$$\frac{1}{B} \sum_{s \in B} Q_\theta(s, \pi_\omega(s)) \rightarrow \max_\omega.$$

7. Для каждого перехода:

$$T := (s, a, r, s', done)$$

посчитать target:

$$y(T) := r + \gamma(1 - done)Q_{\theta^-}(s', \pi_{\omega^-}(s')).$$

8. Сделать шаг градиентного спуска по θ :

$$\frac{1}{B} \sum_T (Q_\theta(s, a) - y(T))^2 \rightarrow \min_\theta.$$

9. Если:

$$t \bmod K = 0,$$

обновить target-сети через soft update:

$$\theta^- \leftarrow (1 - \beta)\theta^- + \beta\theta,$$

$$\omega^- \leftarrow (1 - \beta)\omega^- + \beta\omega.$$

11.17 Twin Delayed DDPG

Twin Delayed DDPG, или **TD3**, является улучшением DDPG.

В нём используются три основные идеи:

1. два критика вместо одного;
2. задержанное обновление стратегии;
3. добавление шума к target action.

11.18 Идея двух критиков

Как и в Twin Q-learning, используются две Q -сети:

$$Q_{\theta_1}(s, a), \quad Q_{\theta_2}(s, a).$$

Target считается через минимум двух оценок:

$$\min_{i \in \{1,2\}} Q_{\theta_i^-}(s', a').$$

Это борется с overestimation bias.

Если один критик случайно завысил оценку, минимум двух критиков даёт более осторожный target.

11.19 Target policy smoothing

В TD3 к действию target-актора добавляется шум:

$$a' = \pi_{\omega^-}(s') + \varepsilon',$$

где:

$$\varepsilon' \sim \text{clip}(\mathcal{N}(0, \hat{\sigma}I), -c, c).$$

Target:

$$y(T) := r + \gamma(1 - \text{done}) \min_{i \in \{1,2\}} Q_{\theta_i^-}(s', \pi_{\omega^-}(s') + \varepsilon').$$

Интуиция. Критик не должен переобучаться на очень узкие пики Q -функции.

Если небольшое изменение действия резко меняет оценку, то actor может эксплуатировать ошибку критика.

Добавление шума к target action сглаживает target и делает обучение устойчивее.

11.20 Delayed policy updates

В TD3 critic обновляется чаще, чем actor.

Actor обновляется только если:

$$t \bmod N = 0,$$

где N — периодичность обновления стратегии.

Интуиция. Актор должен учиться по относительно адекватному критику.

Если actor обновлять слишком часто, он начинает эксплуатировать текущие ошибки критика.

Поэтому сначала дают критикам немного догнать данные, а потом обновляют astor.

11.21 Алгоритм TD3

Гиперпараметры:

- B — размер мини-батча;
- N — периодичность обновления весов стратегии;
- α, σ — параметры exploration noise;
- $\hat{\sigma}, c$ — параметры target action noise;
- β — коэффициент экспоненциального сглаживания для target-сетей;
- $Q_{\theta_1}(s, a), Q_{\theta_2}(s, a)$ — критики;
- $\pi_{\omega}(s)$ — детерминированная стратегия;
- SGD-оптимизаторы.

Инициализация. Инициализировать:

$$\theta_1, \theta_2, \omega$$

произвольно.

Инициализировать target-сети:

$$\theta_1^- := \theta_1, \quad \theta_2^- := \theta_2, \quad \omega^- := \omega.$$

Инициализировать шум:

$$\varepsilon_0 := 0.$$

Пронаблюдать:

$$s_0.$$

На очередном шаге t :

1. Посчитать exploration noise:

$$\varepsilon_t := \alpha \varepsilon_{t-1} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

2. Выбрать действие:

$$a_t := \pi_{\omega}(s_t) + \varepsilon_t.$$

3. Пронаблюдать:

$$r_t, \quad s_{t+1}, \quad done_{t+1}.$$

4. Добавить в replay buffer:

$$(s_t, a_t, r_t, s_{t+1}, done_{t+1}).$$

5. Засэмплировать мини-батч размера B из буфера.

6. Для каждого перехода:

$$T := (s, a, r, s', done)$$

посчитать target action noise:

$$\varepsilon' \sim \text{clip}(\mathcal{N}(0, \hat{\sigma}I), -c, c).$$

7. Посчитать target:

$$y(T) := r + \gamma(1 - done) \min_{i \in \{1, 2\}} Q_{\theta_i^-}(s', \pi_{\omega^-}(s') + \varepsilon').$$

8. Сделать шаг градиентного спуска по θ_1 :

$$\frac{1}{B} \sum_T (Q_{\theta_1}(s, a) - y(T))^2 \rightarrow \min_{\theta_1}.$$

9. Сделать шаг градиентного спуска по θ_2 :

$$\frac{1}{B} \sum_T (Q_{\theta_2}(s, a) - y(T))^2 \rightarrow \min_{\theta_2}.$$

10. Если:

$$t \bmod N = 0,$$

то:

(a) сделать шаг градиентного подъёма по ω :

$$\frac{1}{B} \sum_{s \in B} Q_{\theta_1}(s, \pi_{\omega}(s)) \rightarrow \max_{\omega};$$

(b) обновить target-сети:

$$\theta_1^- \leftarrow (1 - \beta)\theta_1^- + \beta\theta_1,$$

$$\theta_2^- \leftarrow (1 - \beta)\theta_2^- + \beta\theta_2,$$

$$\omega^- \leftarrow (1 - \beta)\omega^- + \beta\omega.$$

11.22 Переход к стохастическим политикам

DDPG и TD3 используют детерминированную стратегию:

$$a = \pi_{\theta}(s).$$

Теперь хотим обучать стохастические политики:

$$a \sim \pi_{\theta}(a \mid s),$$

но при этом сохранить возможность использовать градиент критика по действиям.

Для этого нужен **репараметризационный трюк**.

11.23 Репараметризационный трюк

Определение 22. Скажем, что для параметризации:

$$\pi_{\theta}(a \mid s)$$

применим репараметризационный трюк, если сэмплирование:

$$a \sim \pi_{\theta}(a \mid s)$$

эквивалентно сэмплированию шума из некоторого распределения, не зависящего от параметров:

$$\varepsilon \sim p(\varepsilon),$$

и дальнейшему детерминированному преобразованию:

$$a = f_{\theta}(s, \varepsilon).$$

То есть вся случайность выносится в:

$$\varepsilon,$$

а зависимость от параметров θ находится внутри дифференцируемой функции:

$$f_{\theta}.$$

11.24 Пример: гауссова политика

Пусть стратегия параметризована нормальным распределением:

$$\pi_{\theta}(a \mid s) := \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s)I).$$

Тогда сэмплирование действия можно записать так:

$$a = \mu_{\theta}(s) + \varepsilon \odot \sigma_{\theta}(s),$$

где:

$$\varepsilon \sim \mathcal{N}(0, I),$$

а:

$$\odot$$

— поэлементное умножение.

Это и есть репараметризация.

11.25 Детерминированная политика как частный случай

Детерминированную стратегию тоже можно считать вырожденным случаем репараметризуемой политики.

В этом случае шум:

$$\varepsilon$$

считается пустым, а:

$$f_{\theta}(s, \varepsilon) = \pi_{\theta}(s).$$

Тогда формула для стохастического случая превращается в deterministic policy gradient.

11.26 Policy Gradient с репараметризацией

Теорема 25. Если для:

$$\pi_\theta(a | s)$$

применим репараметризационный трюк, то:

$$\nabla_\theta J(\pi_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_\theta}(s)} \mathbb{E}_{\varepsilon \sim p(\varepsilon)} \left[(\nabla_\theta f_\theta(s, \varepsilon))^\top \nabla_a Q^\pi(s, a)|_{a=f_\theta(s, \varepsilon)} \right].$$

Доказательство. Доказательство полностью повторяет вывод deterministic policy gradient.

Разница только в том, что действие теперь является не просто:

$$a = \pi_\theta(s),$$

а:

$$a = f_\theta(s, \varepsilon).$$

При фиксированном шуме ε это снова дифференцируемое детерминированное преобразование параметров в действие.

11.27 Суррогат для стохастической политики с репараметризацией

Идеи DDPG расширяются на репараметризуемые стохастические политики.

Если убрать из формулы градиента частоты посещения состояний и перейти к policy iteration схеме, получаем функционал:

$$\mathbb{E}_s \mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a)] \rightarrow \max_\theta.$$

Состояния s можно брать из replay buffer.

За счёт репараметризационного трюка градиент считается как:

$$\nabla_\theta \mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a)] = \mathbb{E}_{\varepsilon \sim p(\varepsilon)} [\nabla_\theta Q_\omega(s, f_\theta(s, \varepsilon))].$$

Далее:

$$\nabla_\theta Q_\omega(s, f_\theta(s, \varepsilon)) = (\nabla_\theta f_\theta(s, \varepsilon))^\top \nabla_a Q_\omega(s, a)|_{a=f_\theta(s, \varepsilon)}.$$

11.28 Проблема одномодальной гауссовой политики

Рассмотрим пример с объездом дерева.

Пусть агенту нужно объехать препятствие. Есть два хороших варианта:

- объехать слева;
- объехать справа.

Критик сообщает высокие значения $Q_\omega(s, a)$ и для действий слева, и для действий справа.

Если оптимизировать:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a)] \rightarrow \max_\theta$$

в классе одномодальных гауссиан, может возникнуть плохая стратегия.

Политика может поставить среднее между двумя хорошими модами. Тогда наиболее вероятное действие окажется:

врезаться в дерево.

Смысл. Если хороших действий несколько и они разделены плохой областью, одномодальная гауссова политика может быть неподходящей.

11.29 Смесь гауссиан

Один способ решить проблему мультимодальности — использовать смесь гауссиан.

Пусть используется K компонент смеси.

Для данного состояния s actor выдаёт:

- веса смеси:

$$w_i(s, \theta), \quad i = 1, \dots, K,$$

такие что:

$$\sum_{i=1}^K w_i(s, \theta) = 1;$$

- средние:

$$\mu_i(s, \theta);$$

- стандартные отклонения:

$$\sigma_i(s, \theta).$$

Итоговая политика:

$$\pi_\theta(a | s) := \sum_{i=1}^K w_i(s, \theta) \mathcal{N}(\mu_i(s, \theta), \sigma_i^2(s, \theta) I).$$

Такая политика может представить несколько разных хороших режимов поведения.

11.30 Score-function градиент для стохастического актора

Для функционала:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a)]$$

можно также использовать стандартный score-function trick.

Утверждение 1. Градиент по параметрам actor равен:

$$\nabla_\theta = \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_\theta \ln \pi_\theta(a | s) (Q_\omega(s, a) - b(s))],$$

где:

$$b(s)$$

— baseline, произвольная функция от состояний.

Пояснение. Baseline не меняет математическое ожидание градиента, потому что:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_\theta \ln \pi_\theta(a | s) b(s)] = b(s) \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_\theta \ln \pi_\theta(a | s)] = 0.$$

11.31 Замена переменной в плотности

Во многих алгоритмах для непрерывных действий действие сначала сэмплируется в неограниченном пространстве, а затем пропускается через функцию, ограничивающую диапазон действий.

Например:

$$u \sim \mu_\theta(u | s), \quad a = g(u).$$

Теорема 26 (Формула замены переменной в плотности). *Пусть:*

$$a := g(u),$$

где:

$$g : \mathbb{R}^A \rightarrow \mathbb{R}^A,$$

u :

$$u \sim \mu_\theta(u | s).$$

Тогда:

$$\ln \pi_\theta(a | s) = \ln \mu_\theta(u | s) - \ln |\det \nabla_u g|.$$

Смысл. Если действие a получается детерминированным преобразованием латентной переменной u , то плотность действия меняется с учётом якобиана преобразования.

11.32 Энтропия распределения

Определение 23. *Энтропией распределения $\pi(a)$ называется:*

$$H(\pi(a)) := -\mathbb{E}_{a \sim \pi(a)} [\ln \pi(a)].$$

Энтропия показывает, насколько распределение случайно или разнообразно.

В RL энтропийные добавки часто используют, чтобы поощрять exploration и не давать политике слишком быстро схлопываться в почти детерминированную.

11.33 Пример: преобразование через tanh

Часто непрерывные действия должны лежать в ограниченном диапазоне, например:

$$a \in [-1, 1]^A.$$

Для этого можно взять:

$$a = \tanh(u),$$

где $\tanh$ применяется покомпонентно.

Пусть:

$$g(u) := \tanh(u).$$

Тогда якобиан:

$$\nabla_u g$$

является диагональной матрицей, потому что преобразование покомпонентное.

Его определитель равен произведению диагональных элементов:

$$\det \nabla_u g(u) = \prod_{i=1}^A \nabla_{u_i} \tanh(u_i).$$

Так как:

$$\frac{d}{du} \tanh(u) = 1 - \tanh^2(u),$$

получаем:

$$\det \nabla_u g(u) = \prod_{i=1}^A (1 - \tanh^2(u_i)).$$

Все множители неотрицательны, поэтому модуль в формуле замены переменной можно опустить.

Подставляя в формулу замены переменной, получаем:

$$\ln \pi_\theta(a | s) = \ln \mu_\theta(u | s) - \sum_{i=1}^A \ln (1 - \tanh^2(u_i)).$$

11.34 Итог

В этой лекции были рассмотрены методы для непрерывных пространств действий.

DDPG. DDPG можно понимать двумя эквивалентными способами:

- как DQN, где аргмаксимум по действию аппроксимируется actor-сетью:

$$\pi_\omega(s) \approx \arg \max_a Q_\theta(s, a);$$

- как deterministic policy gradient, где actor обновляется через градиент критика по действию:

$$(\nabla_\theta \pi_\theta(s))^\top \nabla_a Q^\pi(s, a).$$

TD3. TD3 стабилизирует DDPG с помощью:

- двух критиков и минимума их оценок;
- delayed policy updates;
- target policy smoothing через шум в target action.

Стохастические политики. Для стохастических политик используется репараметризационный трюк:

$$a = f_\theta(s, \varepsilon), \quad \varepsilon \sim p(\varepsilon).$$

Он позволяет оптимизировать:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} Q_\omega(s, a)$$

через обычный backpropagation.

Преобразование действий. Если действие получается заменой переменной:

$$a = g(u),$$

то логарифм плотности корректируется якобианом:

$$\ln \pi_{\theta}(a \mid s) = \ln \mu_{\theta}(u \mid s) - \ln |\det \nabla_u g|.$$

Для:

$$g(u) = \tanh(u)$$

получается:

$$\ln \pi_{\theta}(a \mid s) = \ln \mu_{\theta}(u \mid s) - \sum_{i=1}^A \ln (1 - \tanh^2(u_i)).$$

12 Лекция 12. Soft Actor-Critic, Maximum Entropy RL и Soft Q-learning

12.1 Soft Actor-Critic: мотивация

В предыдущей лекции рассматривались методы для непрерывных пространств действий:

- DDPG;
- TD3;
- обучение стохастических политик с помощью репараметризационного трюка;
- преобразование действий, например через $\tanh$.

Теперь рассмотрим алгоритм **Soft Actor-Critic**, или **SAC**.

Главная идея SAC — оптимизировать не только внешнюю награду, но и энтропию политики.

То есть агенту выгодно:

- получать большую награду;
- сохранять достаточно стохастичную стратегию.

Это помогает exploration и делает обучение более устойчивым.

12.2 Энтропия распределения

Определение 24. Энтропией распределения $\pi(a)$ называется величина:

$$H(\pi(a)) := -\mathbb{E}_{\pi(a)} [\ln \pi(a)].$$

Энтропия показывает, насколько распределение случайно.

Если распределение почти детерминированное, то энтропия мала.

Если распределение размазано по действиям, то энтропия велика.

12.3 Maximum Entropy RL

Определение 25. Задачей *Maximum Entropy RL* называется максимизация функционала:

$$J_{\text{soft}}(\pi) := \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} \gamma^t (r_t + \alpha H(\pi(\cdot \mid s_t))) \right] \rightarrow \max_{\pi}.$$

Здесь:

- α — гиперпараметр;
- α называется температурой, или temperature;
- $H(\pi(\cdot \mid s_t))$ — энтропия распределения действий в состоянии s_t .

Чем больше α , тем сильнее агент поощряется за стохастичность политики.

Если:

$$\alpha = 0,$$

то задача превращается в обычную задачу RL без энтропийного бонуса.

12.4 Эквивалентная форма Maximum Entropy RL

Утверждение 2. Задача *Maximum Entropy RL* эквивалентна задаче:

$$J_{\text{soft}}(\pi) := \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} \gamma^t (r_t - \alpha \ln \pi(a_t \mid s_t)) \right] \rightarrow \max_{\pi}.$$

Доказательство. По определению энтропии:

$$H(\pi(\cdot \mid s_t)) = -\mathbb{E}_{a_t \sim \pi(\cdot \mid s_t)} [\ln \pi(a_t \mid s_t)].$$

При этом математическое ожидание по действиям уже входит в математическое ожидание по траекториям:

$$T \sim \pi.$$

Поэтому:

$$\mathbb{E}_{T \sim \pi} [H(\pi(\cdot \mid s_t))] = \mathbb{E}_{T \sim \pi} [-\ln \pi(a_t \mid s_t)].$$

Следовательно:

$$r_t + \alpha H(\pi(\cdot \mid s_t))$$

в среднем эквивалентно:

$$r_t - \alpha \ln \pi(a_t \mid s_t).$$

12.5 Мягкая награда

Можно считать, что в Maximum Entropy RL мы модифицируем награду в среде.

Введём мягкую награду:

$$r_{\text{soft}}(s, a) := r(s, a) - \alpha \ln \pi(a \mid s).$$

Тогда задача Maximum Entropy RL выглядит как обычная задача RL с новой наградой:

$$r_{\text{soft}}(s, a).$$

Замечание 23. *Важно, что эта награда зависит не только от состояния и действия, но и от самой политики π .*

12.6 Зачем нужен энтропийный бонус

Рассмотрим пример.

Пусть у агента есть два пути.

По мере углубления в каждый путь награда растёт одинаково. Первый путь заканчивается тупиком и суммарно позволяет набрать не более:

$$+100.$$

Второй путь заканчивается позже и позволяет набрать:

$$+110.$$

Во время обучения агент может случайно обнаружить первый путь, начать получать там награду и быстро выучить стратегию, которая всегда идёт туда.

После этого он может перестать исследовать второй путь, хотя второй путь лучше.

Энтропийный бонус помогает избежать такой ситуации: в начале эпизода стратегия мотивирована оставаться стохастичной и иногда выбирать разные пути.

То есть энтропийный бонус помогает бороться с плохими локальными максимумами в среде.

12.7 Оптимальная детерминированная стратегия может не существовать

Утверждение 3. *В задаче Maximum Entropy RL оптимальной детерминированной стратегии может не существовать.*

Доказательство. Рассмотрим MDP, где награда всегда равна нулю:

$$r(s, a) = 0.$$

Тогда обычная награда не различает политики.

Но в Maximum Entropy RL оптимизируется ещё и энтропия:

$$H(\pi(\cdot \mid s)).$$

Следовательно, оптимальна стратегия с максимальной энтропией, то есть наиболее случайная стратегия.

Детерминированная стратегия имеет минимальную энтропию, поэтому она не оптимальна.

12.8 Соглашение про температуру

Далее для упрощения записи часто будем считать:

$$\alpha = 1.$$

При необходимости общая форма с произвольной температурой получается заменой:

$$\ln \pi(a \mid s) \longrightarrow \alpha \ln \pi(a \mid s).$$

То есть вместо:

$$Q_{\text{soft}}^{\pi}(s, a) - \ln \pi(a \mid s)$$

в общей форме нужно писать:

$$Q_{\text{soft}}^{\pi}(s, a) - \alpha \ln \pi(a \mid s).$$

12.9 Мягкие функции ценности

В Maximum Entropy RL вводятся мягкие аналоги Q - и V -функций.

Мягкая Q -функция:

$$Q_{\text{soft}}^{\pi}(s, a) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t (r_t - \ln \pi(a_t \mid s_t)) \mid s_0 = s, a_0 = a \right],$$

где первое действие $a_0 = a$ уже зафиксировано.

Мягкая V -функция:

$$V_{\text{soft}}^{\pi}(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t (r_t - \ln \pi(a_t \mid s_t)) \mid s_0 = s \right].$$

12.10 Мягкие уравнения связи

Теорема 27 (Мягкие уравнения связи). *Для фиксированной политики π выполнено:*

$$Q_{\text{soft}}^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V_{\text{soft}}^{\pi}(s')],$$

и:

$$V_{\text{soft}}^{\pi}(s) = \mathbb{E}_{a \sim \pi(a \mid s)} [Q_{\text{soft}}^{\pi}(s, a) - \ln \pi(a \mid s)].$$

Доказательство. Первое уравнение получается из разбиения полной мягкой награды на первый шаг и будущую мягкую ценность:

$$Q_{\text{soft}}^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V_{\text{soft}}^{\pi}(s')].$$

Второе уравнение получается усреднением по первому действию:

$$a \sim \pi(a \mid s).$$

При этом в мягкой награде появляется энтропийный член:

$$- \ln \pi(a \mid s).$$

12.11 Мягкие уравнения Беллмана

Теорема 28 (Мягкие уравнения Беллмана). *Для фиксированной политики π выполнено:*

$$Q_{\text{soft}}^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \mathbb{E}_{a' \sim \pi(a' | s')} [Q_{\text{soft}}^{\pi}(s', a') - \ln \pi(a' | s')],$$

и:

$$V_{\text{soft}}^{\pi}(s) = \mathbb{E}_{a \sim \pi(a | s)} [r(s, a) - \ln \pi(a | s) + \gamma \mathbb{E}_{s'} [V_{\text{soft}}^{\pi}(s')]].$$

Пояснение. Эти уравнения получаются подстановкой мягких уравнений связи друг в друга.

Они являются аналогами обычных уравнений Беллмана, но с дополнительным энтропийным членом:

$$- \ln \pi(a | s).$$

12.12 Сжимаемость мягких операторов Беллмана

Теорема 29. *Операторы, стоящие в правой части мягких уравнений Беллмана, являются сжимающими с коэффициентом γ по метрике:*

$$\rho(V_1, V_2) := \max_{s \in \mathcal{S}} \{|V_1(s) - V_2(s)|\}.$$

Доказательство. Покажем утверждение для мягкой V -функции.

Пусть B_{soft} — оператор, стоящий в правой части мягкого уравнения Беллмана для V -функции.

Пусть даны две функции:

$$V_1, \quad V_2.$$

Тогда:

$$|[B_{\text{soft}} V_1](s) - [B_{\text{soft}} V_2](s)| = \gamma |\mathbb{E}_a \mathbb{E}_{s'} [V_1(s') - V_2(s')]|.$$

Награда и энтропийный член одинаковы для V_1 и V_2 , поэтому они сокращаются.

Далее:

$$\begin{aligned} \gamma |\mathbb{E}_a \mathbb{E}_{s'} [V_1(s') - V_2(s')]| &\leq \gamma \mathbb{E}_a \mathbb{E}_{s'} [|V_1(s') - V_2(s')|] \\ &\leq \gamma \rho(V_1, V_2). \end{aligned}$$

Следовательно:

$$\rho(B_{\text{soft}} V_1, B_{\text{soft}} V_2) \leq \gamma \rho(V_1, V_2).$$

12.13 Сходимость простой итерации

Утверждение 4. *Метод простой итерации сходится к единственному решению мягких уравнений Беллмана из любого начального приближения.*

Пояснение. Это следует из сжимаемости мягкого оператора Беллмана.

Как и в обычном случае, если оператор является сжимающим, то у него есть единственная неподвижная точка, и итерации:

$$V_{k+1} = B_{\text{soft}} V_k$$

сходятся к ней из любого начального приближения.

12.14 Обучение критика в Maximum Entropy RL

Теперь можно построить процедуру обучения критика.

Хотим обучать:

$$Q_{\omega}(s, a) \approx Q_{\text{soft}}^{\pi}(s, a).$$

Если использовать одношаговые оценки в off-policy режиме, можно решать мягкое уравнение Беллмана.

Для перехода:

$$T = (s, a, r, s')$$

целевая переменная может быть записана как:

$$y(T) := r + \gamma \mathbb{E}_{a' \sim \pi(a' | s')} [Q_{\omega}(s', a') - \ln \pi(a' | s')].$$

На практике ожидание по a' часто оценивается сэмплением:

$$a' \sim \pi(a' | s').$$

12.15 Вспомогательная V -функция

В ранней версии SAC использовалась вспомогательная V -сеть:

$$V_{\psi}(s).$$

Тогда таргеты для критика и V -функции можно строить через мягкие уравнения связи.

Для перехода:

$$T = (s, a, r, s')$$

из replay buffer:

$$y_Q(T) := r + \gamma V_{\psi}(s').$$

Для состояния s сэмплируем действие из текущей политики:

$$a_{\pi} \sim \pi(a_{\pi} | s),$$

и сохраняем вероятность:

$$\pi(a_{\pi} | s).$$

Тогда target для V -сети:

$$y_V(T) := Q_{\omega}(s, a_{\pi}) - \ln \pi(a_{\pi} | s).$$

В версии с двумя критиками используют минимум:

$$y_V(T) := \min_{i=1,2} \{Q_{\omega_i}(s, a_{\pi})\} - \alpha \ln \pi_{\theta}(a_{\pi} | s).$$

12.16 Soft Policy Improvement

Теорема 30 (Soft Policy Improvement). Пусть стратегии π_1 и π_2 таковы, что для всех состояний $s \in \mathcal{S}$ выполняется:

$$\mathbb{E}_{a \sim \pi_2(a|s)} [Q_{\text{soft}}^{\pi_1}(s, a)] + H(\pi_2(\cdot | s)) \geq V_{\text{soft}}^{\pi_1}(s).$$

Тогда:

$$\pi_2 \succeq \pi_1$$

с учётом энтропийного бонуса.

Если хотя бы для одного состояния $s \in \mathcal{S}$ неравенство строгое, то:

$$\pi_2 \succ \pi_1.$$

Доказательство. Покажем, что:

$$V_{\text{soft}}^{\pi_2}(s) \geq V_{\text{soft}}^{\pi_1}(s)$$

для любого состояния s .

По условию:

$$V_{\text{soft}}^{\pi_1}(s) \leq \mathbb{E}_{a \sim \pi_2(a|s)} [Q_{\text{soft}}^{\pi_1}(s, a)] + H(\pi_2(\cdot | s)).$$

Используя мягкое уравнение связи:

$$Q_{\text{soft}}^{\pi_1}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V_{\text{soft}}^{\pi_1}(s')],$$

получаем:

$$V_{\text{soft}}^{\pi_1}(s) \leq \mathbb{E}_{a \sim \pi_2(a|s)} [r(s, a) + H(\pi_2(\cdot | s)) + \gamma \mathbb{E}_{s'} [V_{\text{soft}}^{\pi_1}(s')]].$$

Теперь применим это же неравенство рекурсивно для следующего состояния s' .

Получаем:

$$\begin{aligned} V_{\text{soft}}^{\pi_1}(s) &\leq \mathbb{E}_{T \sim \pi_2 | s_0=s} \left[\sum_{t \geq 0} \gamma^t (r_t + H(\pi_2(\cdot | s_t))) \right] \\ &= V_{\text{soft}}^{\pi_2}(s). \end{aligned}$$

Если в условии теоремы для некоторого состояния неравенство было строгим, то в цепочке рассуждений первое неравенство для этого состояния тоже строгое. Поэтому:

$$V_{\text{soft}}^{\pi_2}(s) > V_{\text{soft}}^{\pi_1}(s).$$

12.17 Generalized Policy Iteration для Maximum Entropy RL

Аналог общей схемы **Generalized Policy Iteration** для Maximum Entropy RL выглядит так.

Soft Policy Evaluation. Обучаем аппроксимацию мягкой оценочной функции для текущей стратегии:

$$Q_{\omega} \approx Q_{\text{soft}}^{\pi}.$$

Soft Policy Improvement. Оптимизируем для разных состояний s функционал:

$$\mathbb{E}_{a \sim \pi(a|s)} [Q_\omega(s, a)] + H(\pi(\cdot | s)) \rightarrow \max_{\pi}.$$

С учётом температуры:

$$\mathbb{E}_{a \sim \pi(a|s)} [Q_\omega(s, a)] + \alpha H(\pi(\cdot | s)) \rightarrow \max_{\pi}.$$

Эквивалентно:

$$\mathbb{E}_{a \sim \pi(a|s)} [Q_\omega(s, a) - \alpha \ln \pi(a | s)] \rightarrow \max_{\pi}.$$

12.18 Связь Soft Policy Improvement с KL-дивергенцией

Теорема 31. *Задача:*

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a)] + H(\pi_\theta(\cdot | s)) \rightarrow \max_{\theta}$$

эквивалентна задаче:

$$\text{KL}(\pi_\theta(\cdot | s) \parallel \exp Q_\omega(s, \cdot)) \rightarrow \min_{\theta},$$

где $\exp Q_\omega(s, \cdot)$ — ненормированное распределение над действиями.

Доказательство. Обозначим нормировочную константу распределения:

$$\exp Q_\omega(s, \cdot)$$

как:

$$Z_\omega(s) := \int_{\mathcal{A}} \exp Q_\omega(s, a) da.$$

Тогда нормированное распределение имеет вид:

$$\frac{\exp Q_\omega(s, a)}{Z_\omega(s)}.$$

KL-дивергенция:

$$\text{KL} \left(\pi_\theta(\cdot | s) \parallel \frac{\exp Q_\omega(s, \cdot)}{Z_\omega(s)} \right)$$

равна:

$$\begin{aligned} & \mathbb{E}_{a \sim \pi_\theta(a|s)} \left[\ln \pi_\theta(a | s) - \ln \frac{\exp Q_\omega(s, a)}{Z_\omega(s)} \right] \\ &= \mathbb{E}_{a \sim \pi_\theta(a|s)} [\ln \pi_\theta(a | s) - Q_\omega(s, a) + \ln Z_\omega(s)]. \end{aligned}$$

Домножим на -1 :

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a) - \ln \pi_\theta(a | s)] - \ln Z_\omega(s).$$

Но:

$$-\mathbb{E}_{a \sim \pi_\theta(a|s)} [\ln \pi_\theta(a | s)] = H(\pi_\theta(\cdot | s)).$$

Кроме того:

$$\ln Z_\omega(s)$$

не зависит от параметров θ .

Следовательно, минимизация KL-дивергенции эквивалентна максимизации:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a)] + H(\pi_\theta(\cdot | s)).$$

12.19 Вид жадной стратегии

Теорема 32 (Вид жадной стратегии). *Максимальное значение функционала:*

$$\mathbb{E}_{a \sim \pi(a|s)} [Q_\omega(s, a)] + H(\pi(\cdot | s))$$

достигается на стратегии:

$$\pi(a | s) \propto \exp Q_\omega(s, a).$$

Доказательство. Из предыдущей теоремы задача soft policy improvement эквивалентна минимизации KL-дивергенции:

$$\text{KL} \left(\pi(\cdot | s) \left\| \frac{\exp Q_\omega(s, \cdot)}{Z_\omega(s)} \right. \right).$$

Минимум KL-дивергенции достигается в нуле, когда распределения совпадают.

Следовательно:

$$\pi(a | s) = \frac{\exp Q_\omega(s, a)}{Z_\omega(s)} \propto \exp Q_\omega(s, a).$$

12.20 Пример: гауссова политика

Пусть стратегия параметризована гауссианой:

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta^2(s)I).$$

Для такой политики можно использовать репараметризационный трюк:

$$a = \mu_\theta(s) + \sigma_\theta(s) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Также можно аналитически посчитать энтропию.

С точностью до константы, не зависящей от θ , для диагональной гауссианы:

$$H(\mathcal{N}(\mu, \sigma^2 I)) = \sum_{i=1}^A \ln \sigma_i + \text{const.}$$

Поэтому soft policy improvement можно записать как:

$$\mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} [Q_\omega(s, \mu_\theta(s) + \sigma_\theta(s) \odot \varepsilon)] + \alpha \sum_{i=1}^A \ln \sigma_i(s, \theta) \rightarrow \max_\theta.$$

Если константы энтропии не важны для оптимизации, их можно опускать.

12.21 Пример: смесь гауссиан

Пусть стратегия параметризована смесью гауссиан.

В этом случае:

- репараметризационный трюк в простой форме может быть неприменим;

- аналитически посчитать энтропию сложно.

Тогда можно применять REINFORCE.

Для функционала:

$$\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a) - \alpha \ln \pi_\theta(a | s)]$$

градиент можно оценивать как:

$$\nabla_\theta = \mathbb{E}_{a \sim \pi_\theta(a|s)} [\nabla_\theta \ln \pi_\theta(a | s) (Q_\omega(s, a) - \alpha \ln \pi_\theta(a | s) - b(s))],$$

где:

$$b(s)$$

— baseline, произвольная функция от состояния.

Хорошо выбирать baseline близким к среднему значению:

$$Q_\omega(s, a) - \alpha \ln \pi_\theta(a | s).$$

Например:

$$b(s) := V_\omega(s) = \mathbb{E}_{a \sim \pi_\theta(a|s)} [Q_\omega(s, a) - \alpha \ln \pi_\theta(a | s)].$$

12.22 Алгоритм Soft Actor-Critic

Рассмотрим вариант SAC с двумя критиками и вспомогательной V -сетью.

Гиперпараметры.

- B — размер мини-батчей;
- β — параметр экспоненциального сглаживания target-сети;
- α — температура;
-

$$\pi_\theta(a | s) := \mathcal{N}(\mu_\theta(s), \sigma_\theta^2(s)I)$$

— гауссова стратегия с параметрами θ ;

-

$$Q_{\omega_1}(s, a), \quad Q_{\omega_2}(s, a)$$

— две Q -сети с параметрами ω_1 и ω_2 ;

-

$$V_\psi(s)$$

— V -сеть с параметрами ψ ;

- SGD-оптимизаторы.

Инициализация. Инициализировать параметры:

$$\theta, \omega_1, \omega_2, \psi$$

произвольно.

Инициализировать target-сеть:

$$\psi^- := \psi.$$

Пронаблюдать начальное состояние:

$$s_0.$$

На очередном шаге t :

1. Выбрать действие:

$$a_t \sim \pi_\theta(a_t \mid s_t).$$

2. Пронаблюдать:

$$r_t, \quad s_{t+1}, \quad done_{t+1}.$$

3. Добавить переход в replay buffer:

$$(s_t, a_t, r_t, s_{t+1}, done_{t+1}).$$

4. Засэмплировать мини-батч размера B из буфера.

5. Для каждого состояния s из батча засэмплировать шум:

$$\varepsilon(s) \sim \mathcal{N}(0, I),$$

и посчитать:

$$\mu_\theta(s), \quad \sigma_\theta(s).$$

6. Сформировать репараметризованное действие:

$$a_\theta(s) := \mu_\theta(s) + \sigma_\theta(s) \odot \varepsilon(s).$$

7. Посчитать оценку градиента по параметрам стратегии:

$$\nabla_\theta := \frac{1}{B} \sum_{s \in B} \nabla_\theta \left[\alpha \sum_{i=1}^A \ln \sigma_i(s, \theta) + \min_{j=1,2} \{Q_{\omega_j}(s, a_\theta(s))\} \right].$$

8. Сделать шаг градиентного подъёма по θ , используя:

$$\nabla_\theta.$$

9. Для каждого перехода:

$$T = (s, a, r, s', done)$$

засэмплировать:

$$a_\pi \sim \pi_\theta(a_\pi \mid s),$$

и сохранить:

$$\pi_\theta(a_\pi \mid s).$$

10. Посчитать target для V -сети:

$$y_V(T) := \min_{j=1,2} \{Q_{\omega_j}(s, a_\pi)\} - \alpha \ln \pi_\theta(a_\pi | s).$$

11. Посчитать target для Q -сетей:

$$y_Q(T) := r + \gamma(1 - done)V_{\psi^-}(s').$$

12. Посчитать функции потерь:

$$\text{Loss}_V(\psi) := \frac{1}{B} \sum_T (V_\psi(s) - y_V(T))^2,$$

$$\text{Loss}_{Q_1}(\omega_1) := \frac{1}{B} \sum_T (Q_{\omega_1}(s, a) - y_Q(T))^2,$$

$$\text{Loss}_{Q_2}(\omega_2) := \frac{1}{B} \sum_T (Q_{\omega_2}(s, a) - y_Q(T))^2.$$

13. Сделать шаг градиентного спуска по ψ, ω_1, ω_2 , используя:

$$\nabla_\psi \text{Loss}_V(\psi), \quad \nabla_{\omega_1} \text{Loss}_{Q_1}(\omega_1), \quad \nabla_{\omega_2} \text{Loss}_{Q_2}(\omega_2).$$

14. Обновить target-сеть:

$$\psi^- \leftarrow (1 - \beta)\psi^- + \beta\psi.$$

12.23 Оптимальные мягкие функции ценности

Сформулируем критерий оптимальности Беллмана в Maximum Entropy RL.

По аналогии с обычным случаем введём оптимальные оценочные функции:

$$V_{\text{soft}}^*(s) := \max_{\pi} \{V_{\text{soft}}^\pi(s)\},$$

$$Q_{\text{soft}}^*(s, a) := \max_{\pi} \{Q_{\text{soft}}^\pi(s, a)\}.$$

12.24 Вид оптимальной стратегии

Теорема 33 (Вид оптимальной стратегии для Maximum Entropy RL). *Оптимальной является единственная стратегия:*

$$\pi^*(a | s) \propto \exp Q_{\text{soft}}^*(s, a).$$

Доказательство. Откажемся от стационарности и рассмотрим задачу поиска оптимальной стратегии:

$$\pi_t(a | s)$$

в предположении, что в будущем мы сможем набрать максимально возможную мягкую награду:

$$Q_{\text{soft}}^*(s, a, t).$$

На шаге t нужно решить:

$$\mathbb{E}_{a \sim \pi_t(a|s)} [Q_{\text{soft}}^*(s, a, t) - \ln \pi_t(a | s)] \rightarrow \max_{\pi_t(a|s)}.$$

Как и в теореме про soft policy improvement, это эквивалентно:

$$- \text{KL} \left(\pi_t(a | s) \left\| \frac{\exp Q_{\text{soft}}^*(s, a, t)}{Z(s, t)} \right\| \right) \rightarrow \max_{\pi_t(a|s)},$$

где:

$$Z(s, t)$$

— нормировочная константа:

$$Z(s, t) := \int_{\mathcal{A}} \exp Q_{\text{soft}}^*(s, a, t) da.$$

Оптимум достигается, когда KL-дивергенция равна нулю, то есть:

$$\pi_t(a | s) = \frac{\exp Q_{\text{soft}}^*(s, a, t)}{Z(s, t)}.$$

Дальше рассуждение строится как в обычном случае: Q_{soft}^* по определению не зависит от времени, поэтому оптимальная стратегия получается стационарной:

$$\pi^*(a | s) \propto \exp Q_{\text{soft}}^*(s, a).$$

Поскольку $Q_{\text{soft}}^*(s, a)$ задана однозначно, такая стратегия единственна.

12.25 Оптимальная мягкая V -функция через log-sum-exp

Теорема 34. Для оптимальной мягкой V -функции выполнено:

$$V_{\text{soft}}^*(s) = \ln \int_{\mathcal{A}} \exp Q_{\text{soft}}^*(s, a) da.$$

Доказательство. Мы знаем, что оптимальная стратегия имеет вид:

$$\pi^*(a | s) = \frac{\exp Q_{\text{soft}}^*(s, a)}{Z(s)},$$

где:

$$Z(s) := \int_{\mathcal{A}} \exp Q_{\text{soft}}^*(s, a) da.$$

Посчитаем энтропию такого распределения:

$$\begin{aligned} -\mathbb{E}_{a \sim \pi^*(a|s)} [\ln \pi^*(a | s)] &= -\mathbb{E}_{a \sim \pi^*(a|s)} [Q_{\text{soft}}^*(s, a) - \ln Z(s)] \\ &= \ln Z(s) - \mathbb{E}_{a \sim \pi^*(a|s)} [Q_{\text{soft}}^*(s, a)]. \end{aligned}$$

Подставим оптимальную стратегию в мягкое VQ -уравнение:

$$V_{\text{soft}}^*(s) = \mathbb{E}_{a \sim \pi^*(a|s)} [Q_{\text{soft}}^*(s, a) - \ln \pi^*(a | s)].$$

Получаем:

$$\begin{aligned} V_{\text{soft}}^*(s) &= \mathbb{E}_{\pi^*} [Q_{\text{soft}}^*(s, a)] + \ln Z(s) - \mathbb{E}_{\pi^*} [Q_{\text{soft}}^*(s, a)] \\ &= \ln Z(s). \end{aligned}$$

То есть:

$$V_{\text{soft}}^*(s) = \ln \int_{\mathcal{A}} \exp Q_{\text{soft}}^*(s, a) da.$$

12.26 Мягкое уравнение оптимальности Беллмана

Из мягкого уравнения связи:

$$Q_{\text{soft}}^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V_{\text{soft}}^*(s')],$$

и формулы:

$$V_{\text{soft}}^*(s) = \ln \int_{\mathcal{A}} \exp Q_{\text{soft}}^*(s, a) da$$

получаем мягкое уравнение оптимальности Беллмана:

$$Q_{\text{soft}}^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\ln \int_{\mathcal{A}} \exp Q_{\text{soft}}^*(s', a') da' \right].$$

Это soft-аналог обычного уравнения оптимальности:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a'} Q^*(s', a') \right].$$

Здесь оператор максимума заменён на:

$$\ln \int_{\mathcal{A}} \exp Q(s, a) da,$$

то есть на непрерывный аналог log-sum-exp.

12.27 Сжимаемость soft Bellman optimality operator

Теорема 35. *Оператор, стоящий в правой части мягкого уравнения оптимальности Беллмана, является сжимающим с коэффициентом γ .*

Следовательно, метод простой итерации решения этой системы уравнений сходится из любого начального приближения к единственной неподвижной точке.

Доказательство. Пусть даны две Q -функции:

$$Q_1, \quad Q_2.$$

Пусть:

$$\rho(Q_1, Q_2) := \max_{s \in \mathcal{S}, a \in \mathcal{A}} \{|Q_1(s, a) - Q_2(s, a)|\} \leq \varepsilon.$$

Тогда для всех s, a :

$$Q_1(s, a) \leq Q_2(s, a) + \varepsilon.$$

Следовательно:

$$\begin{aligned} \ln \int_{\mathcal{A}} \exp Q_1(s, a) da &\leq \ln \int_{\mathcal{A}} \exp (Q_2(s, a) + \varepsilon) da \\ &= \ln \left(e^\varepsilon \int_{\mathcal{A}} \exp Q_2(s, a) da \right) \\ &= \varepsilon + \ln \int_{\mathcal{A}} \exp Q_2(s, a) da. \end{aligned}$$

Аналогично:

$$Q_1(s, a) \geq Q_2(s, a) - \varepsilon$$

и поэтому:

$$\ln \int_{\mathcal{A}} \exp Q_1(s, a) da \geq -\varepsilon + \ln \int_{\mathcal{A}} \exp Q_2(s, a) da.$$

Значит:

$$\left| \ln \int_{\mathcal{A}} \exp Q_1(s, a) da - \ln \int_{\mathcal{A}} \exp Q_2(s, a) da \right| \leq \varepsilon.$$

Пусть B_{soft} — оператор, стоящий в правой части мягкого уравнения оптимальности Беллмана.

Тогда:

$$\begin{aligned} & |[B_{\text{soft}}Q_1](s, a) - [B_{\text{soft}}Q_2](s, a)| \\ &= \gamma \left| \mathbb{E}_{s'} \left[\ln \int_{\mathcal{A}} \exp Q_1(s', a') da' - \ln \int_{\mathcal{A}} \exp Q_2(s', a') da' \right] \right| \\ &\leq \gamma \varepsilon. \end{aligned}$$

Следовательно:

$$\rho(B_{\text{soft}}Q_1, B_{\text{soft}}Q_2) \leq \gamma \rho(Q_1, Q_2).$$

12.28 Критерий оптимальности через мягкую жадность

Утверждение 5. Если:

$$\pi(a | s) \propto \exp Q_{\text{soft}}^{\pi}(s, a),$$

то стратегия π оптимальна.

Доказательство. Если:

$$\pi(a | s) \propto \exp Q_{\text{soft}}^{\pi}(s, a),$$

то Q -функция такой стратегии удовлетворяет мягкому уравнению оптимальности Беллмана.

Но мягкое уравнение оптимальности Беллмана имеет единственное решение.

Следовательно:

$$Q_{\text{soft}}^{\pi}(s, a) = Q_{\text{soft}}^*(s, a).$$

Значит, стратегия π оптимальна.

12.29 Soft Q-learning

Теперь можно построить аналог DQN для Maximum Entropy RL.

Такой алгоритм называется:

Soft Q-learning.

В Soft Q-learning нет отдельной модели актора.

Текущая стратегия полагается мягко-жадной по отношению к текущему критику:

$$\pi(a | s) \propto \exp Q_{\theta}(s, a).$$

Это можно интерпретировать как моделирование **Soft Value Iteration**, то есть решение мягкого уравнения оптимальности Беллмана.

Для перехода:

$$T = (s, a, r, s')$$

target строится как:

$$y(T) := r + \gamma \ln \int_{\mathcal{A}} \exp Q_{\theta^-}(s', a') da',$$

где:

$$\theta^-$$

— параметры target-сети.

Затем критик обучается регрессией:

$$(Q_{\theta}(s, a) - y(T))^2 \rightarrow \min_{\theta}.$$

12.30 Связь Soft Q-learning и SAC

Soft Q-learning напрямую использует мягко-жадную стратегию:

$$\pi(a | s) \propto \exp Q_{\theta}(s, a).$$

Но в непрерывных пространствах действий нормировочная константа:

$$Z(s) = \int_{\mathcal{A}} \exp Q_{\theta}(s, a) da$$

часто не считается аналитически.

Кроме того, из такого распределения может быть трудно сэмплировать.

SAC решает эту проблему иначе: он вводит параметризованного актора:

$$\pi_{\theta}(a | s),$$

который учится приближать soft-жадную стратегию через задачу:

$$\text{KL} \left(\pi_{\theta}(\cdot | s) \left\| \frac{\exp Q_{\omega}(s, \cdot)}{Z_{\omega}(s)} \right) \right) \rightarrow \min_{\theta}.$$

12.31 Soft Policy Gradient

Теорема 36 (Soft Policy Gradient). *Для soft-функционала выполнено:*

$$\nabla_{\theta} J_{\text{soft}}(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_{\theta}}(s)} \mathbb{E}_{a \sim \pi_{\theta}(a | s)} [\nabla_{\theta} \ln \pi_{\theta}(a | s) Q_{\text{soft}}^{\pi}(s, a)] + \alpha \nabla_{\theta} H(\pi_{\theta}(\cdot | s)).$$

Пояснение. Эта формула является soft-аналогом обычной теоремы Policy Gradient.

В обычном Policy Gradient градиент зависит от:

$$\nabla_{\theta} \ln \pi_{\theta}(a | s) Q^{\pi}(s, a).$$

В Maximum Entropy RL появляется дополнительное слагаемое, связанное с изменением энтропии политики:

$$\alpha \nabla_{\theta} H(\pi_{\theta}(\cdot | s)).$$

12.32 Итог

В этой лекции была рассмотрена идея Maximum Entropy RL.

Maximum Entropy RL. Оптимизируется не только награда, но и энтропия политики:

$$J_{\text{soft}}(\pi) = \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} \gamma^t (r_t + \alpha H(\pi(\cdot | s_t))) \right].$$

Эквивалентно:

$$J_{\text{soft}}(\pi) = \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} \gamma^t (r_t - \alpha \ln \pi(a_t | s_t)) \right].$$

Soft Bellman equations. Обычные уравнения Беллмана модифицируются энтропийным членом:

$$Q_{\text{soft}}^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \mathbb{E}_{a' \sim \pi} [Q_{\text{soft}}^{\pi}(s', a') - \ln \pi(a' | s')].$$

Soft Policy Improvement. Улучшение политики задаётся максимизацией:

$$\mathbb{E}_{a \sim \pi(a|s)} [Q_{\omega}(s, a)] + H(\pi(\cdot | s)).$$

Это эквивалентно приближению распределения:

$$\pi(a | s) \propto \exp Q_{\omega}(s, a).$$

SAC. Soft Actor-Critic реализует эту идею через:

- стохастического актора;
- два критика;
- энтропийный бонус;
- off-policy обучение из replay buffer;
- мягкие targets для Q - и V -функций.

Soft Q-learning. Soft Q-learning является аналогом DQN для Maximum Entropy RL:

$$y(T) = r + \gamma \ln \int_{\mathcal{A}} \exp Q_{\theta-}(s', a') da'.$$

В нём оператор максимума заменён на log-sum-exp.

13 Лекция 13. Imitation Learning, Inverse RL, GAIL и GAIfo

13.1 Общая мотивация

Если рассматривать RL как попытку создания алгоритма искусственного интеллекта, то нужно дополнительно учитывать несколько фактов.

Во-первых, в одной и той же среде агент может ставить себе совершенно разные задачи.

Поэтому интеллектуальное обучение должно позволять:

- обобщать решения одних задач на другие;
- решать сложные задачи, состоящие из составных частей;
- самостоятельно ставить себе промежуточные задачи.

Во-вторых, в общем случае текущее наблюдение среды не описывает её состояние полностью.

Поэтому агент должен:

- обладать модулем памяти для запоминания предыдущих наблюдений;
- действовать в условиях неопределённости.

В-третьих, в среде могут присутствовать другие агенты.

Эти агенты могут:

- иметь схожие цели;
- иметь противоположные цели;
- передавать вспомогательную информацию;
- играть роль эксперта, демонстрирующего оптимальное или просто полезное поведение.

13.2 Клонирование поведения

13.2.1 Мотивация

Иногда проще получить демонстрации эксперта, чем вручную задавать функцию награды.

Пример: self-driving car. Чтобы обучить self-driving car, проще посадить реального водителя за руль и попросить собрать примеры траекторий, чем вручную описывать функцию награды, которая полностью кодирует правила дорожного движения.

Пример: робот и переливание воды. Чтобы обучить робота переливать воду из стакана в стакан, проще не придумывать функцию награды, описывающую такую задачу, а взять руку робота и несколько раз, «держа его за ручку», выполнить нужное движение.

13.2.2 Определение behavioral cloning

Определение 26. Клонированием поведения, или *behavioral cloning*, называется обучение стратегии воспроизводить действия эксперта:

$$\sum_T \sum_{(s,a) \in T} \ln \pi_\theta(a | s) \rightarrow \max_\theta.$$

Здесь:

- T — экспертные траектории;
- $(s, a) \in T$ — пары состояние–действие из экспертной траектории;
- $\pi_\theta(a | s)$ — обучаемая стратегия.

То есть behavioral cloning сводит задачу к supervised learning: состояние s играет роль входа, а действие эксперта a — роль правильного ответа.

13.3 Проблема behavioral cloning

Главная проблема behavioral cloning — distribution shift.

Экспертные данные покрывают только те состояния, которые посещал эксперт.

Обученная стратегия может немного ошибиться и попасть в состояние, которого не было в экспертной выборке. Тогда она не знает, как из него восстановиться.

На слайде с примером траектории это изображено как ситуация:

learned policy

съезжает с экспертной траектории, а данных о восстановлении нет:

no data on how to recover.

Интуиция. Маленькие ошибки стратегии могут накапливаться.

Эксперт в таких состояниях не бывал, поэтому supervised learning не учит агента возвращаться обратно на правильную траекторию.

13.4 DAgger

Одна универсальная идея борьбы с distribution shift — **DAgger**.

Схема:

1. Сначала обучить стратегию клонированием поведения по экспертным данным.
2. Запустить полученную стратегию в среду.
3. Собрать состояния, которые она реально посещает.
4. Попросить эксперта разметить эти состояния, то есть выбрать оптимальные действия.
5. Добавить новые размеченные пары (s, a) в датасет.
6. Дообучить стратегию.

Смысл. Мы хотим обучать стратегию не только на состояниях, которые посещает эксперт, но и на состояниях, в которые попадает сама обучаемая политика.

Ограничение. DAgger предполагает, что у нас есть средство разметки: эксперт может в любой момент сказать, какое действие оптимально в новом состоянии.

Именно за счёт этого задача снова сводится к обучению с учителем.

13.5 Пример: Quadcopter Navigation in the Forest

Иногда можно придумать способ получить в RL «правильные ответы».

В задаче **Quadcopter Navigation in the Forest** квадрокоптер учится лететь вдоль лесной тропинки.

На каждом шаге у него есть три действия:

влево, вперёд, вправо.

Чтобы собрать обучающую выборку, человек надевает шлем с тремя камерами:

- камера смотрит влево;
- камера смотрит вперёд;
- камера смотрит вправо.

Человек идёт по центру лесной тропинки.

После этого можно построить датасет:

- для камеры, смотрящей влево, правильный ответ — действие «вправо»;
- для центральной камеры правильный ответ — действие «вперёд»;
- для камеры, смотрящей вправо, правильный ответ — действие «влево».

Так задача сводится к обучению классификатора.

13.6 Обратное обучение с подкреплением

Определение 27. Задачей обратного обучения с подкреплением, или *inverse reinforcement learning*, *IRL*, называется задача по набору траекторий:

$$T$$

оптимального агента восстановить функцию награды, которую он максимизирует.

В обычном RL задана награда:

$$r(s, a),$$

и мы ищем оптимальную стратегию:

$$\pi^*.$$

В IRL наоборот: есть демонстрации оптимальной или хорошей стратегии, и нужно понять, какую награду эта стратегия могла оптимизировать.

13.7 Пример IRL в клеточном мире

Рассмотрим клеточный мир, где агент может ходить:

вправо, влево, вниз, вверх.

Для простоты допустим, что функция награды:

- детерминированная;
- зависит только от состояния;
- на клетках одного цвета имеет одинаковые значения.

Имея одну экспертную траекторию, порождённую оптимальной стратегией, мы хотим понять, что можно сказать о наградах клеток разных цветов.

На картинке в лекции эксперт идёт к зелёной клетке, проходя через часть поля.

13.7.1 Зелёные клетки

Агент, видимо, стремился в зелёную клетку.

Поэтому естественно предположить, что за зелёную клетку полагается положительная награда:

$$r_{\text{green}} > 0.$$

13.7.2 Красные клетки

Агент прошёл через красную клетку, хотя мог бы её обойти, но тогда попал бы в зелёную клетку позже.

Из этого можно сделать слабый вывод:

штраф за красную клетку, если он есть, не слишком большой.

То есть красная клетка могла иметь небольшую отрицательную награду:

$$r_{\text{red}} < 0,$$

но по модулю она, видимо, меньше выгоды от более быстрого попадания в зелёную клетку.

Могла ли красная награда быть положительной?

Если бы она была сильно положительной, эксперт, возможно, походил бы по красным клеткам. Поэтому, если красная награда положительна, то награда за зелёную всё равно должна быть сильно выгоднее.

13.7.3 Синие клетки

Синие клетки агент избегал.

Возможные объяснения:

- синие клетки дают штраф;

- синие клетки дают нулевую награду, но путь через них невыгоден;
- если бы они давали заметный бонус, то было бы выгодно добираться до зелёной клетки через них.

Поэтому можно предположить:

$$r_{\text{blue}} \leq 0$$

или, по крайней мере, не слишком положительная.

13.7.4 Жёлтые клетки

Про жёлтые клетки сказать почти ничего нельзя.

Агенту пришлось бы в любом случае пройти через них, чтобы добраться до зелёной клетки.

Поэтому в них может быть:

- небольшой штраф;
- небольшая положительная награда;
- нулевая награда.

Главное: по одной экспертной траектории награда восстанавливается неоднозначно.

13.8 Некорректность задачи IRL

Пример с клеточным миром показывает, что задача IRL плохо определена.

Одна и та же экспертная траектория может быть оптимальной для многих разных функций награды.

Например, если эксперт идёт по некоторому пути, это может означать:

- он стремится к цели;
- он избегает штрафов;
- он предпочитает короткие пути;
- он проходит через обязательные клетки;
- или несколько этих факторов одновременно.

Поэтому в IRL нужно вводить дополнительные предположения о том, какая функция награды считается предпочтительной.

13.9 Maximum Entropy предположение в IRL

Для упрощения формул далее положим:

$$\gamma = 1.$$

Введём предположение: оптимальная стратегия π^* стохастична и генерирует такие траектории, что:

$$p(T \mid \pi^*) \propto e^{R(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t).$$

Здесь:

$$R(T) := \sum_{t \geq 0} r(s_t, a_t)$$

— суммарная награда траектории.

Интуиция. Чем выше награда траектории, тем вероятнее эксперт её сгенерирует.

Но вероятность траектории также зависит от динамики среды:

$$\prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t).$$

13.10 Связь с Maximum Entropy RL

Откуда берётся предположение:

$$p(T \mid \pi^*) \propto e^{R(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t)?$$

Это уже знакомый фреймворк Maximum Entropy RL.

Рассмотрим задачу поиска стратегии, которая порождает траектории из распределения:

$$p(T \mid \pi^*).$$

То есть:

$$\text{KL}(p(T \mid \pi) \parallel p(T \mid \pi^*)) \rightarrow \min_{\pi}.$$

Теорема 37. Задача:

$$\text{KL}(p(T \mid \pi) \parallel p(T \mid \pi^*)) \rightarrow \min_{\pi}$$

эквивалентна задаче Maximum Entropy RL.

Доказательство. Распишем KL-дивергенцию:

$$\text{KL}(p(T \mid \pi) \parallel p(T \mid \pi^*)) = \mathbb{E}_{T \sim \pi} [\ln p(T \mid \pi) - \ln p(T \mid \pi^*)].$$

Вероятность траектории под стратегией π :

$$p(T \mid \pi) = \prod_{t \geq 0} \pi(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t).$$

Поэтому:

$$\ln p(T \mid \pi) = \sum_{t \geq 0} [\ln \pi(a_t \mid s_t) + \ln p(s_{t+1} \mid s_t, a_t)].$$

По Maximum Entropy предположению:

$$p(T \mid \pi^*) = \frac{e^{R(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t)}{Z},$$

где Z — нормировочная константа, не зависящая от π .

Тогда:

$$\ln p(T \mid \pi^*) = R(T) + \sum_{t \geq 0} \ln p(s_{t+1} \mid s_t, a_t) - \ln Z.$$

Подставим:

$$\begin{aligned} & \text{KL}(p(T \mid \pi) \parallel p(T \mid \pi^*)) \\ &= \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} (\ln \pi(a_t \mid s_t) + \ln p(s_{t+1} \mid s_t, a_t)) - R(T) - \sum_{t \geq 0} \ln p(s_{t+1} \mid s_t, a_t) + \ln Z \right]. \end{aligned}$$

Логарифмы вероятностей переходов сокращаются.

Остаётся:

$$\text{KL} = \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} \ln \pi(a_t \mid s_t) - \sum_{t \geq 0} r(s_t, a_t) \right] + \ln Z.$$

Так как $\ln Z$ не зависит от π , минимизация KL эквивалентна максимизации:

$$\mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} (r(s_t, a_t) - \ln \pi(a_t \mid s_t)) \right] \rightarrow \max_{\pi}.$$

Это ровно Maximum Entropy RL при $\gamma = 1$.

13.11 Ноль в KL-задаче может быть недостижим

Утверждение 6. *Ноль в задаче:*

$$\text{KL}(p(T \mid \pi) \parallel p(T \mid \pi^*)) \rightarrow \min_{\pi}$$

не обязательно достижим.

Контрпример. Рассмотрим MDP с единственным состоянием.

Пусть от действий ничего не зависит, но:

- с вероятностью 0.5 агент получает награду:

$$+ \ln 2;$$

- с вероятностью 0.5 агент получает награду:

$$0;$$

- после этого игра заканчивается.

Распределение:

$$p(T \mid \pi^*) \propto e^{R(T)}$$

говорит, что траектория с наградой $+\ln 2$ должна встречаться с вероятностью:

$$\frac{e^{\ln 2}}{e^{\ln 2} + e^0} = \frac{2}{3},$$

а траектория с наградой 0 — с вероятностью:

$$\frac{1}{3}.$$

Но любая стратегия будет получать эти две траектории с вероятностями:

$$0.5 \quad \text{и} \quad 0.5,$$

потому что исход определяется средой, а не действиями агента.

Следовательно, точно совпасть с распределением $p(T \mid \pi^*)$ невозможно.

13.12 Guided Cost Learning

Теперь аппроксимируем функцию награды нейросетью:

$$r_\theta(s, a).$$

Тогда суммарная награда траектории:

$$R_\theta(T) := \sum_{(s,a) \in T} r_\theta(s, a).$$

В предположении Maximum Entropy вероятность одной экспертной траектории равна:

$$p_\theta(T \mid \pi^*) = \frac{e^{R_\theta(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t)}{Z(\theta)}.$$

Здесь нормировочная константа зависит от параметров функции награды:

$$Z(\theta) := \int_T e^{R_\theta(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t) dT.$$

13.13 Логарифм правдоподобия

Логарифм правдоподобия экспертной траектории:

$$\ln p_\theta(T \mid \pi^*) = R_\theta(T) + \sum_{t \geq 0} \ln p(s_{t+1} \mid s_t, a_t) - \ln Z(\theta).$$

Второе слагаемое:

$$\sum_{t \geq 0} \ln p(s_{t+1} \mid s_t, a_t)$$

не зависит от параметров награды θ .

Поэтому при оптимизации по θ важны только:

$$R_\theta(T) - \ln Z(\theta).$$

13.14 Оптимальная стратегия для текущей награды

Пусть у нас есть некоторая функция награды с параметрами θ .

Обозначим:

$$\pi^*[\theta]$$

стратегию, которая оптимально в Maximum Entropy смысле оптимизирует текущую награду:

$$r_\theta(s, a).$$

Такая стратегия по определению генерирует траектории из распределения:

$$p(T \mid \pi^*[\theta]) := \frac{e^{R_\theta(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t)}{Z(\theta)}.$$

13.15 Теорема Guided Cost Learning

Теорема 38 (Guided Cost Learning). *Градиент для оптимизации правдоподобия экспертных траекторий по параметрам функции награды θ равен:*

$$\mathbb{E}_{T \sim \pi^*} [\nabla_\theta R_\theta(T)] - \mathbb{E}_{T \sim \pi^*[\theta]} [\nabla_\theta R_\theta(T)].$$

Доказательство. Рассмотрим градиент логарифма правдоподобия одной траектории:

$$\nabla_\theta \ln p_\theta(T \mid \pi^*) = \nabla_\theta R_\theta(T) - \nabla_\theta \ln Z(\theta).$$

Нужно оптимизировать правдоподобие в среднем по траекториям эксперта:

$$T \sim \pi^*.$$

Поэтому:

$$\mathbb{E}_{T \sim \pi^*} [\nabla_\theta \ln p_\theta(T \mid \pi^*)] = \mathbb{E}_{T \sim \pi^*} [\nabla_\theta R_\theta(T)] - \nabla_\theta \ln Z(\theta).$$

Осталось вычислить:

$$\nabla_\theta \ln Z(\theta).$$

По определению:

$$\nabla_\theta \ln Z(\theta) = \frac{1}{Z(\theta)} \nabla_\theta Z(\theta).$$

Далее:

$$\begin{aligned} \nabla_\theta Z(\theta) &= \nabla_\theta \int_T e^{R_\theta(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t) dT \\ &= \int_T \nabla_\theta e^{R_\theta(T)} \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t) dT \\ &= \int_T e^{R_\theta(T)} \nabla_\theta R_\theta(T) \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t) dT. \end{aligned}$$

Значит:

$$\begin{aligned} \nabla_\theta \ln Z(\theta) &= \frac{1}{Z(\theta)} \int_T e^{R_\theta(T)} \nabla_\theta R_\theta(T) \prod_{t \geq 0} p(s_{t+1} \mid s_t, a_t) dT \\ &= \mathbb{E}_{T \sim p(T \mid \pi^*[\theta])} [\nabla_\theta R_\theta(T)]. \end{aligned}$$

Следовательно:

$$\mathbb{E}_{T \sim \pi^*} [\nabla_\theta \ln p_\theta(T \mid \pi^*)] = \mathbb{E}_{T \sim \pi^*} [\nabla_\theta R_\theta(T)] - \mathbb{E}_{T \sim \pi^*[\theta]} [\nabla_\theta R_\theta(T)].$$

13.16 Интерпретация Guided Cost Learning

Градиент имеет вид:

$$\mathbb{E}_{T \sim \pi^*} [\nabla_{\theta} R_{\theta}(T)] - \mathbb{E}_{T \sim \pi^*[\theta]} [\nabla_{\theta} R_{\theta}(T)] .$$

Это означает:

- награда экспертных траекторий должна увеличиваться;
- награда траекторий, которые порождает текущая оптимальная стратегия для нашей награды, должна уменьшаться.

Иначе говоря, функция награды обучается так, чтобы экспертные траектории выглядели лучше, чем траектории, которые текущая модель считает хорошими.

13.17 Переход к GAIL

Generative Adversarial Imitation Learning, или **GAIL**, связывает обратное обучение с подкреплением и GAN-подобную оптимизацию.

Утверждение 7. *Оптимизация функции награды по формуле Guided Cost Learning соответствует оптимизации функционала:*

$$\mathbb{E}_{T \sim \pi^*} [R(T)] - \max_{\pi} \left\{ \mathbb{E}_{T \sim \pi} \sum_{t \geq 0} [r(s_t, a_t) + H(\pi(\cdot \mid s_t))] \right\} \rightarrow \max_r .$$

Пояснение. Градиент максимума функции равен градиенту этой функции в точке максимума.

При этом энтропия:

$$H(\pi(\cdot \mid s_t))$$

не зависит от функции награды r .

Поэтому градиент второго слагаемого совпадает со вторым слагаемым в формуле Guided Cost Learning:

$$\mathbb{E}_{T \sim \pi^*[\theta]} [\nabla_{\theta} R_{\theta}(T)] .$$

13.18 Occupancy measure

Определение 28. *Occupancy measure для стратегии π называется:*

$$\rho_{\pi}(s, a) := \pi(a \mid s) d_{\pi}(s),$$

где $d_{\pi}(s)$ — частоты посещения состояний.

Смысл:

$$\rho_{\pi}(s, a)$$

описывает, насколько часто стратегия π посещает пару состояние–действие (s, a) .

13.19 Ожидание через occupancy measure

По определению можно записывать:

$$\mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} f(s_t, a_t) \right] = \mathbb{E}_{\rho_\pi} [f(s, a)].$$

То есть вместо ожидания по траекториям можно работать с распределением посещаемых пар:

$$(s, a).$$

13.20 Минимаксная форма через occupancy measure

Используем occupancy measure в функционале для GAIL.

Раскрывая математические ожидания и переписывая оптимизацию по π как максимизацию, получаем игру:

$$\min_r \max_\pi \left\{ \mathbb{E}_{\rho_\pi} [r(s, a)] - \mathbb{E}_{\rho_{\pi^*}} [r(s, a)] + \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} H(\pi(\cdot | s_t)) \right] \right\}.$$

Это можно читать как поиск седловой точки.

Функция награды пытается отличить экспертные пары от пар, порождённых текущей стратегией.

Стратегия пытается максимизировать награду и энтропию.

13.21 Стратегия через occupancy measure

Утверждение 8. Для occupancy measure выполнено:

$$\pi(a | s) = \frac{\rho_\pi(s, a)}{\int_{\mathcal{A}} \rho_\pi(s, a) da}.$$

Пояснение. По определению:

$$\rho_\pi(s, a) = \pi(a | s) d_\pi(s).$$

Интегрируем по действиям:

$$\int_{\mathcal{A}} \rho_\pi(s, a) da = d_\pi(s) \int_{\mathcal{A}} \pi(a | s) da = d_\pi(s).$$

Следовательно:

$$\frac{\rho_\pi(s, a)}{\int_{\mathcal{A}} \rho_\pi(s, a) da} = \frac{\pi(a | s) d_\pi(s)}{d_\pi(s)} = \pi(a | s).$$

13.22 Энтروпийный бонус через occupancy measure

По предыдущему утверждению стратегию можно восстановить из occupancy measure.

Значит, суммарный энтропийный бонус тоже можно переписать в терминах ρ_π .

Обозначим:

$$\tilde{H}(\rho_\pi) := \mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} H(\pi(\cdot \mid s_t)) \right].$$

Тогда:

$$\begin{aligned} \tilde{H}(\rho_\pi) &= \mathbb{E}_{\rho_\pi} [-\ln \pi(a \mid s)] \\ &= -\mathbb{E}_{\rho_\pi} \left[\ln \left(\frac{\rho_\pi(s, a)}{\int_{\mathcal{A}} \rho_\pi(s, a) da} \right) \right]. \end{aligned}$$

Поэтому минимаксную задачу можно переписать как:

$$\min_r \max_{\rho_\pi} \left\{ \mathbb{E}_{\rho_\pi} [r(s, a)] - \mathbb{E}_{\rho_{\pi^*}} [r(s, a)] + \tilde{H}(\rho_\pi) \right\}.$$

13.23 Регуляризация функции награды

Как обсуждалось ранее, задача обратного обучения с подкреплением некорректна и может иметь много решений.

Поэтому добавим регуляризатор для функции награды:

$$\psi(r).$$

Он штрафует выдаваемую функцию награды и выбирает из множества возможных решений более предпочтительное.

Получаем задачу:

$$\min_r \max_{\rho_\pi} \left\{ \psi(r) + \mathbb{E}_{\rho_\pi} [r(s, a)] - \mathbb{E}_{\rho_{\pi^*}} [r(s, a)] + \tilde{H}(\rho_\pi) \right\}.$$

Оказывается, vanilla GAN, различающий сэмплы пар (s, a) из распределений:

$$p(T \mid \pi^*)$$

и:

$$p(T \mid \pi),$$

соответствует специальному выбору регуляризатора.

13.24 Регуляризатор, приводящий к GAIL

Теорема 39. *Выберем регуляризатор:*

$$\psi(r) := \mathbb{E}_{\rho_{\pi^*}} [r(s, a) + \ln(1 - e^{-r(s, a)})],$$

где штраф полагается бесконечно большим, если:

$$r(s, a) \leq 0.$$

Тогда задача:

$$\min_r \max_{\rho_\pi} \left\{ \psi(r) + \mathbb{E}_{\rho_\pi} [r(s, a)] - \mathbb{E}_{\rho_{\pi^*}} [r(s, a)] + \tilde{H}(\rho_\pi) \right\}$$

примет вид:

$$\min_D \max_{\pi} \left\{ -\mathbb{E}_{\rho_{\pi^*}} [\ln(1 - D(s, a))] - \mathbb{E}_{\rho_\pi} [\ln D(s, a)] + \tilde{H}(\rho_\pi) \right\},$$

где:

$$D(s, a) \in (0, 1).$$

Доказательство. Подставим регуляризатор:

$$\begin{aligned} & \psi(r) + \mathbb{E}_{\rho_\pi} [r(s, a)] - \mathbb{E}_{\rho_{\pi^*}} [r(s, a)] + \tilde{H}(\rho_\pi) \\ &= \mathbb{E}_{\rho_{\pi^*}} [r(s, a) + \ln(1 - e^{-r(s, a)})] + \mathbb{E}_{\rho_\pi} [r(s, a)] - \mathbb{E}_{\rho_{\pi^*}} [r(s, a)] + \tilde{H}(\rho_\pi) \\ &= \mathbb{E}_{\rho_{\pi^*}} [\ln(1 - e^{-r(s, a)})] + \mathbb{E}_{\rho_\pi} [r(s, a)] + \tilde{H}(\rho_\pi). \end{aligned}$$

Сделаем замену переменных:

$$D(s, a) := e^{-r(s, a)}.$$

Так как:

$$r(s, a) > 0,$$

то:

$$D(s, a) \in (0, 1).$$

Также:

$$r(s, a) = -\ln D(s, a).$$

Тогда:

$$\mathbb{E}_{\rho_{\pi^*}} [\ln(1 - e^{-r(s, a)})] = \mathbb{E}_{\rho_{\pi^*}} [\ln(1 - D(s, a))],$$

и:

$$\mathbb{E}_{\rho_\pi} [r(s, a)] = -\mathbb{E}_{\rho_\pi} [\ln D(s, a)].$$

С учётом исходной минимизации по D получаем:

$$\min_D \max_{\pi} \left\{ -\mathbb{E}_{\rho_{\pi^*}} [\ln(1 - D(s, a))] - \mathbb{E}_{\rho_\pi} [\ln D(s, a)] + \tilde{H}(\rho_\pi) \right\}.$$

13.25 Интерпретация GAIL

В GAIL вместо явного обучения награды обучается дискриминатор:

$$D(s, a) \in (0, 1).$$

Он решает задачу бинарной классификации:

- пары (s, a) из ρ_{π^*} — экспертный класс;
- пары (s, a) из ρ_π — класс текущей стратегии.

При фиксированной стратегии π оптимизация по D выглядит как:

$$\mathbb{E}_{\rho_{\pi^*}} [\ln(1 - D(s, a))] + \mathbb{E}_{\rho_\pi} [\ln D(s, a)] \rightarrow \max_D.$$

То есть дискриминатор учится отличать пары состояние–действие, встреченные экспертом, от пар, которые встречает текущая стратегия.

Замечание о метках. В этой записи класс эксперта соответствует:

$$D(s, a) \approx 0,$$

а класс текущей стратегии соответствует:

$$D(s, a) \approx 1.$$

Это просто соглашение о том, какой класс как кодируется.

13.26 Оптимизация стратегии в GAIL

При фиксированном дискриминаторе D оптимизация по стратегии выглядит так:

$$\mathbb{E}_{T \sim \pi} \left[\sum_{t \geq 0} (-\ln D(s_t, a_t) + H(\pi(\cdot \mid s_t))) \right] \rightarrow \max_{\pi}.$$

То есть можно рассматривать внутреннюю награду:

$$r_D(s, a) = -\ln D(s, a).$$

Стратегия обучается так, чтобы посещать такие пары (s, a) , которые дискриминатор считает похожими на экспертные.

13.27 GAIL как состязательная схема

GAIL можно понимать как GAN-подобную игру:

1. Дискриминатор $D(s, a)$ учится отличать экспертные пары от пар текущей стратегии.
2. Стратегия π учится обманывать дискриминатор, то есть посещать пары, похожие на экспертные.
3. Дополнительно стратегия получает энтропийный бонус, чтобы сохранять стохастичность и exploration.

Схематически:

$$\pi \longleftrightarrow D$$

где:

$$D$$

играет роль learned reward model, а:

$$\pi$$

играет роль генератора поведения.

13.28 Generative Adversarial Imitation from Observation

Часто эксперт предоставляет траектории, в которых отсутствует информация о совершённых действиях.

То есть у нас есть только цепочки состояний:

$$s_0, s_1, s_2, \dots$$

Такая задача называется:

imitation learning from observations.

Generative Adversarial Imitation from Observation, или **GAIfO**, решает задачу имитационного обучения только по наблюдениям.

13.29 Пример GAIfO

Допустим, есть кадровые анимации того, как персонаж делает сальто.

Мы хотим научить робота с такими же конечностями делать то же самое.

В анимациях понятно:

- в каких координатах находились конечности персонажа;
- в какой момент времени они там находились.

Можно считать, что робот должен посещать похожие цепочки состояний.

Но в анимации нет действий:

$$a_t.$$

Мы не знаем, как именно нужно управлять роботом, чтобы получить такую же траекторию в реальной среде.

13.30 GAIL без действий

Если действий нет, можно пытаться различать только состояния.

Дискриминатор:

$$D(s) \in (0, 1)$$

учится различать:

$$s \sim d_{\pi^*}(s)$$

и:

$$s \sim d_{\pi}(s).$$

Тогда минимаксная задача:

$$\min_D \max_{\pi} \left\{ -\mathbb{E}_{d_{\pi^*}(s)} [\ln(1 - D(s))] - \mathbb{E}_{d_{\pi}(s)} [\ln D(s)] + \mathbb{E}_{d_{\pi}(s)} [H(\pi(\cdot | s))] \right\}.$$

Проблема. Одного состояния может быть недостаточно: важно не только где находится агент, но и как он туда пришёл и куда движется дальше.

13.31 GAIfo: дискриминация пар соседних состояний

В GAIfo предлагается различать не отдельные состояния, а пары:

$$(s, s').$$

То есть награда считается зависящей от пары соседних состояний:

$$r(s, s'),$$

а дискриминатор:

$$D(s, s')$$

учится различать пары:

$$(s, s')$$

из экспертных траекторий и из траекторий текущей стратегии.

13.32 Оссипансу measure для пар состояний

Формально вводится альтернативное определение оссипансу measure.

Определение 29. *Оссипансу measure для пар состояний определяется как вероятность встретить пару:*

$$(s, s')$$

в траекториях стратегии π :

$$\nu_\pi(s, s') := d_\pi(s) \int_{\mathcal{A}} p(s' \mid s, a) \pi(a \mid s) da.$$

Здесь:

- $d_\pi(s)$ — частота посещения состояния s ;
- $\pi(a \mid s)$ — стратегия;
- $p(s' \mid s, a)$ — динамика среды.

13.33 Минимаксная задача GAIfo

С использованием:

$$\nu_\pi(s, s')$$

минимаксная задача GAIfo принимает вид:

$$\min_D \max_\pi \left\{ -\mathbb{E}_{\nu_\pi^*} [\ln(1 - D(s, s'))] - \mathbb{E}_{\nu_\pi} [\ln D(s, s')] + \mathbb{E}_{d_\pi(s)} [H(\pi(\cdot \mid s))] \right\}.$$

Смысл. Стратегия учится не просто посещать похожие состояния, а воспроизводить похожие переходы:

$$s \rightarrow s'.$$

Это лучше подходит для имитации движения, потому что пара соседних состояний несёт информацию о динамике поведения.

13.34 Сравнение BC, IRL, GAIL и GAIfo

Метод	Идея
Behavioral Cloning	Обучить стратегию напрямую воспроизводить действия эксперта: $\ln \pi_{\theta}(a s) \rightarrow \max$.
Dagger	Запускать текущую стратегию, собирать посещённые ею состояния и просить эксперта разметить правильные действия.
IRL	Восстановить функцию награды, которую мог оптимизировать эксперт.
Guided Cost Learning	Обучать награду так, чтобы экспертные траектории имели большую вероятность, чем траектории текущей оптимальной стратегии.
GAIL	Заменить явную награду дискриминатором, который отличает экспертные пары (s, a) от пар текущей стратегии.
GAIfo	Когда действий эксперта нет, отличать пары соседних состояний (s, s') вместо пар (s, a) .

13.35 Итог

В этой лекции были рассмотрены методы обучения по демонстрациям.

Behavioral cloning. Самый прямой подход: использовать экспертные пары:

$$(s, a)$$

как supervised dataset и максимизировать:

$$\sum_T \sum_{(s,a) \in T} \ln \pi_{\theta}(a | s).$$

Главная проблема — стратегия может попасть в состояния, которых нет в экспертной выборке.

Inverse RL. В IRL по экспертным траекториям пытаются восстановить награду.

Задача неоднозначна: одна и та же траектория может быть оптимальной для многих разных наград.

Maximum Entropy IRL. Maximum Entropy предположение задаёт распределение экспертных траекторий:

$$p(T | \pi^*) \propto e^{R(T)} \prod_{t \geq 0} p(s_{t+1} | s_t, a_t).$$

Это приводит к Maximum Entropy RL.

Guided Cost Learning. Градиент обучения награды имеет вид:

$$\mathbb{E}_{T \sim \pi^*} [\nabla_{\theta} R_{\theta}(T)] - \mathbb{E}_{T \sim \pi^*[\theta]} [\nabla_{\theta} R_{\theta}(T)].$$

Экспертные траектории получают большую награду, а траектории текущей модели — меньшую.

GAIL. GAIL переписывает IRL как GAN-подобную игру:

$$D(s, a)$$

отличает экспертные пары от пар текущей стратегии, а стратегия пытается обмануть дискриминатор.

GAIfO. GAIfO применяется, когда у эксперта есть только состояния, но нет действий.

Вместо пар:

$$(s, a)$$

используются пары соседних состояний:

$$(s, s').$$

14 Лекция 14. Multi-Armed Bandit, UCB, Thompson Sampling, World Models и современные результаты

14.1 Multi-Armed Bandit Problem

14.1.1 Постановка задачи

Задача многорукого бандита, или **multi-armed bandit problem**, является упрощённой RL-задачей.

В ней агент на каждом эпизоде выбирает действие:

$$a \in \mathcal{A}$$

согласно некоторой политике:

$$\pi(a),$$

и получает награду:

$$r \sim p(r \mid a).$$

В отличие от полноценной MDP, здесь нет состояния, которое меняется во времени. Формально такую задачу можно считать MDP, зависящей только от выбора действия.

Ценность действия:

$$Q(a) = \mathbb{E}_{p(r|a)} [r].$$

Оптимальное значение:

$$V^* = \max_{a \in \mathcal{A}} \{Q(a)\}.$$

Оптимальное действие:

$$a^* \in \arg \max_{a \in \mathcal{A}} Q(a).$$

14.1.2 Обновление стратегии

Стратегия меняется каждый эпизод.

На k -ом эпизоде:

$$\begin{aligned} a_k &\sim \pi_k(a), \\ r_k &\sim p(r \mid a_k). \end{aligned}$$

После этого политика обновляется:

$$\pi_k \longrightarrow \pi_{k+1}.$$

Для T итераций оптимально было бы каждый раз выбирать действие, дающее максимальную среднюю награду:

$$V^*.$$

Тогда суммарная оптимальная награда в ожидании равна:

$$TV^*.$$

Но агент не знает заранее, какое действие оптимально, поэтому иногда выбирает неоптимальные действия.

14.2 Regret

Определение 30. *Regret*, или *функция сожаления*, за T итераций определяется как:

$$\mathfrak{R}^T = TV^* - \sum_{k=0}^{T-1} Q(a_k).$$

Здесь:

$$Q(a_k)$$

— истинная средняя награда действия, выбранного на шаге k .

Цель задачи многорукого бандита:

$$\mathfrak{R}^T \rightarrow \min.$$

Интуитивно regret показывает, сколько агент потерял из-за того, что не всегда выбирал оптимальную руку.

Если агент всегда выбирает оптимальное действие:

$$a_k = a^*,$$

то:

$$\mathfrak{R}^T = 0.$$

Если агент часто выбирает плохие действия, regret растёт быстро.

14.3 Exploration–exploitation dilemma

На каждой итерации агент сталкивается с выбором:

- проверить новую или недостаточно изученную руку — **exploration**;
- продолжать играть той рукой, которая сейчас кажется лучшей — **exploitation**.

Если слишком много исследовать, агент теряет награду на заведомо плохих действиях.

Если слишком быстро начать эксплуатировать текущую лучшую оценку, агент может застрять на неоптимальной руке.

Это и есть дилемма:

exploration vs exploitation.

14.4 Простая Monte-Carlo оценка $Q(a)$

Можно оценивать $Q(a)$ по Монте-Карло.

После k шагов:

$$Q_k(a) = \frac{\sum_{i=0}^k r_i \mathbb{I}[a_i = a]}{\sum_{i=0}^k \mathbb{I}[a_i = a]}.$$

Здесь:

$$\mathbb{I}[a_i = a]$$

— индикатор того, что на шаге i было выбрано действие a .

Обозначим:

$$n_k(a)$$

число раз, сколько действие a было выбрано за первые $k + 1$ итераций.

Тогда для выбранного действия a_k можно обновлять оценку инкрементально:

$$Q_k(a_k) = \left(1 - \frac{1}{n_k(a_k)}\right) Q_{k-1}(a_k) + \frac{1}{n_k(a_k)} r_k.$$

Эквивалентная форма:

$$Q_k(a_k) = Q_{k-1}(a_k) + \frac{1}{n_k(a_k)} (r_k - Q_{k-1}(a_k)).$$

Для действий:

$$a \neq a_k$$

оценка не меняется:

$$Q_k(a) = Q_{k-1}(a).$$

14.5 Линейный рост regret

Верхняя оценка. При любом алгоритме скорость роста сожалений не более чем линейная: для некоторой константы C :

$$\mathbb{E} [\mathfrak{R}^T] \leq CT.$$

Это естественно: на каждом шаге потери ограничены, значит за T шагов невозможно накопить больше, чем линейный regret.

Жадный выбор. При жадном выборе сожаления могут расти с линейной скоростью: для некоторой константы C :

$$\mathbb{E} [\mathfrak{R}^T] \geq CT.$$

Жадный алгоритм всегда выбирает действие с максимальной текущей оценкой:

$$a_k \in \arg \max_{a \in \mathcal{A}} Q_k(a).$$

Проблема в том, что если в начале оценка неоптимальной руки случайно оказалась завышенной, агент может продолжать её выбирать и не исследовать остальные действия.

14.6 ε -жадная стратегия

Наивное решение проблемы исследования — использовать ε -жадную стратегию.

На шаге k :

$$a_k = \begin{cases} \text{случайное действие из } \mathcal{A}, & \text{с вероятностью } \varepsilon(k), \\ \arg \max_{a \in \mathcal{A}} Q_k(a), & \text{с вероятностью } 1 - \varepsilon(k). \end{cases}$$

Если:

$$\varepsilon(k) > 0,$$

то агент иногда исследует случайные действия.

Если:

$$\varepsilon(k) \rightarrow 0,$$

то агент постепенно переходит к эксплуатации.

14.7 Алгоритм: наивный бандит

Гиперпараметры.

$$\varepsilon(k)$$

— стратегия исследования.

Инициализация. Инициализировать:

$$Q_0(a)$$

произвольно.

Обнулить счётчики:

$$n_0(a) := 0.$$

На очередном шаге k :

1. Выбрать действие:

$$a_k$$

случайно с вероятностью $\varepsilon(k)$, иначе:

$$a_k := \arg \max_{a \in \mathcal{A}} Q_k(a).$$

2. Увеличить счётчик:

$$n_k(a) := n_{k-1}(a) + \mathbb{I}[a_k = a].$$

3. Пронаблюдать награду:

$$r_k.$$

4. Обновить оценку:

$$Q_k(a) := Q_{k-1}(a) + \mathbb{I}[a_k = a] \frac{1}{n_k(a)} (r_k - Q_{k-1}(a)).$$

14.8 Regret при ε -жадном выборе

Утверждение. При ε -жадном выборе с постоянным ε сожаления всё равно растут с линейной скоростью: для некоторой константы C :

$$\mathbb{E} [\mathfrak{R}^T] \geq CT.$$

Доказательство. Пусть существует неоптимальное действие a :

$$Q(a) < V^*.$$

Его regret-gap:

$$\Delta(a) := V^* - Q(a) > 0.$$

При ε -жадной стратегии на каждом шаге с вероятностью:

$$\frac{\varepsilon}{|\mathcal{A}|}$$

мы выбираем именно это неоптимальное действие a .

Тогда в среднем regret содержит слагаемое:

$$T \frac{\varepsilon}{|\mathcal{A}|} (V^* - Q(a)).$$

Следовательно можно взять:

$$C = \frac{\varepsilon}{|\mathcal{A}|} (V^* - Q(a)),$$

и получить:

$$\mathbb{E} [\mathfrak{R}^T] \geq CT.$$

Вывод. Постоянная ε -жадность гарантирует постоянное исследование, но из-за этого агент продолжает выбирать неоптимальные действия даже тогда, когда уже всё понял.

Поэтому regret остаётся линейным.

14.9 Нестационарный бандит

В нестационарных бандитах распределения наград могут меняться со временем:

$$p(r \mid a) = p_k(r \mid a).$$

То есть рука, которая раньше была хорошей, позже может стать плохой, и наоборот.

В этом случае:

- ε нельзя уменьшать к нулю, потому что нужно постоянно проверять разные действия;
- можно выставить ε константой;
- вместо обычного среднего по всем прошлым наблюдениям лучше использовать экспоненциальное сглаживание.

Обновление:

$$Q_k(a_k) = Q_{k-1}(a_k) + \alpha (r_k - Q_{k-1}(a_k)),$$

где:

$$0 < \alpha < 1.$$

Здесь α — постоянный шаг обновления.

Интуиция. Если использовать:

$$\frac{1}{n_k(a)}$$

как шаг, то старые наблюдения будут иметь слишком большой вес.

В нестационарной задаче это плохо: старые награды могут уже не отражать текущую реальность.

Экспоненциальное сглаживание забывает старые наблюдения.

14.10 Теорема Лаи–Роббинса

Regret можно переписать через количество выборов каждого действия:

$$\mathfrak{R}^T = \sum_{a \in \mathcal{A}} n_{T-1}(a) (V^* - Q(a)).$$

Здесь:

$$n_{T-1}(a)$$

— сколько раз действие a было выбрано за первые T итераций.

Теорема 40 (Теорема Лаи–Роббинса). *Для задачи многорукого бандита справедлива нижняя оценка:*

$$\mathbb{E} [\mathfrak{R}^T] \geq \ln T \sum_{\substack{a \in \mathcal{A} \\ a \neq a^*}} \frac{V^* - Q(a)}{\text{KL}(p(\cdot \mid a) \parallel p(\cdot \mid a^*))}.$$

Смысл. Даже оптимальный алгоритм не может иметь regret, растущий существенно медленнее:

$$\ln T.$$

Алгоритм называется **асимптотически оптимальным**, если он имеет логарифмическую скорость роста среднего regret:

$$\mathbb{E} [\mathfrak{R}^T] = O(\ln T).$$

14.11 Upper Confidence Bound

Upper Confidence Bound, или **UCB**, — это семейство алгоритмов, основанных на принципе оптимизма перед неопределённостью.

На очередном шаге k выбирается действие:

$$a_k := \arg \max_{a \in \mathcal{A}} [Q_k(a) + U_k(a)].$$

Здесь:

$$U_k(a) > 0$$

— бонус за исследование, или **exploration bonus**.

Принцип оптимизма. Если мы не уверены в ценности действия, нужно считать, что оно может оказаться хорошим.

Поэтому к текущей оценке:

$$Q_k(a)$$

добавляется бонус:

$$U_k(a).$$

14.12 Идея confidence interval

Идея UCB-алгоритмов:

$$Q(a) \leq Q_k(a) + U_k(a).$$

То есть строится доверительный интервал для истинного значения:

$$Q(a),$$

и агент выбирает верхнюю границу этого интервала.

Бонус:

$$U_k(a)$$

должен быть обратно связан с числом посещений:

$$n_k(a).$$

Если действие выбиралось мало раз, uncertainty большая, значит бонус большой.

Если действие выбиралось много раз, эмпирическое среднее надёжнее, и бонус сжимается.

14.13 Неравенство Хёфдинга

Теорема 41 (Неравенство Хёфдинга). Пусть:

$$X_1, \dots, X_n$$

— *i.i.d.* выборка из распределения на домене:

$$[0, 1]$$

с истинным средним:

$$\mu.$$

Пусть:

$$\hat{\mu} := \frac{1}{n} \sum_{i=1}^n X_i$$

— выборочная оценка среднего.

Тогда для любого:

$$u > 0$$

выполнено:

$$\mathbb{P}(\mu \geq \hat{\mu} + u) \leq e^{-2nu^2}.$$

Замечание. В бандитах обычно предполагается ограниченность наград. Для удобства часто считают:

$$r \in [0, 1].$$

14.14 Вывод UCS-бонуса

Утверждение. Для любого δ с вероятностью $1 - \delta$ истинное значение $Q(a)$ не превосходит:

$$Q_k(a) + U_k(a),$$

где:

$$U_k(a) := \sqrt{\frac{-\ln \delta}{2n_k(a)}}.$$

Доказательство. По неравенству Хёфдинга:

$$\mathbb{P}(Q(a) \geq Q_k(a) + U_k(a)) \leq \exp(-2n_k(a)U_k^2(a)).$$

Хотим, чтобы правая часть была равна δ :

$$\exp(-2n_k(a)U_k^2(a)) = \delta.$$

Берём логарифм:

$$-2n_k(a)U_k^2(a) = \ln \delta.$$

Отсюда:

$$U_k^2(a) = \frac{-\ln \delta}{2n_k(a)}.$$

Следовательно:

$$U_k(a) = \sqrt{\frac{-\ln \delta}{2n_k(a)}}.$$

14.15 Практическая форма UCB

В практическом алгоритме часто выбирают:

$$\delta$$

зависящим от номера шага k .

Одна стандартная форма:

$$a_k = \arg \max_{a \in \mathcal{A}} \left[Q_k(a) + c \sqrt{\frac{\ln k}{n_k(a)}} \right].$$

Здесь:

$$c > 0$$

— гиперпараметр, управляющий силой exploration bonus.

Если:

$$n_k(a)$$

мал, бонус большой.

Если:

$$k$$

растёт, логарифм:

$$\ln k$$

медленно увеличивает требуемую уверенность.

14.16 Сэмплирование Томпсона

14.16.1 Байесовская постановка

Предположим, что распределения наград:

$$p(r \mid a)$$

принадлежат некоторому параметрическому семейству:

$$\theta_a \in \Theta.$$

Награда генерируется из распределения:

$$p(r \mid \theta_a).$$

Для каждого действия a зададим априорное распределение:

$$p(\theta_a).$$

После очередного эпизода, где выбрали автомат a и получили награду r_k , обновляем распределение над θ_a по формуле Байеса:

$$p(\theta_a) \leftarrow p(\theta_a \mid r_k) \propto p(r_k \mid \theta_a)p(\theta_a).$$

Средний выигрыш из автомата a :

$$\mathbb{E} [p(r \mid \theta_a)]$$

теперь является случайной величиной, потому что:

$$\theta_a$$

случайна и имеет распределение:

$$p(\theta_a).$$

14.17 Probability matching

Согласно текущим распределениям:

$$p(\theta_a),$$

жадный выбор действия можно было бы записать как:

$$a := \arg \max_{a \in \mathcal{A}} \left\{ \mathbb{E}_{\theta_a \sim p(\theta_a)} [\mathbb{E} [p(r \mid \theta_a)]] \right\}.$$

Но байесовская неопределённость позволяет посчитать вероятность того, что действие a является оптимальным.

Для действия a появляется вероятность того, что хотя бы для какого-то другого автомата:

$$\hat{a} \neq a$$

выполнено:

$$\mathbb{E} [p(r \mid \theta_{\hat{a}})] > \mathbb{E} [p(r \mid \theta_a)].$$

14.18 Thompson Sampling

Определение 31. *Сэмплирование Томпсона, или **Thompson Sampling**, — процедура, при которой на k -ом шаге при решении задачи многоруких бандитов действие a выбирается с вероятностью того, что оно оптимально в рамках выученных моделей $p(\theta_a)$:*

$$\pi(a) := \mathbb{P} \left(\mathbb{E} [p(r \mid \theta_a)] = \max_{\hat{a} \in \mathcal{A}} \{ \mathbb{E} [p(r \mid \theta_{\hat{a}})] \} \right).$$

Практическая реализация. Обычно Thompson Sampling делают так:

1. Для каждого действия a сэмпляют параметр:

$$\theta_a \sim p(\theta_a).$$

2. Считают соответствующую среднюю награду:

$$\hat{Q}(a) = \mathbb{E} [p(r \mid \theta_a)].$$

3. Выбирают:

$$a_k = \arg \max_{a \in \mathcal{A}} \hat{Q}(a).$$

4. Получают награду и обновляют posterior выбранной руки.

Интуиция. Если мы не уверены в руке, posterior широкий, и у неё есть шанс быть засэмплированной как лучшая.

Если мы уверены, что рука плохая, posterior сосредоточен на малых значениях, и она редко выбирается.

14.19 Bernoulli-бандиты

Рассмотрим случай, где автоматы выдают только награду:

$$0 \text{ или } 1.$$

Для каждого действия a награда имеет распределение Бернулли с вероятностью успеха:

$$\theta_a.$$

То есть:

$$p(r \mid a) := \text{Bernoulli}(r \mid \theta_a) = \theta_a^{\mathbb{I}[r=1]}(1 - \theta_a)^{\mathbb{I}[r=0]}.$$

Так как:

$$r \in \{0, 1\},$$

можно записать:

$$p(r \mid a) = \theta_a^r (1 - \theta_a)^{1-r}.$$

14.20 Beta prior

Для параметра:

$$\theta_a \in [0, 1]$$

удобно взять Beta-распределение:

$$p(\theta_a) := \text{Beta}(\theta_a \mid \alpha, \beta) \propto \theta_a^{\alpha-1} (1 - \theta_a)^{\beta-1}.$$

Здесь:

$$\alpha, \beta$$

— параметры, свои для каждого автомата a .

Beta-распределение является сопряжённым prior для распределения Бернулли.

14.21 Bayesian update для Bernoulli-бандита

После наблюдения:

$$r_k \sim p(r \mid a)$$

применяем формулу Байеса:

$$p(\theta_a \mid r_k) \propto p(r_k \mid \theta_a) p(\theta_a).$$

Подставляем:

$$\begin{aligned} p(r_k \mid \theta_a) &= \theta_a^{r_k} (1 - \theta_a)^{1-r_k}, \\ p(\theta_a) &\propto \theta_a^{\alpha-1} (1 - \theta_a)^{\beta-1}. \end{aligned}$$

Получаем:

$$\begin{aligned} p(\theta_a \mid r_k) &\propto \theta_a^{r_k} (1 - \theta_a)^{1-r_k} \theta_a^{\alpha-1} (1 - \theta_a)^{\beta-1} \\ &= \theta_a^{\alpha+r_k-1} (1 - \theta_a)^{\beta+1-r_k-1}. \end{aligned}$$

Следовательно:

$$p(\theta_a \mid r_k) = \text{Beta}(\theta_a \mid \alpha + r_k, \beta + 1 - r_k).$$

Для обновления параметров достаточно:

$$\alpha \leftarrow \alpha + r_k,$$

$$\beta \leftarrow \beta + 1 - r_k.$$

Если получен успех:

$$r_k = 1,$$

то увеличивается α .

Если получена неудача:

$$r_k = 0,$$

то увеличивается β .

14.22 Среднее значение выигрыша в Bernoulli-бандите

При заданном:

$$\theta_a$$

ценность автомата:

$$\hat{Q}(a) = \mathbb{E}[p(r \mid \theta_a)] = \theta_a.$$

Если:

$$\theta_a \sim \text{Beta}(\alpha, \beta),$$

то математическое ожидание:

$$\mathbb{E}_{\theta_a} [\hat{Q}(a)] = \mathbb{E}_{\text{Beta}(\theta_a \mid \alpha, \beta)} [\theta_a] = \frac{\alpha}{\alpha + \beta}.$$

Интерпретация параметров. Если starting prior:

$$\text{Beta}(1, 1),$$

то это равномерное распределение на $[0, 1]$.

После наблюдений:

$$\alpha = 1 + \text{число успехов},$$

$$\beta = 1 + \text{число неудач}.$$

На слайде с Beta-распределениями видно, как posterior постепенно становится более узким: чем больше успехов и неудач наблюдалось, тем меньше неопределённость относительно θ_a .

14.23 Планирование

Определение 32. Для данного состояния s_0 набор будущих действий:

$$a_0, a_1, a_2, \dots$$

называется **планом**, или *plan*.

Пусть агент находится в состоянии:

$$s_0.$$

Он хочет найти хорошее действие:

$$a_0.$$

После выбора:

$$a_0$$

сэмплируется следующее состояние:

$$s_1,$$

затем выбирается:

$$a_1,$$

и так строится план.

Такие алгоритмы могут не обучать непосредственно зависимость:

$$\pi(a \mid s),$$

а планировать будущие действия в момент принятия решения.

Определение 33. При доступной функции переходов:

$$p(s' \mid s, a)$$

и функции награды:

$$R(T)$$

по траектории T задача выбора плана называется **планированием**, или *planning*.

Формально:

$$\arg \max_{(a_0, a_1, \dots) \in \mathcal{A}^\infty} \left\{ \mathbb{E}_{T|s_0, a_0, a_1, \dots} [R(T)] \right\}.$$

Отличие planning от policy learning. В policy learning мы учим:

$$\pi(a \mid s).$$

В planning мы можем каждый раз заново искать лучший план:

$$a_0, a_1, \dots$$

для текущего состояния.

14.24 World Models

Определение 34. *Моделью мира, или **world model**, называется любая модель, явно или неявно обучающаяся модели динамики среды.*

Обычно модель мира пытается предсказывать:

$$s', \quad r,$$

по текущим:

$$s, a.$$

То есть:

$$p_{\theta}(s', r \mid s, a).$$

14.25 Общая схема model-based подхода

Гиперпараметры.

- планировщик;
- модель мира.

Инициализация. Проинициализировать стратегию:

$$\pi_0(a \mid s),$$

модель мира случайно, а датасет:

$$\mathcal{D}$$

пустым множеством.

На k -ом шаге:

1. Повзаимодействовать со средой при помощи:

$$\pi_k,$$

добавив встреченные переходы:

$$(s, a, r, s')$$

в датасет.

2. Провести дообучение модели мира на собранном датасете:

$$\mathcal{D}.$$

3. Получить новую стратегию:

$$\pi_{k+1}$$

из планировщика, используя текущую модель мира.

14.26 Модель прямой динамики

Определение 35. Модель функции переходов:

$$p_{\theta}(s' \mid a, s)$$

называется *моделью прямой динамики*, или *forward dynamics model*.

Учить полную генеративную модель:

$$p_{\theta}(s', r \mid s, a)$$

может быть дорого.

Более простой вариант — обучать детерминированное приближение:

$$s' \approx f_{\theta}(a, s).$$

Это можно делать по любым доступным траекториям, собранным любым способом.

Функция потерь для предсказания следующего состояния:

$$\sum_{s,a,s'} \|f_{\theta}(a, s) - s'\|^2 \rightarrow \min_{\theta}.$$

Отдельно можно обучать модель награды:

$$r_{\psi}(s, a) \approx r.$$

Функция потерь:

$$\sum_{s,a,r} (r_{\psi}(s, a) - r)^2 \rightarrow \min_{\psi}.$$

14.27 Сновидения

Если есть модель прямой динамики:

$$p_{\theta}(s', r \mid a, s),$$

то можно генерировать траектории с помощью текущей стратегии:

$$\pi.$$

При этом реальная внешняя среда не используется.

Определение 36. *Сновидениями*, или *dreaming*, называется обучение агента на опыте, собранном при помощи приближения динамики среды:

$$p_{\theta}(s', r \mid a, s).$$

Интуиция. Агент как будто тренируется во внутреннем симуляторе.

Плюс: можно получить много дешёвого опыта.

Минус: если модель мира ошибается, агент может выучить стратегию, которая эксплуатирует ошибки модели, но плохо работает в реальной среде.

14.28 Интересные результаты

Во второй части лекции обсуждаются современные статьи:

- *Optimistic Posterior Sampling for Reinforcement Learning with Few Samples and Tight Guarantees*, Daniil Tiapkin, Denis Belomestny et al., 2022;
- *From Dirichlet to Rubin: Optimistic Exploration in RL without Bonuses*, Daniil Tiapkin, Denis Belomestny et al., 2022;
- *Federated UCBVI: Communication-Efficient Federated Regret Minimization with Heterogeneous Agents*, Safwan Labbi, Daniil Tiapkin et al.

14.29 Optimistic Posterior Sampling for Reinforcement Learning

Рассматривается RL в среде, моделируемой эпизодическим конечным stage-dependent MDP.

Параметры задачи:

- горизонт эпизода:

$$H;$$

- число состояний:

$$S;$$

- число действий:

$$A.$$

Качество агента измеряется regret после взаимодействия со средой в течение:

$$T$$

эпизодов.

Предложен алгоритм:

$$\text{OPSRL},$$

то есть **Optimistic Posterior Sampling for Reinforcement Learning**.

Это вариант posterior sampling, которому нужно только логарифмическое по:

$$H, S, A, T$$

число posterior samples на каждую пару состояние–действие.

14.30 Regret bound для OPSRL

Для OPSRL гарантируется high-probability regret bound порядка:

$$\tilde{O}\left(\sqrt{H^3 SAT}\right),$$

если игнорировать полилогарифмические множители:

$$\text{polylog}(HSAT).$$

Этот bound совпадает с нижней оценкой порядка:

$$\Omega\left(\sqrt{H^3 SAT}\right).$$

Тем самым решаются open problems из работы Agrawal and Jia для episodic setting.

Ключевой технический ингредиент — новая sharp anti-concentration inequality для линейных форм.

В частности, lower bound на основе normal approximation для Beta-распределений расширяется на Dirichlet-распределения.

14.31 Теорема: regret bound for OPSRL

Теорема 42 (Regret bound for OPSRL). *Рассмотрим параметр:*

$$\delta \in (0, 1).$$

Пусть:

$$\begin{aligned}\kappa &\triangleq 2 \left(\log \frac{12SAH}{\delta} + 3 \log(e\pi(2T+1)) \right), \\ n_0 &\triangleq \lceil \kappa (c_0 + \log_{17/16}(T)) \rceil, \quad r_0 \triangleq 2,\end{aligned}$$

где c_0 — абсолютная константа:

$$c_0 \triangleq \left(\frac{4}{\sqrt{\log(17/16)}} + 8 + \frac{49 \cdot 4\sqrt{6}}{9} \right)^2 \frac{8}{\pi} + \log_{17/16} \left(\frac{20}{32} \right) + 1.$$

Тогда для OPSRL с вероятностью хотя бы $1 - \delta$:

$$\mathfrak{R}^T = \tilde{O} \left(\sqrt{H^3 SATL^3} + H^3 S^2 AL^3 \right),$$

где:

$$L \triangleq \tilde{O} \left(\log \frac{HSAT}{\delta} \right).$$

14.32 Алгоритм OPSRL

Input.

- семейство распределений:

$$\rho : \mathbb{N}_+^{S+1} \rightarrow \Delta_{S'}$$

над переходами;

- initial pseudo-count:

$$n_h^0;$$

- число posterior samples:

$$J.$$

Для $t \in [T]$: Для всех:

$$(s, a, h) \in \mathcal{S} \times \mathcal{A} \times [H]$$

сэмплировать J независимых переходов:

$$\tilde{p}_h^{t-1,j}(s, a) \sim \rho \left(\left(n_h^{t-1}(s' \mid s, a) \right)_{s' \in \mathcal{S}'} \right), \quad j \in [J].$$

14.32.1 Optimistic backward induction

Положить:

$$\bar{V}_{H+1}^{t-1}(s) = 0.$$

Рекурсивно для:

$$h \in [H]$$

вычислять:

$$\bar{Q}_h^{t-1}(s, a) = r_h(s, a) + \max_{j \in [J]} \left\{ \tilde{p}_h^{t-1,j} \bar{V}_{h+1}^{t-1}(s, a) \right\},$$

$$\bar{V}_h^{t-1}(s) = \max_{a \in \mathcal{A}} \bar{Q}_h^{t-1}(s, a),$$

$$\pi_h^t(s) \in \arg \max_{a \in \mathcal{A}} \bar{Q}_h^{t-1}(s, a).$$

Далее для:

$$h \in [H]$$

выполнять:

1. сыграть:

$$a_h^t = \pi_h^t(s_h^t);$$

2. пронаблюдать:

$$s_{h+1}^t \sim p_h(s_h^t, a_h^t);$$

3. увеличить pseudo-count:

$$n_h^t(s_{h+1}^t \mid s_h^t, a_h^t).$$

14.33 Экспериментальные результаты OPSRL

На grid-world среде со:

$$100$$

состояниями и:

$$4$$

действиями, при:

$$H = 50$$

и transition noise:

$$0.2,$$

сравнивались:

$$\text{OPSRL}, \quad \text{PSRL}, \quad \text{RLSVI}, \quad \text{UCBVI}, \quad \text{UCBVI-B}.$$

График показывает regret по эпизодам, усреднённый по 4 seeds.

Основной смысл: OPSRL сравнивается с классическими и posterior-sampling baseline-алгоритмами по скорости накопления regret.

14.34 From Dirichlet to Rubin: Optimistic Exploration in RL without Bonuses

В этой работе предложен алгоритм:

BayesUCBVI.

Он рассматривается для tabular, stage-dependent, episodic MDP и является естественным расширением алгоритма:

BayesUCB

для multi-armed bandits.

Для BayesUCBVI доказывается regret bound порядка:

$$\tilde{O}\left(\sqrt{H^3 SAT}\right),$$

где:

- H — длина эпизода;
- S — число состояний;
- A — число действий;
- T — число эпизодов.

Этот bound совпадает с lower bound:

$$\Omega\left(\sqrt{H^3 SAT}\right)$$

с точностью до poly-log множителей по:

$$H, S, A, T$$

для достаточно большого T .

14.35 Идея BayesUCBVI

Метод использует квантиль posterior распределения Q -функции как upper confidence bound на оптимальную Q -функцию.

То есть вместо явного бонуса:

$$Q + \text{bonus}$$

берётся высокий квантиль posterior по Q .

Важное утверждение авторов: это первый алгоритм, который получает оптимальную зависимость от горизонта H и числа состояний S без сложного Bernstein-like bonus или искусственного шума.

Ключевой элемент анализа — fine-grained anticoncentration bound для weighted Dirichlet sum.

Также обсуждается связь BayesUCBVI с Bayesian bootstrap Rubin.

14.36 Сравнение regret upper bounds

Для episodic, non-stationary, tabular MDPs в лекции приводится сравнение:

Алгоритм	Upper bound
UCBVI, UCBAAdvantage, RLSVI	$\tilde{O}(\sqrt{H^3 SAT})$
PSRL, Agrawal and Jia	$\tilde{O}(H^2 S \sqrt{AT})$
BayesUCBVI	$\tilde{O}(\sqrt{H^3 SAT})$
Lower bound	$\Omega(\sqrt{H^3 SAT})$

Главная идея: BayesUCBVI достигает оптимального порядка regret, matching lower bound с точностью до логарифмических множителей.

14.37 Теорема для BayesUCBVI

Теорема 43. Рассмотрим параметр:

$$\delta > 0.$$

Пусть:

$$n_0 \triangleq \lceil c_{n_0} + \log_{17/16}(T) \rceil, \quad r_0 \triangleq 2,$$

где:

$$c_{n_0} = \frac{1}{(\sqrt{2\pi} - 1)^2} \left(\frac{2\sqrt{2}}{\sqrt{\log(17/16)}} + \frac{98\sqrt{6}}{9} \right)^2 + \frac{\log(10\pi)}{\log(17/16)}.$$

Тогда для BayesUCBVI с вероятностью хотя бы $1 - \delta$:

$$\mathfrak{R}^T = \tilde{O} \left(\sqrt{H^3 SAT L} + H^3 S^2 AL^2 \right),$$

где:

$$L \triangleq \tilde{O} \left(\log \frac{HSAT}{\delta} \right).$$

14.38 Алгоритм BayesUCBVI

Input.

- quantile functions:

$$(\kappa^t)_{t \in [T]};$$

- prior distribution:

$$\rho^0.$$

Для $t \in [T]$:

1. Выполнить optimistic planning.
2. Для:

$$h \in [H]$$

сыграть:

$$a_h^t \in \arg \max_{a \in \mathcal{A}} Q_h^{t-1}(s_h^t, a).$$

3. Пронаблюдать:

$$s_{h+1}^t \sim p_h(s_h^t, a_h^t).$$

4. Обновить posterior distributions:

$$\rho_h^t(s_h^t, a_h^t)$$

с использованием перехода:

$$(s_h^t, a_h^t, s_{h+1}^t).$$

14.39 Эксперименты BayesUCBVI

На графиках лекции сравниваются:

BayesUCBVI, IncrBayesUCBVI,

и baseline-алгоритмы.

Параметры одного из экспериментов:

$$H = 30,$$

transition noise:

$$0.1,$$

усреднение по:

$$4$$

seeds.

Также приводится сравнение deep RL алгоритмов на Atari-57 по median human normalized score:

DoubleDQN, BootDQN, BayesUCBDQN.

Графики показывают average $\pm$ standard deviation по 3 seeds.

14.40 Federated UCBVI

Federated UCBVI — это communication-efficient federated regret minimization with heterogeneous agents.

В работе представлен алгоритм:

Fed-UCBVI.

Он является расширением UCBVI для federated learning setting.

Основная гарантия: regret Fed-UCBVI масштабируется как:

$$O\left(\sqrt{\frac{H^3 |\mathcal{S}| |\mathcal{A}| T}{M}}\right),$$

с небольшим дополнительным членом из-за неоднородности.

Здесь:

$$M$$

— число агентов.

В одноагентном случае bound совпадает с minimax lower bound с точностью до polylog множителей.

В многоагентном случае появляется линейное ускорение по числу агентов:

$$M.$$

14.41 Setting: policies and value functions

Рассматривается M агентов.

Детерминированная политика:

$$\pi$$

задаётся набором функций:

$$\pi_h : \mathcal{S} \rightarrow \mathcal{A}, \quad h \in [H].$$

То есть:

$$\pi_h(s) \in \mathcal{A}.$$

Для агента i value function политики π на шаге h :

$$V_{i,\pi}^h(s) = \mathbb{E}_\pi \left[\sum_{h'=h}^H r_i^{h'}(s_i^{h'}, a_i^{h'}) \mid s_i^h = s \right].$$

Здесь:

$$\begin{aligned} a_i^{h'} &\sim \pi_i^{h'}(\cdot \mid s_i^{h'}), \\ s_i^{h'+1} &\sim P_i^{h'}(\cdot \mid s_i^{h'}, a_i^{h'}). \end{aligned}$$

14.42 Q-функция и уравнения Беллмана в Fed-UCBVI

Q-функция политики π для агента i на шаге h :

$$Q_{i,\pi}^h(s, a) := \mathbb{E}_\pi \left[\sum_{h'=h}^H r_i^{h'}(s_i^{h'}, a_i^{h'}) \mid s_i^h = s, a_i^h = a \right].$$

Уравнения Беллмана:

$$Q_{i,\pi}^h(s, a) = r_i^h(s, a) + P_i^h V_{i,\pi}^{h+1}(s, a),$$

$$V_{i,\pi}^{h+1}(s) = Q_{i,\pi}^h(s, \pi_h(s)).$$

Оптимальная Q-функция удовлетворяет:

$$Q_{i,\diamond}^h(s, a) = r_i^h(s, a) + P_i^h V_{i,\diamond}^{h+1}(s, a),$$

$$V_{i,\diamond}^h(s) = \max_{a \in \mathcal{A}} Q_{i,\diamond}^h(s, a).$$

14.43 Learning protocol

В начале каждого эпизода:

$$t \in [T]$$

все агенты выбирают общую политику:

$$\pi^t.$$

Эта политика вычисляется на основе информации, которой агенты обменялись до эпизода t .

Затем каждый агент генерирует независимую траекторию длины:

$$H.$$

На шаге h агент i :

1. наблюдает состояние:

$$s_i^{t,h} \in \mathcal{S};$$

2. выбирает действие:

$$a_i^{t,h} = \pi_t^h(s_i^{t,h});$$

3. наблюдает следующее состояние:

$$s_i^{t,h+1} \sim P_i^h(\cdot \mid s_i^{t,h}, a_i^{t,h});$$

4. получает детерминированную награду:

$$r_i^{t,h} = r_i^h(s_i^{t,h}, a_i^{t,h}).$$

После генерации траекторий агенты могут обмениваться информацией через центральный сервер.

14.44 Federated regret

Определение 37. *Federated regret определяется как:*

$$R(T) := \max_{\pi} \frac{1}{M} \sum_{i=1}^M \sum_{t=1}^T (V_{i,\pi}^1(s_i^{t,1}) - V_{i,\pi^t}^1(s_i^{t,1})).$$

Эта величина измеряет cumulative difference между:

- средним значением оптимальной collaborative policy;
- политиками, которые использовались во время обучения.

14.45 Communication complexity and cost

Communication complexity:

$$C(T)$$

— число эпизодов, в которых происходит коммуникация между центральным сервером и агентами.

Communication cost — общее число битов, которыми обменялись центральный сервер и агенты во время обучения.

Цель federated RL алгоритма — одновременно минимизировать:

$$R(T)$$

и:

$$C(T).$$

14.46 Environmental heterogeneity

Среды, в которых живут разные агенты, могут отличаться.

Но так как агенты должны обучить общую политику, неоднородность сред должна быть небольшой.

Вводится новая мера heterogeneity: переходное ядро каждого агента раскладывается на:

- общую часть, shared by all agents;
- индивидуальную часть, отражающую особенности среды агента.

14.46.1 Assumption A-1

Существуют:

- non-homogeneous transition kernel:

$$\{P_c^h\}_{h \in [H]};$$

- индивидуальные transition kernels:

$$\{P_{\text{ind},i}^h\}_{h \in [H]}, \quad i \in [M];$$

- константа:

$$\varepsilon_p \in [0, 1);$$

такие что для любого:

$$i \in [M], \quad (s, a, s', h) \in \mathcal{S} \times \mathcal{A} \times \mathcal{S} \times [H],$$

выполнено:

$$P_i^h(s' \mid s, a) = (1 - \varepsilon_p)P_c^h(s' \mid s, a) + \varepsilon_p P_{\text{ind},i}^h(s' \mid s, a).$$

Из этого следует:

$$\max_{(s,a,h) \in \mathcal{S} \times \mathcal{A} \times [H]} \|P_c^h(\cdot \mid s, a) - P_i^h(\cdot \mid s, a)\|_1 \leq \varepsilon_p.$$

14.46.2 Assumption A-2

Существует константа:

$$\varepsilon_r \in [0, 1)$$

такая что для всех:

$$(i, j) \in [M]$$

и всех:

$$h \in [H]$$

выполнено:

$$\|r_i^h - r_j^h\|_\infty \leq \varepsilon_r.$$

14.47 Fed-UCBVI algorithm

Fed-UCBVI расширяет UCBVI на federated learning framework.

Алгоритм работает через несколько communication rounds с центральным сервером.

Число эпизодов в каждом communication round, или epoch:

$$r,$$

является случайным.

Каждая epoch состоит из трёх фаз:

1. Data collection.

Каждый агент взаимодействует со своей средой, используя политику:

$$\pi^{(r)}$$

от центрального сервера, и собирает trajectory data.

2. Synchronization.

Когда любой агент выполняет synchronization condition, он отправляет synchronization signal на центральный сервер.

Центральный сервер рассылает информацию всем остальным агентам.

3. Policy update.

Все агенты участвуют в H последовательных коммуникациях с центральным сервером.

На каждом шаге:

$$h = H, H - 1, \dots, 1$$

агенты отправляют локальные оценки Q -values и связанную информацию для шага h .

В ответ получают глобальную оценку V -values, обновлённую политику и связанную информацию.

14.48 Data collection

В начале раунда:

$$r$$

каждый агент:

$$i \in [M]$$

следует политике:

$$\pi^{(r)}$$

и собирает новые траектории.

Для:

$$\ell \in \mathbb{N}$$

обозначим:

$$n_i^{(r,\ell),h}(s, a)$$

число посещений пары:

$$(s, a)$$

на шаге h после ℓ эпизодов в раунде r .

Также:

$$n_i^{(r,\ell),h}(s, a, s')$$

— число переходов:

$$(s, a) \rightarrow s'$$

на шаге h после ℓ эпизодов в раунде r .

14.49 Synchronization

В начале epoch r все агенты получают текущие global counters:

$$N^{(r),h}(s, a) := \sum_{i=1}^M n_i^{(r),h}(s, a),$$

где:

$$n_i^{(r),h}(s, a) := n_i^{(r,0),h}(s, a)$$

— число посещений пары (s, a) агентом i до раунда r .

Во время epoch r , после ℓ эпизодов, агент i отправляет synchronization signal, если обнаружена новая visited state-action-step triplet:

$$(s, a, h),$$

и выполнено одно из двух условий.

Эти условия зависят от того, превышает ли:

$$N^{(r),h}(s, a)$$

порог:

$$\nu(\delta, T) = O(\varepsilon_p THM + M).$$

14.49.1 Local Doubling Condition

Если:

$$N^{(r),h}(s, a) \leq \nu(\delta, T),$$

агент i отправляет synchronization signal, если:

$$n_i^{(r,\ell),h}(s, a) > 2n_i^{(r),h}(s, a).$$

14.49.2 Globally Estimated Doubling Condition

Если:

$$N^{(r),h}(s, a) > \nu(\delta, T),$$

агент i отправляет synchronization signal, если:

$$\widehat{N}_i^{(r,\ell),h}(s, a) > 2N^{(r),h}(s, a).$$

Здесь:

$$\widehat{N}_i^{(r,\ell),h}(s, a)$$

— оценка:

$$\sum_{i=1}^M n_i^{(r,\ell),h}(s, a)$$

на основе информации, доступной агенту i .

14.50 Policy update

После synchronization signal каждый агент вычисляет локальные оценки переходов:

$$P_i^{(r+1),h}(s' \mid s, a) := \frac{n_i^{(r+1),h}(s, a, s')}{n_i^{(r+1),h}(s, a)},$$

если:

$$n_i^{(r+1),h}(s, a) > 0.$$

Если:

$$n_i^{(r+1),h}(s, a) = 0,$$

то используется равномерная оценка:

$$P_i^{(r+1),h}(s' \mid s, a) := \frac{1}{|\mathcal{S}|}.$$

Далее агенты и центральный сервер обмениваются оценками Q - и V -значений.

Для:

$$h = H, \dots, 1$$

каждый агент вычисляет local Q-value estimate:

$$\widehat{Q}_i^{(r+1),h}(s, a) := r_i^h + P_i^{(r+1),h} \widehat{V}^{(r+1),h+1}(s, a),$$

используя global value estimate:

$$\widehat{V}^{(r+1),h+1},$$

полученную от центрального сервера.

14.51 Aggregation in Fed-UCBVI

Агрегированная Q -оценка:

$$\widehat{Q}^{(r+1),h}(s, a) := \min \left\{ T_{\omega}^{(r+1),h}(s, a) + b_i^{(r+1),h}(s, a), H \right\}.$$

Здесь:

$$T_{\omega}^{(r+1),h}(s, a) = \sum_{i=1}^M \omega_i^{(r+1),h}(s, a) \widehat{Q}_i^{(r+1),h}(s, a),$$

а веса:

$$\omega_i^{(r+1),h}(s, a) := \frac{n_i^{(r+1),h}(s, a)}{N^{(r+1),h}(s, a)}.$$

Центральный сервер обновляет value function:

$$V^{(r+1),h}(s) := \max_{a \in \mathcal{A}} \widehat{Q}^{(r+1),h}(s, a),$$

и политику:

$$\pi^{(r+1),h}(s) := \arg \max_{a \in \mathcal{A}} \widehat{Q}^{(r+1),h}(s, a).$$

Обновлённые значения рассылаются всем агентам.

Процесс продолжается для всех:

$$h = H, \dots, 1.$$

Когда достигается:

$$h = 1,$$

начинается новая epoch:

$$r + 1.$$

14.52 Communication complexity: идея doubling condition

Дизайн Fed-UCBVI похож на работы по RL с low switching cost.

Число раз, когда меняется локальная data collection policy, называется:

switching cost.

В federated framework это соответствует числу communication rounds:

$$C(T).$$

Простейшая doubling condition:

$$\exists(s, a, h) : N^{(r,\ell),h}(s, a) > 2N^{(r),h}(s, a),$$

где:

$$N^{(r,\ell),h}(s, a) := \sum_{i=1}^M n_i^{(r,\ell),h}(s, a).$$

Но в federated setting это условие нельзя напрямую проверить: глобальный счётчик:

$$N^{(r,\ell),h}(s, a)$$

не доступен отдельному агенту.

14.53 Почему нужен globally estimated doubling

Можно использовать более слабое local doubling condition.

Но тогда communication complexity масштабируется линейно по числу агентов:

$$M,$$

что непрактично для больших federated systems.

Поэтому предлагается строить оценку глобального счётчика:

$$\widehat{N}_i^{(r,\ell),h}(s,a)$$

и использовать её как plug-in estimate.

Такие оценки могут быть неточными в начале обучения.

Но после того, как число посещений превышает порог:

$$\nu(\delta, T) = O(\varepsilon_p T H M + M),$$

они становятся достаточно надёжными.

14.54 Main results for Fed-UCBVI

Теорема 44 (Communication Complexity). *C вероятностью хотя бы $1 - \delta$ число communication rounds Fed-UCBVI ограничено:*

$$C(T) \leq O(|\mathcal{S}||\mathcal{A}|H \log T + M|\mathcal{S}||\mathcal{A}|H \log \log T + M|\mathcal{S}||\mathcal{A}|H \log(1 + \varepsilon_p T)),$$

где логарифмическая зависимость от:

$$|\mathcal{S}|, \quad |\mathcal{A}|, \quad H, \quad 1/\delta, \quad M$$

опущена.

Теорема 45 (Regret bound for Fed-UCBVI). *C вероятностью хотя бы $1 - \delta$ для Fed-UCBVI выполнено:*

$$R(T) = O \left(\sqrt{\frac{H^3 |\mathcal{S}| |\mathcal{A}| T}{M}} + H^3 |\mathcal{S}|^2 |\mathcal{A}| + O(T H (H \varepsilon_p + \varepsilon_r)) \right).$$

Смысл. Первое слагаемое:

$$\sqrt{\frac{H^3 |\mathcal{S}| |\mathcal{A}| T}{M}}$$

показывает ускорение за счёт M агентов.

Последнее слагаемое:

$$O(T H (H \varepsilon_p + \varepsilon_r))$$

отражает цену неоднородности сред и наград.

Если:

$$\varepsilon_p = 0, \quad \varepsilon_r = 0,$$

агенты полностью однородны, и этот штраф исчезает.

14.55 Итог

В этой лекции были рассмотрены три большие темы.

Multi-Armed Bandits. Задача многорукого бандита — простейшая RL-задача без состояния:

$$Q(a) = \mathbb{E}[r \mid a].$$

Качество измеряется regret:

$$\mathfrak{R}^T = TV^* - \sum_{k=0}^{T-1} Q(a_k).$$

Главная проблема — exploration–exploitation trade-off.

ε -greedy и UCB. ε -greedy прост, но при постоянном ε имеет линейный regret.

UCB использует optimism in the face of uncertainty:

$$a_k = \arg \max_a \left[Q_k(a) + c \sqrt{\frac{\ln k}{n_k(a)}} \right].$$

Thompson Sampling. Thompson Sampling поддерживает posterior над параметрами наград:

$$p(\theta_a),$$

и выбирает действие с вероятностью того, что оно оптимально.

Для Bernoulli-бандита удобно использовать Beta prior:

$$\text{Beta}(\alpha, \beta),$$

с обновлением:

$$\alpha \leftarrow \alpha + r_k, \quad \beta \leftarrow \beta + 1 - r_k.$$

World Models. Model-based RL обучает модель динамики:

$$p_\theta(s', r \mid s, a)$$

или детерминированное приближение:

$$s' \approx f_\theta(s, a).$$

После этого можно планировать или обучаться на сгенерированных траекториях, то есть использовать dreaming.

Современные результаты. В лекции также обсуждались современные regret-optimal подходы:

- OPSRL;
- BayesUCBVI;
- Fed-UCBVI.

Их общая тема — эффективное исследование, posterior uncertainty, optimism и строгие regret-гарантии.

15 Лекция 15. Планирование: MCTS, LQR и iLQR

15.1 В предыдущих сериях

В предыдущей лекции обсуждалась задача многорукого бандита.

Для неё была введена функция сожаления:

$$\mathfrak{R}^T := TV^* - \sum_{k=0}^{T-1} Q(a_k) \rightarrow \min_{(a_k)_{k=0}^{T-1} \in \mathcal{A}^T}.$$

Были рассмотрены:

1. ε -жадный бандит;
2. нестационарный бандит;
3. UCB-бандит;
4. сэмплирование Томпсона;
5. Bernoulli-бандиты.

Также была поставлена задача планирования.

Определение 38. Для данного состояния s_0 набор действий:

$$a_0, a_1, a_2, \dots$$

называется **планом**.

Если известны функция переходов:

$$p(s' \mid s, a)$$

и функция награды, то задача планирования записывается как:

$$\arg \max_{(a_0, a_1, \dots) \in \mathcal{A}^\infty} \left\{ \mathbb{E}_{T|s_0, a_0, a_1, \dots} [R(T)] \right\}.$$

Также была введена модель мира.

Определение 39. Модель функции переходов:

$$p_\theta(s' \mid s, a)$$

называется **моделью прямой динамики**.

Определение 40. Сновидениями называется обучение агента на опыте, собранном при помощи приближения динамики среды:

$$p_\theta(s', r \mid s, a).$$

15.2 MCTS: постановка задачи

Пусть у нас имеется идеальный симулятор среды для MDP с дискретным пространством действий:

$$|\mathcal{A}| < \infty.$$

Хотим построить алгоритм планирования для задачи:

$$\arg \max_{(a_0, a_1, \dots) \in \mathcal{A}^\infty} \left\{ \mathbb{E}_{T|s_0, a_0, a_1, \dots} [R(T)] \right\}.$$

Здесь:

$$s_0$$

— текущее состояние, из которого нужно выбрать хорошее действие.

15.3 Наивное построение дерева среды

Если среда детерминированная, и известна функция переходов:

$$s' = f(s, a),$$

то можно было бы построить полное дерево среды.

В этом дереве:

- корень соответствует состоянию s_0 ;
- каждое ребро соответствует действию a ;
- каждый следующий узел соответствует состоянию, полученному после применения действия.

Если построить всё дерево до конца эпизода, то можно гарантированно получить оптимальный план.

Проблемы.

1. В сложных средах дерево растёт экспоненциально быстро:

$$|\mathcal{A}|^H,$$

где H — горизонт планирования.

2. Реальные среды часто стохастичны:

$$s', r \sim p(s', r \mid s, a).$$

Поэтому полное дерево обычно строить невозможно.

15.4 Дерево MDP

Определение 41. *Деревом MDP с корнем в состоянии s_0 будем называть дерево, где:*

- каждой дуге соответствует действие a ;
- узел на t -ом уровне соответствует плану:

$$a_0, a_1, \dots, a_{t-1},$$

то есть пути от корня до этого узла.

В MCTS дерево строится не полностью, а постепенно и эффективно.

Для каждого узла N и действия a из него храним:

- счётчик прохождения по дуге:

$$n(N, a);$$

- оценку ценности дуги:

$$Q(N, a).$$

Смысл:

$$Q(N, a)$$

— текущая Monte-Carlo оценка того, насколько хорошим оказывается выбор действия a в узле N .

15.5 Monte-Carlo Tree Search

Один шаг процедуры **Monte-Carlo Tree Search**, или **MCTS**, состоит из четырёх этапов:

1. Selection;
2. Expansion;
3. Simulation, или Evaluation;
4. Update, или Backpropagation.

15.6 MCTS: Selection

На этапе **Selection** мы выбираем ветку дерева, которую будем расширять.

Процедура.

1. Стартуем в корне дерева, которому соответствует текущее состояние реальной игры:

$$s_0.$$

2. Далее идём по дереву, пока не попадём в лист дерева на некотором уровне t .

3. В каждом внутреннем узле используем стратегию:

TreePolicy.

4. TreePolicy выбирает действие:

$$a_i.$$

5. После выбора действия спускаемся по дереву на уровень глубже по дуге, соответствующей a_i .

6. Так как среда может быть стохастической, сэмплируем:

$$s_{i+1}, r_i \sim p(s_{i+1}, r_i \mid s_i, a_i).$$

В результате мы получаем траекторию от корня до листа:

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_t.$$

Фактически мы выбираем начало некоторого плана:

$$a_0, a_1, \dots, a_{t-1}.$$

15.7 TreePolicy как бандитная задача

Задача TreePolicy — выбрать, в какую ветку дерева идти дальше.

В каждом узле N , который не является листом, известны:

- оценки ценности действий:

$$Q(N, a), \quad a \in \mathcal{A};$$

- числа посещений дуг:

$$n(N, a), \quad a \in \mathcal{A}.$$

Нужно выбрать наиболее перспективное действие, но при этом иногда исследовать недостаточно изученные ветки.

Это ровно аналог задачи многорукого бандита.

Поэтому в качестве TreePolicy хорошо подходит UCB.

15.8 UCB внутри MCTS

В узле N обозначим:

$$n(N) := \sum_{a \in \mathcal{A}} n(N, a).$$

Тогда TreePolicy может выбирать действие:

$$a := \arg \max_{a \in \mathcal{A}} \left\{ Q(N, a) + C \sqrt{\frac{\log n(N)}{n(N, a)}} \right\}.$$

Здесь:

$$C > 0$$

— гиперпараметр UCSB.

Первое слагаемое:

$$Q(N, a)$$

отвечает за exploitation.

Второе слагаемое:

$$C \sqrt{\frac{\log n(N)}{n(N, a)}}$$

отвечает за exploration.

Если дуга редко посещалась, то:

$$n(N, a)$$

мало, и exploration bonus большой.

15.9 MCTS: Expansion

После Selection мы оказываемся в листе дерева.

На этапе **Expansion** в выбранном листе создаём новые листья.

Для каждого действия:

$$a_t \in \mathcal{A}$$

создаём новый узел-ребёнок, соответствующий выбору этого действия.

Так дерево расширяется вдоль выбранной ветки на один шаг вперёд.

Замечание 24. Если $|\mathcal{A}|$ очень велико, то можно расширять дерево не по всем действиям, а по случайному подмножеству действий.

15.10 MCTS: Simulation, или Evaluation

На этапе **Simulation** нужно оценить reward-to-go для новых листьев.

Идея: из нового листа запускается симуляция до конца эпизода.

Для этого используется стратегия:

Default Policy.

Часто Default Policy — просто случайная стратегия.

Например, для каждого действия:

$$a_t$$

из состояния:

$$s_t$$

можно просимулировать одну или несколько игр до конца эпизода, выбирая дальнейшие действия по Default Policy.

Полученная суммарная награда:

$$R(T_a)$$

используется как Monte-Carlo оценка ценности нового листа.

Почему Default Policy не использует дерево. После Expansion новые узлы ещё не содержат статистики:

$$Q(N, a), \quad n(N, a).$$

Поэтому дальше из них нельзя продолжать TreePolicy. Вместо этого используется простая эвристическая политика.

15.11 MCTS: Update, или Backpropagation

На этапе **Update** обновляются счётчики и оценки во всех дугах дерева, по которым мы прошли на данном шаге.

Пусть для некоторой посещённой дуги:

$$(N, a)$$

получена Monte-Carlo оценка reward-to-go:

$$\hat{V}.$$

Тогда можно обновить:

$$Q(N, a) \leftarrow Q(N, a) + \frac{1}{n(N, a)} \left(\hat{V} - Q(N, a) \right),$$

$$n(N, a) \leftarrow n(N, a) + 1.$$

Это такое же инкрементальное Monte-Carlo среднее, как в задаче многорукого бандита.

15.12 Применение MCTS: хорошие практики

Практически MCTS используют так.

1. Предобучение дерева обычно не проводится.

Дерево постоянно обновляется в процессе реального использования в среде.

2. Перед каждым выбором действия для реальной среды делается много шагов MCTS-процедуры.

Например:

$$K = 1000$$

симуляций.

3. После очередного реального шага агент переходит в новое состояние.

Тогда в дереве можно спуститься в соответствующий узел и оставить только поддерево этого узла.

Это экономит уже накопленную статистику.

4. Для выбора действия в реальной среде не обязательно использовать TreePolicy.

TreePolicy содержит exploration, который нужен для построения дерева, но не всегда нужен на инференсе.

Часто действие для реальной среды выбирают по числу посещений:

$$\pi(a_0 \mid s_0) \propto n(N_0, a_0)^T,$$

где:

$$T$$

— температура.

Если T большая, распределение более резко концентрируется на наиболее посещаемом действии.

15.13 Итоговый алгоритм MCTS

Вход.

- текущее состояние реальной среды:

$$s_0;$$

- симулятор:

$$p(s', r \mid s, a);$$

- гиперпараметр UCB:

$$C;$$

- корень текущего дерева:

$$N_0;$$

- в дереве хранятся счётчики:

$$n(N, a)$$

и оценки:

$$Q(N, a);$$

- число шагов MCTS:

$$K;$$

- температура:

$$T.$$

Выход. Стратегия выбора первого действия:

$$\pi(a_0 \mid s_0) \propto n(N_0, a_0)^T.$$

15.13.1 Один шаг MCTS

На k -ом шаге из K :

1. Садимся в корень:

$$N := N_0, \quad s := s_0.$$

2. Инициализируем траекторию симуляции:

$$\mathcal{T} := (s_0).$$

3. Пока N не является листом:

(a) считаем:

$$n(N) = \sum_{a \in \mathcal{A}} n(N, a);$$

(b) выбираем ветку:

$$a := \arg \max_{a \in \mathcal{A}} \left\{ Q(N, a) + C \sqrt{\frac{\log n(N)}{n(N, a)}} \right\};$$

(c) генерируем:

$$s', r \sim p(s', r \mid s, a);$$

(d) сохраняем:

$$a, r, s'$$

в траекторию $\mathcal{T}$;

(e) спускаемся по дереву:

$$N \leftarrow \text{child}(N, a), \quad s \leftarrow s'.$$

4. Для каждого действия:

$$a \in \mathcal{A}$$

выполнить Expansion и Simulation:

(a) создать узел:

$$\hat{N}$$

— ребёнка N с дугой для действия a ;

(b) симулировать траекторию:

$$T_a \sim \pi_{\text{random}}(a \mid s);$$

(c) инициализировать:

$$n(\hat{N}, a) := 1, \quad Q(\hat{N}, a) := R(T_a).$$

5. Для каждой посещённой дуги:

$$(N, a)$$

выполнить Update:

(a) посчитать:

$$\hat{V}$$

— суммарный reward-to-go в траектории $\mathcal{T}$, полученный после посещения данной дуги;

(b) награду после посещения листа оценить как:

$$\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} R(T_a);$$

(c) обновить:

$$Q(N, a) \leftarrow Q(N, a) + \frac{1}{n(N, a)} \left(\hat{V} - Q(N, a) \right);$$

(d) обновить счётчик:

$$n(N, a) \leftarrow n(N, a) + 1.$$

Итог. MCTS — дорогостоящая по времени, разумная по памяти и практически работающая процедура поиска хороших действий в игре.

15.14 Планирование для непрерывного управления

Теперь перейдём к планированию в непрерывном пространстве действий.

Пусть:

- модель динамики среды известна;
- функция награды известна;
- обе функции дифференцируемы;
- среда детерминированная;
- эпизод состоит из T шагов;
- дисконтирование:

$$\gamma = 1;$$

- пространство действий непрерывно:

$$\mathcal{A} \equiv \mathbb{R}^d.$$

Функции награды и динамики могут дополнительно зависеть от дискретного времени:

$$t \in \{1, \dots, T\}.$$

Так как динамика детерминированная, выбор действий полностью определяет траекторию.

Поэтому нужно искать оптимальное управление, то есть набор действий:

$$a_1, a_2, \dots, a_T,$$

которые приведут к наилучшей траектории.

15.15 Математическая формулировка задачи управления

Задача:

$$\begin{cases} \sum_{t=1}^T r_t(s_t, a_t) \rightarrow \max_{(a_1, \dots, a_T) \in \mathcal{A}^T}, \\ s_t = f_{t-1}(s_{t-1}, a_{t-1}), \quad t = 2, \dots, T. \end{cases}$$

Здесь:

$$s_1$$

считается известным начальным состоянием.

15.16 Прямое дифференцирование

Один вариант решения — построить вычислительный граф, похожий на RNN.

Схематически:

$$s_1 \rightarrow a_1 \rightarrow s_2 \rightarrow a_2 \rightarrow \dots \rightarrow s_T \rightarrow a_T.$$

Действия можно получать из некоторой параметризованной политики:

$$a_t = \pi_\theta(s_t, t),$$

а состояния через динамику:

$$s_{t+1} = f_t(s_t, a_t).$$

После этого суммарная награда:

$$\sum_{t=1}^T r_t(s_t, a_t)$$

дифференцируема по параметрам политики или непосредственно по действиям.

Проблема. Так же как в RNN, при большом T градиенты могут затухать или взрываться.

Поэтому прямое backpropagation-through-time может быть неэффективным.

15.17 Linear Quadratic Regulator: мотивация

Рассмотрим упрощённую постановку.

Если заменить:

- функцию динамики линейным приближением;
- функцию награды квадратичным приближением;

то задачу можно аналитически решить методом динамического программирования.

Такой подход называется:

Linear Quadratic Regulator,

или:

LQR.

15.18 LQR: обозначения

Пусть динамика линейна по состоянию и действию:

$$f_t(s, a) = F_t \begin{bmatrix} s \\ a \end{bmatrix} + f_t.$$

Пусть награда квадратична:

$$r_t(s, a) = \frac{1}{2} \begin{bmatrix} s \\ a \end{bmatrix}^\top R_t \begin{bmatrix} s \\ a \end{bmatrix} + \begin{bmatrix} s \\ a \end{bmatrix}^\top r_t.$$

Матрицы:

$$F_t, \quad R_t$$

и векторы:

$$r_t, \quad f_t$$

считаются известными.

Матрицу R_t и вектор r_t разбиваем на блоки:

$$R_t := \begin{bmatrix} R_{t,s,s} & R_{t,s,a} \\ R_{t,a,s} & R_{t,a,a} \end{bmatrix}, \quad r_t := \begin{bmatrix} r_{t,s} \\ r_{t,a} \end{bmatrix}.$$

Для упрощения считаем:

$$R_{t,s,a} = R_{t,a,s}^\top.$$

Для существования и единственности решения предполагаем отрицательную определённость матриц:

$$R_t, \quad t = 1, \dots, T.$$

15.19 LQR: последний шаг

Теорема 46. *Оптимальное действие в последний момент времени:*

$$a_T^*$$

является линейной формой от состояния:

$$s_T.$$

Доказательство. На последнем шаге нет будущих наград, поэтому:

$$Q_T^*(s_T, a_T) = r_T(s_T, a_T).$$

То есть:

$$Q_T^*(s_T, a_T) = \frac{1}{2} \begin{bmatrix} s_T \\ a_T \end{bmatrix}^\top R_T \begin{bmatrix} s_T \\ a_T \end{bmatrix} + \begin{bmatrix} s_T \\ a_T \end{bmatrix}^\top r_T.$$

Оптимальное действие:

$$a_T^* = \arg \max_{a_T \in \mathcal{A}} Q_T^*(s_T, a_T).$$

Ищем максимум квадратичной формы, приравнявая градиент по a_T к нулю:

$$\nabla_{a_T} Q_T^*(s_T, a_T) = R_{T,a,a} a_T + R_{T,a,s} s_T + r_{T,a} = 0.$$

Отсюда:

$$a_T^* = -R_{T,a,a}^{-1} (R_{T,a,s} s_T + r_{T,a}).$$

Введём обозначения:

$$K_T := -R_{T,a,a}^{-1} R_{T,a,s},$$

$$k_T := -R_{T,a,a}^{-1} r_{T,a}.$$

Тогда:

$$a_T^* = K_T s_T + k_T.$$

То есть оптимальное действие действительно линейно зависит от состояния.

15.20 LQR: ценность последнего состояния

Теорема 47. *Ценность последнего состояния:*

$$V^*(s_T)$$

является квадратичной формой от состояния:

$$s_T.$$

Доказательство. По определению и детерминированности:

$$V^*(s_T) = Q^*(s_T, a_T^*).$$

Подставим:

$$a_T^* = K_T s_T + k_T$$

в квадратичную форму.

Получаем:

$$V^*(s_T) = \frac{1}{2} \begin{bmatrix} s_T \\ K_T s_T + k_T \end{bmatrix}^\top R_T \begin{bmatrix} s_T \\ K_T s_T + k_T \end{bmatrix} + \begin{bmatrix} s_T \\ K_T s_T + k_T \end{bmatrix}^\top r_T.$$

Подстановка линейной функции в квадратичную функцию даёт квадратичную функцию.

Поэтому:

$$V^*(s_T) = \frac{1}{2} s_T^\top V_T s_T + v_T^\top s_T + \text{const}.$$

Если отбросить константу, которая не влияет на выбор действий на предыдущих шагах, можно записать:

$$V^*(s_T) = \frac{1}{2} s_T^\top V_T s_T + v_T^\top s_T.$$

Матрица V_T выражается как:

$$V_T := R_{T,s,s} + K_T^\top R_{T,a,a} K_T + K_T^\top R_{T,a,s} + R_{T,s,a} K_T.$$

Линейный коэффициент v_T получается из линейных слагаемых после подстановки.

15.21 LQR: индуктивный шаг

Теорема 48. Для всех моментов времени t выполнено:

- оптимальные оценочные функции:

$$Q_t^*$$

являются квадратичными формами от:

$$\begin{bmatrix} s_t \\ a_t \end{bmatrix};$$

- оптимальные действия:

$$a_t^*$$

являются линейными формами от:

$$s_t;$$

- оптимальные функции ценности:

$$V_t^*$$

являются квадратичными формами от:

$$s_t.$$

Доказательство. Рассмотрим шаг:

$$T - 1.$$

По определению:

$$Q_{T-1}^*(s_{T-1}, a_{T-1}) = r_{T-1}(s_{T-1}, a_{T-1}) + V_T^*(s_T).$$

Так как динамика линейна:

$$s_T = F_T \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + f_T.$$

Тогда:

$$\begin{aligned} Q_{T-1}^*(s_{T-1}, a_{T-1}) &= r_{T-1}(s_{T-1}, a_{T-1}) \\ &\quad + V_T^* \left(F_T \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + f_T \right). \end{aligned}$$

Поскольку:

$$V_T^*(s) = \frac{1}{2} s^\top V_T s + v_T^\top s + \text{const},$$

получаем:

$$\begin{aligned} &V_T^* \left(F_T \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + f_T \right) \\ &= \frac{1}{2} \left(F_T \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + f_T \right)^\top V_T \left(F_T \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + f_T \right) \\ &\quad + v_T^\top \left(F_T \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + f_T \right) + \text{const}. \end{aligned}$$

Раскрывая скобки и отбрасывая константу, получаем квадратичную форму:

$$Q_{T-1}^*(s_{T-1}, a_{T-1}) = \frac{1}{2} \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix}^\top Q_{T-1} \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix} + \begin{bmatrix} s_{T-1} \\ a_{T-1} \end{bmatrix}^\top q_{T-1} + \text{const.}$$

При этом:

$$\begin{aligned} Q_{T-1} &:= R_{T-1} + F_T^\top V_T F_T, \\ q_{T-1} &:= r_{T-1} + F_T^\top V_T f_T + F_T^\top v_T. \end{aligned}$$

Так как Q_{T-1}^* снова квадратична по состоянию и действию, оптимальное действие находится из условия:

$$\nabla_{a_{T-1}} Q_{T-1}^* = 0.$$

Отсюда:

$$a_{T-1}^* = K_{T-1} s_{T-1} + k_{T-1},$$

где:

$$\begin{aligned} K_{T-1} &:= -Q_{T-1,a,a}^{-1} Q_{T-1,a,s}, \\ k_{T-1} &:= -Q_{T-1,a,a}^{-1} q_{T-1,a}. \end{aligned}$$

Подстановка этого действия в:

$$V_{T-1}^*(s_{T-1}) = Q_{T-1}^*(s_{T-1}, a_{T-1}^*)$$

снова даёт квадратичную форму по s_{T-1} .

Дальше аналогично раскручиваем назад:

$$T-2, T-3, \dots, 1.$$

15.22 LQR: обратный проход

Вход.

- параметры динамики:

$$F_t, f_t, \quad t = 1, \dots, T;$$

- параметры награды:

$$R_t, r_t, \quad t = 1, \dots, T.$$

Выход. Линейная стратегия:

$$\pi_t(s) := K_t s + k_t, \quad t = 1, \dots, T.$$

Алгоритм. Инициализировать:

$$V_{T+1} = 0_{|\mathcal{S}| \times |\mathcal{S}|}, \quad v_{T+1} = 0_{|\mathcal{S}|}.$$

Для:

$$t = T, T-1, \dots, 1$$

выполнить:

1. Считаём Q -функцию:

$$Q_t := R_t + F_{t+1}^\top V_{t+1} F_{t+1},$$

$$q_t := r_t + F_{t+1}^\top V_{t+1} f_{t+1} + F_{t+1}^\top v_{t+1}.$$

2. Считаём оптимальную стратегию:

$$K_t := -Q_{t,a,a}^{-1} Q_{t,a,s},$$

$$k_t := -Q_{t,a,a}^{-1} q_{t,a}.$$

3. Считаём V -функцию:

$$V_t := Q_{t,s,s} + K_t^\top Q_{t,a,a} K_t + K_t^\top Q_{t,a,s} + Q_{t,s,a} K_t.$$

Линейный коэффициент:

$$v_t$$

получается из линейных слагаемых после подстановки:

$$a_t = K_t s_t + k_t$$

в квадратичную форму $Q_t^*(s_t, a_t)$.

Замечание 25. В лекционных обозначениях для v_t используется компактная запись через блоки Q_t и q_t . Практически v_t проще получать систематическим раскрытием линейных членов после подстановки $a_t = K_t s_t + k_t$.

15.23 LQR: прямой проход

После обратного прохода известны:

$$K_t, \quad k_t, \quad t = 1, \dots, T.$$

Так как стартовое состояние известно, можно получить оптимальный план.

Вход.

- стратегия с обратного прохода:

$$K_t, k_t, \quad t = 1, \dots, T;$$

- функция динамики:

$$f_t.$$

Выход. План:

$$a_1, a_2, \dots, a_T.$$

Алгоритм. Для:

$$t = 1, \dots, T$$

ВЫПОЛНИТЬ:

$$a_t = K_t s_t + k_t,$$

$$s_{t+1} = f_t(s_t, a_t).$$

15.24 LQR для шумной динамики

LQR обобщается на случай недетерминированных сред.

Пусть динамика задаётся нормальным распределением:

$$p(s_t \mid s_{t-1}, a_{t-1}) := \mathcal{N}(f_t(s_{t-1}, a_{t-1}), \Sigma_t).$$

Здесь:

$$f_t(s_{t-1}, a_{t-1})$$

— линейная функция от предыдущего состояния и действия, а:

$$\Sigma_t$$

— фиксированная матрица ковариации, зависящая только от времени t , но не от состояния и действия.

Теорема 49. В этом предположении схема LQR остаётся неизменной.

Доказательство. В детерминированном случае:

$$Q_{t-1}^*(s_{t-1}, a_{t-1}) = r_{t-1}(s_{t-1}, a_{t-1}) + V_t^*(s_t).$$

В стохастическом случае меняется только связь Q - и V -функций:

$$Q_{t-1}^*(s_{t-1}, a_{t-1}) = r_{t-1}(s_{t-1}, a_{t-1}) + \mathbb{E}_{s_t} [V_t^*(s_t)].$$

Пусть:

$$V_t^*(s_t) = \frac{1}{2} s_t^\top V_t s_t + v_t^\top s_t + \text{const}.$$

Тогда:

$$\mathbb{E}_{s_t} [V_t^*(s_t)] = \mathbb{E}_{s_t} \left[\frac{1}{2} s_t^\top V_t s_t + v_t^\top s_t + \text{const} \right].$$

Если:

$$s_t \sim \mathcal{N}(\mu_t, \Sigma_t),$$

то:

$$\mathbb{E} [s_t^\top V_t s_t] = \text{Tr}(V_t \Sigma_t) + \mu_t^\top V_t \mu_t.$$

Здесь:

$$\mu_t = f_t(s_{t-1}, a_{t-1}).$$

Следовательно:

$$\begin{aligned} \mathbb{E}_{s_t} [V_t^*(s_t)] &= \frac{1}{2} \text{Tr}(V_t \Sigma_t) + \frac{1}{2} \mu_t^\top V_t \mu_t + v_t^\top \mu_t + \text{const} \\ &= V_t^*(\mu_t) + \text{const}. \end{aligned}$$

Член:

$$\frac{1}{2} \text{Tr}(V_t \Sigma_t)$$

не зависит от действия, если Σ_t фиксирована.

Значит, он меняет только константу Q -функции и не влияет на оптимальное управление.

Поэтому формулы для:

$$Q_t, \quad q_t, \quad K_t, \quad k_t$$

остаются теми же.

15.25 Iterative LQR: мотивация

Вернёмся к детерминированному случаю.

До сих пор LQR требовал:

- линейной динамики;
- квадратичной награды.

Но в реальности функции:

$$f, \quad r$$

часто не являются линейными и квадратичными.

Идея **Iterative LQR**, или **iLQR**, — использовать локальные приближения этих функций около текущего плана.

- Динамику приближаем разложением Тейлора первого порядка.
- Награду приближаем разложением Тейлора второго порядка.

После этого локальная задача становится LQR-задачей.

15.26 iLQR: локальная аппроксимация

Возьмём некоторый текущий план и честным прямым проходом посчитаем траекторию:

$$\hat{s}_1, \hat{a}_1, \hat{s}_2, \hat{a}_2, \dots, \hat{s}_T, \hat{a}_T.$$

В окрестности этой траектории разложим динамику:

$$f_t(s_{t-1}, a_{t-1}) \approx f_t(\hat{s}_{t-1}, \hat{a}_{t-1}) + \nabla_{s,a} f_t(\hat{s}_{t-1}, \hat{a}_{t-1})^\top \begin{bmatrix} s_{t-1} - \hat{s}_{t-1} \\ a_{t-1} - \hat{a}_{t-1} \end{bmatrix}.$$

Награду разложим до второго порядка:

$$\begin{aligned} r_t(s_t, a_t) \approx & \frac{1}{2} \begin{bmatrix} s_t - \hat{s}_t \\ a_t - \hat{a}_t \end{bmatrix}^\top \nabla_{s,a}^2 r_t(\hat{s}_t, \hat{a}_t) \begin{bmatrix} s_t - \hat{s}_t \\ a_t - \hat{a}_t \end{bmatrix} \\ & + \nabla_{s,a} r_t(\hat{s}_t, \hat{a}_t)^\top \begin{bmatrix} s_t - \hat{s}_t \\ a_t - \hat{a}_t \end{bmatrix}. \end{aligned}$$

15.27 iLQR: обозначения

Вводим обозначения по аналогии с LQR:

$$F_t := \nabla_{s,a} f_t(\hat{s}_{t-1}, \hat{a}_{t-1}),$$

$$f_t := f_t(\hat{s}_{t-1}, \hat{a}_{t-1}),$$

$$R_t := \nabla_{s,a}^2 r_t(\hat{s}_t, \hat{a}_t),$$

$$r_t := \nabla_{s,a} r_t(\hat{s}_t, \hat{a}_t).$$

Теперь можно применить LQR к локальной задаче.

15.28 Перецентрирование в iLQR

В iLQR пространства состояний и действий перецентрированы:

$$s_t - \hat{s}_t, \quad a_t - \hat{a}_t.$$

После LQR получаем локальную стратегию в координатах отклонений:

$$a_t - \hat{a}_t = K_t(s_t - \hat{s}_t) + k_t.$$

Отсюда:

$$a_t = K_t(s_t - \hat{s}_t) + k_t + \hat{a}_t.$$

Раскрывая:

$$a_t = K_t s_t - K_t \hat{s}_t + k_t + \hat{a}_t.$$

Значит, если хотим записать стратегию в обычном виде:

$$a_t = K_t s_t + \tilde{k}_t,$$

то нужно заменить:

$$k_t \leftarrow k_t - K_t \hat{s}_t + \hat{a}_t.$$

15.29 Алгоритм iLQR

Вход.

- функция награды:

$$r;$$

- функция динамики:

$$f.$$

Выход. План:

$$\hat{a}_1, \hat{a}_2, \dots, \hat{a}_T.$$

Алгоритм. Проинициализировать траекторию случайно:

$$\hat{s}_1, \hat{a}_1, \hat{s}_2, \dots, \hat{s}_T, \hat{a}_T.$$

На каждом шаге:

$$j = 1, \dots, N$$

выполнить:

1. Получить локальные параметры:

$$F_t, \quad f_t, \quad R_t, \quad r_t, \quad t = 1, \dots, T,$$

по формулам разложения Тейлора.

2. Получить матрицы:

$$K_t, \quad k_t, \quad t = 1, \dots, T,$$

при помощи алгоритма LQR с матрицами динамики:

$$F_t, f_t$$

и награды:

$$R_t, r_t.$$

3. Учесть перецентрирование:

$$k_t \leftarrow k_t - K_t \hat{s}_t + \hat{a}_t, \quad t = 1, \dots, T.$$

4. При помощи стратегии:

$$\pi_t(s_t) = K_t s_t + k_t$$

заспавнить новую траекторию:

$$\hat{s}_1, \hat{a}_1, \hat{s}_2, \dots, \hat{s}_T, \hat{a}_T,$$

используя честный прямой проход с точными функциями:

$$f, \quad r.$$

15.30 Сравнение MCTS, LQR и iLQR

Метод	Когда подходит
MCTS	Дискретные действия, есть симулятор среды. Хорошо подходит для игр и задач, где можно много раз симулировать будущее из текущего состояния.
Прямое дифференцирование	Дифференцируемые динамика и награда, непрерывные действия. Но при большом горизонте могут затухать или взрываться градиенты.
LQR	Линейная динамика и квадратичная награда. В этом случае задача решается аналитически обратным проходом динамического программирования.
iLQR	Нелинейная динамика и неквадратичная награда, но функции дифференцируемы. Метод итеративно строит локальные LQR-приближения около текущего плана.

15.31 Итог

В этой лекции были рассмотрены методы планирования.

MCTS. MCTS строит дерево MDP не целиком, а постепенно.

Один шаг MCTS состоит из:

$$\text{Selection} \rightarrow \text{Expansion} \rightarrow \text{Simulation} \rightarrow \text{Update}.$$

В Selection задача выбора ветки сводится к бандитной задаче, поэтому удобно использовать UCB:

$$a = \arg \max_{a \in \mathcal{A}} \left\{ Q(N, a) + C \sqrt{\frac{\log n(N)}{n(N, a)}} \right\}.$$

После многих симуляций действие для реальной среды можно выбирать по числу посещений:

$$\pi(a_0 \mid s_0) \propto n(N_0, a_0)^T.$$

LQR. LQR решает задачу непрерывного управления при линейной динамике:

$$f_t(s, a) = F_t \begin{bmatrix} s \\ a \end{bmatrix} + f_t$$

и квадратичной награде:

$$r_t(s, a) = \frac{1}{2} \begin{bmatrix} s \\ a \end{bmatrix}^\top R_t \begin{bmatrix} s \\ a \end{bmatrix} + \begin{bmatrix} s \\ a \end{bmatrix}^\top r_t.$$

Оптимальная стратегия имеет линейный вид:

$$a_t^* = K_t s_t + k_t.$$

Шумная динамика. Если динамика гауссова:

$$p(s_t \mid s_{t-1}, a_{t-1}) = \mathcal{N}(f_t(s_{t-1}, a_{t-1}), \Sigma_t),$$

и Σ_t не зависит от состояния и действия, то схема LQR не меняется: добавляется только константа, не влияющая на оптимальное управление.

iLQR. iLQR обобщает LQR на нелинейные задачи.

Метод итеративно:

- берёт текущий план;
- линеаризует динамику около него;
- квадратично приближает награду;
- решает локальную LQR-задачу;
- делает новый прямой проход с точной динамикой.

Главная идея:

нелинейную задачу $\longrightarrow$ последовательность локальных LQR-задач.